Faster and Smaller Anonymous Credentials from Standardized Signatures

Chad Sharp
University of Michigan

Paul Grubbs
University of Michigan

Abstract

We present GordieHowe, a system comprising a language, a certifying compiler, and a zero-knowledge proof protocol. GordieHowe is especially suitable for building anonymous credentials on top of standardized signature schemes like ECDSA, RSA, and ML-DSA.

GordieHowe is the first system to achieve the following three properties simultaneously: provable security against quantum attackers, efficiency for statements spanning multiple fields—including proving and verifying in parallel—and automatic batching of structured computations.

GordieHowe is high-assurance by design. The language is designed to enable static checking of bounds safety, preventing integer overflows. Its compiler emits a Lean certificate that its optimization decisions are semantics-preserving for the input program. The proof system's security theorems are machine-checked in Lean.

GordieHowe is faster and has smaller proofs than comparable prior work. It proves, in parallel on four cores, an ECDSA-signed mobile driving license presentation in 64.8 ms with a 276 KiB proof, 10.1× faster and 22% smaller than Google's Longfellow. With at least 101 bits of knowledge soundness in the random oracle model, GordieHowe proves knowledge of a 1 KiB SHA3-256 preimage in 5.0 ms with a 60 KiB proof, and of an RSA-2048 signature on a private 2 KiB message in 25.9 ms with a 210 KiB proof. With at least 116 bits against quantum adversaries, GordieHowe proves knowledge of an ML-DSA-44 signature on a private message in 87.3 ms with a 549 KiB proof, 70.2× faster and 29% smaller than an optimized Longfellow circuit.

In building GordieHowe we make a number of contributions of independent interest. We define a new notion of an interactive oracle proof (IOP) whose messages are partially ordered, which we call a *series-parallel* IOP. We extend the BCS transformation to series-parallel IOPs, and give knowledge soundness and ZK proofs in the QROM. We also show our BCS extension satisfies a form of simulation extractability in the QROM; we believe this is the first simulation extractability result of any kind for BCS in the QROM.

Contents

1 Introduction **3**
1.1 Technical overview . . . 4
2 Preliminaries **8**
2.1 Notation . . . 8
2.2 Polynomial identities . . . 9
2.3 Extension fields and packing . . . 9
2.4 Reed–Solomon codes . . . 11
2.5 Series–parallel IOPs . . . 12
2.6 Interactive oracle reductions . . . 15
2.7 Longfellow . . . 18
2.8 BCS . . . 18
3 The GordieHowe language **19**
3.1 Programs . . . 19
3.2 Phases and the execution of a program . . . 23
3.3 Library functions . . . 24
4 The compiler **25**
4.1 Realizations . . . 25
4.2 Subprotocols, tables and packing . . . 31
4.3 Realized programs . . . 35
5 The proof system **36**
5.1 The protocol . . . 36
5.2 Lookups . . . 40
5.3 Phases and late values . . . 41
5.4 Proving the equalities . . . 42
5.5 Zero-knowledge ring switching . . . 45
5.6 Ligero . . . 48
5.7 Parameter selection . . . 51
6 Series–parallel BCS **53**
6.1 Security notions . . . 53
6.2 Series–parallel BCS . . . 56
6.3 Security of spBCS . . . 58
6.4 Security of GordieHowe . . . 60
7 Applications and evaluation **62**
7.1 Fields . . . 63
7.2 Proofs of possession of RSA signatures . . . 63
7.3 SHA-256 preimages . . . 66
7.4 Proofs of possession of ML-DSA-44 signatures . . . 67
7.5 Mobile driving licenses . . . 69
7.6 Comparison with Binius64 . . . 72
7.7 Soundness and parameters . . . 72
7.8 Ablations . . . 74
8 Formalization **75**
8.1 Results . . . 75
8.2 The model . . . 76
8.3 Assumptions . . . 76
8.4 Problems in published analyses . . . 77
9 Related work **77**
10 Future work **81**
References **82**
Appendices **94**

1 Introduction

Zero-knowledge proofs let a prover convince a verifier that a statement holds while revealing nothing else about the prover's secret inputs [GMR85]. For decades they were mainly a theoretical tool. Proof systems are now fast enough to deploy. Cryptocurrencies use them to hide the parties and amounts of payments [BCG+14], and digital identity systems now use them to show a fact about a person without revealing the document that attests it [Sta25b].

A digital credential, such as a mobile driving license (mDL) [ISO21] or an electronic passport [Int21], is a document signed by its issuer. To show a property of the holder, for example that the holder is over 18, without revealing the credential, the holder proves in zero knowledge that it knows a credential whose signature verifies and whose fields have the property. Google deploys Longfellow for such presentations [Fs24, Sta25a, Sta25b]. These relations are hard to prove efficiently, because their signature schemes were not designed for zero-knowledge proofs. Verification hashes the document with SHA-256 [Nat15a] or SHA-3 [Nat15b], which compute on bits, and then checks an equation modulo a prime for ECDSA [Nat23], modulo a product of two unknown primes for RSA [MKJR16], or computes in a polynomial ring and hashes the result again for ML-DSA [Nat24]. One relation therefore spans several fields, or it uses arithmetic that is native to none of them. The prover often runs on a phone, and both proving and verification happen while the holder waits, so we prioritize prover time, then verifier time, then proof size.

However, these relations often have structure that a proof system can exploit. A hash applies one compression function to every block of the document, and most of its values are bits. ML-DSA multiplies polynomials with small coefficients, and RSA multiplies large integers that split into small limbs. Both products are convolutions, and a public linear map, the number-theoretic transform, turns each into many independent products.

NIST has standardized post-quantum signatures [Nat24], and a proof about such a signature must also be sound against quantum adversaries. Otherwise a quantum prover could present a credential that it could not forge directly. The proof's parameters must therefore come from a concrete soundness bound in the quantum random oracle model (QROM). Such bounds exist for proofs compiled with the BCS transformation [CMS19, CDHZ25].

No existing system both exploits this structure and gives parameters against quantum adversaries. Longfellow proves the ECDSA mDL relation in two fields, linked by a MAC [Fs24]. It proves every constraint of a field in one GKR circuit [GKR15], without batching repeated constraints or using lookups, so it exploits little of the structure. Its security analyses are classical and give no parameters for quantum adversaries. Vega exploits the structure: it folds the repeated SHA-256 steps into one step and uses lookups to parse the credential [KS25]. But it proves the whole relation in one field, and its commitments rest on discrete logarithms, so it is not post-quantum. Binius64 exploits the structure of hashing and is plausibly post-quantum [Irr26b], but no analysis bounds its soundness against quantum adversaries. It also computes only over binary fields, so it emulates arithmetic modulo a prime. To our knowledge, the only machine-checked security analysis among them is Google's concurrent analysis of Longfellow in the random oracle model (ROM) [Goo26b].

We present GordieHowe, a language, a compiler and a proof system for such relations. A GordieHowe program states a relation over several fields, with variables typed by field and subfield and with predicates such as congruences, lookups and equalities between fields. The compiler chooses how to lower each predicate to constraints, links the fields with a single polynomial MAC, and rejects a program in which an integer equality or congruence could wrap around the characteristic of its field. It groups repeated constraints, such as the round constraint of a hash, into families and dispatches each family to a subprotocol chosen by an empirically calibrated cost model. The proof system commits values in a subfield densely, so that one element of $\mathbb{F}_{2^{128}}$ holds 128 bits. It

composes the subprotocols of different fields, and the batches within a field, in parallel, so the prover can compute them on separate cores. The proof system turns the compiler's output, a realized program, into a transparent non-interactive argument of knowledge for the program's relation, which is knowledge sound in the QROM and has everlasting zero knowledge. We machine-check in Lean that the argument has these properties for every realized program that Lean checks to be valid, whenever its source program is complete and round-by-round knowledge sound. For our ML-DSA-44 proofs of possession, Lean also proves this condition.

At our ROM parameters (Table 3), our applications have at least 99 bits of knowledge soundness against classical adversaries in the ROM. The applications that we evaluate at our QROM parameters have at least 116 bits there against quantum adversaries in the QROM. At the ROM parameters on four cores, GordieHowe proves knowledge of a 1 KiB SHA3-256 preimage in 5.0 ms with a 59.8 KiB proof. It proves possession of an RSA-2048 signature on a 2 KiB message in 25.9 ms with a 209.8 KiB proof. It proves Longfellow's ECDSA mDL presentation in 64.8 ms with a 275.5 KiB proof, 10.1× faster than Longfellow. At the QROM parameters, GordieHowe proves possession of an ML-DSA-44 signature in 87.3 ms with a 549.2 KiB proof, 70.2× faster than our optimized Longfellow baseline. The companion paper [SWPG26] describes our program for ML-DSA-44.

We present the following contributions:

- a language for relations over several fields, and a compiler that lowers its predicates, chooses the subprotocol that proves each constraint and links each pair of fields with a single polynomial MAC (Sections 3, 4 and 4.1.4),
- a proof system that proves repeated constraints with a batched zerocheck and commits subfield values densely, reducing claims on them by zero-knowledge ring switching (Sections 5.4.2 and 5.5),
- a round-by-round analysis of Ligero's [AHIV22] linear and quadratic tests up to the Johnson radius, and a mask test that closes a gap in the published analyses of Ligero (Section 5.6),
- series–parallel BCS (spBCS), which compiles interactive oracle proofs (IOPs) with subprotocols composed in parallel into arguments that are knowledge sound and, to our knowledge for the first time for BCS, weakly online simulation extractable in the QROM (Section 6),
- an evaluation on six applications against Longfellow, Binius64 and the proof of Baum and Beckmann [BB26], with an estimated comparison against Vega (Section 7), and
- a Lean formalization of the security of the proof system in the ROM and the QROM, with an end-to-end machine-checked proof for ML-DSA-44 proofs of possession (Section 8).

Open source availability GordieHowe's compiler and proof system and our Lean formalization are available as open source at `https://github.com/cmlsharp/gordiehowe`.

1.1 Technical overview

GordieHowe works in two steps. Offline, the programmer states a relation as a program in the GordieHowe language, and the compiler turns the program into a realized program, which fixes the interactive protocol that proves the relation. Online, the GordieHowe proof system uses the realized program to compute and verify proofs. We first describe the two steps and then the ideas behind each component.

The offline step: the GordieHowe language and its compiler. A program in the GordieHowe language (Section 3) declares variables and states predicates over them. Like the bellman library [zkc26], the language is embedded in a host language, here C++. Each variable has a field, a type that may restrict it to a subfield or to bits, and one of four kinds: part of the statement, a value the prover chooses, a value the verifier computes, or a verifier challenge. The language has four core predicates: equality, sharing of bits between two fields, lookup in a table, and congruence modulo an integer. A standard library builds SHA-256, Keccak, permutation checks [GWC19] and offline memory checking [BEG+91] from these predicates. Figure 1 shows a program.

The compiler (Section 4) has two jobs. It transforms a source program into a realized program, and it exports both programs to Lean, with certificates for its choices. Lean's kernel then checks that the realized program meets the conditions under which our proof system is secure, and it checks each rewriting of a source predicate. The translation works as follows. First it expresses every predicate with equalities and lookups (Section 4.1). It rewrites each congruence as an equality with a range-checked quotient, and it checks equalities between fields with a single polynomial MAC (Section 4.1.4). Then it groups the equalities into constraint families, each the equalities that apply one polynomial to different affine operands, and chooses the subprotocol that proves each family (Section 4.2). Affine families go to Ligero's linear test. For every other family, the compiler estimates the sum of prover and verifier time of each subprotocol that admits it (GKR, a batched zerocheck and Ligero's quadratic test) with an empirically calibrated cost model, and it chooses the subprotocol with the lowest estimated cost. With the same cost model, the compiler chooses the lookup tables of each field and the packing of subfield values into each field's commitment. From the program's dependencies on challenges, it infers the phases in which the verifier draws them (Section 3.2). Last, for each field it chooses the Ligero parameters to minimize estimated proof size subject to the per-round soundness target (Section 4.2.4).

A Lean theorem shows that whenever the kernel's checks pass, the predicates of the realized program imply those of the source program, except with small probability over the challenges of randomized checks such as the MAC, and every witness of the source program extends to one of the realized program. We show that if the source program is complete and round-by-round knowledge sound for a relation, then the non-interactive argument that GordieHowe produces from a valid realized program whose Ligero parameters meet the conditions of Theorem 5.3 is complete, knowledge sound and zero knowledge for that relation (Theorem 6.7). These guarantees rest on the assumptions of Section 8.3, chiefly three assumptions about the GKR circuits that Longfellow's circuit compiler produces.

The online step: proving with GordieHowe. The GordieHowe proof system (Section 5) builds an interactive oracle proof (IOP) from the realized program and makes it non-interactive. The messages of this IOP are ordered by a series–parallel partial order (Definition 2.18), so in place of the standard BCS transformation the proof system uses our series–parallel BCS transformation (spBCS, Section 6).

The prover first commits, for each field, its witness values and one-time pads in one Ligero tableau, which packs subfield values densely into field elements. The verifier then draws the program's challenges in phases, and the prover sends each value that depends on them masked with a committed pad (Section 5.3). Next, the subprotocols that the compiler chose reduce the predicates to linear claims and products on committed values. A batched zerocheck proves each batch of constraint families, and GKR, on a circuit that Longfellow's circuit compiler produces, proves the remaining equalities and the lookups of each field. The prover masks each of its messages with a committed pad, so the verifier's checks are linear in the pads and become claims on the committed values.

This design follows Ben-Or et al. [BGG+90], whose prover hides its messages and proves that the verifier would accept them. Longfellow [Fs24] and Hyrax [WTs+18] follow the same design. Ring switching [DP26, Sou24] reduces the claims on subfield values to claims on the packed elements. Last, Ligero's linear and quadratic tests prove all these claims by opening some columns of each tableau, and on the same columns our subfield test checks that the subfield masks of ring switching lie in their subfield (Section 5.5.2 and Protocol 5.1).

1.1.1 Key ideas of GordieHowe

We now describe the key ideas of the three components, comparing them mostly with Longfellow. Section 9 compares GordieHowe with other work.

The GordieHowe language. Lookups are predicates of the language, and the compiler proves them with LogUp [Hab22]. Longfellow has no lookup argument, and it range-checks values bit by bit. Programs can draw verifier challenges, so permutation checks [GWC19] and offline memory checking [BEG+91] are ordinary library functions. GordieHowe locates the fields of an mDL credential with a permutation check, where Longfellow shifts the whole signed mobile security object through a barrel shifter [Goo26a]. Because congruences are predicates, the compiler bounds every integer expression, and so it can reject programs whose equalities could wrap around modulo the field characteristic. Chaliasos et al. list such overflows as a root cause of under-constrained circuits among the 141 SNARK vulnerabilities that they collect from audit reports and bug trackers [CET+24, §5, p. 3862]. Finally, our language's support for explicitly verified subfield types, combined with zero-knowledge ring switching (Section 5.5), lets us pack subfield values together to shrink the tableau and improve efficiency.

Longfellow marks subfield values only to blind them with subfield randomness and to shorten its opened columns. Its verifier does not check their membership [Fs26, §4.5 and §7.4], and each entry of its tableau holds one subfield value.

The compiler. An important design choice of the compiler is how it checks equalities between fields. Longfellow links its two fields with a MAC that needs a committed key share and a tag for every 128-bit chunk of a shared value (Section 2.7). Each tag is a multiplication in $\mathbb{F}_{2^{128}}$, which Longfellow's circuit over the field of odd characteristic emulates. This is expensive: the MACs account for 13.4% of Longfellow's prover time for its ECDSA mDL, which shares six chunks, and 51.0% for our Longfellow baseline for ML-DSA-44, which shares 52 chunks. The tags also reveal whether a chunk is zero, which is harmless for Longfellow's digests and keys but breaks zero knowledge in general. For example, tags on an ML-DSA-44 challenge polynomial with two bits per coefficient reveal a zero chunk with probability about $2^{-15.8}$ (Section 4.1.4), so our ML-DSA-44 baseline adds a blind to each tag. Our compiler links all values that two fields share with a single polynomial MAC (Section 4.1.4), whose masked tag hides every chunk, including zero chunks. In characteristic 2 its check is linear in the committed bits. In odd characteristic it is a convolution, which Ligero's linear test checks, and table lookups range-check its quotient digits. On one core, replacing our MAC with Longfellow's per-chunk MAC makes GordieHowe's prover 1.7× slower in the ECDSA mDL and 6.2× slower in the ML-DSA-44 proof of possession (Section 7.8).

The compiler also derives the interactive structure of the proof from the program. In Longfellow, the application author writes the link between its two fields by hand, including the order of its commitments and challenges [Fs24, Goo26a]. Our compiler places every challenge and variable in its least phase, across all fields (Section 3.2). For each value that depends on a challenge, the prover

commits a pad with its phase-0 values and later sends the value masked by the pad (Section 4.1.5), so each field keeps a single Ligero tableau.

The GordieHowe proof system. The BCS transformation [BCS16] and its follow-ups derive each challenge from every earlier message. This order of derivation serializes the prover across parallel subprotocols, and the analyses of these transformations do not cover transcripts forked across subprotocols. One of Longfellow's authors writes that it is unclear how to formulate the round-by-round argument for such forked transcripts, since the parallel subprotocols may have different numbers of rounds [as26]. We resolve this by defining series–parallel IOPs, whose messages are ordered by a series–parallel partial order, and our spBCS transformation, which derives each challenge from the messages that precede it in that order (Section 6). We prove that spBCS gives arguments that are knowledge sound and weakly online simulation extractable in the ROM and the QROM, with concrete errors (Theorems 6.6 and I.17). To our knowledge, this is the first proof of weak online simulation extractability in the QROM for arguments compiled with the BCS transformation. We also prove everlasting zero knowledge and unconditional witness indistinguishability.

Our batched zerocheck (Section 5.4.2) combines the zerocheck of Setty and of Chen et al. [Set20, CBBZ23] with Gruen's univariate skip [Gru24], and it masks its messages with committed pads as in Longfellow. Longfellow's GKR can prove copies of one circuit, but its mDL circuits use a single copy and lay out the SHA-256 rounds as separate gates [Goo26a], so its proofs do not exploit the repetition. When GordieHowe proves SHA-256 on a 2 KiB message with Longfellow's circuit in GKR, its prover is 10.7× slower than with the batched rounds that the compiler chooses (Section 7.3).

As in FRI-Binius, Binius64 and Flock [DP26, Irr26b, BRW26], ring switching reduces the claims of our batched zerocheck on packed values to claims on the packed elements. We make ring switching zero knowledge by adding a committed mask to each of the prover's messages. The verifier's check then raises each mask to a power of the Frobenius map, and our proof system checks committed values only through linear claims, products and subfield membership. When a subfield contains the evaluation points, the masks lie in it, so the power is linear in their entries, and our subfield test checks their membership on the opened columns at no cost in proof size (Lemma 5.10). For a smaller subfield in characteristic 2, such as $\mathbb{F}_2$ packed into $\mathbb{F}_{2^{16}}$, the prover commits each mask with its chain of squares, which product constraints check (Section 5.5.2).

Finally, we modify Ligero and give an improved analysis of it. To our knowledge, we give the first published round-by-round analysis of Ligero's linear and quadratic tests beyond the unique-decoding radius (Section 5.6.2), where fewer opened columns meet the soundness target. Chiesa, Fenzi and Weissenberg [CFW26] and Arnon, Boneh and Fenzi [ABF26] analyze Ligero-style linear tests in this regime, and Google's concurrent Lean analysis of Longfellow, released as a repository, also bounds Ligero's rounds beyond this radius [Goo26b]. We also identify a gap in published soundness analyses of Ligero. Ames et al.'s soundness lemma for the quadratic test, for fewer corrupted columns than half the code's distance [AHIV22, Lem. B.9], assumes that the mask rows of the masked tests have low degree, but no test enforces this, and the lemma fails for mask rows of higher degree. The main bounds of Ames et al., for fewer corrupted columns than a quarter of the distance, still hold. The gap also affects Cascudo et al.'s analysis of Longfellow, whose Theorem 4.4 fails for the same reason [CHVY25].

Two modifications let us get the full benefit of this regime. Beyond unique decoding, the error from the untested mask rows can dominate, so we introduce an additional mask test, which checks the mask rows' degree (Section 5.6.2). With the mask test in place, the error of the low-degree test can dominate instead over 128-bit fields. That error is an exceptional count divided by the field size, and the count grows as the agreement threshold falls. The compiler therefore repeats the

low-degree test with independent challenges when this lets it lower the threshold and open fewer columns (Section 4.2.4). Without our mask test, proofs of the ECDSA mDL and the ML-DSA-44 proof of possession are 30% larger (Section 7.8).

1.1.2 Formalization

The proof system composes many subprotocols, its analysis in the QROM has many steps, and published QROM proofs have contained gaps [DFMS21, App. A] [BBD+23a]. We therefore formalize our results in Lean. For every series–parallel IOP, Lean proves the security of its spBCS transformation in the ROM and the QROM, including weak online simulation extractability, and for every realized program that passes the kernel's checks it proves the security of the resulting argument (Theorem 6.7), under the assumptions of Section 8.3. For ML-DSA-44 proofs of possession, Lean also proves that the source program is complete and round-by-round knowledge sound for the verification relation of FIPS 204, so the security of these proofs is machine-checked end to end. The formalization has about 190,000 lines of statements and proofs and declares no axioms. The Lean kernel checks every proof, including proofs by computation (Section 8). To our knowledge, no earlier mechanized proof covers the BCS transformation for a general class of IOPs in any model, a Fiat–Shamir-style transformation [FS87] of an IOP in the QROM, or zero knowledge of such a transformation for a general class of IOPs (Section 9). Our proofs of extraction in the QROM rest on Zhandry's compressed oracle [Zha18], which we formalize on what is, to our knowledge, the first model of the QROM in Lean. Formalizing the analysis of Ligero is also how we found the gap in its published analyses (Section 5.6.1).

Organization. Section 2 introduces the notation and the protocols that GordieHowe builds on. Section 3 describes the GordieHowe language and Section 4 its compiler. Section 5 gives the proof system, and Section 6 makes it non-interactive with spBCS and proves the security of GordieHowe. Section 7 applies GordieHowe to proofs of possession of RSA and ML-DSA-44 signatures, SHA-256 preimages and mobile driving licenses, and compares it with Longfellow, Binius64 and other systems. Section 8 describes our formalization. We discuss related work in Section 9 and future work in Section 10.

2 Preliminaries

This section fixes notation, recalls the definitions and lemmas that later sections use, and then describes Longfellow and the BCS transformation.

2.1 Notation

We denote the natural numbers including 0 as $\mathbb{N}$. We write $[n] = \{0, \dots, n-1\}$, and for integers $a \le b$, $[a, b] = \{a, \dots, b\}$ and $[a, b) = \{a, \dots, b-1\}$. The falling factorial is $(x)_t = x(x-1)\cdots(x-t+1)$. For a finite set S, $|S|$ is its number of elements. For a string or vector s, $|s|$ is its length, and $s \| s'$ is the concatenation of s and s'. We write $A \subseteq B$ when A is a subset of B, and $A \subset B$ when it is a proper subset. The same notation applies to subfields.

For a set R and a finite set S, R^S is the set of vectors with entries in R indexed by S. For a ring R and $a, y \in R^S$, $\langle a, y \rangle = \sum_{i \in S} a_i y_i$. The Boolean hypercube of dimension m is $\{0,1\}^m$. We index a table of length 2^m by the points of $\{0,1\}^m$, identifying $b \in \{0,1\}^m$ with the integer $\sum_{j \in [m]} 2^j b_j$.

For a prime power n, $\mathbb{F}_n$ is the field with n elements. In this section, $\mathbb{F}$ is an arbitrary finite field. For a subfield $\mathbb{F} \subseteq K$, $[K : \mathbb{F}]$ is the degree of K over $\mathbb{F}$. The ring of polynomials in n variables over $\mathbb{F}$ is $\mathbb{F}[X_0, \ldots, X_{n-1}]$, and $\deg p$ is the total degree of a polynomial p.

For a finite set S, $x \xleftarrow{\$} S$ denotes sampling x uniformly from S. We write $y \leftarrow A(x)$ for running the algorithm A on x and assigning its output to y. We write $\Pr_{x \xleftarrow{\$} S}[E]$ for the probability of an event E over the choice of x, and $\Pr[E : \mathrm{G}]$ for the probability of E in the experiment G. The statistical distance of random variables X and Y is $\mathsf{SD}(X, Y) = \frac{1}{2}\sum_z |\Pr[X = z] - \Pr[Y = z]|$. The trace distance of density operators ρ and σ is $\mathsf{TD}(\rho, \sigma) = \frac{1}{2}\|\rho - \sigma\|_1$, with $\|\cdot\|_1$ the trace norm. The trace distance of two diagonal density operators is the statistical distance of the distributions on their diagonals.

A *relation* is a set $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ of pairs of a *statement* $\mathsf{x} \in \mathcal{X}$ and a *witness* $\mathsf{w} \in \mathcal{W}$. Its *language* is $\mathcal{L}(\mathcal{R}) = \{\mathsf{x} \in \mathcal{X} : \exists\, \mathsf{w} \in \mathcal{W},\ (\mathsf{x}, \mathsf{w}) \in \mathcal{R}\}$.

2.2 Polynomial identities

Many of our soundness arguments use the fact that a nonzero polynomial of low degree has few roots.

Lemma 2.1 (Schwartz–Zippel [Sch79, Zip79], as stated in [Tha23, Lem. 3.3])**.** *Let $\mathbb{F}$ be a field and let $h \in \mathbb{F}[X_0, \ldots, X_{n-1}]$ be a nonzero polynomial of total degree at most d. Then for every finite set $S \subseteq \mathbb{F}$, $\Pr_{x \xleftarrow{\$} S^n}[h(x) = 0] \leq d/|S|$.*

Lemma 2.2 (Polynomial-evaluation hash)**.** *Let $\mathbb{F}$ be a finite field, $n \geq 1$, and $x, y \in \mathbb{F}^{n+1}$ with $x \neq y$. Then $\Pr_{\kappa \xleftarrow{\$} \mathbb{F}}\left[\sum_{c=0}^{n} x_c \kappa^c = \sum_{c=0}^{n} y_c \kappa^c\right] \leq n/|\mathbb{F}|$.*

Proof. The difference $\sum_{c=0}^{n}(x_c - y_c)\kappa^c$ is a nonzero polynomial of degree at most n in κ, so Lemma 2.1 with $S = \mathbb{F}$ gives the bound. □

Wegman and Carter build message authentication from strongly universal hash families, composing them with hash stages whose collision probability is small [WC81].

Lemma 2.3 (LogUp, following the proof of [Hab22, Lem. 5])**.** *Let $\mathbb{F}$ be a finite field, $\mathsf{tb} \in \mathbb{F}^{|\mathsf{tb}|}$ a table and $v \in \mathbb{F}^N$, with N below the characteristic of $\mathbb{F}$. Every v_i is an entry of tb if and only if there is $m \in \mathbb{F}^{|\mathsf{tb}|}$ with*

$$\sum_{i=0}^{N-1} \frac{1}{X - v_i} = \sum_{j=0}^{|\mathsf{tb}|-1} \frac{m_j}{X - \mathsf{tb}_j}$$

as rational functions. If every v_i is an entry of tb, the identity holds with m_j the number of i with $v_i = \mathsf{tb}_j$ if j is the first index at which the entry tb_j occurs, and $m_j = 0$ otherwise.

Haböck writes the identity with $X + a_i$ and $X + b_j$. Setting $a_i = -v_i$ and $b_j = -\mathsf{tb}_j$ gives the identity of Lemma 2.3.

2.3 Extension fields and packing

Like Binius [DP25], GordieHowe packs several elements of a subfield into one element of an extension field, and it turns linear claims on the packed values into claims on the extension field by ring switching [DP26]. Packing and ring switching both use the trace and dual bases, so we recall them first.

Definition 2.4 (Frobenius map and trace [Gao93, §2.1])**.** Let r be a prime power and $L \supseteq \mathbb{F}_r$ a finite field. The *Frobenius map* of L over $\mathbb{F}_r$ is $\mathrm{Frob}_{\mathbb{F}_r} : L \to L$, $\mathrm{Frob}_{\mathbb{F}_r}(x) = x^r$. For $k \in \mathbb{N}$, $\mathrm{Frob}_{\mathbb{F}_r}^k$ is its k-fold composite and $\mathrm{Frob}_{\mathbb{F}_r}^{-k}$ the inverse of $\mathrm{Frob}_{\mathbb{F}_r}^k$. For $L = \mathbb{F}_{r^n}$, the *trace* of $\mathbb{F}_{r^n}$ over $\mathbb{F}_r$ is $\mathrm{Tr}_{\mathbb{F}_{r^n}/\mathbb{F}_r} : \mathbb{F}_{r^n} \to \mathbb{F}_r$,

$$\mathrm{Tr}_{\mathbb{F}_{r^n}/\mathbb{F}_r}(x) = \sum_{i=0}^{n-1} x^{r^i} = \sum_{i=0}^{n-1} \mathrm{Frob}_{\mathbb{F}_r}^i(x).$$

The Frobenius map is a field automorphism of L and generates the Galois group of L over $\mathbb{F}_r$ [Gao93, §2.1], so its fixed points are the elements of $\mathbb{F}_r$. The trace is an $\mathbb{F}_r$-linear map to $\mathbb{F}_r$ [Gao93, §2.1].

Definition 2.5 (Dual basis [Gao93, §2.1])**.** Let $\mathfrak{b}_0, \ldots, \mathfrak{b}_{n-1}$ and $\mathfrak{b}_0^*, \ldots, \mathfrak{b}_{n-1}^*$ be bases of $\mathbb{F}_{r^n}$ over $\mathbb{F}_r$. The basis $\mathfrak{b}_0^*, \ldots, \mathfrak{b}_{n-1}^*$ is the *dual basis* of $\mathfrak{b}_0, \ldots, \mathfrak{b}_{n-1}$ if $\mathrm{Tr}_{\mathbb{F}_{r^n}/\mathbb{F}_r}(\mathfrak{b}_i\, \mathfrak{b}_j^*)$ is 1 for $i = j$ and 0 for $i \neq j$.

Every basis has a unique dual basis [Gao93, §2.1]. The dual basis recovers coordinates: if $U = \sum_i x_i \mathfrak{b}_i$ with $x_i \in \mathbb{F}_r$, then linearity of the trace gives $x_j = \mathrm{Tr}_{\mathbb{F}_{r^n}/\mathbb{F}_r}(\mathfrak{b}_j^*\, U)$.

Definition 2.6 (Packing, after [DP25, §1.3, p. 7, and §4.3])**.** Let $B \subseteq T$ be finite fields with $d = [T : B]$, and fix a basis $\mathfrak{b}_0, \ldots, \mathfrak{b}_{d-1}$ of T over B with its dual basis $\mathfrak{b}_0^*, \ldots, \mathfrak{b}_{d-1}^*$ (Definition 2.5). The maps $\mathsf{pack}_{T/B} : B^d \to T$ and $\mathsf{unpack}_{T/B} : T \to B^d$ are

$$\mathsf{pack}_{T/B}(u_0, \ldots, u_{d-1}) = \sum_{j=0}^{d-1} \mathfrak{b}_j\, u_j, \qquad \mathsf{unpack}_{T/B}(U) = \big(\mathrm{Tr}_{T/B}(\mathfrak{b}_0^*\, U), \ldots, \mathrm{Tr}_{T/B}(\mathfrak{b}_{d-1}^*\, U)\big).$$

The subscript is omitted when the fields are clear from the context. The j-th value packed in $U \in T$ is $\mathsf{unpack}(U)_j$. Both maps extend componentwise to vectors: for $U \in T^m$, $\mathsf{unpack}(U) \in B^{dm}$ is the concatenation of $\mathsf{unpack}(U_0), \ldots, \mathsf{unpack}(U_{m-1})$.

The maps pack and unpack are mutually inverse B-linear bijections, so each element of T packs d elements of B. We follow Hashcaster [Sou24] in recovering each packed value as a trace against the dual basis.

Lemma 2.7 (Transitivity of unpacking)**.** *Let $A \subseteq B \subseteq T$ be finite fields. Fix a basis $\mathfrak{a}_0, \ldots, \mathfrak{a}_{d'-1}$ of B over A with dual basis $\mathfrak{a}_0^*, \ldots, \mathfrak{a}_{d'-1}^*$, and a basis $\mathfrak{b}_0, \ldots, \mathfrak{b}_{d-1}$ of T over B with dual basis $\mathfrak{b}_0^*, \ldots, \mathfrak{b}_{d-1}^*$. The products $\mathfrak{a}_i \mathfrak{b}_j$, ordered by j and then i, form a basis of T over A with dual basis $(\mathfrak{a}_i^* \mathfrak{b}_j^*)_{i,j}$. For this basis, $\mathsf{unpack}_{T/A} = \mathsf{unpack}_{B/A} \circ \mathsf{unpack}_{T/B}$, with $\mathsf{unpack}_{B/A}$ applied componentwise.*

The proof is in Appendix A.

Since unpack is only B-linear, a linear claim on the values packed in U is not a T-linear claim on U. The twist identity rewrites such a claim as a sum of d Frobenius images of T-linear forms in U.

Definition 2.8 (Twist, after Hashcaster's Frobenius twists [Sou24])**.** In the setting of Definition 2.6, for a finite field $L \supseteq T$, coefficients $a \in L^d$ and $0 \leq k < d$, the k-th *twist* of a is

$$f_k(a) = \mathrm{Frob}_B^{-k}\Big(\sum_{j=0}^{d-1} \mathrm{Frob}_B^k(\mathfrak{b}_j^*)\, a_j\Big),$$

with Frob_B the Frobenius map of L over B (Definition 2.4).

Lemma 2.9 (Twist identity)**.** *In the setting of Definition 2.6, with $L \supseteq T$ and the twists of Definition 2.8, for every m, all $U \in T^m$ and all coefficients $a_{g,j} \in L$, $g \in [m]$, $j \in [d]$, with $a_g = (a_{g,0}, \dots, a_{g,d-1})$,*

$$\sum_{g=0}^{m-1} \sum_{j=0}^{d-1} a_{g,j}\, \mathsf{unpack}(U_g)_j = \sum_{k=0}^{d-1} \mathrm{Frob}_B^k\Big(\sum_{g=0}^{m-1} f_k(a_g)\, U_g\Big).$$

Proof. By Definitions 2.4 and 2.6, $\mathsf{unpack}(U_g)_j = \sum_{k<d} \mathrm{Frob}_B^k(\mathfrak{b}_j^*)\, \mathrm{Frob}_B^k(U_g)$. Substituting, exchanging the sums and using that Frob_B^k is an automorphism of L gives the right side. □

2.4 Reed–Solomon codes

GordieHowe commits to its tableaus with Ligero. Ligero encodes each row of a tableau with a Reed–Solomon code. We recall the code and the notions of agreement that the analysis of Ligero uses.

Definition 2.10 (Reed–Solomon code [GRS25, Def. 5.2.1])**.** Let K be a finite field, $\mathcal{D} \subseteq K$ a set of n points and $1 \le k \le n$. The *Reed–Solomon code* $\mathrm{RS}[K, \mathcal{D}, k] \subseteq K^{\mathcal{D}}$ is the set of words $(p(x))_{x \in \mathcal{D}}$ for the polynomials $p \in K[X]$ of degree below k. Its *block length* is n and its *dimension* is k. For words $u, v \in K^{\mathcal{D}}$, the *agreement set* is $\mathrm{Agr}(u, v) = \{x \in \mathcal{D} : u_x = v_x\}$. The *distance* of u and v is $n - |\mathrm{Agr}(u, v)|$ [GRS25, Def. 1.3.3]. For a polynomial p, $\mathrm{Agr}(p, u) = \mathrm{Agr}((p(x))_{x \in \mathcal{D}}, u)$.

Two distinct polynomials of degree below k agree on at most $k-1$ points, so each codeword is the word of exactly one such polynomial. We identify the codeword with that polynomial. The values of a codeword on every set of k points determine it, and the code has distance $n-k+1$ [GRS25, Claim 5.2.3].

The verifier of Ligero makes fewer queries for the same soundness when its analysis holds up to a larger distance from the code.

Definition 2.11 (Unique-decoding radius [GRS25, Prop. 1.4.2, Rem. 1.4.3])**.** The *unique-decoding radius* of $\mathrm{RS}[K, \mathcal{D}, k]$ is $e_{\mathrm{U}} = \lfloor (n-k)/2 \rfloor$, and its *unique-decoding threshold* is $n - e_{\mathrm{U}}$.

A word agrees with at most one codeword on at least $n - e_{\mathrm{U}}$ positions. Two such codewords agree with each other on at least $n - 2e_{\mathrm{U}} \ge k$ positions, so they are equal.

Definition 2.12 (Johnson radius [DKT26, §1, p. 4, and p. 120])**.** The *Johnson radius* of $\mathrm{RS}[K, \mathcal{D}, k]$ is $1 - \sqrt{(k-1)/n}$. An *agreement threshold* is an integer A with $k \le A \le n$. The threshold A is *within the Johnson radius* if $A^2 > n(k-1)$, that is, if the relative distance $1 - A/n$ is below $1 - \sqrt{(k-1)/n}$.

The Johnson radius exceeds the relative unique-decoding radius e_{U}/n. Section 5.6.2 analyzes Ligero at every agreement threshold within the Johnson radius. This analysis uses the following property of lines. Dao, Kominers and Thaler call it full agreement-set line MCA (mutual correlated agreement).

Definition 2.13 (Mutual correlated agreement [DKT26, §3.2, Def. 3.1 with $\ell = 1$, p. 26])**.** Let $C = \mathrm{RS}[K, \mathcal{D}, k]$, let A be an agreement threshold and $E \ge 0$. The code C has *full agreement-set line MCA at threshold A with exceptional count E* if for all $u_0, u_1 \in K^{\mathcal{D}}$ there is a set $\mathcal{Z} \subseteq K$ of at most E points such that, for every $z \in K \setminus \mathcal{Z}$ and every polynomial p of degree below k with $|\mathrm{Agr}(p, u_0 + z u_1)| \ge A$, there are polynomials p_0, p_1 of degree below k with

$$p = p_0 + z p_1, \qquad \mathrm{Agr}(p, u_0 + z u_1) = \mathrm{Agr}(p_0, u_0) \cap \mathrm{Agr}(p_1, u_1).$$

The analysis of Section 5.6.2 uses the property in two regimes.

Lemma 2.14 (Mutual correlated agreement for Reed–Solomon codes)**.** *Let $n = |\mathcal{D}|$. The code* $\mathrm{RS}[K, \mathcal{D}, k]$ *has full agreement-set line MCA*

1. *at every agreement threshold A within the Johnson radius, for $2 \le k \le n-1$, with an exceptional count less than $8nt^3/(3\rho^-)$, where $\rho^- = (k-1)/n$, $\eta = A/n - \sqrt{\rho^-}$ and $t = \max(\lceil \sqrt{\rho^-}/(2\eta) \rceil, 3) + 1/2$, and*
2. *at every agreement threshold $A \ge n - e_{\mathrm{U}}$, with exceptional count n.*

The proof is in Appendix A.

The parameter search (Section 5.7) takes the exceptional count from a finite certificate in place of the closed form of Lemma 2.14(1). The certificate takes three integers: a multiplicity m, a bound B on the degree in Y of an interpolating polynomial $Q(X, Z, Y)$, and a bound H on its degree in the challenge variable Z. Dao, Kominers and Thaler call B the jet-degree bound [DKT26, §3.7, p. 30] and H the height [DKT26, Lem. 3.12, p. 30].

Lemma 2.15 (Weighted Johnson certificate)**.** *Let $n = |\mathcal{D}|$, let $2 \le k \le n-1$ and $D = k-1$, and let A be an agreement threshold. Let $m \ge 1$, $1 \le B \le 2D+1$ and $H \ge B$ be integers with $DB < mA$, and let $B' = \min(B, m-1)$. Suppose that*

$$\sum_{j=0}^{B} (mA - Dj)(H + 1 - j) > n \sum_{b=0}^{B'} (m - b)(H + 1 - b).$$

Then $\mathrm{RS}[K, \mathcal{D}, k]$ *has full agreement-set line MCA at threshold A with exceptional count*

$$E(n, k, A; m, B, H) = (2B-1)H + \frac{n-D}{A-D}\big(H + B + (2D-1)(2B-1)H\big) + (n-D-1)B.$$

The proof is in Appendix A.

At the parameters of [DKT26, Thm. 5.12], whenever their degree bound B_{jet} [DKT26, eq. (66)] is at most $2D+1$, the count of Lemma 2.15 is below the closed form of Lemma 2.14(1) (Remark A.1).

2.5 Series–parallel IOPs

GordieHowe's proof system compiles a program into an interactive oracle proof (IOP), and the series–parallel BCS transformation (spBCS, Protocol 6.5) turns this IOP into a non-interactive argument. Chiesa and Yogev [CY26] present the BCS transformation for public-coin IOPs in which each round has one oracle string and each message may depend on every earlier message. The proof system's IOP commits several tableaus in one message and composes subprotocols in parallel, each prover message depending only on the earlier messages of its own subprotocol. We recall Chiesa and Yogev's model and then extend it.

Definition 2.16 (Interactive oracle proof [CY26, §23.1, Defs. 23.1.1–23.1.3, pp. 257–259])**.** Let $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ be a relation. An *interactive oracle proof* (IOP) for $\mathcal{R}$ is a pair $\mathsf{IOP} = (\mathbf{P}, \mathbf{V})$ of probabilistic algorithms with the following structure.

- It has $\mathsf{k} \in \mathbb{N}$ rounds. Each round $i \in [\mathsf{k}]$ has a finite alphabet $\mathcal{A}_i$, a length $\mathsf{l}_i \in \mathbb{N}$, a finite set $\mathcal{M}_i$ of non-oracle messages, a singleton when round i has none, and a finite set $\mathcal{C}_i$ of verifier messages.

- The prover $\mathbf{P}$ receives a statement $\mathsf{x} \in \mathcal{X}$ and a witness $\mathsf{w} \in \mathcal{W}$, and $\mathbf{V}$ receives x. In round $i \in [\mathsf{k}]$, $\mathbf{P}$ sends a proof string $O_i \in \mathcal{A}_i^{\mathsf{l}_i}$ and a non-oracle message $a_i \in \mathcal{M}_i$, and then $\mathbf{V}$ sends $\theta_i \in \mathcal{C}_i$.
- The verifier $\mathbf{V}$ reads every a_i in full, makes queries (j, s) with $j \in [\mathsf{k}]$ and $s \in [\mathsf{l}_j]$, each answered by $O_j[s]$, and outputs a bit.

The IOP IOP has *perfect completeness* if $\Pr[\langle \mathbf{P}(\mathsf{x}, \mathsf{w}), \mathbf{V}(\mathsf{x})\rangle = 1] = 1$ for every $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$. It has *soundness error* ε, a function from $\mathcal{X}$ to the real interval $[0, 1]$, if $\Pr[\langle \tilde{\mathbf{P}}, \mathbf{V}(\mathsf{x})\rangle = 1] \leq \varepsilon(\mathsf{x})$ for every $\mathsf{x} \notin \mathcal{L}(\mathcal{R})$ and every malicious prover $\tilde{\mathbf{P}}$. The IOP IOP is *public-coin* if every θ_i is a uniform element of $\mathcal{C}_i$, independent of everything else, and $\mathbf{V}$ has no other randomness. The queries of a public-coin IOP are *challenge-determined* if they depend only on the challenges $(\theta_i)_i$.

The decision of the verifier of a public-coin IOP is a function only of x, $(\theta_i)_i$, the non-oracle messages $(a_i)_i$ and the answers to its queries. We restrict our attention to public-coin IOPs with challenge-determined queries. Reingold, Rothblum and Rothblum's input-oblivious queries [RRR20, Def. 4.7, p. 33] are independent of the input only in interactions with the honest prover, whereas challenge-determined queries are a function of the challenges against every prover. We extend this model in two ways. First, a prover message may carry several oracle strings. Second, a partial order on the messages records the earlier messages on which each message depends. The proof system's IOP needs only series–parallel orders.

Definition 2.17 (Series–parallel order, after [VTL79, Def. 1, p. 2])**.** The *series–parallel* partial orders are defined recursively. The empty order and the order on a single element are series–parallel, and the empty order is the unit of both compositions. If $\sqsubset_1$ and $\sqsubset_2$ are series–parallel partial orders on disjoint sets A_1 and A_2, then so are

1. their *series composition* $A_1; A_2$, the order on $A_1 \cup A_2$ whose restriction to each A_k is $\sqsubset_k$ and that has $j \sqsubset i$ for all $j \in A_1$ and $i \in A_2$, and no other relations, and
2. their *parallel composition* $A_1 \sqcup A_2$, the order on $A_1 \cup A_2$ whose restriction to each A_k is $\sqsubset_k$ and that relates no element of A_1 to an element of A_2.

The series–parallel orders are the transitive closures of the directed acyclic graphs that Valdes, Tarjan and Lawler call minimal series parallel.

Definition 2.18 (Series–parallel IOP)**.** A *series–parallel IOP* generalizes a public-coin IOP with challenge-determined queries (Definition 2.16) as follows.

- Its messages are numbered $i \in [N_{\mathrm{msg}}]$, and $[N_{\mathrm{msg}}]$ is partitioned into prover messages J_P and verifier messages J_V. Consecutive messages may have the same sender.
- A prover message $i \in J_P$ is

$$M_i = (a_i, O_{i,0}, \ldots, O_{i,m_i-1}) \in \mathcal{M}_i \times \prod_{j<m_i} \mathcal{A}_{i,j}^{\mathsf{l}_{i,j}},$$

  the non-oracle message a_i of Definition 2.16 together with $m_i \geq 0$ oracle strings, each over its own alphabet.
- The oracle positions are $\mathcal{P}_{\mathrm{or}} = \{(i, j, q) : i \in J_P,\ j < m_i,\ q \in [\mathsf{l}_{i,j}]\}$, and $n_{\mathrm{pos}} = |\mathcal{P}_{\mathrm{or}}|$. The verifier's queries are a list $\mathsf{qry}(\theta) \in \mathsf{List}(\mathcal{P}_{\mathrm{or}})$ without repetitions that depends only on the challenges $\theta = (\theta_i)_{i \in J_V}$, and in particular not on the statement.

- The messages carry a series–parallel partial order $\sqsubset$ on $[N_{\mathrm{msg}}]$ (Definition 2.17) compatible with message order: $j \sqsubset i$ implies $j < i$. The *past* of a message i is $\mathrm{past}(i) = \{j \in [N_{\mathrm{msg}}] : j \sqsubset i\}$. The honest prover's message i is a function of the statement, the witness, its coins and the messages of $\mathrm{past}(i)$.

We write $q_{\mathsf{IOP}} = \max_\theta |\mathsf{qry}(\theta)|$, $m = \max_i m_i$ and $\mathsf{l}_{\max} = \max_{i,j} \mathsf{l}_{i,j}$, with empty maxima 0.

The total order with alternating messages and one oracle string per prover message recovers exactly the public-coin IOPs with challenge-determined queries. The restriction to the past applies only to the honest prover, and a malicious prover may read the whole transcript.

Definition 2.19 (Completeness error, after [DFMS22, p. 10])**.** A series–parallel IOP for $\mathcal{R}$ has *completeness error* ε_{c}, a function from $\mathcal{X}$ to the real interval $[0, 1]$, if $\Pr[\langle \mathbf{P}(\mathsf{x}, \mathsf{w}), \mathbf{V}(\mathsf{x})\rangle = 1] \geq 1 - \varepsilon_{\mathrm{c}}(\mathsf{x})$ for every $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$, over the challenges and the prover's coins.

Chiesa and Yogev define only perfect completeness for IOPs, the case $\varepsilon_{\mathrm{c}} = 0$.

The BCS transformation turns round-by-round knowledge soundness of an IOP into knowledge soundness of the non-interactive argument. We adapt Chiesa, Manohar and Spooner's definition [CMS19, Defs. 8.3 and 8.5, pp. 39–40] to series–parallel IOPs.

Definition 2.20 (Causal round-by-round knowledge soundness, after [CMS19, Defs. 8.3 and 8.5])**.** Let IOP be a series–parallel IOP for $\mathcal{R}$. For a set $D \subseteq [N_{\mathrm{msg}}]$, a *transcript of* D assigns a message to each element of D, and $\mathsf{tr}|_D$ is the restriction of a transcript tr to D. A *state function* for IOP maps a statement x, a verifier message $i \in J_V$ and a transcript of $\mathrm{past}(i)$ or of $\mathrm{past}(i) \cup \{i\}$ to a bit. An *extractor* for IOP maps a statement x and a transcript of $\mathrm{past}(i)$ for a verifier message $i \in J_V$ to a candidate witness. The IOP IOP has *causal round-by-round knowledge errors* $(\varepsilon_i^{\mathrm{k}})_{i \in J_V}$, functions from $\mathcal{X}$ to the real interval $[0, 1]$, if there exist a polynomial-time extractor $\mathbf{E}$ and a state function State such that for every statement x

(a) for every full transcript tr that $\mathbf{V}$ accepts on x with $(\mathsf{x}, \mathbf{E}(\mathsf{x}, \mathsf{tr}|_{\mathrm{past}(i)})) \notin \mathcal{R}$ for every $i \in J_V$, some $i \in J_V$ has $\mathsf{State}(\mathsf{x}, i, \mathsf{tr}|_{\mathrm{past}(i)}) = 0$ and $\mathsf{State}(\mathsf{x}, i, \mathsf{tr}|_{\mathrm{past}(i)\cup\{i\}}) = 1$, and

(b) for every $i \in J_V$ and every transcript tr of $\mathrm{past}(i)$ with $\mathsf{State}(\mathsf{x}, i, \mathsf{tr}) = 0$,

$$\Pr_{\theta_i \xleftarrow{\$} \mathcal{C}_i}\left[\mathsf{State}(\mathsf{x}, i, \mathsf{tr} \| \theta_i) = 1\right] > \varepsilon_i^{\mathrm{k}}(\mathsf{x}) \implies (\mathsf{x}, \mathbf{E}(\mathsf{x}, \mathsf{tr})) \in \mathcal{R}.$$

We call Definition 2.20 *causal* because the state and the extractor at each challenge read only the past of that challenge. For a total order it is equivalent to Chiesa, Manohar and Spooner's definition (Lemma 2.21). Their definition evaluates the state on prefixes of the transcript and requires that the state be 0 on the empty transcript, stay 0 across prover messages and be 1 on every accepted full transcript. Along an accepted transcript, these conditions imply that the state moves from 0 to 1 at some challenge. Under a partial order the pasts of two challenges need not be nested, so the conditions no longer imply this. Condition (a) therefore asks that every accepted transcript from which the extractor fails contain a challenge at which the state moves from 0 to 1. The proof of knowledge soundness (Theorem I.16) uses condition (a). Condition (b) is their round-by-round condition, with the state at each challenge a function of that challenge's past. They use one error for all rounds, and we use one error per round, indexed as in Chiesa and Yogev [CY26, Def. 31.1.6, p. 386]. In the rest of the paper, round-by-round knowledge soundness of a series–parallel IOP means the causal notion.

Lemma 2.21. *Under the total order, a series–parallel IOP has round-by-round knowledge errors $(\varepsilon_i^{\mathrm{k}})_{i\in J_V}$ in the sense of Definition 2.20 if and only if it has them in the sense of [CMS19, Def. 8.5] with per-round errors as in [CY26, Def. 31.1.6, p. 386], with the same extractor.*

The proof is in Appendix A.

Our proof that the non-interactive argument is zero knowledge uses honest-verifier zero knowledge of the IOP.

Definition 2.22 (Honest-verifier zero knowledge [CY26, Defs. 23.1.7 and 23.1.8, p. 260])**.** Let IOP be a series–parallel IOP for $\mathcal{R}$. The verifier's *view* on (x,w), denoted $\mathrm{View}(\mathbf{P},\mathbf{V},\mathsf{x},\mathsf{w})$, is the random variable $(\mathsf{x},\theta,(a_i)_{i\in J_P},\mathsf{qry}(\theta),\mathsf{ans})$ in an interaction of $\mathbf{P}(\mathsf{x},\mathsf{w})$ with $\mathbf{V}(\mathsf{x};\theta)$ for uniform challenges $\theta=(\theta_i)_{i\in J_V}$. Here $(a_i)_{i\in J_P}$ are the non-oracle messages and ans are the answers at the positions $\mathsf{qry}(\theta)$. The IOP IOP has *honest-verifier zero-knowledge error* $\varepsilon_{\mathrm{hvzk}}$, a function from $\mathcal{X}$ to the real interval $[0,1]$, if there exists a polynomial-time probabilistic algorithm $\mathbf{S}$, the *HVZK simulator*, such that $\mathsf{SD}\big(\mathrm{View}(\mathbf{P},\mathbf{V},\mathsf{x},\mathsf{w}),\mathbf{S}(\mathsf{x})\big)\le\varepsilon_{\mathrm{hvzk}}(\mathsf{x})$ for every $(\mathsf{x},\mathsf{w})\in\mathcal{R}$.

The definition follows Chiesa and Yogev's, with the non-oracle messages added to the view.

2.6 Interactive oracle reductions

GordieHowe's proof system builds its IOP by composing interactive oracle reductions [BCG+19]. Each reduction turns claims about an *oracle input* into claims about the same oracle input. Chiesa, Fenzi and Weissenberg call this data the implicit instance, and their verifier has query access to it [CFW26, §3.1]. In our relations the oracle input is also the witness, and it stays the same across a composition. The verifier of Definition 2.23 makes no queries. Ligero's column query is the only query to the oracle input, and Lemma 5.13 treats it as a challenge whose output relation checks the oracle input at the queried columns. We follow Chiesa, Di, Hu and Zheng [CDHZ25, §3.4] and Chiesa, Fenzi and Weissenberg [CFW26, Defs. 4.1–4.2].

Definition 2.23 (Interactive oracle reduction)**.** Let $T_0,\dots,T_{h-1}$ be finite fields and $\mathcal{U}=T_0^{n_0}\times\cdots\times T_{h-1}^{n_{h-1}}$. An element $\boldsymbol{U}\in\mathcal{U}$ is an *oracle input*. A *relation on oracle inputs* is a relation $\mathcal{R}\subseteq\mathcal{X}\times\mathcal{U}$ whose witness is the oracle input. A *reduction* Π from $\mathcal{R}_1\subseteq\mathcal{X}_1\times\mathcal{U}$ to $\mathcal{R}_2\subseteq\mathcal{X}_2\times\mathcal{U}$ with r rounds consists of the following.

- A series–parallel order $\sqsubset$ on the rounds $[r]$ (Definition 2.17) with $j\sqsubset i$ only if $j<i$.
- In round i, the prover computes a message a_i from $(\mathsf{x},\boldsymbol{U})$ and the rounds of $\mathrm{past}(i)$. Its randomness is the masks in the oracle input (item (c) of Definition 2.24). The message lies in a finite nonempty set S_i, a product of powers of subfields of $T_0,\dots,T_{h-1}$. The verifier reads the message in full and then sends a uniform challenge θ_i in a finite nonempty set $\mathcal{C}_i$.
- An output map out from the statement and the transcript to a statement of $\mathcal{R}_2$. The map does not read $\boldsymbol{U}$.

Definition 2.24 (Security of a reduction)**.** Let Π be a reduction from $\mathcal{R}_1$ to $\mathcal{R}_2$ with r rounds. A *transcript of* a set D of rounds assigns a message and a challenge to each round of D.

(a) A *state function* State maps x, $\boldsymbol{U}$, a round i and a transcript of $\mathrm{past}(i)$ followed by the message a_i, or by a_i and θ_i, to a bit. The reduction Π has *round-by-round soundness errors* $\varepsilon_0,\dots,\varepsilon_{r-1}$, real numbers in $[0,1]$, if there is a state function State such that for every x and $\boldsymbol{U}$

(i) for every transcript tr of all rounds with $(\mathsf{x},\boldsymbol{U})\notin\mathcal{R}_1$ and $(\mathsf{out}(\mathsf{x},\mathsf{tr}),\boldsymbol{U})\in\mathcal{R}_2$, some round i has $\mathsf{State}(\mathsf{x},\boldsymbol{U},i,\mathsf{tr}|_{\mathrm{past}(i)}\|a_i)=0$ and $\mathsf{State}(\mathsf{x},\boldsymbol{U},i,\mathsf{tr}|_{\mathrm{past}(i)}\|a_i\|\theta_i)=1$, and

(ii) for every round i and every transcript p of past(i) followed by a message of round i with $\mathsf{State}(\mathsf{x}, \boldsymbol{U}, i, p) = 0$,

$$\Pr_{\theta_i \xleftarrow{\$} \mathcal{C}_i} \left[\mathsf{State}(\mathsf{x}, \boldsymbol{U}, i, p \| \theta_i) = 1\right] \leq \varepsilon_i.$$

(b) The reduction Π is *complete on* $\mathcal{K} \subseteq \mathcal{U}$ *with error* ε_{c}, a real number in $[0,1]$, if for every $\mathsf{x} \in \mathcal{X}_1$ and every $\boldsymbol{U} \in \mathcal{K}$ with $(\mathsf{x}, \boldsymbol{U}) \in \mathcal{R}_1$, the transcript tr of $\mathbf{P}$ with uniform challenges has $\Pr[(\mathsf{out}(\mathsf{x}, \mathsf{tr}), \boldsymbol{U}) \notin \mathcal{R}_2] \leq \varepsilon_{\mathrm{c}}$. The reduction Π is *complete on* $\mathcal{K}$ if it is complete on $\mathcal{K}$ with error 0.

(c) Let the oracle input be a function $\boldsymbol{U} = \boldsymbol{U}(\mathsf{w}, m)$ of an element w of a set $\mathcal{W}$ and *masks* m. Let μ be a distribution on the masks. The honest prover draws m from μ. The *view* of Π at $(\mathsf{x}, \boldsymbol{U})$ is the random variable $((\theta_i)_{i \in [r]}, (a_i)_{i \in [r]})$ of the uniform challenges and the prover's messages. The *uniform simulator* is the algorithm that, on x, draws $(\theta_i)_i$ uniformly from $\prod_i \mathcal{C}_i$ and $(a_i)_i$ uniformly from $\prod_i S_i$, independently. The reduction Π is *honest-verifier zero knowledge with error* 0 *with respect to the uniform simulator*, with masks drawn from μ, if for every $\mathsf{x} \in \mathcal{X}_1$ and every $\mathsf{w} \in \mathcal{W}$, the view at $(\mathsf{x}, \boldsymbol{U}(\mathsf{w}, m))$ and the output of the uniform simulator on x have statistical distance 0. Here m is drawn from μ.

The *soundness error* of Π is the supremum, over $(\mathsf{x}, \boldsymbol{U}) \notin \mathcal{R}_1$ and over provers, of the probability over the challenges that $(\mathsf{out}(\mathsf{x}, \mathsf{tr}), \boldsymbol{U}) \in \mathcal{R}_2$. Round-by-round soundness errors $\varepsilon_0, \ldots, \varepsilon_{r-1}$ bound the soundness error by $\sum_i \varepsilon_i$. By (a)(i), every transcript tr with $(\mathsf{out}(\mathsf{x}, \mathsf{tr}), \boldsymbol{U}) \in \mathcal{R}_2$ has a round at which the state moves from 0 to 1. By (a)(ii) and a union bound over the rounds, such a move occurs with probability at most $\sum_i \varepsilon_i$. Conditions (i) and (ii) can also be read at one fixed oracle input $\boldsymbol{U}$, and Lemma 2.27 composes them at each oracle input separately.

Item (a) is Chiesa, Di, Hu and Zheng's round-by-round soundness [CDHZ25, Def. 3.4] at proximity bound 0, adapted to series–parallel orders as in Definition 2.20. Item (c) strengthens Chiesa, Fenzi and Weissenberg's honest-verifier zero knowledge [CFW26, Def. 4.2, p. 32] at error 0 against a distinguisher with no queries. It fixes the simulator to the uniform one and holds for every statement and every witness, with the masks drawn from μ.

The reductions of GordieHowe's proof system output claims of three kinds: linear claims, product constraints and membership claims. Ring switching outputs membership claims for its mask entries (Section 5.5.2). Ligero checks linear claims with its linear test, product constraints with its quadratic test, and membership claims with its subfield test (Protocol G.7).

Definition 2.25 (Linear, product and membership relations)**.** Let $\mathcal{U}$ be as in Definition 2.23, K one of the fields $T_0, \ldots, T_{h-1}$, I a finite set and $\mathsf{val} \colon \mathcal{U} \to K^I$ a map, the *value map*.

1. The *linear relation* $\mathcal{R}_{\mathrm{lin}}^{\mathsf{val}}$ has statements $(a, e) \in K^I \times K$, and $((a, e), \boldsymbol{U}) \in \mathcal{R}_{\mathrm{lin}}^{\mathsf{val}}$ if and only if $\langle a, \mathsf{val}(\boldsymbol{U}) \rangle = e$. A statement of $\mathcal{R}_{\mathrm{lin}}^{\mathsf{val}}$ is a *linear claim* on val. Without a superscript, val is the projection of $\boldsymbol{U}$ onto its entries in K.

2. The *product relation* $\mathcal{R}_{\mathrm{prod}}$ has statements (i, j, l), three positions of entries of $\boldsymbol{U}$ in one field, and $((i, j, l), \boldsymbol{U}) \in \mathcal{R}_{\mathrm{prod}}$ if and only if $\boldsymbol{U}_i \boldsymbol{U}_j = \boldsymbol{U}_l$. A statement of $\mathcal{R}_{\mathrm{prod}}$ is a *product constraint*.

3. The *membership relation* $\mathcal{R}_{\mathrm{mem}}$ has statements (i, E), a position i and a subfield E of the field of $\boldsymbol{U}_i$, and $((i, E), \boldsymbol{U}) \in \mathcal{R}_{\mathrm{mem}}$ if and only if $\boldsymbol{U}_i \in E$. A statement of $\mathcal{R}_{\mathrm{mem}}$ is a *membership claim*.

4. For relations $\mathcal{R}_1 \subseteq \mathcal{X}_1 \times \mathcal{U}$ and $\mathcal{R}_2 \subseteq \mathcal{X}_2 \times \mathcal{U}$, the *conjunction* $\mathcal{R}_1 \wedge \mathcal{R}_2 \subseteq (\mathcal{X}_1 \times \mathcal{X}_2) \times \mathcal{U}$ contains $((\mathsf{x}_1, \mathsf{x}_2), \boldsymbol{U})$ if and only if $(\mathsf{x}_1, \boldsymbol{U}) \in \mathcal{R}_1$ and $(\mathsf{x}_2, \boldsymbol{U}) \in \mathcal{R}_2$. For a relation $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{U}$, $\mathcal{R}^*$ has

as statements the finite lists of statements of $\mathcal{R}$, and $(\vec{\mathsf{x}}, \boldsymbol{U}) \in \mathcal{R}^*$ if and only if $(\mathsf{x}, \boldsymbol{U}) \in \mathcal{R}$ for every entry x of $\vec{\mathsf{x}}$.

We use three value maps: the projection onto the elements of the oracle input in one field, the values packed in them (Definition 2.6), and the values stored in a packing tree (Definition 4.6).

Definition 2.26 (Series and parallel composition)**.** Let Π_1 and Π_2 be reductions on the same space of oracle inputs.

1. Let Π_1 be a reduction to a conjunction $\mathcal{R}_{\text{in}} \wedge \mathcal{R}_{\text{out}}$ (Definition 2.25) with output map $(\mathsf{out}_{\text{in}}, \mathsf{out}_{\text{out}})$, and Π_2 a reduction from $\mathcal{R}_{\text{in}}$ with output map out_2. The *series composition* $\Pi_2 \circ \Pi_1$ consists of the messages of Π_1 followed by those of Π_2, with the rounds of Π_2 numbered after those of Π_1 and the series composition of their orders (Definition 2.17). It gives Π_2 the statement $\mathsf{out}_{\text{in}}(\mathsf{x}, \mathsf{tr}_1)$, and its output statement is $\big(\mathsf{out}_2(\mathsf{out}_{\text{in}}(\mathsf{x}, \mathsf{tr}_1), \mathsf{tr}_2),\ \mathsf{out}_{\text{out}}(\mathsf{x}, \mathsf{tr}_1)\big)$.
2. Let Π_1 and Π_2 have output maps out_1 and out_2. The *parallel composition* $\Pi_1 \sqcup \Pi_2$ on a statement $(\mathsf{x}_1, \mathsf{x}_2)$ consists of Π_1 on x_1 and Π_2 on x_2, with the rounds of Π_2 numbered after those of Π_1 and the parallel composition of their orders. Its output statement is $(\mathsf{out}_1(\mathsf{x}_1, \mathsf{tr}_1), \mathsf{out}_2(\mathsf{x}_2, \mathsf{tr}_2))$. For a finite family $(\Pi_i)_i$, $\bigsqcup_i \Pi_i$ is the parallel composition of its members.

Lemma 2.27 (Composition of reductions)**.** *Let all reductions below be on the same space $\mathcal{U}$ of oracle inputs (Definition 2.23).*

1. Series. *Let Π_1 be a reduction from $\mathcal{R}_1$ to $\mathcal{R}_{\text{in}} \wedge \mathcal{R}_{\text{out}}$ (Definition 2.25) with output map $(\mathsf{out}_{\text{in}}, \mathsf{out}_{\text{out}})$, and Π_2 a reduction from $\mathcal{R}_{\text{in}}$ to $\mathcal{R}_2$ with output map out_2. The series composition $\Pi_2 \circ \Pi_1$ (Definition 2.26) is a reduction from $\mathcal{R}_1$ to $\mathcal{R}_2 \wedge \mathcal{R}_{\text{out}}$.*
2. Parallel. *Let Π_1 be a reduction from $\mathcal{R}_1$ to $\mathcal{R}_2$ and Π_2 a reduction from $\mathcal{R}'_1$ to $\mathcal{R}'_2$, with output maps out_1 and out_2. The parallel composition $\Pi_1 \sqcup \Pi_2$ (Definition 2.26) is a reduction from $\mathcal{R}_1 \wedge \mathcal{R}'_1$ to $\mathcal{R}_2 \wedge \mathcal{R}'_2$.*

In both cases, the following hold for the composed reduction.

(a) If Π_1 and Π_2 have round-by-round soundness errors $(\varepsilon_i)_i$ and $(\varepsilon'_i)_i$ (Definition 2.24), then the composition has round-by-round soundness errors ε_i at the rounds of Π_1 and ε'_i at the rounds of Π_2.

(b) If Π_1 is complete on $\mathcal{K}_1$ with error $\varepsilon_{\text{c}}^{(1)}$ and Π_2 on $\mathcal{K}_2$ with error $\varepsilon_{\text{c}}^{(2)}$, then the composition is complete on $\mathcal{K}_1 \cap \mathcal{K}_2$ with error $\varepsilon_{\text{c}}^{(1)} + \varepsilon_{\text{c}}^{(2)}$.

(c) Let the oracle input be $\boldsymbol{U}(\mathsf{w}, m_1, m_2)$, with masks m_1 of Π_1 and m_2 of Π_2. Suppose that the following three conditions hold. First, Π_1 is honest-verifier zero knowledge with error 0 with respect to the uniform simulator, with m_1 drawn from μ_1, at every value of m_2. Second, Π_2 is honest-verifier zero knowledge with error 0 with respect to the uniform simulator, with m_2 drawn from μ_2, at every value of m_1. Third, the messages of Π_1 do not depend on m_2. Then the composition is honest-verifier zero knowledge with error 0 with respect to the uniform simulator, with (m_1, m_2) drawn from $\mu_1 \times \mu_2$.

The proof is in Appendix A.

Lemma 2.27 extends Chiesa, Fenzi and Weissenberg's composition theorem [CFW26, Thm. 4.5].

2.7 Longfellow

GordieHowe shares the basic architecture of Longfellow and improves on it. Longfellow proves statements expressed as arithmetic circuits. Frigo and shelat present it for mobile driving licenses signed with ECDSA [Fs24]. Longfellow's proof for these licenses uses two fields: $\mathbb{F}_{2^{128}}$ for hashing and the base field $\mathbb{F}_{p_{256}}$ of P-256 for the signature.

For each field, Longfellow's circuit compiler builds one GKR circuit from the constraints over that field [Fs24, Goo26a]. The prover commits the field's witness in one Ligero tableau [Fs26, AHIV22], and it makes GKR [GKR15] zero knowledge with pads that it commits together with the witness [Fs24, §2.3, Protocol 2.5]. Ligero then proves the claims that GKR leaves on the committed values. Ligero's linear test proves the linear constraints, its quadratic test proves the product constraints, and its low-degree test checks that the rows of the tableau are close to codewords [Fs26].

When witness values are statically known to lie in a subfield that contains the evaluation points of the code, Longfellow stores them in *subfield rows*, whose entries and blinding values lie in the subfield, so that each queried column carries smaller elements and the proof is smaller [Fs26, §4.3, p. 22]. Longfellow also packs two to four bits into one element with its bit plucker, and its GKR circuit unpacks them [Goo26a].

A value that both fields use, such as the digest that the signature signs, is committed once in each field. Longfellow checks that the two copies agree with an information-theoretic MAC over $\mathbb{F}_{2^\omega}$ with $\omega = 128$ [Fs24, §3.4]. The prover commits its share of the key, the verifier then sends its share, and the proof over each field checks the MAC of its own copy against the same public tag. Longfellow's implementation [Goo26a] splits the shared bits into chunks $x_1, \ldots, x_n$ of ω bits, commits a key share a_i for each chunk, and uses the tag $(\kappa + a_i)\, x_i$, with κ the verifier's share, a form that Frigo and shelat suggest for nonzero values [Fs24, footnote 5]. Because the committed share multiplies the chunk, a zero chunk gives the tag 0, so the tag does not hide it. In the ECDSA mDL, the shared values are the SHA-256 digest of the issuer-signed document and the device's public key, which are nonzero except with negligible probability, but a chunk of a sparse vector, such as the challenge polynomial of ML-DSA, can be zero (Section 4.1.4). Each tag also needs the product $a_i x_i$ of two committed elements of $\mathbb{F}_{2^\omega}$. The circuit over the field of odd characteristic evaluates it by a Karatsuba multiplication of bit vectors followed by a reduction modulo the field polynomial, and in odd characteristic each exclusive or in this computation costs a multiplication, as each product of bits does. Google's zero-knowledge theorem for Longfellow's MAC gives its simulator the pattern of zero chunks [Goo26b].

Frigo and shelat analyze the soundness and zero knowledge of Longfellow [Fs24], and Cascudo et al. analyze its Ligero at the unique-decoding radius (Definition 2.11) [CHVY25, Thm. 4.3]. Google's concurrent Lean proofs cover knowledge soundness, zero knowledge and witness indistinguishability for a model of Longfellow version 7, including its Ligero past the unique-decoding radius, and their analysis bounds the untested mask rows by a list-size bound on the response words that agree with a mask [Goo26b] (Sections 5.6.1 and 9). To compare GordieHowe with Longfellow, we bound Longfellow's Ligero round by round (Lemma G.12) and use this bound to set the per-round target (Section 7). The concurrent analysis bounds the two Ligero rounds jointly for a fixed commitment. It assumes a weaker adversary than ours, and it is unclear how to extend it to the QROM.

2.8 BCS

GordieHowe makes its IOP non-interactive with an extension of the BCS transformation [BCS16] (Section 6). BCS turns a public-coin IOP into a non-interactive argument in the random oracle model. The prover commits to each oracle string with a Merkle tree [Mer88] and derives each

verifier challenge by hashing the commitments that precede it [BCS16, §6]. The proof contains the roots, the leaves at the queried positions with their authentication paths, and the final value of the hash chain, and Chiesa and Yogev's formulation also places the non-oracle messages in the proof [CY26, Rem. 25.1.2, p. 276].

The BCS hash chain absorbs each Merkle root and no challenge [BCS16, §6, p. 25], and the ROM analysis of BCS covers this chain [BCS16, §7, Thm. 7.1, p. 27]. Chiesa, Manohar and Spooner prove soundness and knowledge soundness in the QROM, from round-by-round soundness and round-by-round knowledge soundness of the IOP respectively, for a variant whose chain also absorbs each challenge [CMS19, Rem. 8.2, §8.2, §8.6 and Thm. 8.6, pp. 39–45]. Fractal gives adaptive knowledge soundness in the QROM from round-by-round knowledge soundness, with an asymptotic error [COS19, Thm. 10.1], and Chiesa, Di, Hu and Zheng give it with explicit constants, for interactive oracle reductions [CDHZ25, Thms. 6.10 and 11.3, pp. 46 and 78]. In the classical ROM, Chiesa and Fenzi prove that BCS is weakly simulation extractable with a straight-line extractor [CF24, Def. 5.16, p. 36, and Lem. 9.12, p. 76].

Ben-Sasson, Chiesa and Spooner prove zero knowledge in the classical ROM for one proof of a statement and witness fixed before the oracle is sampled [BCS16, §2.3, p. 13, and Thm. 7.1, p. 27]. Their distinguisher makes no hash queries before the proof and any number after it, so their bound is on the statistical distance between the real and the simulated pairs of oracle and proof. Chiesa and Yogev prove adaptive zero knowledge of BCS in the classical ROM for one proof [CY26, Def. 7.1.8, p. 74, and Thm. 26.2.1, p. 297]. Their adversary makes at most t hash queries in all and chooses the statement and witness after its first queries. Chiesa and Fenzi prove a multi-instance form in the classical ROM, in which the adversary also programs the random oracle [CF24, Def. 5.10, p. 32, and Lem. 9.9, p. 74]. Chiesa, Manohar and Spooner observe that the bound of BCS holds against quantum distinguishers that receive the whole oracle, and they state zero knowledge of BCS in the QROM in this form, with an asymptotic error [CMS19, §2.9, pp. 12–13, §3.3, p. 15, and Thm. 8.6(3), p. 40]. Fractal sketches zero knowledge in the QROM for one proof of a statement chosen after hash queries, with an error bound stated up to constant factors [COS19, §10.6].

3 The GordieHowe language

GordieHowe expresses relations as programs in a small constraint language. A program declares variables, each over one of several finite fields, and states predicates over them. The language has four core predicates: equality, sharing of bits between two fields, lookup in a table, and congruence modulo an integer. A program may also draw verifier challenges. A predicate that contains a challenge is a *randomized test*, such as the permutation check of Section 3.3.

3.1 Programs

The basic value of a program is a tensor, a k-dimensional array of field elements. A Keccak state, for example, is a tensor of $5 \times 5 \times 64$ bits, and a polynomial is the vector of its coefficients.

Definition 3.1 (Tensors)**.** A *tensor* of shape $s = (s_0, \ldots, s_{k-1})$ over a set S is a function $[s_0] \times \cdots \times [s_{k-1}] \to S$. An *index map* from a shape s' to a shape s is a function from the indices of s' to those of s. For a tensor T of shape s and an index map I from s' to s, the tensor $T[I]$ of shape s' is $j \mapsto T(I(j))$.

Different computations are cheaper to express in different fields. Hashing on bits is cheapest over a binary field, and arithmetic modulo q over a field of odd characteristic (Fig. 1). A program therefore declares each variable over one of several fields.

Definition 3.2 (Variables and assignments)**.** A program has a finite list of finite fields. A *variable* v has a field K_v of the program, a shape s_v, a type $\mathsf{type}(v) \in \{\mathsf{Bit}, \mathsf{Sub}(E), \mathsf{Field}\}$ for a subfield E of K_v, and a *kind*. There are four kinds. A stmt variable holds part of the statement. A priv variable holds a value that the prover chooses. A pub variable holds a value that the verifier computes from public values and challenges. A chal variable holds a challenge that the verifier samples. The verifier sees every value except those of priv variables. The *value space* $\mathcal{X}_v$ of v is the set of tensors of shape s_v over K_v. The *type set* $\mathcal{X}_v^{\mathsf{type}} \subseteq \mathcal{X}_v$ of v is the set of those tensors with entries in $\{0, 1\}$ if $\mathsf{type}(v) = \mathsf{Bit}$, in E if $\mathsf{type}(v) = \mathsf{Sub}(E)$ and in K_v if $\mathsf{type}(v) = \mathsf{Field}$. An *assignment* of a set V of variables maps each $v \in V$ to an element of $\mathcal{X}_v$.

In characteristic 2, the type Bit is the type $\mathsf{Sub}(\mathbb{F}_2)$. In odd characteristic p, the type Bit is the type $\mathsf{Sub}(\mathbb{F}_p)$ together with the Boolean check $x(x-1) = 0$, which the compiler adds to the realized program (Section 4.2.3). The subfield of Bit is therefore $\mathbb{F}_2$ in characteristic 2 and $\mathbb{F}_p$ in odd characteristic.

Definition 3.3 (Expressions)**.** The *expressions* over a field of the program are

$$e ::= v \mid c \mid e + e \mid e \cdot e \mid e[I] \mid \Lambda(e) \mid \mathsf{prod}_a(e),$$

for variables v and constant tensors c over that field, index maps I, linear maps Λ whose coefficients are constants or pub variables over that field, and axes a. The operations $+$ and $\cdot$ are pointwise on tensors of one shape, and prod_a is the product along axis a. We write $e(\mathsf{as})$ for the value of e at an assignment as.

Many relations check equations over the integers modulo an integer q that is not the characteristic of a field of the program. RSA verification multiplies modulo the public modulus, and ML-DSA checks $w \equiv \hat{A}z - 2^{13}ct_1 \pmod{q}$. An integer expression lets a program state such an equation directly over $\mathbb{Z}$. The compiler realizes the equation by field equalities and lookups (Section 4.1.3).

Definition 3.4 (Integer expressions)**.** An *integer expression* over a field K of odd characteristic p is an expression built from variables of type Bit or $\mathsf{Sub}(\mathbb{F}_p)$ and constants in $\mathbb{F}_p$ by $+$, $\cdot$, $e[I]$, and $\Lambda(e)$ for linear maps Λ. Each coefficient of Λ is a constant in $\mathbb{F}_p$ or a pub variable of type $\mathsf{Sub}(\mathbb{F}_p)$ whose declaration states an integer interval: a declaration of form 3 or 4 (Definition 3.6), or a public value declared by a MAC (Section 4.1.4). For $c \in \mathbb{F}_p$, its *least nonnegative representative* is the integer in $[0, p)$ congruent to c, and its *centered representative* is the integer in $[-(p-1)/2, (p-1)/2]$ congruent to c. The *integer value* $[\![e]\!]_{\mathsf{as}}$ of an integer expression e at an assignment as is the value of e over $\mathbb{Z}$, with each entry of a variable replaced by its least nonnegative representative and each constant and each coefficient, including a pub coefficient, replaced by its centered representative.

The centered representative keeps the interval bounds of Section 4.1.3 small: a coefficient -1 contributes $-x$, whereas its least nonnegative representative would contribute $(p-1)x$. The integer value of an expression can differ from the least nonnegative representative of its value in $\mathbb{F}_p$ when the evaluation over $\mathbb{Z}$ leaves $[0, p)$.

For expressions e and e' of lengths n and n', let the index maps I and J send $(i, j) \in [n] \times [n']$ to i and to j. Let Λ map a tensor of shape (n, n') to the tensor of length $n + n' - 1$ whose entry k is the sum of the entries (i, j) with $i + j = k$. The *convolution* $\mathsf{conv}(e, e')$ of e and e' is $\Lambda(e[I] \cdot e'[J])$. It is an integer expression when e and e' are, since Λ has constant coefficients. For tensors whose further leading axes have a common shape, conv acts along the last axis at each index of the leading axes.

The proof system proves two kinds of predicate directly, equalities and lookups, and the compiler reduces the other two to these (Section 4.1).

Definition 3.5 (Predicates)**.** Let as be an assignment.

1. For expressions e, e' of one shape over one field, $\mathsf{equal}(e, e')$ holds at as if and only if $e(\mathsf{as}) = e'(\mathsf{as})$.
2. For variables x, y of type Bit and one shape over two fields, the predicate $\mathsf{link}(x, y)$ holds at as if and only if every entry of $\mathsf{as}(x)$ and of $\mathsf{as}(y)$ lies in $\{0, 1\}$ and $\mathsf{as}(x)_i = \mathsf{as}(y)_i$, as elements of $\{0, 1\}$, for every index i.
3. For expressions e and e' over one field, $\mathsf{lookup}(e, e')$ holds at as if and only if every entry of $e(\mathsf{as})$ is an entry of $e'(\mathsf{as})$.
4. For an integer $q \geq 0$ and integer expressions e, e' of one shape over a field of odd characteristic p, $\mathsf{mod}_q(e, e')$ holds at as if and only if $[\![e]\!]_{\mathsf{as}}(i) - [\![e']\!]_{\mathsf{as}}(i) \in q\mathbb{Z}$ for every index i. For $q = 0$ the predicate states the equality $[\![e]\!]_{\mathsf{as}} = [\![e']\!]_{\mathsf{as}}$ of integer values. We write $\mathsf{int_eq}(e, e')$ for $\mathsf{mod}_0(e, e')$. If an entry of a variable of e or e' lies outside $\mathbb{F}_p$, the predicate does not hold.

The compiler accepts a program only when the interval bounds of Section 4.1.3 make the equalities and lookups that realize each mod_q predicate imply the congruence of the integer values.

A program fixes the order in which its values are determined: each declaration refers only to earlier variables. The compiler assigns the values to phases using these dependencies and the predicates that contain challenges (Section 3.2).

Definition 3.6 (Programs)**.** A *program* $\mathcal{P}$ is a finite sequence of declarations of distinct variables, followed by a finite set Preds of predicates. Each declaration refers only to earlier variables $\vec{y}$. A declaration is one of:

1. $v := \mathsf{priv}$, an *input*, or $v := \mathsf{priv}(f, \vec{y})$, for a randomized function $f\colon \prod_{y \in \vec{y}} \mathcal{X}_y \to \mathcal{X}_v^{\mathsf{type}}$ fixed by the program;
2. $v := \mathsf{stmt}$, or $v := \mathsf{stmt}(N)$ for v over a prime field $\mathbb{F}_p$ of odd characteristic and an integer $1 \leq N \leq p$, the *declared bound* of v;
3. $v := \mathsf{pub}(e)$, for an expression e over stmt, pub and chal variables and constants, with every value of e in $\mathcal{X}_v^{\mathsf{type}}$;
4. $v := \mathsf{pub}(f, \vec{y})$, for a function $f\colon \prod_{y \in \vec{y}} \mathcal{X}_y \to \mathcal{X}_v^{\mathsf{type}}$ fixed by the program, with every $y \in \vec{y}$ a stmt variable or a pub variable declared in this form;
5. $v := \mathsf{chal}$, with a finite set S_v of elements of K_v of the type of v. The challenge is uniform on $S_v^{s_v}$, the tensors of shape s_v with entries in S_v.

A pub declaration of form 3 or 4 over a field of odd characteristic p may state an integer interval $[l, u] \subseteq [0, (p-1)/2]$. Then every entry of every value of v has its centered representative in $[l, u]$. Since $[l, u] \subseteq [0, (p-1)/2]$, this centered representative is also the least nonnegative representative. The *dependencies* $\mathsf{Dep}(v)$ of v are the variables $\vec{y}$ of its declaration, or the variables of e for $v := \mathsf{pub}(e)$, together with their own dependencies. We write $\mathsf{priv}(\vec{y})$ for $\mathsf{priv}(f, \vec{y})$ when the predicates determine the value of v from the values of $\vec{y}$ and f computes that value.

Figure 1 shows a program over two fields. In the figure, $H.\mathsf{priv}$ declares a priv variable over the field H, and Bit^s gives the type Bit and the shape s. The program declares the bits of the witness twice. The input x holds them over H, and the declaration $y := \mathsf{priv}(x)$ copies them to A. The link predicate proves that x and y hold the same bits. The variable v holds the short vector $B(y)$. Three bits bound an entry of v only by 7, so the lookup predicate bounds each entry by 2η with a

Relation. For an odd prime p, an odd prime $q \leq (p-1)/2$, $\eta \leq 3$, $3n \leq 1084$ and a matrix $M \in [0,q)^{m \times n}$,

$$\mathcal{R} = \big\{\big((h,t),x\big) : x \in \{0,1\}^{n \times 3},\ B(x) \in [0,2\eta]^n,\ MB(x) \equiv t \pmod q,\ \mathsf{SHA3\text{-}256}(x) = h\big\},$$

with $h \in \{0,1\}^{256}$, $t \in [0,q)^m$ and $B(x)_i = x_{i,0} + 2x_{i,1} + 4x_{i,2}$. The hash SHA3-256 reads x row by row. The bound on $3n$ makes x and its padding fit in one block of SHA3-256.

Program. Fields $H = \mathbb{F}_{2^{1280}}$ and $A = \mathbb{F}_p$.

$$\begin{aligned}
h &:= H.\mathsf{stmt} && : \mathsf{Bit}^{256}\\
t &:= A.\mathsf{stmt}(q) && : \mathsf{Sub}(\mathbb{F}_p)^m\\
x &:= H.\mathsf{priv} && : \mathsf{Bit}^{n\times 3}\\
y &:= A.\mathsf{priv}(x) && : \mathsf{Bit}^{n\times 3}\\
v &:= A.\mathsf{priv}(y) && : \mathsf{Sub}(\mathbb{F}_p)^n\\
&\mathsf{equal}\big(\mathsf{Hash}(x), h\big)\\
&\mathsf{link}(x,y)\\
&\mathsf{equal}\big(v, B(y)\big)\\
&\mathsf{lookup}\big(v, (0,1,\ldots,2\eta)\big)\\
&\mathsf{mod}_q\big(Mv, t\big)
\end{aligned}$$

The library function Hash**.** SHA3-256 of x, one block of Keccak-f on a state of shape $(5,5,64)$.

$$\begin{aligned}
a_1 &:= H.\mathsf{priv}(x) && : \mathsf{Bit}^{5\times 5\times 64}\\
a_{r+1} &:= H.\mathsf{priv}(a_r) && : \mathsf{Bit}^{5\times 5\times 64},\ \text{for } r = 1,\ldots,23\\
&\mathsf{equal}\big(a_1, R_0(\mathsf{pad}(x))\big)\\
&\mathsf{equal}\big(a_{r+1}, R_r(a_r)\big),\ \text{for } r = 1,\ldots,23\\
&\textbf{return } a_{24}[I_{256}]
\end{aligned}$$

The round is $R_r(a) = \iota_r(\chi(\theta(a)[I_{\rho\pi}]))$. The map θ is $\theta(a)_{k,j,z} = a_{k,j,z} + c_{j-1,z} + c_{j+1,z-1}$ for the column sums $c_{j,z} = \sum_k a_{k,j,z}$. The index map $I_{\rho\pi}$ applies ρ and π. The map χ is $\chi(b)_{k,j,z} = b_{k,j,z} + (1 + b_{k,j+1,z})\, b_{k,j+2,z}$. The map ι_r adds the round constant. The function pad places x and the padding constants in the state. The index map I_{256} selects the first 256 bits.

Figure 1: A program for knowledge of a short preimage under M with a public hash.

lookup in a constant table. The declared bound q of t restricts each entry of t in a statement to $[0,q)$ (Definition 3.9). The mod_q predicate states that the integer values of Mv and t are congruent modulo q. When the integer values of Mv can exceed p, the compiler realizes the congruence by parts (Section 4.1.3).

The library function Hash (Fig. 1) computes SHA3-256 [Nat15b] with the core predicates over H. Addition in characteristic 2 is exclusive or, so the linear map θ and the degree-2 map χ compute Keccak's θ and χ on bits. The function declares the state after every round as a priv variable, so each of its 24 predicates has degree 2 and shape $(5,5,64)$. The compiler can therefore batch these predicates (Section 5.4.2).

The program of Fig. 1 draws no challenges, but the realization of link adds one challenge, the key of its MAC (Section 4.1.4). The proof system draws one LogUp challenge for the lookups of each field (Section 5.2).

3.2 Phases and the execution of a program

A randomized test is sound only if the prover's values in it are fixed before its challenge is drawn. The phases of a program order its values and challenges so that every randomized test meets this condition.

For a set S of variables, the degree $\deg_S(e)$ of an expression e in S (Definition B.1) bounds the degree of e as a polynomial in the variables of S, after each variable declared $\mathsf{pub}(e')$ is replaced by e'. The *degree* $\deg_S(P)$ of a predicate P in S is the largest degree in S of its operands. We write $\deg_\gamma$ for $\deg_{\{\gamma\}}$.

Let γ be a chal variable and P an equality. Replace each variable declared $\mathsf{pub}(e')$ in P by e', and fix every other value in P other than γ. If the two operands of P are then distinct polynomials in the entries of γ, P holds with probability at most $\deg_\gamma(P)/|S_\gamma|$ over γ (Lemma 2.1). A prover value computed from γ is chosen after γ, so γ does not test it.

Definition 3.7 (Phases)**.** A *phase function* of a program $\mathcal{P}$ is a function ph from its variables to $\mathbb{N}$ with the following properties.

1. $\mathsf{ph}(v) \geq \mathsf{ph}(y)$ for every variable v and every $y \in \mathsf{Dep}(v)$.
2. $\mathsf{ph}(\gamma) \geq 1$ for every chal variable γ.
3. For every predicate P and every chal variable γ with $\deg_\gamma(P) > 0$, every priv variable v in P with $\gamma \notin \mathsf{Dep}(v)$ has phase below $\mathsf{ph}(\gamma)$.
4. $\mathsf{ph}(v) = 0$ for every input v.

We consider only programs that have a phase function, and we fix one phase function ph with each program. The *phases* of $\mathcal{P}$ are the values of ph, and n_{ph} is the largest phase.

Condition (3) constrains the predicates of the source program. For the predicates that a realization adds, the compiler checks the phase order stated with that realization, in Section 4.1.4 for a MAC and in Section 5.2 for LogUp. A MAC realization may also declare its key and some public values outside the five declaration forms of Definition 3.6 (Section 4.1.4). Such a public value states an integer interval in $[0, (p-1)/2]$, with the meaning of Definition 3.6. A realized program is a program in this extended sense.

The pointwise minimum of two phase functions is a phase function, so a program with a phase function has a least one. The compiler computes the least phase function by a topological sort of the conditions of Definition 3.7, with the orders of the realizations in place of condition (3) for the predicates they add. It also assigns phases to the challenges that the realizations add, and it fixes the phase of each LogUp challenge (Section 5.2). It rejects the program if these conditions have no solution, which happens exactly when they contain a cycle through a strict inequality. Our results hold for every phase function.

Definition 3.8 (Late variables)**.** A priv variable v of a program $\mathcal{P}$ is *late* if $\mathsf{ph}(v) \geq 1$ for the phase function ph of $\mathcal{P}$. A *late value* is a value of a late variable. The set of late variables is $\mathsf{Late}(\mathcal{P})$.

Definition 3.9 (Execution of a program)**.** Let $\mathcal{P}$ be a program. Its *statements* $\mathcal{X}^{\mathrm{st}}_{\mathcal{P}}$ are the assignments of its stmt variables with values in their type sets such that every entry of a variable with a declared bound N has its least nonnegative representative in $[0, N)$. Its *witnesses* $\mathcal{W}_{\mathcal{P}}$ are the assignments of its inputs. An assignment of all variables of $\mathcal{P}$ is *accepting* if every predicate of $\mathcal{P}$ holds at it and the value of every priv variable v lies in $\mathcal{X}_v^{\mathsf{type}}$. The *execution* $\mathsf{Exec}_{\mathcal{P}}(\mathsf{x}, \mathsf{w})$ on $\mathsf{x} \in \mathcal{X}^{\mathrm{st}}_{\mathcal{P}}$ and $\mathsf{w} \in \mathcal{W}_{\mathcal{P}}$ is the random assignment of all variables, made in the order of declaration. It assigns

each stmt variable by x, each input by w, each chal variable γ uniformly on $S_\gamma^{s_\gamma}$ and each variable $v := \mathsf{pub}(e)$ by the value of e. It assigns each other variable by the function of its declaration, with fresh coins for each randomized function.

In the interactive proof $\Pi_\mathcal{P}$ of $\mathcal{P}$ (Definition B.2), the prover first sends the values of the priv variables of phase 0. For each later phase, the verifier sends the challenges of that phase, one per message, and the prover then sends the values of the priv variables of that phase. A final verifier message ends the proof.

Definition 3.10 (Completeness and soundness of a program)**.** Let $\mathcal{P}$ be a program with phases $0, \ldots, n_{\mathrm{ph}}$, $\mathcal{R} \subseteq \mathcal{X}_\mathcal{P}^{\mathrm{st}} \times \mathcal{W}_\mathcal{P}$ a relation, and $\Pi_\mathcal{P}$ the interactive proof of $\mathcal{P}$ (Definition B.2).

1. The program $\mathcal{P}$ is *complete for* $\mathcal{R}$ *with error* $\varepsilon_{\mathrm{c}} \in [0,1]$ if for every $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$, $\mathsf{Exec}_\mathcal{P}(\mathsf{x}, \mathsf{w})$ is accepting with probability at least $1 - \varepsilon_{\mathrm{c}}$.
2. The program $\mathcal{P}$ is *round-by-round knowledge sound for* $\mathcal{R}$ *with errors* $(\varepsilon_\gamma)_\gamma$ *and extractor* $\mathbf{E}$ if $\mathbf{E}$ is a polynomial-time algorithm whose output is a function of the statement and of the phase-0 values, and $\Pi_\mathcal{P}$ has round-by-round knowledge errors for $\mathcal{R}$ with extractor $\mathbf{E}$ in the sense of Chiesa, Manohar and Spooner [CMS19, Defs. 8.3 and 8.5, pp. 39–40], with error ε_γ at the message of each γ and error 0 at the final message, per round as in Chiesa and Yogev [CY26, Def. 31.1.6, p. 386].

In round-by-round knowledge soundness, the state function can change from 0 to 1 only at a verifier message, so the final message lets it change after the last prover message. A program without challenges, whose phase function is 0, is round-by-round knowledge sound for the relation of the statements and inputs that extend to an accepting assignment (Section B.2).

3.3 Library functions

GordieHowe also provides a standard library of functions expressed in the core language. Each library function declares variables and predicates in the program that calls it. The library includes SHA-256 [Nat15a], Keccak-f with the SHA3-256 and SHAKE sponges [Nat15b], word arithmetic modulo 2^w, the permutation check, offline memory checking and ordered selection.

Most library functions are deterministic: their predicates imply the property of the function at every assignment. The other library functions draw challenges, and their predicates imply the property except with small probability over the challenges.

The library's *permutation check* proves that two lists L and R of records are permutations of each other. A record of arity ℓ is a tuple of ℓ elements of a field F. The check uses a grand product, as in PLONK's permutation argument [GWC19]. It encodes each record r as one element $\mathsf{enc}(r)$ of F and draws a challenge δ. Its predicate states that the product of $\delta - \mathsf{enc}(r)$ over the records r of L equals the product over the records of R. If L and R have at most N records each and are not permutations of each other, the predicate holds with probability at most $N\ell/|F|$, or $N/|F|$ when the encoding has fixed coefficients (Lemma B.4 in Section B.3). The proof system computes the two products with a GKR circuit (Section 5.4.3).

A *filtered permutation check* takes a flag $a_i \in \{0,1\}$ for each record l_i of L and proves that R is a permutation of the flagged records of L, those with $a_i = 1$. Its product over L gives each unflagged record the factor 1 (Section B.3). Its error is that of the permutation check on the flagged records of L and on R, with $N = \max(|L|, |R|)$.

For a public integer k, *ordered selection* takes a list L and a predicate on each entry of L and on the number of entries selected before it, and proves that a list R consists of the first k entries of L

that the predicate accepts, in the order of L. The rejection sampling of SampleInBall in ML-DSA has this form. Ordered selection labels each entry of L with the number of entries selected before it. Its predicates check that this count advances exactly at the entries that the predicate accepts, until k entries are selected. A filtered permutation check then proves that R, with each entry labeled by its position, is a permutation of the selected entries of L with their labels. *Offline memory checking* [BEG+91] combines two checks. A permutation check proves that the reads and the final cells are a permutation of the initial cells and the writes. A second check proves that each read follows its write. The library function performs the permutation check, and the calling program states the second check. Our ML-DSA-44 programs use ordered selection and offline memory checking for SampleInBall (Section 7.4).

4 The compiler

The compiler turns a program into a realized program (Definition 4.7) and chooses the Ligero parameters of each field (Section 4.2.4). These are the inputs of the proof system. The proof system proves only equalities and lookups. It commits the values of each field in a single Ligero commitment and proves each equality with one of four subprotocols (Section 5).

Section 4.1 describes the realizations. Section 4.2 describes the compiler's choices of subprotocols, lookup tables and packing, based on an empirically calibrated model of the proof system, and of the Ligero parameters. Section 4.3 defines the realized program that results and the conditions under which it is valid. The compiler exports the realized program to Lean with a certificate for each realization, and Lean's kernel checks from the certificates that each realization implies its predicate (Section 8).

4.1 Realizations

The compiler reduces every predicate other than an equality or a lookup to equalities and lookups (Section 5). Two core predicates need a reduction, mod_q and link. For each of them, the compiler chooses a realization (Definition 4.1). We realize a congruence by a quotient, or by a field equality when the modulus is the characteristic, and link by a MAC. The compiler can also realize a lookup into an interval by a lookup of its complement, or by a decomposition into digits and lookups of the digits in a smaller table.

Definition 4.1 (Realization)**.** A *realization* of a predicate expresses the predicate by equalities and lookups and may introduce additional priv variables. Each realization declares its prover variables as $v := \mathsf{priv}(f, \vec{y})$, where f computes the value that the description of the realization below specifies, so a realization adds no inputs. A realization by a MAC also declares one challenge, its key. Each realization has a *condition*, which the compiler checks. When the condition holds, the predicates of the realization imply the predicate that they replace, and for a MAC they imply it unless the key is a root of a nonzero binding polynomial (Lemma 4.5). A realization of a range check (Section 4.1.1) *establishes* a bound B if, when its condition holds, its predicates imply that every entry of the checked expression lies in $[0, B)$.

4.1.1 Range checks

The most common table is an interval $[0, n)$ of the prime subfield $\mathbb{F}_p$ of a field K of odd characteristic. Such a table bounds, for example, the coefficients of a short vector. A *range check* is the predicate $\mathsf{lookup}(e, [0, n))$ for an expression e with N entries, and its *bound* is n. The realization by a *direct*

lookup keeps the lookup of every entry of e in a table $[0, n)$ of the field. Its condition is that the compiler declares this table, and it establishes the bound n. The realization by the *complement* looks up every entry of $n - 1 - e$ in a table $[0, T)$ with $n \leq T$. Its condition is $I(e) \subseteq [0, p - T]$ (Definition 4.2). Under this condition, an entry $e_i \geq n$ makes $n - 1 - e_i$ a field element whose least nonnegative representative is $p + n - 1 - e_i \geq T$, so its lookup fails. The honest entries of $n - 1 - e$ lie in $[0, n) \subseteq [0, T)$, so their lookups succeed. The realization establishes the bound n. The interval of e (Definition 4.2) does not use this bound, because the condition of the complement depends on the interval of e. A direct lookup needs a table of n rows, and for most bounds this table is too large. The realization by k *digits* with bounds $b_0, \ldots, b_{k-1} < n$ introduces priv variables $d_0, \ldots, d_{k-1}$ of the shape of e to hold the digits of e in the mixed radix $(b_j)_j$. With $w_0 = 1$ and $w_{j+1} = w_j b_j$, the realization asserts $e = \sum_{j<k} w_j d_j$ and looks up every entry of d_j in a table $[0, b_j)$. Digits in a single base b are the case $b_j = b$, with $w_k = b^k$. When $w_k \leq p$, the sum does not wrap modulo p, so the realization proves $e \in [0, w_k)$ and establishes the bound w_k (Definition 4.1). When $n < w_k$, the realization may use a *second digit set*, which also introduces the digits of $n - 1 - e$ in the same radix. Suppose also that $2(w_k - 1) < p$. If $n \leq e < w_k$, the field element $n - 1 - e$ has the representative $p + n - 1 - e \geq p + n - w_k \geq w_k$, so its digit sum, which lies in $[0, w_k)$, cannot equal it. Hence $e < n$, and the realization establishes the bound n. The realization by digits introduces kN digits with one digit set and $2kN$ digits with two, and it looks up each digit once. The condition of the realization by digits is $w_k \leq p$ and, with one digit set, that w_k is an admissible bound of the range check (Section 4.2.2), or, with a second digit set, that $n < w_k$ and $2(w_k - 1) < p$.

A range check is *implied* when the interval of e (Definition 4.2) already lies in $[0, n)$. Its realization as implied introduces no priv variables, equalities or lookups. Its condition is that the range check is implied.

4.1.2 Lookup counts

For the lookups into a table over a field K, the realized program declares the *counts*, the priv variables over K that hold the number of lookups into each row of the table. A count is a field element, so it determines the number of lookups only when that number is below $\operatorname{char} K$. The realized program therefore splits the lookups into each table into *groups* of fewer than $\operatorname{char} K$ lookups, each with its own counts. The counts of the lookups of a group whose looked-up values all have phase 0 are phase-0 variables. The counts of the other lookups of the group depend on the late values that these lookups look up, so the realized program declares them after those values, with the largest phase among those values. When the cost model rates it cheaper, the compiler instead declares one set of counts with that largest phase for all lookups of the group. Counts of phase above 0 are late variables and have pads (Section 4.1.5).

4.1.3 Integer congruences

The predicate $\mathsf{mod}_q(e, e')$ holds when the integer values $[\![e]\!]$ and $[\![e']\!]$ are congruent modulo q (Definition 3.5). The proof system proves equalities in the field K of e. The realization of $\mathsf{mod}_q(e, e')$ by a quotient introduces a priv variable z, the *quotient*. It asserts $e = e' + qz$ in K and range-checks z. Both e and $e' + qz$ take values in the prime subfield $\mathbb{F}_p$ of K. The equality implies the congruence of the integer values if $[\![e]\!]$ and $[\![e']\!] + q[\![z]\!]$ differ by less than p. When q is the characteristic p of K, the realization of $\mathsf{mod}_p(e, e')$ is instead the equality $e = e'$ in K, and its condition is $q = p$. The integer values $[\![e]\!]$ and $[\![e']\!]$ reduce modulo p to the values of e and e', so this equality holds if and only if the congruence holds. The realization of $\mathsf{int_eq}(e, e')$ is the equality $e = e'$ in K. This equality implies the integer equality when $[\![e]\!]$ and $[\![e']\!]$ differ by less than p.

The compiler bounds the integer value of every expression by an interval (Definition 4.2).

Definition 4.2 (Intervals)**.** Let K be a field of odd characteristic p, and let v be a variable over K of type Bit or $\mathsf{Sub}(\mathbb{F}_p)$. The *interval* of v is $I(v) = [0, B_v - 1]$, where B_v is the least of the following bounds. The first is the bound of its type, 2 for Bit and p for $\mathsf{Sub}(\mathbb{F}_p)$. The second is the declared bound of v, if v is a stmt variable with one. The others are the bounds that the realizations of range checks on v by a direct lookup or by digits establish (Definition 4.1 and Section 4.1.1). For an integer expression e whose variables have intervals, the interval $I(e)$ is the tensor of integer intervals with entries

$$I(e)(i) = \begin{cases} I(v) & \text{if } e = v, \\ \{\tilde{c}_i\} & \text{if } e = c, \text{ for the centered representative } \tilde{c}_i \text{ of } c_i, \\ I(e_1)(i) + I(e_2)(i) & \text{if } e = e_1 + e_2, \\ I(e_1)(i) \cdot I(e_2)(i) & \text{if } e = e_1 \cdot e_2, \\ I(e_1)(J(i)) & \text{if } e = e_1[J], \\ \sum_j a_{ij}\, I(e_1)(j) & \text{if } e = \Lambda(e_1), \end{cases}$$

where a_{ij} is the centered representative of a constant coefficient of Λ, or the stated interval of a pub coefficient. Intervals add as $[l, u] + [l', u'] = [l + l', u + u']$. The product $[l, u] \cdot [l', u']$ is the smallest interval that contains the four products of endpoints, and for an integer c, $c\,[l, u]$ is $[cl, cu]$ if $c \geq 0$ and $[cu, cl]$ if $c < 0$.

The conditions of the direct lookup and of the realization by digits do not depend on any interval, so the intervals of the variables are well defined. By induction on e, if each variable of e lies in its interval and each pub coefficient of e has its centered representative in its stated interval, then $[\![e]\!]$ lies in $I(e)$. The compiler also replaces an interval by a smaller one that contains the integer value at every assignment that satisfies the realized program, and the induction still holds. It does so in the two cases below and for the accumulators of a MAC (Section 4.1.4). Let c be a constant with centered representative $\tilde{c}$, and let a realization of a range check on $e + c$ by a direct lookup or by digits establish a bound B. If $I(e)$ and $[-\tilde{c}, B - 1 - \tilde{c}]$ both lie in $(-p/2, p/2)$, then $[\![e]\!]$ is the centered representative of e, and it lies in $[-\tilde{c}, B - 1 - \tilde{c}]$. If a predicate states $(m - v)(m - (c - v)) = 0$ for a variable m with $I(m) \subseteq [0, \tilde{c}]$, then v is m or $c - m$, so $[\![v]\!]$ lies in $[0, \tilde{c}]$. Lean's kernel checks these intervals for every exported program (Section 8.3).

For integer expressions L and R of one shape and an index i, with $I(L)(i) = [l_L, u_L]$ and $I(R)(i) = [l_R, u_R]$, the *span* of L and R at i is $\mathsf{span}_i(L, R) = \max(u_L, u_R) - \min(l_L, l_R)$. For an integer q with $1 \leq q < p$, the span of e and $e' + qz$ computes $I(e' + qz)$ as $I(e') + q\, I(z)$, with the integer q in place of its centered representative. The condition of the realization of $\mathsf{mod}_q(e, e')$ by a quotient is $1 \leq q < p$ and $\mathsf{span}_i(e, e' + qz) < p$ at every index i. The condition of the realization of $\mathsf{int_eq}(e, e')$ is $\mathsf{span}_i(e, e') < p$ at every index i. Under the first condition, the equality in K implies the congruence of the integer values, and under the second it implies the integer equality (Lemma 4.3).

Lemma 4.3. *Let K be a field of odd characteristic p, e, e' integer expressions over K of one shape (Definition 3.4), and as an assignment with the following properties. The entries of every variable of e and e', and of every variable of the predicates from which the compiler derives the replaced intervals of e, e' and their subexpressions, lie in their intervals (Definition 4.2), and these predicates hold. Every pub coefficient of e and e' has its centered representative in its stated interval.*

1. *Let $1 \leq q < p$ be an integer, and let z be a variable over K of type $\mathsf{Sub}(\mathbb{F}_p)$ and of e's shape whose entries lie in their intervals at as, with $\mathsf{span}_i(e, e' + qz) < p$ for every index i, where $I(e' + qz) = I(e') + q\, I(z)$. If $e(\mathsf{as}) = e'(\mathsf{as}) + q\, \mathsf{as}(z)$ in K, then $\mathsf{mod}_q(e, e')$ holds at as.*

2. *Let* $1 \le q < p$ *be an integer, and let* z *be a variable over* K *of type* $\mathsf{Sub}(\mathbb{F}_p)$ *and of* e*'s shape, not a variable of* e*,* e' *or the predicates above, with* $\mathsf{span}_i(e, e' + qz) < p$ *for every index* i*, where* $I(e' + qz) = I(e') + q\,I(z)$*. If* $[\![e]\!]_{\mathsf{as}} - [\![e']\!]_{\mathsf{as}} = qy$ *for a tensor* y *of integers with every entry* $y_i \in I(z)$*, then setting* $\mathsf{as}(z) = y \bmod p$ *puts the entries of* z *in their intervals and gives* $e(\mathsf{as}) = e'(\mathsf{as}) + q\,\mathsf{as}(z)$ *in* K*.*

3. *Let* $\mathsf{span}_i(e, e') < p$ *for every index* i*. Then* $\mathsf{int_eq}(e, e')$ *holds at* as *if and only if* $e(\mathsf{as}) = e'(\mathsf{as})$ *in* K*.*

4. *Let* $q = p$*. Then* $\mathsf{mod}_q(e, e')$ *holds at* as *if and only if* $e(\mathsf{as}) = e'(\mathsf{as})$ *in* K*.*

Proof. Let L and R be integer expressions of one shape whose variables lie in their intervals at as, with $\mathsf{span}_i(L, R) < p$ at every index. The values $L(\mathsf{as})$ and $R(\mathsf{as})$ lie in the prime subfield $\mathbb{F}_p$ of K. The integer value $[\![L]\!]_{\mathsf{as}}$ evaluates L over $\mathbb{Z}$ with each entry of a variable replaced by its least nonnegative representative and each constant and coefficient by its centered representative (Definition 3.4). Each replacement is congruent modulo p to the element it replaces, so $[\![L]\!]_{\mathsf{as}}$ reduces modulo p to $L(\mathsf{as})$, and likewise for R. Write $I(L)(i) = [l_L, u_L]$ and $I(R)(i) = [l_R, u_R]$. By the induction after Definition 4.2, $[\![L]\!]_{\mathsf{as}}(i)$ and $[\![R]\!]_{\mathsf{as}}(i)$ both lie in $[\min(l_L, l_R), \max(u_L, u_R)]$, so their difference has absolute value less than p. A multiple of p of absolute value less than p is 0. So $L(\mathsf{as}) = R(\mathsf{as})$ in $\mathbb{F}_p$ if and only if $[\![L]\!]_{\mathsf{as}} = [\![R]\!]_{\mathsf{as}}$. Part (3) is this claim for $L = e$ and $R = e'$. For part (1), let $L = e$ and $R = e' + qz$, and read the constant q of R as the integer q, its least nonnegative representative. Then $[\![R]\!]_{\mathsf{as}} = [\![e']\!]_{\mathsf{as}} + q[\![z]\!]_{\mathsf{as}}$ reduces modulo p to $R(\mathsf{as})$ and lies in $I(e') + q\,I(z)$, so the claim applies to L and R. It gives $[\![e]\!]_{\mathsf{as}} - [\![e']\!]_{\mathsf{as}} = q[\![z]\!]_{\mathsf{as}} \in q\mathbb{Z}$. For part (2), $I(z) \subseteq [0, p-1]$, so the least nonnegative representative of $y_i \bmod p$ is y_i, which lies in $I(z)$. Reducing $[\![e]\!]_{\mathsf{as}} - [\![e']\!]_{\mathsf{as}} = qy$ modulo p gives $e(\mathsf{as}) = e'(\mathsf{as}) + q\,\mathsf{as}(z)$. For part (4), the reduction of integer values above needs no span condition, so $[\![e]\!]_{\mathsf{as}} - [\![e']\!]_{\mathsf{as}} \in p\mathbb{Z}$ if and only if $e(\mathsf{as}) = e'(\mathsf{as})$ in $\mathbb{F}_p$. □

The interval of a large sum can be wider than p even when each of its terms is small. For such a sum, the compiler realizes the congruence by parts.

For a sum $e = \sum_{j<N} u_j$ of integer expressions u_j of one shape and a partition $G_1, \ldots, G_\ell$ of $[N]$ into consecutive intervals, write $e_g = \sum_{j \in G_g} u_j$. The *realization of* $\mathsf{mod}_q(e, e')$ *by the parts* $G_1, \ldots, G_\ell$ introduces, for each part g, priv variables r_g and z_g of the shape of e. It asserts $e_g = r_g + qz_g$ in K, range-checks r_g in $[0, q)$ and z_g as a quotient, and realizes $\mathsf{mod}_q(\sum_g r_g, e')$ with one quotient.

The condition of the realization by parts is $1 \le q < p$, and the span of the two expressions of every equality it asserts is below p at every index. Each equality then holds over the integers, so $[\![e_g]\!] \equiv [\![r_g]\!] \pmod q$ for every part, and $\sum_g [\![r_g]\!] \equiv [\![e']\!] \pmod q$. Since $[\![e]\!] = \sum_g [\![e_g]\!]$, the sum of these congruences gives $\mathsf{mod}_q(e, e')$. No partition meets the condition if a single term u_j has an interval of width at least p, such as the term $2x$ for a variable x without a range check. The compiler then rejects the program and reports the term.

The range check on the quotient may establish any of several bounds. The compiler chooses one that lets this range check share a lookup table with other range checks (Section 4.2.2). It first shifts z and e' by constants, so that the honest quotient lies in $[0, n_z)$ and $I(e')$ lies in $[0, W]$. Completeness requires a bound of at least n_z. If the range check establishes $z < \bar{n}$, the right-hand side $e' + qz$ is at most $W + q(\bar{n} - 1)$, and this maximum is below p if and only if $\bar{n} \le \bar{n}^+$ for $\bar{n}^+ = \lfloor (p - 1 - W)/q \rfloor + 1$. The compiler fixes $\bar{n}^+$ before it chooses the tables, so that every bound it may choose keeps the right-hand side below p. When $I(e) \subseteq [0, p-1]$, such a bound also meets the span condition of Lemma 4.3. A bound $\bar{n}$ is *admissible* if $n_z \le \bar{n} \le \bar{n}^+$ and the equality $e = e' + qz$ meets the span condition of Lemma 4.3 under $z < \bar{n}$. Every admissible bound gives a complete and sound realization.

For example, each entry of Mv in Fig. 1 is a sum of n products of an entry of M in $[0, q)$ and an entry of v in $[0, 2\eta]$. When the interval of such a sum is wider than p, the compiler realizes

$\mathsf{mod}_q(Mv, t)$ by parts.

4.1.4 Cross-field links

A relation may need the same witness value in computations over two fields. For example, a hash may compute a digest over a binary field, and a signature verification may use the digest over a field of odd characteristic. The prover commits a copy of the value over each field, and the proof must show that the copies agree. The predicate $\mathsf{link}(x, y)$ states this agreement (Definition 3.5).

Longfellow's per-chunk tags do not hide zero chunks (Section 2.7). In relations other than Longfellow's, a chunk can be zero with noticeable probability. For example, the challenge polynomial of ML-DSA-44 has 39 nonzero coefficients among 256 [Nat24, Table 1]. With two bits per coefficient, each of its four 128-bit chunks holds 64 coefficients and is zero with probability $\binom{192}{39}/\binom{256}{39} \approx 2^{-17.8}$, so some chunk is zero with probability about $2^{-15.8}$.

The realization batches the shared values of each pair of fields and each phase into one polynomial MAC, masked by one committed pad. For a key $\kappa \in \mathbb{F}_{2^\omega}$, a pad $Z \in \mathbb{F}_{2^\omega}$ and chunks $X = (X_0, \ldots, X_{n_{\mathrm{ch}}-1})$ of the shared bits, viewed as elements of $\mathbb{F}_{2^\omega}$, let

$$\mathsf{MAC}_\kappa(Z, X) = Z + \sum_{c<n_{\mathrm{ch}}} \kappa^{c+1} X_c.$$

The prover commits Z, and the verifier then draws κ. The prover replies with the masked tag $\mathsf{MAC}_\kappa(Z, X)$. Once κ is drawn, each chunk is multiplied only by the publicly computable element $r_c = \kappa^{c+1}$.

We express a product rx by a publicly computable element $r \in \mathbb{F}_{2^\omega}$ as an integer matrix applied to the bits of x.

Definition 4.4 (Bit coordinates and convolution matrices)**.** Let $\mathbb{F}_{2^\omega} = \mathbb{F}_2[X]/(\varphi)$ for an irreducible polynomial φ of degree ω, and let $\mathrm{bits}\colon \mathbb{F}_{2^\omega} \to \{0,1\}^\omega$ give the coefficients in the basis $1, X, \ldots, X^{\omega-1}$. We identify a bit vector with the element of $\mathbb{F}_{2^\omega}$ whose coefficients it lists. For $r \in \mathbb{F}_{2^\omega}$, $\mathrm{Conv}(r)$ is the $\omega \times \omega$ integer matrix that multiplies the polynomial $\sum_j v_j X^j$ by $\sum_i \mathrm{bits}(r)_i X^i$ over $\mathbb{Z}$ and replaces each monomial X^i of the product with $i \geq \omega$ by the coefficient vector $\mathrm{bits}(X^i \bmod \varphi)$. We write $\mathrm{Conv}_{\max}$ for the largest row sum of $\mathrm{Conv}(r)$ over all r.

For every $x \in \mathbb{F}_{2^\omega}$, $\mathrm{Conv}(r)\,\mathrm{bits}(x) \equiv \mathrm{bits}(rx) \pmod 2$. Modulo 2, the product over $\mathbb{Z}$ in Definition 4.4 is the carry-less product of r and x, and the replacement of each monomial X^i with $i \geq \omega$ is the reduction modulo φ.

The compiler realizes the link predicates between two fields K_1 and K_2 of a program by a MAC over $\mathbb{F}_{2^\omega}$, whose key, tag and pad are values of $\mathbb{F}_{2^\omega}$. If one of the two fields is $\mathbb{F}_{2^\omega}$, we call it K_1, and K_1 *hosts* the MAC. Otherwise $\mathbb{F}_{2^\omega}$ need not be a subfield of K_1 or K_2. Let j be a phase, and let $\mathsf{link}(x_l, y_l)$ be the predicates of the program, each with x_l in K_1 and y_l in K_2, whose variables have largest phase j. The realization fixes a number n_{ch} of chunks, with $n_{\mathrm{ch}}\omega$ at least the number of entries of the x_l. The entries of the x_l, padded with zeros to $n_{\mathrm{ch}}\omega$ entries, form the chunks $X_{1,c} \in \{0,1\}^\omega$ for $c < n_{\mathrm{ch}}$. The entries of the y_l, in the same order, form the chunks $X_{2,c}$.

In both cases, the realization declares a pad $Z^{(1)} := \mathsf{priv}(f)$ in K_1, for a function f that draws it uniformly. It also declares its bit copy $Z^{(2)} := \mathsf{priv}(f', Z^{(1)})$ in K_2, for the function f' that writes the bits of $Z^{(1)}$, with type Bit, shape (ω) and phase 0. If K_1 hosts the MAC, $Z^{(1)}$ is one element of K_1. The key is then a challenge of K_1, and the tag is a late value of K_1 whose pad is $Z^{(1)}$. If no field hosts the MAC, $Z^{(1)}$ also has type Bit, shape (ω) and phase 0, and the pad of the tag is the element of $\mathbb{F}_{2^\omega}$ whose bits are $Z^{(1)}$. We then count $\mathbb{F}_{2^\omega}$ among the fields $T_0, \ldots, T_{h-1}$ of Definition 2.23, with no

committed elements, so the key and the masked tag are a challenge and a message of Definition 2.23. Over a field of odd characteristic, the compiler adds a Boolean check for each entry of the bit vectors among $Z^{(1)}$ and $Z^{(2)}$ (Section 4.2.3). In both cases, the *key* $\kappa := \mathsf{chal}$ is uniform on $\mathbb{F}_{2^\omega}$ and has phase $j+1$. The *tag* $\mathsf{tag} = \sum_{c<n_{\mathrm{ch}}} \kappa^{c+1} X_{1,c} \in \mathbb{F}_{2^\omega}$ is a late value of the phase of κ (Section 4.1.5). We write Z for Z_{tag}, and we identify $Z^{(1)}$ with Z.

The prover therefore sends the masked tag $\overline{\mathsf{tag}} = \mathsf{MAC}_\kappa(Z^{(1)}, X_1)$.

For $k \in \{1,2\}$, the proof over the field K_k checks the masked tag against the copies in K_k. For $r_c = \kappa^{c+1}$, the *accumulator* of K_k is the expression

$$\mathsf{acc}_k = Z^{(k)} + \sum_{c<n_{\mathrm{ch}}} \mathrm{Conv}(r_c)\, X_{k,c}$$

of shape (ω). If K_k hosts the MAC, its check is $\mathsf{equal}\big(\overline{\mathsf{tag}}, Z^{(k)} + \sum_{c<n_{\mathrm{ch}}} r_c X_{k,c}\big)$ over $\mathbb{F}_{2^\omega}$. If K_k has characteristic 2 and does not host the MAC, its check is $\mathsf{equal}\big(\mathsf{acc}_k, \mathrm{bits}(\overline{\mathsf{tag}})\big)$ entrywise. This check is affine over $\mathbb{F}_2 \subseteq K_k$. The entries of the chunks and of $\mathrm{bits}(\overline{\mathsf{tag}})$ lie in $\mathbb{F}_2$, so the check puts every entry of $Z^{(k)}$ in $\{0,1\}$. If K_k has odd characteristic, its check is $\mathsf{mod}_2\big(\mathsf{acc}_k, \mathrm{bits}(\overline{\mathsf{tag}})\big)$. The compiler realizes this check as in Section 4.1.3. The realized program declares acc_k, or its sums over groups of chunks, as variables, together with the affine equalities that define them (Definition 4.7). These variables and the digits of the quotients are late variables of the phase of κ. Once the key is drawn and the masked tag is sent, both are public, so the coefficients of acc_k and the bits $\mathrm{bits}(\overline{\mathsf{tag}})$ are pub values of K_k. The check contains no exclusive or, and its equalities are affine in the committed bits, the committed accumulators and the quotient digits, so Ligero's linear test proves them. The GKR circuit proves only the lookups of the quotient digits.

The linear map of the check is a sum of convolutions followed by a sparse reduction modulo the field polynomial. If the prime subfield of the field of odd characteristic has a root of unity of order 2ω, the prover and the verifier evaluate the convolutions with number-theoretic transforms of length 2ω (Section 5.4.1).

Two sets of copies that differ satisfy both checks only if the key κ is a root of a nonzero polynomial of degree at most n_{ch} (Lemma 4.5). Over a field of odd characteristic p, the coefficients of acc_k depend on κ and are integers in $[0, \omega]$. The compiler requires $p > 2\omega$, so each coefficient is its own centered representative. Hence acc_k is an integer expression (Definition 3.4) whose pub coefficients lie in the stated interval $[0, \omega]$, and the integer value of acc_k uses these integers as its coefficients. At every key, each entry of acc_k lies in $[0, \mathsf{acc}_{\max}]$ for $\mathsf{acc}_{\max} = 1 + n_{\mathrm{ch}}\mathrm{Conv}_{\max}$, and the compiler uses this interval for acc_k in place of the interval of Definition 4.2. Lemma 4.3 applies at each key. The quotient realization with the least quotient bound $\lfloor \mathsf{acc}_{\max}/2 \rfloor + 1$ meets its span condition when $\mathsf{acc}_{\max} < p - 1$. Otherwise, or when no table of the field can realize the range check on this quotient, the compiler realizes the check by parts (Section 4.1.3).

Lemma 4.5. *Let* $\omega, n_{\mathrm{ch}} \geq 1$, $\mathbb{F}_{2^\omega} = \mathbb{F}_2[X]/(\varphi)$ *with* bits *as above, and for* $k \in \{1,2\}$ *let* $X_{k,0}, \ldots, X_{k,n_{\mathrm{ch}}-1}, Z^{(k)} \in \mathbb{F}_{2^\omega}$*, each identified with its bits in* $\{0,1\}^\omega$*. For* $\kappa \in \mathbb{F}_{2^\omega}$ *let* $r_c = \kappa^{c+1}$ *and* $\mathsf{acc}_k(\kappa) = Z^{(k)} + \sum_{c<n_{\mathrm{ch}}} \mathrm{Conv}(r_c)\, X_{k,c} \in \mathbb{Z}^\omega$*, and let*

$$G(Y) = (Z^{(1)} - Z^{(2)}) + \sum_{c<n_{\mathrm{ch}}} Y^{c+1}(X_{1,c} - X_{2,c}) \in \mathbb{F}_{2^\omega}[Y],$$

the binding polynomial.

1. *For every* $\kappa \in \mathbb{F}_{2^\omega}$ *and every* $t \in \mathbb{F}_{2^\omega}$*: if* $\mathsf{acc}_1(\kappa) \equiv \mathrm{bits}(t) \pmod 2$ *and* $\mathsf{acc}_2(\kappa) \equiv \mathrm{bits}(t) \pmod 2$*, then* $X_{1,c} = X_{2,c}$ *for every* c *and* $Z^{(1)} = Z^{(2)}$*, or* $G \neq 0$ *and* $G(\kappa) = 0$*.*

2. If $G \neq 0$, then $\Pr_{\kappa \xleftarrow{\$} \mathbb{F}_{2^\omega}}[G(\kappa) = 0] \leq n_{\mathrm{ch}}/2^\omega$.

Proof. (1) By Definition 4.4, $\mathrm{Conv}(r)\,\mathrm{bits}(x) \equiv \mathrm{bits}(rx) \pmod 2$, and bits is additive modulo 2, so $\mathsf{acc}_k(\kappa) \equiv \mathrm{bits}\big(Z^{(k)} + \sum_c r_c X_{k,c}\big) \pmod 2$, and the hypotheses give $t = Z^{(k)} + \sum_c r_c X_{k,c}$ for both k. Subtracting, $G(\kappa) = 0$. If $G = 0$, its coefficients vanish: $Z^{(1)} = Z^{(2)}$ and $X_{1,c} = X_{2,c}$ for every c. (2) The polynomial G is nonzero of degree at most n_{ch}, so it has at most n_{ch} roots. □

The masked tag $\overline{\mathsf{tag}}$ hides the shared bits. Its pad Z is uniform, and every other masked value that depends on Z follows the tag in the order of the late values and is masked by its own pad (condition (V1) of Definition 4.8), so the masked tag is uniform on $\mathbb{F}_{2^\omega}$ for every witness (Lemma 5.7(3)). In a field of characteristic 2, the check of the tag gives the hypothesis of Lemma 4.5(1) on the bits. In a field of odd characteristic, the mod_2 check gives it by Lemma 4.3.

4.1.5 Late values

A late value may depend on a verifier challenge (Definition 3.8), so the prover cannot include it in the Ligero commitment.

GordieHowe commits a one-time pad for each late value in the phase-0 commitment of a field, and the prover sends the late value masked by its pad. Each late value costs one committed element and one masked element in the proof. Two properties keep this cost small. First, GKR and LogUp-GKR compute many of the intermediate values that depend on challenges as internal wires, so the prover never commits them. Second, Ligero's proofs are already large, so the masked elements add a small fraction to the proof size.

We estimate that a second Ligero tableau, committed after the challenges to hold the late values, would make our proofs 7 to 14% larger, because it needs its own low-degree test, queried columns and Merkle paths. Such a tableau would also not make the prover faster.

For each late variable v, the compiler declares a *pad* $Z_v := \mathsf{priv}(f)$. The pad is a phase-0 variable of the field and shape of v, except for the tag of a MAC that no field hosts, whose pad is $Z^{(1)}$ (Section 4.1.4). Its type is $\mathsf{Sub}(E)$ when v has type $\mathsf{Sub}(E)$, $\mathsf{Sub}(F)$ for the prime subfield F of the field when v has type Bit, and Field when v has type Field. Its function f draws it uniformly from the set given below. The prover commits Z_v in phase 0 and, after the challenges of v's phase, sends the *masked value* $\overline{v} = v + Z_v$. Each predicate on v then uses $\overline{v} - Z_v$, the difference of a sent value and a committed value, in place of v. The masked value reveals nothing about v, provided that v depends neither on Z_v nor on the pad of a masked value sent after it. Condition (V1) of validity (Definition 4.8) requires this independence, and Section 5.3 proves the hiding.

The pad of a value of type $\mathsf{Sub}(E)$ is uniform on E, so its masked value is a prover message in E. The pad of a value of type Field is uniform on its field. The pad of a value of type Bit is uniform on $\mathbb{F}_2$ in characteristic 2 and, in odd characteristic p, on the prime subfield $\mathbb{F}_p$, since $\{0, 1\}$ is not a subfield. In every characteristic, a late value of type Bit has its Boolean check (condition (V3) of Definition 4.8).

4.2 Subprotocols, tables and packing

The compiler chooses the subprotocol of each equality, the lookup tables of each field and the packing tree of each field with a cost model of the proof system's prover and verifier time, calibrated on measurements. For each field, the model assigns a cost per committed element of the field, a cost per committed packed value of a proper subfield, and a cost per quadratic term (a product of two wires) of the GKR circuit. For Ligero's quadratic test, it assigns a cost per product, a fixed cost, and a cost per tableau row that the products add, which includes hashing that row. For a batched

zerocheck (Section 5.4.2), it assigns each evaluator a fixed cost plus a cost per field operation of the first round and of the operand map.

4.2.1 Dispatching equality constraints

The compiler dispatches each equality to one of four subprotocols: a row of Ligero's linear test, Ligero's quadratic test (Section 5.4.1), a batched zerocheck (Section 5.4.2), or the GKR circuit of its field (Section 5.4.3). We call an equality *affine* if it has degree at most 1 (Definition B.1) in the priv variables. The linear test admits the affine equalities. The compiler dispatches every affine equality to the linear test.

The compiler dispatches the remaining equalities in *constraint families*. A constraint family consists of the equalities over one field that have the form $Q(L_1, \ldots, L_{n_{\text{op}}}) = 0$ for a single polynomial Q with constant coefficients, shared by the family, and whose operands read values stored at a single node of the packing tree. The operands L_j are affine and may differ between the equalities of a family. For example, the equalities of one constraint applied to every round of a hash form a family. The Boolean checks $x(x-1) = 0$ on phase-0 values (Section 4.2.3) form a constraint family of $Q = X^2 - X$. Among the subprotocols that admit a family, the compiler finds the one of least estimated cost. It dispatches the family to that subprotocol if its cost is more than 5% below the cost of the GKR circuit, and to the GKR circuit otherwise. It dispatches an equality outside every constraint family on its own, by the same rule. The quadratic test admits equalities $\mathsf{equal}(\lambda(a \cdot b - c), 0)$ for a constant $\lambda \neq 0$ and phase-0 priv variables a, b, c of one shape, stored at the root of the field's packing tree (Definition 4.6). A batched zerocheck admits instances of polynomials Q_i with $Q_i(0) = 0$. The operands of each instance must be affine in phase-0 values stored at one node of the packing tree, excluding pads. Their public coefficients may differ between instances. The degree of the Q_i must be small enough that the field has a skip domain for the batch (Definition D.2). The compiler forms each batch from the equalities of one constraint family, so the batch's polynomial is the family's polynomial and its instances are the family's equalities (Definition D.1). The GKR circuit admits every equality.

The subprotocols that admit a constraint family depend on the values that the program commits. For example, the library function of Fig. 1 commits the state of a hash after every round. Its round constraints have degree 2, so a batched zerocheck admits them. A program that commits the state only every few rounds has constraints of high degree, and only the GKR circuit admits them. Committing every round enlarges the commitment, and in exchange no layers of the GKR circuit compute the intermediate states. The first sumcheck round of the zerocheck reads every instance. When the operands lie in a subfield, this round computes in that subfield, whereas the GKR circuit computes every gate in the full field. Our library functions commit at the granularity that admits batching. In our applications, the compiler batches the rounds of Keccak-f, the Ch, Maj and word additions of SHA-256, and the products of the RSA convolutions (Section 7.2). In the ML-DSA applications, it also batches the Boolean checks over the field of odd characteristic (Section 5.4.2).

4.2.2 Lookup tables

The compiler chooses the lookup tables of each field: their number and their sizes. A field can have several lookup tables $[0, b)$ of different sizes, and the realization of each range check uses one of them. The proof system proves lookups with LogUp-GKR [PH23] (Section 5.4.3). Each committed digit adds one committed value, each lookup adds six quadratic terms to the field's GKR circuit, and each table row adds one quadratic term and one committed count, the number of lookups into that row. The cost model prices these items, and c_{T} denotes its cost of a table row. A range check

Let the range checks over the field be $\mathsf{lookup}(e_i, [0, n_i))$ for $i \in [r]$, each with admissible bounds $[n_i, m_i]$, and let $\rho_i = \mathsf{none}$ mean that range check i gets no realization because the other realizations imply it. Let $R_i(b)$ be the set of realizations of range check i in the table $[0, b)$ (Section 4.1.1) whose conditions hold, apart from the condition of the complement, which depends on β. A realization ρ establishes a bound $\beta(\rho)$. Its cost $\mathsf{cost}(\rho)$ is the cost of its committed digits and lookups under the cost model.

$$\min_{\mathcal{B},\, \rho_1, \ldots, \rho_r} \quad c_{\mathrm{T}} \textstyle\sum_{b \in \mathcal{B}} b \;+\; \sum_{i:\ \rho_i \neq \mathsf{none}} \mathsf{cost}(\rho_i)$$

$$\begin{aligned}
\text{s.t.} \quad & \rho_i \in \textstyle\bigcup_{b \in \mathcal{B}} R_i(b) \text{ or } \rho_i = \mathsf{none} && \text{for every } i, \\
& \beta(\rho_i) \in [n_i, m_i] && \text{for every } i \text{ with } \rho_i \neq \mathsf{none}, \\
& I_\beta(e_i) \subseteq [0, m_i) && \text{for every } i \text{ with } \rho_i = \mathsf{none}, \\
& I_\beta(e_i) \subseteq [0, p - b] && \text{for every } i \text{ whose } \rho_i \text{ is the complement in a table } [0, b), \\
& \mathsf{span}_\beta(L, R) < p && \text{for every realized congruence or integer equality } L = R,
\end{aligned}$$

where I_β and span_β are the interval of Definition 4.2 and the span of Section 4.1.3 at every index, computed with the bounds $\beta(\rho_i)$ of the realizations by a direct lookup or by digits.

Figure 2: The choice of lookup tables over one field.

$\mathsf{lookup}(e, [0, n))$ on N entries, realized with $k = \lceil \log_b n \rceil$ digits in $[0, b)$ (Section 4.1.1), commits kN digits and makes kN lookups. If b^k is not an admissible bound of the range check, the realization needs a second digit set, the digits of $n - 1 - e$ (Section 4.1.1), which doubles both counts. A larger table costs more rows but lowers k, and so reduces both the digits and the lookups of each range check that uses it. For one range check alone, the best table size therefore grows with the number N of entries checked.

Each range check restricts the tables that can realize it. Its *admissible bounds* are the bounds that its realization may establish. A range check with bound n must establish n itself. The range check on a quotient (Section 4.1.3) may establish any bound in $[n_z, \bar{n}^+]$ that meets the span condition of Lemma 4.3.

The best realization of one range check depends on the realizations of the others, for two reasons. First, range checks over one field can share tables, so a table that one range check already uses costs another range check only its own digits and lookups. Second, the bound that a realization establishes on a variable changes the intervals of the expressions that contain the variable (Section 4.1.3). These intervals decide whether other range checks are implied and whether each realized equality keeps its span below p.

The compiler therefore chooses the realizations of all range checks over a field together. Figure 2 states the optimization problem.

The compiler restricts the table sizes to the candidates $\lceil n_i^{1/k} \rceil$, for the bounds n_i of the range checks over the field and $1 \leq k \leq k_{\max}$, where $k_{\max}$ is the largest number of digits that a realization may use. Some choice of least cost uses only these sizes. To see this, take a choice of least cost and replace each table by the largest of the sizes $\lceil n_i^{1/k} \rceil$ over the realizations that use it. A realization of range check i by k digits keeps its number of digits and stays valid in the new table, whose size b' lies between $\lceil n_i^{1/k} \rceil$ and the old size. Its bound becomes b'^k, which lies between n_i and the old bound and so stays admissible. It needs a second digit set only if it needed one before. A direct or complement lookup stays valid in a table of n_i rows. The new size therefore still admits each realization that uses the table, so the replacement removes rows and adds no digits or lookups. The replacement raises no bound that a realization establishes, so intervals only narrow. Implied range checks stay implied, and a realized equality whose span is below p keeps a span below p.

The compiler chooses the set of table sizes by branch and bound over the subsets of these candidates, deciding the candidates from the largest down and pruning with a lower bound on the cost of each partial choice. For each subset, it computes the cheapest way to realize every range check with tables from that subset. Most range checks are independent: each gets its cheapest realization, or none if it is already implied. Range checks linked through an expression are not independent. For example, let a program range-check x and y into $[0, 16)$ and $x + 16y$ into $[0, 256)$. Once the checks on x and y are realized, the interval of $x + 16y$ lies in $[0, 256)$, so its check is implied and costs nothing. If the compiler chose the realization of this check separately, it would commit range-checked digits of $x + 16y$. The compiler therefore chooses the realizations of linked range checks together. The cost of a subset is the cost of its tables plus the cost of these realizations, and among the subsets of nearly least cost the compiler prefers one with fewer tables.

4.2.3 Packing subfield elements

A program can declare a value of type $\mathsf{Sub}(E)$ for every subfield E of the value's field (Definition 3.2). The proof system packs $[K : E]$ values of E into each committed element of K (Section 5.5). Packing divides the number of committed elements by $[K : E]$, compared with committing each value as its own element of K. Packing also guarantees that every packed value lies in its subfield, so a program can rely on the type $\mathsf{Sub}(E)$ of a packed value without checking that the value lies in E. For example, the permutation check encodes a record of ℓ components in a subfield E as one element $r_0 + \mathring{u} r_1 + \cdots + \mathring{u}^{\ell-1} r_{\ell-1}$, for a fixed $\mathring{u}$ with $1, \mathring{u}, \ldots, \mathring{u}^{\ell-1}$ independent over E (Section B.3). This encoding is injective only because every component lies in E.

For each field, the compiler builds a packing tree to decide how the values of the field are packed. The nodes of the tree are subfields, and each edge packs the values of a node into elements of its parent.

Definition 4.6 (Packing tree)**.** Let K be a finite field. A *packing tree* $\mathcal{T}$ over K consists of:

- a finite rooted tree whose nodes are subfields of K, with root K, in which every non-root node B is a proper subfield of its parent $\mathsf{par}(B)$;
- for every non-root node B, a basis of $\mathsf{par}(B)$ over B together with its dual basis (Definition 2.5);
- for every node T, a number n_T and a partition of $[n_T]$ into a set O_T and one set P_B for each child B of T, with $n_B = [T : B]\,|P_B|$.

For $U \in K^{n_K}$, the *values at the nodes* are $X_K = U$ and $X_B = \mathsf{unpack}_{T/B}\big((X_T)_{P_B}\big)$ for each child B of a node T (Definition 2.6), from the root down. The entries $(X_T)_{P_B}$ are the *packed elements* of B. The entries of U are the *committed elements* over K. The entries $(X_T)_{O_T}$, over all nodes T, are the *stored values*, written $\mathsf{st}(U)$.

For every vector that the prover commits, every value at a node B lies in B, by induction from the root (Definition 2.6). Every stored value therefore lies in the subfield of its node. A value of type $\mathsf{Sub}(B)$ is stored at the node B, and a value of type Field is stored at the root (condition (V3) of Definition 4.8). In odd characteristic, a value of type Bit is stored at the prime subfield and has its Boolean check. The tree must therefore contain the root and every subfield that the program uses. The compiler makes the remaining choices of the tree by cost.

The parent of each node is a minimal node among the other nodes that contain it. The cost of an edge is the number of its mask and chain cells (Section F.1) plus its degree. When several nodes are minimal, as $\mathbb{F}_{q^2}$ and $\mathbb{F}_{q^3}$ are for $\mathbb{F}_q$ inside $\mathbb{F}_{q^6}$, the compiler takes the one whose path to the root has the least total edge cost. The compiler then adds intermediate subfields to split long edges. It

splits an edge of degree above 16 whose degree is a product of factors of at most 16 into a chain of edges of degree at most 16. Let the evaluation subfield S_0 be the smallest subfield that contains the evaluation points of the field's code (Section 5.5.2). The compiler prefers chains in which the base field of every edge contains S_0 or is $\mathbb{F}_2$. Among these, it takes the chain with the fewest edges and, among those, the one whose largest edge degree is smallest. For example, an edge of degree 20 becomes edges of degrees 4 and 5. For the edge from S_0 to K, the compiler also compares the candidate chains by their measured prover and verifier time, and it keeps the chain of the rule above unless another chain costs more than 5% less. For our binary hashing fields $S_0 = \mathbb{F}_{2^{16}}$, so bits are packed into $\mathbb{F}_{2^{16}}$ and then, possibly through an intermediate subfield such as $\mathbb{F}_{2^{64}}$, into the field.

4.2.4 Ligero parameters

Last, the compiler chooses the parameters of each field's Ligero commitment (Section 5.6). For each field, it considers the choices of the tableau, the number of low-degree tests and the mask test at which every Ligero round error is at most the per-round target of the parameter set, or at the ROM parameters the larger error of another round of the field when there is one (Section 5.7). Among these choices, it takes the one of least estimated proof size.

4.3 Realized programs

A realized program collects the outputs of Sections 4.1 and 4.2: the program with its realizations, and the compiler's choices for each field.

Definition 4.7 (Realized program)**.** A *realized program* is a pair $(\mathcal{P}, \mathcal{F})$ of a program $\mathcal{P}$ whose predicates are equalities and lookups and an *annotation* $\mathcal{F} = ((\mathcal{T}_K)_K, \mathsf{node}, \mathsf{pad}, (\mathcal{B}_j)_{j<n_\mathrm{b}}, \mathsf{sub})$. Write $\mathsf{Vars}_0(\mathcal{P})$ for the phase-0 priv variables of $\mathcal{P}$ and $\mathsf{Late}(\mathcal{P})$ for its late variables (Definition 3.8). The annotation consists of:

1. a packing tree $\mathcal{T}_K$ for each field K of $\mathcal{P}$ (Definition 4.6), and a map node that sends each $v \in \mathsf{Vars}_0(\mathcal{P})$ to a node $\mathsf{node}(v)$ of $\mathcal{T}_{K_v}$, the node at which v is stored;
2. a map pad on $\mathsf{Late}(\mathcal{P})$, $v \mapsto Z_v$, the *pad* of v (Section 4.1.5), that sends the tag of a MAC that no field hosts to the element Z of $\mathbb{F}_{2^\omega}$ whose bits are $Z^{(1)}$ (Section 4.1.4), and every other late variable to a variable $Z_v \in \mathsf{Vars}_0(\mathcal{P})$ of its field and shape;
3. disjoint sets $\mathcal{B}_0, \ldots, \mathcal{B}_{n_\mathrm{b}-1}$ of equalities of $\mathcal{P}$, each over one field, the *batches*, each with a skip width $k_j \geq 1$, the number of instances that the first round of its zerocheck handles together (Definition D.1);
4. a map sub from the equalities of $\mathcal{P}$ to $\{\mathsf{affine}, \mathsf{quadratic}, \mathsf{gkr}\} \cup \{\mathsf{batch}(j) : j < n_\mathrm{b}\}$, the subprotocol that proves each equality (Section 4.2): Ligero's linear test, Ligero's quadratic test, the GKR circuit of its field, or the batched zerocheck of a batch, with $\mathsf{sub}(\psi) = \mathsf{batch}(j)$ if and only if $\psi \in \mathcal{B}_j$.

The realized program *realizes* a program $\mathcal{P}_0$, its *source program*, if $\mathcal{P}$ is obtained from $\mathcal{P}_0$ by the following steps. Each predicate other than an equality or a lookup is replaced by one of its realizations (Definition 4.1) whose condition holds, until only equalities and lookups remain. A realization may also commit an expression e as a new variable a with the equality $a = e$, and use a in place of e. Conversely, it may replace a priv variable a that an equality $a = e$ defines by the expression e, and drop that equality. Each range check is realized by a direct lookup, by the complement lookup, by digits, or as implied (Section 4.1.1). The Boolean checks of Section 4.2.3 are added. The counts of each group of lookups are declared (Section 4.1.2). A pad is declared for each late variable.

The compiler outputs, on a program $\mathcal{P}_0$, a realized program that realizes $\mathcal{P}_0$, and it chooses the realization of each range check (Section 4.2.2). No variable of $\mathcal{P}_0$ reads a variable that the realizations add. The restriction of $\mathsf{Exec}_{\mathcal{P}}$ to the variables of $\mathcal{P}_0$ therefore has the distribution of $\mathsf{Exec}_{\mathcal{P}_0}$.

The proof system's guarantees hold for every valid realized program that realizes its source program, at Ligero parameters that meet the conditions of Theorem 5.3. Lean's kernel checks from the compiler's certificates that each exported realized program is valid and realizes its source program (Section 8).

Definition 4.8 (Valid realized program)**.** We order the late variables of a realized program $(\mathcal{P}, \mathcal{F})$ by $\prec$: by phase, then in order of declaration. A realized program $(\mathcal{P}, \mathcal{F})$ is *valid* if it meets the following conditions.

(V1) For all late variables u and v of $\mathcal{P}$, if the pad of u lies in $\mathsf{Dep}(v)$, then $u \prec v$.

(V2) Every equality has the form that its subprotocol admits (Section 4.2.1).

(V3) Every phase-0 priv variable of type Field is stored at the root K. Every phase-0 priv variable of type Bit or $\mathsf{Sub}(E)$, other than the pad of a late variable, is stored at the node of the subfield of its type or, for type Bit in characteristic 2, has its Boolean check. The pad of every late variable of type $\mathsf{Sub}(E)$ or Bit is stored at the node of the subfield of its type. Every late variable of type Bit has its Boolean check (Section 4.2.3).

(V4) Every packing tree $\mathcal{T}_K$ of $\mathcal{F}$ is admissible (Definition 5.11).

Condition (V1) makes the masked values hide the late values (Lemma 5.7(3)), and condition (V2) gives the subprotocols of Section 5.4 their input forms. When the predicates hold and every masked value lies in its message set, condition (V3) puts the value of every priv variable v of type Bit or $\mathsf{Sub}(E)$ in its type set $\mathcal{X}_v^{\mathsf{type}}$. Condition (V4) gives every edge of every packing tree a mask construction, so ring switching applies along it (Theorem F.6).

5 The proof system

A realized program $(\mathcal{P}, \mathcal{F})$ (Definition 4.7) describes an interactive oracle proof $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ (Protocol 5.1). Its statements and witnesses are those of the source program $\mathcal{P}_0$, and its oracles are Ligero tableaus, one for each field of $\mathcal{P}$. GordieHowe applies the spBCS transformation (Section 6) to $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ to obtain a non-interactive argument. Theorem 5.3 states the security of $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$, and Theorem 6.7 states the security of the non-interactive argument.

5.1 The protocol

The prover first commits one Ligero tableau for each field, and the verifier applies Ligero's low-degree test to each tableau. Ligero masks its linear and quadratic tests with one row each, and its low-degree test does not cover these mask rows. We augment Ligero with a low-degree test on these masks, the *mask test* (Section 5.6.2). Neither Ligero as published nor Longfellow has this test (Section 5.6.1).

The rest of $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ is a composition of interactive oracle reductions (Definition 2.23) on the oracle input that the tableaus encode. Each reduction takes as input the claims that earlier reductions output. The phase step Π_{ph} reduces the relation $\mathcal{R}_{\mathcal{R}}^{\mathbf{E}}$ on the phase-0 values to the relation $\mathcal{R}_{\mathrm{pred}}$ (Lemma 5.7). For a relation $\mathcal{R}$ and an extractor $\mathbf{E}$, the relation $\mathcal{R}_{\mathcal{R}}^{\mathbf{E}}$ holds when $\mathbf{E}$ maps the statement and the phase-0 values to a witness of the statement in $\mathcal{R}$. The relation $\mathcal{R}_{\mathrm{pred}}$ holds when the predicates of the realized program hold at the sent challenges and masked values. Definition 5.6

defines both relations. The subprotocols Π_{zc} and Π_{GKR} reduce the batched predicates and the predicates of the GKR circuits to linear claims and product constraints (Section 5.4). Their linear claims are on stored values and on elements of the oracle input. Their product constraints are on elements of the oracle input. The equalities of E_K^{aff} are already linear claims, and the equalities of E_K^{quad} are already product constraints. Step 3 passes the linear claims of E_K^{aff} to ring switching and the product constraints of E_K^{quad} to Ligero unchanged. In the series composition with ring switching (Lemma 2.27), these product constraints and the claims of Π_{zc} and Π_{GKR} on elements of the oracle input form the part $\mathcal{R}_{\mathrm{out}}$ that passes through ring switching unchanged. Ring switching $\Pi_{\mathrm{RS}}(\mathcal{T}_K)$ reduces the linear claims on stored values to linear claims on elements of the oracle input, product constraints and membership claims (Theorem F.6). Ligero's proof phase $\Pi_{\mathrm{Lig}}(U_K)$ reduces the claims on elements of the oracle input to the verifier's checks at the queried columns (Lemma 5.13). Figure 3 gives $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$.

By condition (V2) of Definition 4.8 and the construction of C_K (Section 5.4.3), $\mathcal{R}_{\mathrm{pred}}$ is a conjunction over the fields, and each conjunct is the input of one of these reductions. For each field K, the conjuncts are $\mathcal{R}_{\mathrm{batch}}$ at each batch of K, the linear claims of E_K^{aff} on the stored values, the product constraints of E_K^{quad}, and, when K has a GKR circuit, $\mathcal{R}_{\mathrm{GKR}}$ at C_K, which covers the equalities with $\mathsf{sub}(\psi) = \mathsf{gkr}$ and the lookups.

Let ph' be the phases that $\mathcal{P}$ assigns to the variables of $\mathcal{P}_0$. A verifier message of $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ can carry the challenges of several consecutive verifier messages of $\Pi_{\mathcal{P}_0,\mathsf{ph}'}$ (Definition B.2) or of one reduction, with no prover message between them. For example, one message carries the chal variables and the LogUp challenges of one phase. Lemma 5.2 bounds the round error of such a message by the sum of the errors of its challenges.

Lemma 5.2 (Merged verifier messages)**.** *Let Π be a series–parallel IOP or a reduction, and let Π' be obtained from Π by merging consecutive verifier messages $i_1 < \cdots < i_r$ of Π, with no prover message between them and with $\mathrm{past}(i_{j+1}) = \mathrm{past}(i_j) \cup \{i_j\}$, into one verifier message i. Let a state function of Π (Definition 2.20 for an IOP, Definition 2.24(a) for a reduction) have errors $\varepsilon^{\mathrm{k}}_{i_1}, \ldots, \varepsilon^{\mathrm{k}}_{i_r}$ at these messages. For an IOP, let the output of $\mathbf{E}$ on $\mathsf{tr}|_{\mathrm{past}(i_j)}$ be the same for every j. Then the states at $i_1, \ldots, i_r$ give a state function of Π' with error $\sum_j \varepsilon^{\mathrm{k}}_{i_j}$ at i and the errors of Π at its other verifier messages.*

Proof. The order of Π' is that of Π on the messages other than $i_2, \ldots, i_r$, with i in place of i_1, so it is series–parallel. The challenge set of i is the product of those of the i_j. The extractor of Π' at i outputs $\mathbf{E}(\mathsf{x}, \mathsf{tr}|_{\mathrm{past}(i_1)})$, which by hypothesis equals $\mathbf{E}(\mathsf{x}, \mathsf{tr}|_{\mathrm{past}(i_j)})$ for every j. At every other verifier message, the state and the extractor are those of Π on the restriction of the transcript to the message's past in Π. Let the state of i be 0 on every transcript of $\mathrm{past}(i)$. On a transcript of $\mathrm{past}(i) \cup \{i\}$, let it be 1 if the state moves from 0 to 1 at some i_j. Two facts give condition (a) of Definition 2.20. First, a move at some i_j is a move at i. Second, $\mathbf{E}$ fails on $\mathrm{past}(i)$ exactly when it fails on every $\mathrm{past}(i_j)$. For condition (b), let $\mathbf{E}$ fail on $\mathrm{past}(i)$, and hence on every $\mathrm{past}(i_j)$. By (b) at i_j, at every value of the earlier challenges at which the state at i_j is 0, the state moves to 1 with probability at most $\varepsilon^{\mathrm{k}}_{i_j}$. A union bound over j bounds the probability of a move at i by $\sum_j \varepsilon^{\mathrm{k}}_{i_j}$. For a reduction, the same argument gives conditions (i) and (ii) of Definition 2.24(a), without an extractor. □

Theorem 5.3 (Security of $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$)**.** *Let $(\mathcal{P}, \mathcal{F})$ be a valid realized program (Definition 4.8) that realizes $\mathcal{P}_0$ (Definition 4.7), and let $\mathcal{R} \subseteq \mathcal{X}^{\mathrm{st}}_{\mathcal{P}_0} \times \mathcal{W}_{\mathcal{P}_0}$. For each field, let the Ligero parameters (block length n_{RS}, dimension k_{RS}, n_{col} queried columns, ℓ entries per tableau row, n_{rep} low-degree challenges, and an agreement threshold A at which the code has full agreement-set line MCA with*

Protocol 5.1 (The interactive oracle proof $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$)**.** *Parameters.* A valid realized program $(\mathcal{P},\mathcal{F})$ that realizes $\mathcal{P}_0$, with fields $K_0,\dots,K_{h-1}$. For each field K, $\mathcal{F}$ gives the packing tree $\mathcal{T}_K$, the set $\mathcal{B}_K$ of the batches $\mathcal{B}_j$ whose equalities are over K, each with its skip width k_j, and the sets E_K^{aff} and E_K^{quad} of the equalities of K assigned to Ligero's linear test ($\mathsf{sub}(\psi)=\mathsf{affine}$) and to its quadratic test ($\mathsf{sub}(\psi)=\mathsf{quadratic}$). The compiler gives the Ligero parameters of each field (Section 4.2.4). For each field K that has a GKR circuit, the proof system gives the GKR circuit C_K with its input layout lay_K (Section 5.4.3). For $v\in\mathsf{Late}(\mathcal{P})$, $\mathcal{X}_v^{\text{pad}}$ is the pad set of v (Definition 5.6).
Inputs. $\mathbf{P}$ gets $(\mathsf{x},\mathsf{w})\in\mathcal{X}_{\mathcal{P}_0}^{\text{st}}\times\mathcal{W}_{\mathcal{P}_0}$, and $\mathbf{V}$ gets x.

1. Π_{com}.
   (a) For each field K, $\mathbf{P}$ chooses the stored values of the packing tree $\mathcal{T}_K$ (Definition 4.6): each phase-0 priv variable v of K, pads included, at distinct positions of $O_{\mathsf{node}(v)}$, and the pads, product elements and masks of Π_{zc}, Π_{GKR} and Π_{RS} on K at positions of O_K. It packs them into the elements $U_K\in K^{n_K}$ of the oracle input.
   (b) $\mathbf{P}$ sends the tableaus of $U_{K_0},\dots,U_{K_{h-1}}$ as the oracle strings of one message.
   (c) For $K=K_0,\dots,K_{h-1}$: $\mathbf{V}$ sends the low-degree challenges χ_K, and $\mathbf{P}$ sends the low-degree responses w_K.
2. Π_{ph}. For each phase $k\geq 1$ of a variable of $\mathcal{P}$ or of a challenge α_K (Section 5.2), in increasing order:
   (a) $\mathbf{V}$ sends the chal variables of phase k and the challenges α_K of phase k.
   (b) $\mathbf{P}$ sends $\overline{v}=v+Z_v\in\mathcal{X}_v^{\text{pad}}$ for each $v\in\mathsf{Late}(\mathcal{P})$ of phase k (Definition 5.6).
3. For each field K: $\Pi_{\text{zc}}(\mathcal{B}_j)$ for each $\mathcal{B}_j\in\mathcal{B}_K$, and $\Pi_{\text{GKR}}(C_K,\mathsf{lay}_K)$ if K has a GKR circuit.
4. For each field K: $\Pi_{\text{RS}}(\mathcal{T}_K)$ on the linear claims on stored values of K from step 3 and from E_K^{aff}.
5. For each field K: $\Pi_{\text{Lig}}(U_K)$, the proof phase of Ligero, with or without the mask test, on the tableau of U_K, with the claims on U_K from steps 3 and 4 and with E_K^{quad}.

The verifier $\mathbf{V}$ accepts if and only if no subprotocol rejects. The protocol is composed (Definition 2.26) as

$$\mathsf{IOP}_{\mathcal{P},\mathcal{F}}=\Big(\bigsqcup_K\Pi_{\text{Lig}}(U_K)\circ\Pi_{\text{RS}}(\mathcal{T}_K)\circ\Big(\text{Id}(E_K^{\text{aff}},E_K^{\text{quad}})\sqcup\Pi_{\text{GKR}}(C_K,\mathsf{lay}_K)\sqcup\bigsqcup_{\mathcal{B}_j\in\mathcal{B}_K}\Pi_{\text{zc}}(\mathcal{B}_j)\Big)\Big)\circ\Pi_{\text{ph}}\circ\Pi_{\text{com}},$$

where $\text{Id}(E_K^{\text{aff}},E_K^{\text{quad}})$ is the reduction with no rounds whose output statement is its input, the claims of E_K^{aff} and E_K^{quad}, and the term $\Pi_{\text{GKR}}(C_K,\mathsf{lay}_K)$ is omitted for each field K without a GKR circuit.

Figure 3: The interactive oracle proof $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ for a valid realized program $(\mathcal{P},\mathcal{F})$. The subprotocols Π_{zc}, Π_{GKR}, Π_{RS} and Π_{Lig} are the masked batched zerocheck, masked GKR, ring switching along the packing tree, and Ligero's proof phase.

exceptional count n_{bad}, as in Definition 5.12) satisfy $2k_{\mathrm{RS}} < n_{\mathrm{RS}}$, $n_{\mathrm{col}} \le k_{\mathrm{RS}} - \ell$ and, for a tableau with the mask test, $2n_{\mathrm{col}} \le k_{\mathrm{RS}}$ (Lemma 5.13 and Section G.6). Then $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ (Protocol 5.1) is a series–parallel IOP (Definition 2.18) for $\mathcal{R}$, and the following hold.

1. *If $\mathcal{P}_0$ is complete for $\mathcal{R}$ with error ε_{c} (Definition 3.10), then $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ has completeness error at most ε_{c} for $\mathcal{R}$ (Definition 2.19), and honest-verifier zero-knowledge error at most ε_{c} (Definition 2.22) with respect to the simulator of Protocol 5.4.*
2. *If $\mathcal{P}_0$ is round-by-round knowledge sound for $\mathcal{R}$ with errors $(\varepsilon_\gamma)_\gamma$ (Definition 3.10) under the phases ph' that $\mathcal{P}$ assigns to the variables of $\mathcal{P}_0$ (Definition B.2), then $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ is round-by-round knowledge sound for $\mathcal{R}$ (Definition 2.20) with round errors at most $\varepsilon^{\mathcal{F}}$. At each verifier message, $\varepsilon^{\mathcal{F}}$ is the sum of the following errors, over the challenges that the message carries:*
   - *ε_γ at a challenge γ of $\mathcal{P}_0$;*
   - *$\varepsilon_{\mathrm{LogUp}}(K)$ at the LogUp challenge α_K and $\varepsilon_{\mathrm{MAC}}(M)$ at the key of a MAC M (Lemma 5.7);*
   - *$\varepsilon_{\mathrm{zc}}(\mathcal{B},\theta)$ at a challenge θ of the zerocheck of a batch $\mathcal{B}$ (Lemma 5.8), and $\varepsilon_{\mathrm{GKR}}(K,\theta)$ at a challenge θ of the GKR circuit of K (Lemma 5.9);*
   - *$1/|K|$ at each combination challenge of ring switching along $\mathcal{T}_K$ (Theorem F.6);*
   - *$\varepsilon_{\mathrm{com,ldt}}$, $\varepsilon_{\mathrm{prf,con}}$ and $\varepsilon_{\mathrm{prf,col}}$ of the Ligero parameters of K at the message of the low-degree challenges, the constraint challenge and the column query of K's tableau (Definition 5.12).*

We prove Theorem 5.3 by composing the reductions of Sections 5.3 to 5.6 on the oracle input extracted from the tableaus. Fix the tableaus $(\widehat{U}_K)_K$ of step 1. The state at the low-degree challenge of K moves to 1 when $\chi_K \in \mathcal{Z}_{\widehat{U}_K}$, an event of probability at most $\varepsilon_{\mathrm{com,ldt}}$ (Lemma 5.13(1), or Corollary G.11 for a tableau without the mask test). When $\chi_K \notin \mathcal{Z}_{\widehat{U}_K}$ for every K, let $\boldsymbol{U} = (\mathsf{Ext}(\widehat{U}_K, \chi_K, w_K))_K$. The prefix up to the last low-degree response determines $\boldsymbol{U}$. Replace each relation $\mathcal{R}'$ of Π_{ph}, Π_{zc}, Π_{GKR} and Π_{RS} by the relation of the pairs $((\mathsf{x}', (\chi_K, w_K)_K), (\widehat{U}_K)_K)$ with $(\mathsf{x}', \boldsymbol{U}) \in \mathcal{R}'$. These reductions and $\Pi_{\mathrm{Lig}}(U_K)$ are then interactive oracle reductions with the same oracle input $(\widehat{U}_K)_K$. Their series–parallel composition (Lemma 2.27) is a reduction from $\mathcal{R}_{\mathcal{R}}^{\mathbf{E}}$ (Definition 5.6) to the relation $\mathcal{R}_{\mathrm{col}}$ of Lemma 5.13(1), with the round errors of Theorem 5.3(2). By Lemma 5.2, the error at a message that carries several challenges is the sum of the errors of its challenges. Let $\mathbf{E}$ output $\mathbf{E}(\mathsf{x}, \boldsymbol{U})$ on every past that contains the low-degree responses of every field, and a fixed candidate witness on every other past. After the commitment, let the state of $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ be the state of the composite reduction at this $\boldsymbol{U}$. If $\chi_K \in \mathcal{Z}_{\widehat{U}_K}$ for some K, let the state at every later verifier message be 1. Condition (a) of Definition 2.20 then follows from condition (i) of Definition 2.24(a), and condition (b) from condition (ii).

For completeness, fix the coins of the execution of $\mathcal{P}_0$ and the pads and masks of the honest prover. This fixes $\boldsymbol{U}$. The honest commitment passes the low-degree tests. The reductions after Π_{ph} are complete on $\boldsymbol{U}$ with error 0 (Lemmas 5.8, 5.9 and 5.13 and Theorem F.6), so the verifier accepts whenever the output statement of Π_{ph} lies in $\mathcal{R}_{\mathrm{pred}}$. By Lemma 5.7(2), this statement lies outside $\mathcal{R}_{\mathrm{pred}}$ with probability at most ε_{c} over the coins and the challenges. So the verifier rejects with probability at most ε_{c}.

For zero knowledge, the reductions before Π_{Lig} are honest-verifier zero knowledge with error 0 with respect to the uniform simulator. Their pads and masks occupy disjoint positions of the vectors U_K, and the prover draws these pads and masks independently. The messages of each reduction do not depend on the masks of the reductions after it, and its sent values are uniform at every value of the other reductions' masks. Lemma 2.27(c) therefore applies to these reductions. On the event that the claims of step 5 hold, the Ligero view has the distribution of the output of Protocol G.8 on these

claims and uniform Ligero challenges (Lemma 5.13(3)), or of its variant in the proof of Corollary G.11 for a tableau without the mask test. This holds at every oracle input at which the claims hold, and the prover draws Ligero's masks independently of the other pads and masks. So, given the claims, this view is independent of the earlier messages. At every choice of the challenges and coins of $\mathcal{P}_0$ at which the execution of $\mathcal{P}_0$ is accepting, $\mathcal{R}_{\text{pred}}$ holds at every pad value by Lemma 5.7(2), so the claims of step 5 hold at every value of the later challenges and masks by the completeness of Lemma 5.8, Lemma 5.9 and Theorem F.6. At such a choice, the simulator's output has the distribution of the view. The execution fails to be accepting with probability at most ε_{c}, so the zero-knowledge error is ε_{c}. Protocol 5.4 gives the simulator.

Protocol 5.4 (HVZK simulator of Protocol 5.1)**.** On a statement x:

1. draw the verifier's messages of Protocol 5.1 uniformly;
2. draw each masked value $\overline{v}$, for $v \in \mathsf{Late}(\mathcal{P})$, uniformly from $\mathcal{X}_v^{\text{pad}}$, and each element that a batch, a GKR circuit or a ring-switching step sends uniformly from its field;
3. compute the statements that steps 3 and 4 output from x, the verifier's messages and these values, and from these statements the linear claims, product constraints and membership claims of step 5 over each field;
4. for each field, run Protocol G.8, the simulator of Lemma 5.13(3), or its variant in the proof of Corollary G.11 for a tableau without the mask test, on these claims and the Ligero challenges of the field. This simulator outputs the low-degree response, the tests' responses and the queried columns;
5. output the verifier's messages and the values of steps 2 and 4.

The formalization proves the theorem for every valid realized program that passes the exported checks (Section 8), under the three assumptions on the GKR circuits stated in Section 8.3 and the hypothesis that Lean's fields follow their specifications (Appendix J).

5.2 Lookups

The proof system proves the lookups of each field K with LogUp [Hab22]. For each group of lookups (Section 4.1.2), the *LogUp identity* of the group states that $\sum_i 1/(X - v_i) = \sum_j m_j/(X - \mathsf{tb}_j)$ as rational functions, where v_i are the looked-up values, tb_j are the rows of the table and m_j are the counts (Lemma 2.3). The proof system checks every LogUp identity of K at one challenge α_K, and the GKR circuit of K evaluates both sides (Section 5.4.3).

The verifier draws α_K in the phase after the last phase of the values, the table rows and the counts of the lookups over K, and no row is computed from α_K. It draws α_K uniformly from $K \setminus \mathcal{N}_\alpha$, where $\mathcal{N}_\alpha$ is the union of the range tables $[0, b)$ of K, so that no denominator $\alpha_K - \mathsf{tb}_j$ at a row of a range table is zero. The GKR circuit of K checks the identity of every other table with the denominators of both sides cleared, whether the rows of the table are private, computed from the statement or given by an expression. The rows of these tables need not lie in $\mathcal{N}_\alpha$ (Lemma 5.5(3)).

Lemma 5.5 (LogUp soundness, following [Hab22, §3.2, eq. (22), p. 11])**.** *Let F be a finite field, $\mathsf{tb} \in F^{|\mathsf{tb}|}$ a table and $v \in F^N$, with N below the characteristic of F. For $\alpha \in F$, let $w_j = (\alpha - \mathsf{tb}_j)^{-1}$ if $\alpha \neq \mathsf{tb}_j$ and $w_j = 0$ otherwise, and let $N_v(X) = \sum_i \prod_{i' \neq i} (X - v_{i'})$ and $D_v(X) = \prod_i (X - v_i)$, the numerator and the denominator of $\sum_i 1/(X - v_i)$.*

1. *For every set* $\mathcal{N}_\alpha \subseteq F$ *that contains* $\mathsf{tb}_0, \ldots, \mathsf{tb}_{|\mathsf{tb}|-1}$*, every* v *with some* v_i *not an entry of* tb*, and every* $m \in F^{|\mathsf{tb}|}$*,*

$$\Pr_{\alpha \xleftarrow{\$} F \setminus \mathcal{N}_\alpha} \left[N_v(\alpha) = D_v(\alpha) \sum_{j=0}^{|\mathsf{tb}|-1} m_j w_j \right] \leq \frac{N + |\mathsf{tb}| - 1}{|F| - |\mathcal{N}_\alpha|}.$$

2. *If every* v_i *is an entry of* tb *and* m *is as in Lemma 2.3, the equation of (1) holds for every* $\alpha \notin \{\mathsf{tb}_0, \ldots, \mathsf{tb}_{|\mathsf{tb}|-1}\}$*.*
3. *Let* $N_{\mathsf{tb}}(X) = \sum_j m_j \prod_{j' \neq j} (X - \mathsf{tb}_{j'})$ *and* $D_{\mathsf{tb}}(X) = \prod_j (X - \mathsf{tb}_j)$*. For every set* $\mathcal{N}_\alpha \subseteq F$*, every* v *with some* v_i *not an entry of* tb*, and every* $m \in F^{|\mathsf{tb}|}$*,*

$$\Pr_{\alpha \xleftarrow{\$} F \setminus \mathcal{N}_\alpha} \left[N_v(\alpha) D_{\mathsf{tb}}(\alpha) = D_v(\alpha) N_{\mathsf{tb}}(\alpha) \right] \leq \frac{N + |\mathsf{tb}| - 1}{|F| - |\mathcal{N}_\alpha|}.$$

   If every v_i *is an entry of* tb *and* m *is as in Lemma 2.3, this equation holds for every* $\alpha \in F$*.*

The proof clears the denominators of the identity of Lemma 2.3, and the cleared difference is a nonzero polynomial of degree at most $N + |\mathsf{tb}| - 1$. Drawing α off the table's entries keeps every weight defined. Part (3) needs no restriction on α, since the cleared equation has no denominators.

5.3 Phases and late values

The late variables $\mathsf{Late}(\mathcal{P})$ of a realized program $(\mathcal{P}, \mathcal{F})$ include the counts whose phase is above 0 (Section 4.1.2) and the tag of each MAC (Section 4.1.4). Their order is $\prec$ (Definition 4.8).

Definition 5.6 (Relations of the phase step)**.** Let $(\mathcal{P}, \mathcal{F})$ be a realized program that realizes $\mathcal{P}_0$. The *pad subfield* of a type is E for $\mathsf{Sub}(E)$, the field itself for Field, and the prime subfield for Bit (Section 4.1.5). For a late variable v, its *pad set* $\mathcal{X}_v^{\mathrm{pad}}$ is the set of tensors of v's shape over the pad subfield of v's type. The pad set of the tag of a MAC is $\mathcal{X}_{\mathsf{tag}}^{\mathrm{pad}} = \mathbb{F}_{2^\omega}$. The *predicates of the realized program*, $\mathsf{Preds}_{\mathcal{F}}$, are the equalities of $\mathcal{P}$ and, for each group of lookups over a field K, its LogUp identity evaluated at α_K. The evaluated identity is the equation of Lemma 5.5(1) for a range table and the equation of Lemma 5.5(3) for every other table. The *phase-0 values* of an oracle input $\boldsymbol{U} = (U_K)_K$ are the values of the phase-0 priv variables of $\mathcal{P}$ at their positions among the stored values. Let $\mathcal{R} \subseteq \mathcal{X}_{\mathcal{P}_0}^{\mathrm{st}} \times \mathcal{W}_{\mathcal{P}_0}$ be a relation, and let $\mathbf{E}$ be an extractor whose output depends only on a statement and on the phase-0 values of the variables of $\mathcal{P}_0$. The relation $\mathcal{R}_{\mathcal{R}}^{\mathbf{E}}$ contains $(\mathsf{x}, \boldsymbol{U})$ if and only if $(\mathsf{x}, \mathbf{E}(\mathsf{x}, \boldsymbol{U})) \in \mathcal{R}$. The statements of the relation $\mathcal{R}_{\mathrm{pred}}$ are the tuples $(\mathsf{x}, c, \vec{\overline{v}})$. Here c assigns the chal variables of $\mathcal{P}$ and the LogUp challenges α_K (step 2 of Protocol 5.1), and $\vec{\overline{v}}$ is the vector of masked values. The relation $\mathcal{R}_{\mathrm{pred}}$ contains $((\mathsf{x}, c, \vec{\overline{v}}), \boldsymbol{U})$ if and only if every predicate of $\mathsf{Preds}_{\mathcal{F}}$ holds at the assignment formed by x, c, $\vec{\overline{v}}$, the phase-0 values of $\boldsymbol{U}$, the value $\overline{v} - Z_v$ of each late v, and the pub values computed from them.

Each masked value is public once sent, so a predicate on a late v, with $\overline{v} - Z_v$ in place of v, is a predicate on the phase-0 values. Its coefficients depend on x, c and $\vec{\overline{v}}$. The substitution is affine in the pad, so an affine predicate stays affine. The execution of $\mathcal{P}$ (Definition 3.9) draws the pad of each late variable v uniformly from $\mathcal{X}_v^{\mathrm{pad}}$ and draws the MAC pads as declared in Section 4.1.4. The honest prover of $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ carries out the execution of $\mathcal{P}$ at the verifier's challenges and draws the pads, product elements and masks of Π_{zc}, Π_{GKR} and Π_{RS} from their honest distributions. As a reduction (Definition 2.23), Π_{ph} has an empty first prover message, and its last round has a challenge set of one element.

Lemma 5.7 (Security of the phase step)**.** *Let $(\mathcal{P}, \mathcal{F})$ be a valid realized program that realizes $\mathcal{P}_0$, $\mathcal{R} \subseteq \mathcal{X}^{\mathrm{st}}_{\mathcal{P}_0} \times \mathcal{W}_{\mathcal{P}_0}$ a relation, and Π_{ph} step 2 of Protocol 5.1, on the oracle input $\boldsymbol{U}$ of step 1, with output statement $(\mathsf{x}, c, \vec{\overline{v}})$ for the challenges c and the masked values $\vec{\overline{v}}$ that it sends.*

1. *If $\mathcal{P}_0$ is round-by-round knowledge sound for $\mathcal{R}$ with errors $(\varepsilon_\gamma)_\gamma$ and extractor $\mathbf{E}$ (Definition 3.10) under the phases that $\mathcal{P}$ assigns to the variables of $\mathcal{P}_0$ (Definition B.2), then Π_{ph} is a reduction (Definition 2.23) from $\mathcal{R}^{\mathbf{E}}_{\mathcal{R}}$ to $\mathcal{R}_{\mathrm{pred}}$ (Definition 5.6) with round-by-round soundness error $\sum_\gamma \varepsilon_\gamma$ at the message of phase k, over the chal variables and the LogUp challenges γ of phase k. For the chal variables of $\mathcal{P}_0$, ε_γ is the given error. For the LogUp challenges (Section 5.2) and the keys of the MACs, ε_γ is:*
   (a) *for the challenge α_K of a field K: $\varepsilon_{\mathrm{LogUp}}(K) = \sum_{\mathsf{tb}} \sum_g (N_g + |\mathsf{tb}| - 1)/(|K| - |\mathcal{N}_\alpha|)$, over the tables tb of K and their groups g (Section 4.1.2);*
   (b) *for the key of a MAC M with n_{ch} chunks over $\mathbb{F}_{2^\omega}$: $\varepsilon_{\mathrm{MAC}}(M) = n_{\mathrm{ch}}/2^\omega$ (Lemma 4.5).*
2. *Let $(\mathsf{x}, \mathsf{w}) \in \mathcal{X}^{\mathrm{st}}_{\mathcal{P}_0} \times \mathcal{W}_{\mathcal{P}_0}$, and let $S = \mathsf{Exec}_{\mathcal{P}_0}(\mathsf{x}, \mathsf{w})$ be accepting with probability at least $1 - \varepsilon_{\mathrm{c}}$. Let S' extend S by the values of the declarations that the realizations add, at the challenges that the realizations introduce. These values are the honest quotients and remainders (Lemma 4.3(2), Section 4.1.3), digits (Section 4.1.1), counts (Section 4.1.2), the variables that commit expressions (Definition 4.7) and the accumulators of each MAC (Section 4.1.4). The extension S' also adds the tag $\sum_{c<n_{\mathrm{ch}}} r_c X_{1,c}$ of each MAC (Section 4.1.4), at these challenges. Let $\boldsymbol{U}$ be an oracle input whose phase-0 values other than the pads are those of S', whose pads lie in their pad sets, $Z_v \in \mathcal{X}^{\mathrm{pad}}_v$ for each $v \in \mathsf{Late}(\mathcal{P})$, and whose pad copies satisfy $Z^{(2)} = Z^{(1)}$ in each MAC. For each value ρ of the coins of S, the prover $\mathbf{P}_\rho$ of Π_{ph} that sends $\overline{v} = S'_v + Z_v$ for each $v \in \mathsf{Late}(\mathcal{P})$, with S'_v computed at the pads of $\boldsymbol{U}$ and at ρ, gives $((\mathsf{x}, c, \vec{\overline{v}}), \boldsymbol{U}) \in \mathcal{R}_{\mathrm{pred}}$ at every value of the challenges and the coins of S at which S is accepting, and so except with probability at most ε_{c} over them.*
3. *When the pads are drawn as the execution of $\mathcal{P}$ draws them, the prover $\mathbf{P}_\rho$ of (2) is, at every ρ, honest-verifier zero knowledge with error 0 with respect to the uniform simulator (Definition 2.24), whose message sets are the pad sets $\mathcal{X}^{\mathrm{pad}}_v$ (Definition 5.6).*

Appendix C proves Lemma 5.7.

5.4 Proving the equalities

This subsection describes how the proof system proves the equalities of each kind (Section 4.2.1): by Ligero's linear and quadratic tests, by the batched zerocheck Π_{zc} and by the GKR circuit Π_{GKR}. It also gives the claims that Π_{zc} and Π_{GKR} output.

5.4.1 Linear and quadratic tests

Ligero proves linear claims on committed elements with its linear test, and product constraints $x_i \cdot x_j = x_l$ with its quadratic test [AHIV22, §§4.2–4.4].

An equality assigned to the linear test ($\mathsf{sub}(\psi) = \mathsf{affine}$) is affine (Section 4.2.1). Its coefficients may depend on pub values and challenges. Each late value v in the equality enters as $\overline{v} - Z_v$. Since the verifier knows every coefficient and the masked value $\overline{v}$, and the pad Z_v is a stored value, the equality is a linear claim on the stored values (Definitions 2.25 and 4.6). Ring switching reduces it to linear claims on the committed elements (Theorem F.6). For example, when the span of its two sides is below p, the congruence of Fig. 1 is realized as $Mv = t + qz$ (Section 4.1.3). This equality is

affine in v and in the quotient z, so the linear test proves it. Each equality of its realization by parts (Section 4.1.3) is affine in the same way.

An equality assigned to the quadratic test ($\mathsf{sub}(\psi) = \mathsf{quadratic}$) has the form $\mathsf{equal}(\lambda(a \cdot b - c), 0)$ of Section 4.2.1. It holds exactly when the product constraints $a_i b_i = c_i$ hold on the committed elements. Ligero copies the three operands of each product constraint into three rows and adds the copy constraints to the linear claims (Section G.2).

In the linear test, the prover and the verifier both apply the transpose of the constraint matrix to the verifier's random vector. When the matrix is a convolution with a public operand or a number-theoretic transform, GordieHowe applies it in quasi-linear time. Convolutions with a public operand arise in the MAC's carry-less products (Section 4.1.4) and in polynomial products in lattice signatures. The transforms arise in products of large integers (Section 7.2). A transform requires a root of unity of its length in the prime subfield. Every field of odd characteristic in our applications has such roots except the base field of P-256. Over the base field of P-256, GordieHowe applies the matrix directly.

5.4.2 Batched zerocheck

A batched zerocheck proves all instances of a few polynomial constraints at once (Section 4.2.1). Examples are the rounds of Keccak-f [Nat15b] and the steps of SHA-256's compression function [Nat15a].

A batch is a set of instances of polynomials $Q_0, \ldots, Q_{n_Q-1}$ of total degree at most D with $Q_i(0) = 0$ (Definition D.1). A batch may be over any field of the program, and the operands of its instances are affine in values stored at any nodes of the field's packing tree (Section 4.2.3). The compiler forms each batch over phase-0 values that are stored at one node and are not pads (Section 4.2.1). The protocol Π_{zc} that proves a batch combines three known techniques: the zerocheck by sumcheck [Set20, CBBZ23], Gruen's univariate skip [Gru24, §5.1, §6], and the masking of sumcheck messages by pads in the oracle input [Fs24, §2.3]. The skip domain (Definition D.2) consists of integers in odd characteristic, as Dao et al. take in large prime fields [DDB+26, §7], and of bytes of $\mathbb{F}_{2^8}$ in characteristic two.

To make the zerocheck zero knowledge, the oracle input holds a phase-0 pad for each element the prover sends, and the prover sends each element plus its pad. The prover sends a skip message, the coefficients of each sumcheck round and one terminal claim for each of the n_{op} operands. The skip pads, the round pads and the terminal pads $P^{\mathrm{t}}_0, \ldots, P^{\mathrm{t}}_{n_{\mathrm{op}}-1}$ mask these three kinds of elements. The checks of the sumcheck rounds are linear in the skip and round pads. The verifier's final check evaluates each Q_i at the sent operand values minus their terminal pads, so this check has degree D in the terminal pads. For each multiset of two or more operand indices contained in a monomial of some Q_i, the oracle input also holds a *product element*, the product of their terminal pads as one element of K. Each product element is the product of one terminal pad and either a shorter product element or a second terminal pad, and one product constraint checks this factorization for each product element. With these products, the final check becomes a linear claim on the pads and the product elements.

The protocol Π_{zc} outputs, in the order of Lemma 5.8, the terminal constraint, the opening claim and one product constraint per product element. The terminal constraint is the final check, and the opening claim is a linear claim on the stored values and the terminal pads. For a batch that reads bits packed into elements of $\mathbb{F}_{2^{16}}$, the opening claim has one coefficient for each bit, and ring switching (Section 5.5) would compute the twisted coefficients of each packed element (Section F.1) from the sixteen coefficients of its bits. A masked sumcheck over the positions of the bits within each packed element (Protocol D.7) reduces the opening claim to a claim in which the coefficient of each

bit is the product of one value of its packed element and one value of its position. Ring switching then computes the twisted coefficients of each packed element from one value of the element and sixteen values shared by all packed elements. Ring switching combines the opening claim, or the output claim of the bit sumcheck, with the other linear claims on the same values in its own random combination. Section D.1 constructs a batched predicate from each batch $\mathcal{B}_j$ of $\mathcal{F}$, and $\Pi_{\mathrm{zc}}(\mathcal{B}_j)$ is Π_{zc} of Lemma 5.8 on that predicate. Section 7.3 describes the batch of SHA-256.

In Protocol D.6, the verifier first sends a merging challenge $\varrho \in K$ and a point $\xi \in K^m$. The merging challenge combines the n_Q polynomials. The verifier then sends a skip challenge $\tau \in K$, a challenge $\nu_i \in K$ in each of the m sumcheck rounds and an opening combination $\varphi \in K^{n_{\mathrm{op}}}$.

Lemma 5.8. *Let K be a finite field and $\mathcal{B}$ a batched predicate over K (Definition D.1) with polynomials $Q_0, \ldots, Q_{n_Q-1}$ of total degree at most D with $Q_i(0) = 0$, skip width k, m index variables and n_{op} operands, and a skip domain of $D(k-1)+1$ points (Definition D.2). Let $m_{\mathcal{B}} = \ell_{\mathrm{bit}}$ when the values Z that $\mathcal{B}$ reads are bits packed into elements of $\mathbb{F}_{2^{16}}$, with $2^{\ell_{\mathrm{bit}}}$ bits in each element, and $m_{\mathcal{B}} = 0$ otherwise. Let $\Pi_{\mathrm{zc}}(\mathcal{B})$ be Protocol D.6 on $\mathcal{B}$, followed by Protocol D.7 when $m_{\mathcal{B}} > 0$, with its skip, round, terminal and bit-sumcheck pads and its product elements P_S among the elements of the oracle input in K. Then:*

1. *$\Pi_{\mathrm{zc}}(\mathcal{B})$ is a reduction (Definition 2.23) from $\mathcal{R}_{\mathrm{batch}}$ at $\mathcal{B}$ to $\mathcal{R}_{\mathrm{lin}} \wedge \mathcal{R}^{\mathsf{st}}_{\mathrm{lin}} \wedge \mathcal{R}^{*}_{\mathrm{prod}}$ (Definitions 2.25 and 4.6), at the terminal constraint, one linear claim on the stored values and the product constraints $P_S = P_{S'} \cdot P^{\mathrm{t}}_{j_{s-1}}$ of Protocol D.6. The linear claim on the stored values is the opening claim when $m_{\mathcal{B}} = 0$ and the output claim of Protocol D.7 otherwise. The reduction has round-by-round soundness errors $\varepsilon_{\mathrm{zc}}(\mathcal{B}, \cdot)$, which are $(n_Q - 1 + m)/|K|$ at (ϱ, ξ), $D(k-1)/|K|$ at τ, $(D+1)/|K|$ at each ν_i, $1/|K|$ at φ, and $2/|K|$ at each of the $m_{\mathcal{B}}$ challenges ν_i^{bit} of Protocol D.7 (Lemma D.8). The sum of these errors is*

$$\varepsilon^{\Sigma}_{\mathrm{zc}} = \frac{m(D+2) + D(k-1) + n_Q + 2m_{\mathcal{B}}}{|K|}.$$

2. *$\Pi_{\mathrm{zc}}(\mathcal{B})$ is complete on the oracle inputs at which every product element P_S equals $\prod_{j \in S} P^{\mathrm{t}}_j$.*
3. *When the prover draws the pads independently and uniformly from K and sets each $P_S = \prod_{j \in S} P^{\mathrm{t}}_j$, $\Pi_{\mathrm{zc}}(\mathcal{B})$ is honest-verifier zero knowledge with error 0 with respect to the uniform simulator (Definition 2.24), whose prover messages are its $(D-1)(k-1) + mD + n_{\mathrm{op}} + 2m_{\mathcal{B}}$ sent elements of K.*

Appendix D gives the protocols and the source of each error term, and the formalization proves the lemma (Section 8).

5.4.3 The GKR circuit

The proof system builds at most one layered circuit with assertions C_K per field K [Fs26, §§6.3.1–6.3.3] (Definition E.1). A field has this circuit if at least one of its equalities is assigned to the GKR circuit ($\mathsf{sub}(\psi) = \mathsf{gkr}$) or if the field has a lookup. The outputs and assertions of C_K are the equalities of K assigned to the GKR circuit and the LogUp identities of the groups of K's lookups (Section 5.2). Its input layout lay_K maps each input wire to a phase-0 stored value, to 0 or to a public value such as a challenge or a masked value. The circuit computes each late value v as its masked value minus its pad (Section 4.1.5).

GordieHowe proves C_K as Longfellow proves its circuits, with zero-knowledge GKR (Section 2.7). The prover masks each value it sends with its own pad. The protocol Π_{GKR} outputs, for each layer,

a linear claim on the pads of that layer and the previous one and on the layer's product element, a product constraint on its claim pads, and one linear claim on the stored values and the claim pads of the last layer. Ring switching reduces the claim on the stored values to claims on the elements of the oracle input (Section 5.5), and Ligero proves the other claims.

The circuit computes the pub values that depend on challenges as wires, the grand products of the permutation checks of Section 3.3 as product trees [Tha13, Set20], and the sums of the LogUp identities as trees of fractions, as in LogUp-GKR [PH23, §3.1, p. 3].

Lemma 5.9. *Let C be a layered circuit with assertions over K (Definition E.1) with n_{lay} layers of gates, wire layers $0, \ldots, n_{\mathrm{lay}}$, and m_j variables in wire layer j, lay an input layout and pv an assignment of the public values it reads. Let $\mathcal{R}_{\mathrm{GKR}}$ be the relation on oracle inputs that contains $((C, \mathsf{lay}, \mathsf{pv}), \boldsymbol{U})$ if and only if C is satisfied on the input In that lay lays out from pv and the stored values of $\boldsymbol{U}$.*

1. *Protocol E.2 is a reduction (Definition 2.23) from $\mathcal{R}_{\mathrm{GKR}}$ at $(C, \mathsf{lay}, \mathsf{pv})$ to $\mathcal{R}^{*}_{\mathrm{lin}} \wedge \mathcal{R}^{\mathsf{st}}_{\mathrm{lin}} \wedge \mathcal{R}^{*}_{\mathrm{prod}}$ (Definitions 2.25 and 4.6), at its layer checks, its input claim and its product constraints $P^{\mathrm{c}}_{j,0} P^{\mathrm{c}}_{j,1} = P^{\mathrm{c}}_{j,\times}$, with round-by-round soundness errors $\varepsilon_{\mathrm{GKR}}(K, \cdot)$, which are $2m_0/|K|$ at $\nu^{(0)}$, $2/|K|$ at (ζ_j, σ_j) for each layer j, $3/|K|$ at $\nu_{j,k}$ for even k, $2/|K|$ at $\nu_{j,k}$ for odd k, and $1/|K|$ at ζ_{in}. The sum of these errors is*

$$\varepsilon^{\Sigma}_{\mathrm{GKR}} = \frac{2m_0 + 2n_{\mathrm{lay}} + 5\sum_{j=0}^{n_{\mathrm{lay}}-1} m_{j+1} + 1}{|K|}.$$

2. *It is complete on the oracle inputs at which $P^{\mathrm{c}}_{j,\times} = P^{\mathrm{c}}_{j,0} P^{\mathrm{c}}_{j,1}$ for every layer j.*
3. *When the prover draws the round and claim pads independently and uniformly from K and sets each $P^{\mathrm{c}}_{j,\times} = P^{\mathrm{c}}_{j,0} P^{\mathrm{c}}_{j,1}$, it is honest-verifier zero knowledge with error 0 with respect to the uniform simulator (Definition 2.24), whose prover messages are its $\sum_{j=0}^{n_{\mathrm{lay}}-1}(4m_{j+1} + 2)$ sent elements of K.*

Each sumcheck round of a layer has error $2/|K|$, the error of a sumcheck round with a polynomial of degree 2. The challenge $(\zeta_{j+1}, \sigma_{j+1})$ detects a nonzero assertion table of layer $j+1$ unless its multilinear extension vanishes at the left point $\nu^{(j+1,0)}$. This vanishing adds $1/|K|$ to the error of each round at an even k. These rounds bind the coordinates of the left input. The input layer has no assertion table, so at the last layer this term is an upper bound. The term $2m_0/|K|$ at $\nu^{(0)}$ covers V_0 and A_0, with $m_0/|K|$ each. Appendix E gives the protocol, and the formalization proves the lemma (Section 8).

5.5 Zero-knowledge ring switching

Ring switching along the packing tree reduces linear claims on stored values to linear claims on committed elements. Its messages, the twisted sums (Section 5.5.1), are linear functions of the committed values, so ring switching by itself reveals these functions of the committed values. We make it zero knowledge as we do the zerocheck and GKR (Sections 5.4.2 and 5.4.3): the prover masks each message with a committed mask, and Ligero proves the verifier's check on the masked messages. Because the check applies a power of the Frobenius map to each message, the masks need a special form (Section 5.5.2).

5.5.1 Ring switching

Let $B \subseteq K$ be finite fields with $d = [K : B]$, and fix a basis of K over B and its dual basis. A vector $U \in K^m$ packs the values $\mathsf{unpack}_{K/B}(U) \in B^{dm}$ (Definition 2.6). The map $\mathsf{unpack}_{K/B}$ is B-linear but not K-linear, so Ligero's linear test cannot check a linear claim $\langle a, \mathsf{unpack}_{K/B}(U)\rangle = e$ directly. Ring switching [DP26, §1.3] reduces evaluation claims on the multilinear extension of packed values to claims on the packed elements, and the form that we use reduces such a claim to claims on U. We use the form of ring switching from Hashcaster [Sou24], which rewrites the claim with the twist identity (Lemma 2.9). Diamond and Posen show that this form is an instantiation of ring switching [DP26, §1.3]. Write U_h for the entries of U and $a_h \in K^d$ for the part of a on $\mathsf{unpack}(U_h)$. Then

$$\langle a, \mathsf{unpack}(U)\rangle = \sum_{k<d} \mathrm{Frob}_B^k\Big(\sum_h f_k(a_h)\, U_h\Big).$$

We call the inner sums $\sum_h f_k(a_h)\, U_h$ the *twisted sums*. The prover sends the d twisted sums. The verifier applies Frob_B^k to the k-th sum and checks the sum of the results against e. Each claim that a sent value equals its twisted sum is linear in U, so Ligero's linear test proves it with the other linear claims of the field. The same rewriting applies to an edge $B \subset T$ of the packing tree with $T \neq K$ and $d = [T : B]$. The elements $U_h \in T$ are then entries of $\mathsf{unpack}_{K/T}$ of the committed vector (Lemma 2.7), and the twisted-sum claims of the edge are linear claims on the values at T. The step of the edge above T reduces these claims in turn (Section 5.5.2).

5.5.2 Subfield masks and squaring-chain masks

To make ring switching zero knowledge, the prover adds a committed mask R_k to the k-th twisted sum. The verifier's check then contains $\mathrm{Frob}_B^k(R_k) = R_k^{|B|^k}$, a power of degree $|B|^k$ in the mask. The proof system checks committed values only through linear claims, products of two and membership in a subfield. Two kinds of mask keep this check within those forms. Subfield masks make the check linear in entries of a subfield S_0 of K, defined below, and the subfield test checks that these entries lie in S_0. Squaring-chain masks reduce the check to products of two.

For a field K whose Ligero code is $\mathrm{RS}[K, \mathcal{D}, k_{\mathrm{RS}}]$ (Definition 2.10), the *evaluation subfield* S_0 is the smallest subfield of K that contains $\mathcal{D}$. Ligero places the entries and blinding values of each row at interpolation points in S_0 and evaluates the resulting polynomial on $\mathcal{D} \subseteq S_0$ (Protocol G.7). A row whose entries and blinding values lie in a subfield $E \supseteq S_0$ therefore has its codeword in $E^{\mathcal{D}}$. Longfellow uses this to store the values of E in subfield rows (Section 2.7).

Subfield masks, when $S_0 \subseteq B$. The prover packs each mask from $d_0 = [K : S_0]$ entries of S_0. Since Frob_B fixes S_0, the image $\mathrm{Frob}_B^k(R_k)$ is linear in these entries, and uniform entries give a uniform mask. The prover commits the entries in S_0-subfield rows. GordieHowe commits program values in packed form, so the rows of mask entries are its only subfield rows.

The verifier must also check that the entries lie in S_0. In the *subfield test*, our verifier checks that every queried entry of an S_0-subfield row lies in S_0. Longfellow's verifier does not perform this check [Fs26, §4.5 and §7.4]. A row given by a polynomial of degree below k_{RS} with a coefficient outside S_0 has entries in S_0 on at most $k_{\mathrm{RS}} - 1$ columns, so the test rejects such a row except with probability $(k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}/(n_{\mathrm{RS}})_{n_{\mathrm{col}}}$ (Lemma 5.10).

In Lemma 5.10, the rows v_i are the S_0-subfield rows and $\mathcal{D}'$ is a set of columns on which every row agrees with a polynomial of degree below k_{RS}.

Lemma 5.10. *Let $S_0 \subseteq K$ be finite fields, $\mathcal{D} \subseteq S_0$ with $|\mathcal{D}| = n_{\mathrm{RS}}$, and $1 \le n_{\mathrm{col}} \le k_{\mathrm{RS}} \le n_{\mathrm{RS}}$. For all rows $v_0, \dots, v_{r-1} \in K^{\mathcal{D}}$ with $r \ge 1$, every $\mathcal{D}' \subseteq \mathcal{D}$ and all polynomials $p_0, \dots, p_{r-1} \in K[X]$ of degree below k_{RS} with $v_i(x) = p_i(x)$ for every $x \in \mathcal{D}'$ and every i, and with some $p_i \notin S_0[X]$: for*

	subfield masks, $\Pi^{\text{sub}}_{\text{RS}}(B,T)$	squaring-chain masks, $\Pi^{\text{sq}}_{\text{RS}}(B,T)$
condition	$S_0 \subseteq B$	characteristic 2, $S_0 \not\subseteq B$
committed elements added	$d\,[K : S_0]$ entries of S_0	d masks, $\nu\, d(d-1)/2$ squares
checks added	membership of each entry in S_0	$\nu\, d(d-1)/2$ product constraints

Table 1: The two ring-switching protocols, for an edge $B \subset T$ of the packing tree, with $d = [T : B]$ and $\nu = [B : \mathbb{F}_2]$.

n_{col} distinct columns drawn uniformly from $\mathcal{D}$, the probability that every drawn column lies in $\mathcal{D}'$ and every entry of every row at the drawn columns lies in S_0 is at most $(k_{\text{RS}}-1)_{n_{\text{col}}}/(n_{\text{RS}})_{n_{\text{col}}}$.

The proof is in Section G.4.

Hash functions such as SHA-256 and Keccak compute on bits, so in characteristic 2 the programs hold values in subfields smaller than S_0, down to $\mathbb{F}_2$. Subfield rows cannot hold these values. The evaluation points $\mathcal{D}$ lie in S_0, so the codeword of a row with entries in a subfield $B \subset S_0$ has entries in S_0 that need not lie in B. A subfield test for B would therefore reject honest rows. Even for values of S_0, each entry of a subfield row holds one value, so a tableau of subfield rows has as many rows as a tableau that stores each value as one element of K. The prover encodes and hashes every row, the tests combine every row, and the verifier's work at each queried column is linear in the number of rows.

GordieHowe instead packs $[T : B]$ values of every subfield B into one element of its parent T, along the packing tree (Section 4.2.3). Packed values lie in B by construction, so they need no subfield test.

Definition 5.11 (Admissible packing tree)**.** Let K be a field of a program, with evaluation subfield S_0. A packing tree over K is *admissible* if every edge $B \subset \mathsf{par}(B)$ has $S_0 \subseteq B$ or K has characteristic 2.

For every field of odd characteristic that we use, S_0 is the prime subfield, so every packing tree over it is admissible.

Squaring-chain masks, when B does not contain S_0. An admissible tree has such an edge only in characteristic 2 (Definition 5.11). For $B = \mathbb{F}_{2^\nu}$, $\text{Frob}^k_B(x) = x^{2^{\nu k}}$ is the result of νk squarings, so the prover commits each mask together with its chain of νk squares. Product constraints check the squares, and the image $\text{Frob}^k_B(R_k)$ is then itself a committed element. Squaring-chain masks let GordieHowe pack $[K : \mathbb{F}_2]$ bits into each committed element. In our measurements, storing each bit of H as one element of $\mathbb{F}_{2^{16}}$ makes the ML-DSA-44 proof of possession 1.45× larger than with packed bits (Section 7.8). Longfellow's bit plucker also adds gates to its GKR circuit to unpack the bits, while ring switching needs no such gates.

Table 1 compares the two ring-switching protocols. Both send d masked twisted sums, add $d+1$ linear claims, and have perfect soundness and HVZK. The number of squares, $\nu\, d(d-1)/2$, grows with the square of d, so the compiler keeps the degree d of a squaring-chain step small (Section 4.2.3). Section F.1 gives both protocols, $\Pi^{\text{sub}}_{\text{RS}}(B,T)$ and $\Pi^{\text{sq}}_{\text{RS}}(B,T)$, and their lemmas (Lemmas F.2 and F.4).

Ring switching along the packing tree, $\Pi_{\text{RS}}(\mathcal{T}_K)$, applies one ring-switching step to each edge of $\mathcal{T}_K$, from the leaves to the root. A step uses subfield masks if the child of its edge contains S_0, and squaring-chain masks otherwise (Table 1). Before a step whose child has two or more outstanding claims, the verifier sends a challenge that combines them into one claim. The protocol $\Pi_{\text{RS}}(\mathcal{T}_K)$ reduces linear claims on the stored values of K to linear claims on committed elements, product

constraints and membership claims (Theorem F.6). It is round-by-round sound with error $1/|K|$ in each round and perfectly HVZK.

5.6 Ligero

As in Longfellow (Section 2.7), GordieHowe commits the values of each field in one Ligero tableau [AHIV22]. The tableau stores a vector $x \in K^{m\ell}$ in m rows, each holding ℓ entries $x_{g,0}, \ldots, x_{g,\ell-1}$, and adds mask rows for the three tests (Definition G.5). The prover encodes the rows of the tableau with the Reed–Solomon code $\mathrm{RS}[K, \mathcal{D}, k_{\mathrm{RS}}]$ (Definition 2.10). Ligero's linear test proves the field's linear claims, its quadratic test proves its product constraints, and its low-degree test checks that the rows are close to codewords (Protocol G.7). The verifier queries n_{col} columns of the tableau, so the proof size after spBCS (Section 6) grows with n_{col}. This section changes Ligero's analysis and protocol to reduce n_{col} and repairs a flaw in the published analyses. Protocol G.7 differs from Ligero as Longfellow runs it in four places. Three changes let the analysis work at the Johnson radius, where fewer queried columns suffice (Section 5.6.2): the low-degree test moves before the constraint challenge, it is repeated n_{rep} times with its own mask row each time, and the mask test (Section 5.6.2) checks the masks of the linear and quadratic tests. The fourth is the subfield test (Section 5.5.2).

5.6.1 Errors in the published analyses of Ligero's linear and quadratic tests

Ames et al.'s protocol [AHIV22, §4.7, App. B.4–B.5] and Longfellow [Fs26, §§4.3 and 4.4.1] mask the linear test and the quadratic test with one row each, M_{lin} and M_{quad}. An honest prover commits each mask row as a polynomial of degree at most $2k_{\mathrm{RS}} - 2$. The low-degree test does not cover these rows, and neither protocol tests them separately. The mask test that we introduce in Section 5.6.2 adds this test. The published soundness analyses of these two tests, by Ames et al. [AHIV22, Lems. B.7 and B.9, App. C] and by Cascudo et al. for Longfellow [CHVY25, Thms. 4.3–4.4, pp. 17–19], assume that each mask row has low degree. Ames et al.'s Lemma B.7 states this assumption as the hypothesis that the tableau with its mask row is close to a codeword, and no test enforces it. Ames et al. state Lemma B.9 without a proof. We call the response determined by the decoded rows and the mask row the *correct response*. The bound of Lemma B.9 and the quadratic term of their Appendix C count at most $2k_{\mathrm{RS}}$ columns outside the corrupted ones on which a wrong response can agree with the correct one [AHIV22, pp. 44 and 46–47]. The quadratic term follows the argument of their linear test. This count needs the mask row to have low degree. Cascudo et al.'s Theorem 4.4 uses the bound $2k_{\mathrm{RS}} - 2$ on the degree of the mask row in its proof [CHVY25, pp. 18–19]. Under the degree assumption, a wrong response agrees with the correct one on at most $2k_{\mathrm{RS}} - 2$ columns. A cheating prover can instead commit any vector of $K^{\mathcal{D}}$ as a mask row, and for many constraint challenges a suitable mask row lets a wrong response agree with the correct one on more than $2k_{\mathrm{RS}} - 2$ columns.

We give machine-checked counterexamples to Ames et al.'s soundness lemma for the quadratic test [AHIV22, Lem. B.9] and the quadratic term of their Appendix C, to Theorem 4.4 of Cascudo et al., and to the sum of the bounds of Cascudo et al.'s Theorems 4.3 and 4.4 in Longfellow's layout with one entry per row ($\ell = 1$) and $n_{\mathrm{col}} = k_{\mathrm{RS}} - 1$ queried columns (Section G.8). Ames et al.'s headline bounds, for a number of corrupted columns below a quarter of the code's distance [AHIV22, Thm. 4.7, Lem. 4.14], still hold, but their proofs omit one step. We supply it and machine-check it (Section G.8). Cascudo et al.'s round-by-round claim for Ligero [CHVY25, pp. 14–15] bounds the column round with the degree argument of their Theorem 4.4. That argument fails when the mask row has higher degree, so the claimed bound for the column round does not follow.

Our repaired bound replaces the degree argument by the Johnson bound on response polynomials

of degree $2k_{\mathrm{RS}} - 2$. Let A_J be the Johnson agreement of this degree (Definition G.3), with $A_J \approx \sqrt{2k_{\mathrm{RS}}n_{\mathrm{RS}}}$. Outside a set of the test's challenges of density at most $L_{\mathrm{m}}/|K|$, with $L_{\mathrm{m}} \leq n_{\mathrm{RS}}$, a sent response that passes the algebraic check and differs from the correct response agrees with it on at most A_J columns (Lemma G.10). At the column query, a wrong response then passes with probability at most $(A_J)_{n_{\mathrm{col}}}/(n_{\mathrm{RS}})_{n_{\mathrm{col}}} \approx (A_J/n_{\mathrm{RS}})^{n_{\mathrm{col}}}$. We call this probability the *mask term*. At rate 1/7, the Johnson agreement lies below the unique-decoding threshold, so the mask term is negligible there. At the five tableaus of Longfellow version 7, our repaired bound gives a column error within about 0.1 bit of Cascudo et al.'s formula. For the rate-1/4 parameter sets of versions 5 and 6, our repaired bound proves $\approx 3/4$ of the bits that the published bounds give (Section G.6).

Lemma G.12 analyzes Ligero round by round in Longfellow's order of challenges. In this order, one joint challenge carries the low-degree challenge and the constraint challenges. At the joint challenge, the analysis takes a union over the tableaus of the interleaved code that agree with the committed tableau on at least A columns, and the Johnson bound limits the number of these tableaus. Section 7 evaluates this bound at the parameters of Longfellow version 7 and compares it with GordieHowe's per-round target.

5.6.2 Ligero at the Johnson radius

Within the Johnson radius (Definition 2.12), several tableaus of the interleaved code can each agree with the committed tableau on many columns. In Ligero as published and in Longfellow, the low-degree challenge is sent together with the constraint challenge [AHIV22, §4.7, step 1 of the interactive protocol, p. 25] [Fs26, §4.4.4], so the prover chooses its low-degree response w, and with it the candidate tableau, after seeing the constraint challenge. We move the low-degree test into the commit phase, so that its challenge and response fix one candidate tableau before the constraint challenge. The move simplifies the analysis, since every later round is analyzed against one fixed tableau.

The soundness analysis of Ligero in this section (Lemma 5.13) is parametrized by an *agreement threshold* A (Definition 2.12), and this threshold sets the number of queried columns. Each of the n_{rep} low-degree challenges $\chi^{(r)}$ combines the rows of the tableau with its own mask row $M^{(r)}_{\mathrm{ldt}}$. The prover sends a polynomial $w^{(r)}$ of degree below k_{RS} as this combination. The *common agreement set* $\mathcal{D}_w \subseteq \mathcal{D}$ is the set of columns at which every $w^{(r)}$ equals the combination of the rows by $\chi^{(r)}$. The verifier rejects every queried column outside $\mathcal{D}_w$. If $|\mathcal{D}_w| < A$, all n_{col} queried columns fall in $\mathcal{D}_w$ with probability at most $(A-1)_{n_{\mathrm{col}}}/(n_{\mathrm{RS}})_{n_{\mathrm{col}}} \approx (A/n_{\mathrm{RS}})^{n_{\mathrm{col}}}$. Suppose instead that $|\mathcal{D}_w| \geq A$. For each r, mutual correlated agreement (Definition 2.13) gives a set of challenges of density at most $n_{\mathrm{bad}}/(|K|-1)$. If some challenge $\chi^{(r)}$ lies outside its set, every row agrees with a polynomial of degree below k_{RS} on all of $\mathcal{D}_w$ (Theorem G.4). All n_{rep} challenges lie in their sets with probability at most $(n_{\mathrm{bad}}/(|K|-1))^{n_{\mathrm{rep}}}$, and this bound has no factor in the number of rows. The extractor then interpolates each row on $\mathcal{D}_w$.

Ames et al. and Cascudo et al. analyze Ligero's linear and quadratic tests at an agreement threshold of at least the unique-decoding threshold, $A \geq (n_{\mathrm{RS}} + k_{\mathrm{RS}})/2$ (Definition 2.11) [AHIV22, CHVY25]. Mutual correlated agreement holds at every threshold within the Johnson radius, down to $A \approx \sqrt{k_{\mathrm{RS}}n_{\mathrm{RS}}}$ [DKT26]. We apply it to Ligero, so the verifier queries fewer columns for the same error. The exceptional count n_{bad} grows as A approaches $\sqrt{k_{\mathrm{RS}}n_{\mathrm{RS}}}$ (Lemma 2.14).

Without the mask test, however, the mask term (Section 5.6.1) does not shrink with A. Once $A < A_J$, the mask term is the largest term, so the column error stays at $\approx (A_J/n_{\mathrm{RS}})^{n_{\mathrm{col}}}$ however small A is.

We therefore introduce the mask test. Write M for either mask, M_{lin} or M_{quad}, of degree at most $2k_{\mathrm{RS}} - 2$. The prover commits M as three rows M_0, M_1, M_2 of degrees below k_{RS}, below k_{RS}

and below $k_{\mathrm{RS}} - 1$, and these rows enter the low-degree test with the other rows. At each queried column x the verifier recombines

$$M(x) = M_0(x) + x^{k_{\mathrm{RS}} - n_{\mathrm{col}}} M_1(x) + x^{k_{\mathrm{RS}}} M_2(x).$$

The shifts 0, $k_{\mathrm{RS}} - n_{\mathrm{col}}$ and k_{RS} let the pieces recombine to any mask of degree at most $2k_{\mathrm{RS}} - 2$. Every piece has degree below k_{RS}, so the low-degree test covers the pieces of every mask. The mask term becomes $(2k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}} / (n_{\mathrm{RS}})_{n_{\mathrm{col}}} \approx ((2k_{\mathrm{RS}} - 1)/n_{\mathrm{RS}})^{n_{\mathrm{col}}}$. At every tableau of our applications, $2k_{\mathrm{RS}} < A$, so this term is below the proximity term $(A-1)_{n_{\mathrm{col}}} / (n_{\mathrm{RS}})_{n_{\mathrm{col}}}$ and the threshold A again sets the column error. With the mask test, the constraint challenge has error $1/|K|$ instead of $L_{\mathrm{m}}/|K|$.

The split reveals no more than the values of M at the queried columns. The honest prover draws (M_0, M_1, M_2) uniformly among the triples of degrees below k_{RS}, below k_{RS} and below $k_{\mathrm{RS}} - 1$ that recombine to M. When $2n_{\mathrm{col}} \leq k_{\mathrm{RS}}$, for every set of n_{col} distinct columns, the values of M_1 and M_2 at these columns are uniform and independent of M (Lemma G.9). The value of M_0 at each column is then determined by them and the value of M there. Two pieces of degree below k_{RS} cannot hide M this way. They have $2k_{\mathrm{RS}}$ coefficients, while a mask of degree at most $2k_{\mathrm{RS}} - 2$ has $2k_{\mathrm{RS}} - 1$, so given M the pieces have only one free coefficient. Their values at n_{col} columns then lie in an affine space of dimension at most 1, and for $n_{\mathrm{col}} \geq 2$ they cannot be uniform. The three pieces of the split have $3k_{\mathrm{RS}} - 1$ coefficients, which leaves k_{RS} free given M. That is enough to make the values of M_1 and M_2 at n_{col} columns uniform when $2n_{\mathrm{col}} \leq k_{\mathrm{RS}}$. The split follows the randomized split of PLONK's quotient [GWC19, Round 3] and the split of Haböck and Al Kindi for n_{col} queries [HK24, §4.1, Lem. 4].

Definition 5.12 (Ligero's round errors)**.** Let S_0, K, $\mathcal{D}$, k_{RS}, ℓ, n_{col} and n_{rep} be parameters of Protocol G.7, and let A be an agreement threshold of $\mathrm{RS}[K, \mathcal{D}, k_{\mathrm{RS}}]$ at which the code has full agreement-set line MCA with exceptional count n_{bad} (Definition 2.13). The *round errors* of Protocol G.7 are

$$\varepsilon_{\mathrm{com,ldt}} = \Big(\frac{n_{\mathrm{bad}}}{|K| - 1}\Big)^{n_{\mathrm{rep}}}, \qquad \varepsilon_{\mathrm{prf,con}} = \frac{1}{|K|}, \qquad \varepsilon_{\mathrm{prf,col}} = \frac{(A-1)_{n_{\mathrm{col}}} + (2k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}} + (k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}}{(n_{\mathrm{RS}})_{n_{\mathrm{col}}}},$$

at the low-degree challenges, the constraint challenge and the column query. For a tableau without the mask test (Section 5.6.2), $\varepsilon_{\mathrm{com,ldt}}$ is the same, and

$$\varepsilon_{\mathrm{prf,con}} = \frac{L_{\mathrm{m}}}{|K|}, \qquad \varepsilon_{\mathrm{prf,col}} = \frac{(A-1)_{n_{\mathrm{col}}} + (A_J)_{n_{\mathrm{col}}} + (k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}}{(n_{\mathrm{RS}})_{n_{\mathrm{col}}}},$$

for A_J and L_{m} as in Lemma G.10 (Corollary G.11). In both cases, the term $(k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}$ of $\varepsilon_{\mathrm{prf,col}}$ is present only when the tableau has S_0-subfield rows.

The column error sums three terms, one for each case of the column query. The term $(A-1)_{n_{\mathrm{col}}}$ covers proximity: if $|\mathcal{D}_w| < A$, every accepted tuple of queried columns lies in $\mathcal{D}_w$. The term $(2k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}$ covers the mask test. The low-degree test checks M_2 at degree bound k_{RS}, so a cheating prover's recombined mask has degree at most $2k_{\mathrm{RS}} - 1$. A sent response that differs from the correct response then agrees with it on at most $2k_{\mathrm{RS}} - 1$ columns. The term $(k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}$ covers the subfield test: an S_0-subfield row whose polynomial on $\mathcal{D}_w$ has a coefficient outside S_0 has entries in S_0 on at most $k_{\mathrm{RS}} - 1$ columns of $\mathcal{D}_w$ (Lemma 5.10).

Lemma 5.13 (Security of Ligero with the mask test)**.** *Let S_0, K, $\mathcal{D}$, k_{RS}, ℓ, n_{col}, A and n_{bad} be as in Definition 5.12, with $2k_{\mathrm{RS}} < n_{\mathrm{RS}}$, and let $\varepsilon_{\mathrm{com,ldt}}$, $\varepsilon_{\mathrm{prf,con}}$ and $\varepsilon_{\mathrm{prf,col}}$ be the round errors there*

for a tableau with the mask test (Corollary G.11 gives the case without it). A common input cl *of Protocol G.7 is a statement of* $\mathcal{R}^*_{\text{lin}} \wedge \mathcal{R}^*_{\text{prod}} \wedge \mathcal{R}^*_{\text{mem}}$ *on* x *(Definition 2.25). In this statement, a row triple* (g, g', g'') *stands for the product constraints* $x_{g,s}\, x_{g',s} = x_{g'',s}$ *for* $s \in [\ell]$*, and an* S_0*-subfield row stands for the membership claims of its entries. The list* cl holds at x *if* $(\mathsf{cl}, x) \in \mathcal{R}^*_{\text{lin}} \wedge \mathcal{R}^*_{\text{prod}} \wedge \mathcal{R}^*_{\text{mem}}$.

1. *There is a polynomial-time map* Ext*, from a tableau* $\widehat{U} \in K^{(m+6+n_{\text{rep}})\times\mathcal{D}}$ *and the low-degree challenges and responses to* $K^{m\ell}$*, with the following property. For every* $\widehat{U}$ *there is a set* $\mathcal{Z}_{\widehat{U}}$ *of tuples of low-degree challenges of density at most* $\varepsilon_{\text{com,ldt}}$ *such that, for every tuple* $\chi \notin \mathcal{Z}_{\widehat{U}}$ *and all responses* w*, the proof phase of Protocol G.7 is a reduction (Definition 2.23) whose conditions (i) and (ii) of Definition 2.24(a) hold at the oracle input* $\widehat{U}$ *for every statement. It reduces the relation* $\mathcal{R}_{\text{Lig}}$ *of pairs* $(\mathsf{cl}, \widehat{U})$ *such that* cl *holds at* $\mathsf{Ext}(\widehat{U}, \chi, w)$ *to the relation* $\mathcal{R}_{\text{col}}$ *of pairs of an output statement and* $\widehat{U}$ *at which the verifier accepts: the responses pass the algebraic check of the linear test, and the column checks hold at the queried columns of* $\widehat{U}$*. The reduction has round-by-round soundness errors* $\varepsilon_{\text{prf,con}}$ *at the constraint challenge and* $\varepsilon_{\text{prf,col}}$ *at the column query.*
2. *For every statement* cl *and every* x *at which* cl *holds, the verifier accepts the honest prover on the honest encoding of* x *with probability* 1*.*
3. *If* $n_{\text{col}} \le k_{\text{RS}} - \ell$ *and* $2n_{\text{col}} \le k_{\text{RS}}$*, Protocol G.7 has honest-verifier zero-knowledge error* 0 *(Definition 2.22) with respect to the simulator that draws the challenges uniformly and runs Protocol G.8 on them.*

The soundness error is at most $\varepsilon_{\text{com,ldt}} + \varepsilon_{\text{prf,con}} + \varepsilon_{\text{prf,col}}$, and these are Ligero's round errors in Theorem 5.3(2). The proof is in Section G.5.

5.7 Parameter selection

A *parameter set* (Table 3) fixes the fields of the program, the hash function, the lengths of its digests, streams and salts, a per-round error target ε^*, and the rates of the Reed–Solomon codes of H and A. The lengths bound the error terms of spBCS (Theorem 6.6). The rate sets the trade-off between proof size and prover time. At a lower rate, fewer columns are opened and the proof is smaller, while the prover encodes and hashes a larger tableau. We set the rate of each field, including the native field of ML-DSA, from the summed prover and verifier times of our applications over that field, at code rates from 1/3 to 1/24. Each field uses the lowest rate whose summed time is within 5% of that of its fastest rate, so the rate gives up a little time for a smaller proof.

The compiler makes its other choices first and aims to minimize the sum of prover and verifier time estimated by its cost model (Section 4.2). The realized program, with the proof system's choices for its batches (Appendix D), then fixes every round error of $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ other than Ligero's. These are the errors of the source program, of LogUp, of the MAC keys, of the zerochecks, of GKR and of ring switching (Theorem 5.3).

As Section 4.2.4 outlines, for each field K the compiler chooses the dimensions $(n_{\text{RS}}, k_{\text{RS}})$ of the tableau, the number n_{rep} of low-degree tests, the agreement threshold A, the number n_{col} of queried columns, and whether to run the mask test (Section 5.6.2). Every choice gives each Ligero round, the column query included, an error of at most ε^* by itself, with the exception described next. At the ROM parameters, when a round of a field other than Ligero's has an error above ε^*, the compiler replaces ε^* for that field by the largest such error. Among our applications, only A of the ML-DSA mDL has such a round, the LogUp challenge of its range lookups, with error $2^{-112.7}$ (Section 7.5.2). Every choice satisfies $2k_{\text{RS}} < n_{\text{RS}}$ and $n_{\text{col}} \le k_{\text{RS}} - \ell$ (Lemma 5.13), and $2n_{\text{col}} \le k_{\text{RS}}$ when the compiler runs the mask test (Lemma 5.13(3)). For each candidate choice of dimensions,

					queried columns	
K	$(n_{\mathrm{RS}}, k_{\mathrm{RS}})$	ε^*	n_{rep}	$\lvert K\rvert\,(\varepsilon^*)^{1/n_{\mathrm{rep}}}$	w/ mask test	w/o mask test
$\mathbb{F}_{2^{128}}$	$(3392, 282)$	$2^{-113.5}$	1	$2^{14.5}$	107	107
$\mathbb{F}_{2^{128}}$	$(3392, 282)$	$2^{-113.5}$	2	$2^{71.25}$	63	86
$\mathbb{F}_{p_{256}}$	$(1367, 341)$	$2^{-113.5}$	1	$2^{142.5}$	108	205
$\mathbb{F}_{2^{256}}$	$(4069, 339)$	2^{-236}	1	2^{20}	149	176
$\mathbb{F}_{2^{256}}$	$(4069, 339)$	2^{-236}	2	2^{138}	128	176

Table 2: Queried columns with and without the mask test. Each row fixes the dimensions $(n_{\mathrm{RS}}, k_{\mathrm{RS}})$ of a tableau that the compiler chose in our measurements (Table 7). The first two rows use the tableau of the hashing field $\mathbb{F}_{2^{128}}$ in the ML-DSA-44 proof of possession at the ROM parameters, and the last two rows use the tableau of the hashing field $\mathbb{F}_{2^{256}}$ in the same proof at the QROM parameters, both at rate 1/12. The middle row uses the tableau of the algebra field $\mathbb{F}_{p_{256}}$ in the ECDSA mDL at the ROM parameters, at rate 1/4. Three low-degree tests give the same counts as two.

n_{rep} and masking (the mask test or untested masks, Corollary G.11), the compiler first finds the least threshold whose exceptional count n_{bad}, computed as below, makes the error $(n_{\mathrm{bad}}/(|K|-1))^{n_{\mathrm{rep}}}$ of the low-degree tests at most ε^* (Section 5.6.2). The compiler then sets n_{col} to the least number of columns at which the column query at this threshold has error at most ε^*. At this n_{col}, it takes the threshold A that minimizes the larger of the low-degree error and the column error, among the thresholds from the least one up to the largest at which the column error stays at most ε^*. Among the candidates, it takes the one with the smallest estimated proof size. It prefers the mask test on a tie, and it takes $n_{\mathrm{rep}} > 1$ only when the estimated proof is more than 5% smaller than with $n_{\mathrm{rep}} = 1$.

Each queried column adds its entry in every row and a Merkle path to the proof, so the queried columns make up most of the proof. The column error is $\approx (A/n_{\mathrm{RS}})^{n_{\mathrm{col}}}$, so a smaller threshold A means fewer columns. Two limits keep A from falling. First, without the mask test, the untested mask rows keep the column error above $\approx (A_J/n_{\mathrm{RS}})^{n_{\mathrm{col}}}$, where $A_J \approx \sqrt{2k_{\mathrm{RS}}n_{\mathrm{RS}}}$ (Definition G.3 and Lemma G.10). Second, the exceptional count n_{bad} grows as A falls (Lemma 2.15), and the low-degree tests allow a count of at most $\approx |K|\,(\varepsilon^*)^{1/n_{\mathrm{rep}}}$. At our tableaus, n_{bad} at $A = A_J$ is between about 2^{16} and 2^{18} at rate 1/12, between $2^{14.5}$ and $2^{16.5}$ at rates 1/6 and 1/9, and between 2^{10} and $2^{12.5}$ at rates 1/4 and 1/5. Near $A = \sqrt{k_{\mathrm{RS}}n_{\mathrm{RS}}}$, it is between 2^{33} and 2^{43}. The mask test therefore saves columns only once $|K|\,(\varepsilon^*)^{1/n_{\mathrm{rep}}}$ exceeds n_{bad} at $A = A_J$, which is about 2^{16} for the tableaus of the hashing fields. A second low-degree test raises this bound from $|K|\,\varepsilon^*$ to $|K|\,(\varepsilon^*)^{1/2}$, so the second test makes the proof smaller when $|K|\,\varepsilon^*$ is small. Once the bound reaches n_{bad} at $A \approx \sqrt{k_{\mathrm{RS}}n_{\mathrm{RS}}}$, A falls to $\approx \sqrt{k_{\mathrm{RS}}n_{\mathrm{RS}}}$, and the mask test saves a fraction of $\approx 1/\log_2(n_{\mathrm{RS}}/k_{\mathrm{RS}})$ of the columns, 28% at rate 1/12 and half at rate 1/4.

Table 2 gives the queried columns at three tableaus of our measurements, which we estimate with the compiler's parameter search. Over $\mathbb{F}_{2^{128}}$ at the ROM target, $|K|\,\varepsilon^* = 2^{14.5}$, so with one low-degree test the mask test saves no columns. With two tests, the mask test cuts the queried columns from 86 to 63. Over $\mathbb{F}_{p_{256}}$, one test suffices, and the mask test cuts the queried columns from 205 to 108. Over $\mathbb{F}_{2^{256}}$ at the QROM target, $|K|\,\varepsilon^* = 2^{20}$, so the mask test already saves columns with one test, and a second test saves 21 more columns, so the compiler takes $n_{\mathrm{rep}} = 2$. The mask test commits four extra rows and adds no responses. Section 7.8 measures the proofs without the mask test.

In this paragraph and the next, m and B denote the parameters of the certificate of Lemma 2.15.

The compiler takes the exceptional count n_{bad} of a threshold A from this lemma, with $n = n_{\mathrm{RS}}$, $k = k_{\mathrm{RS}}$, and D and B' as in the lemma. The count $E(n, k, A; m, B, H)$ of the lemma increases with H, so the compiler evaluates Lemma 2.15 at the least height $H \geq B$ that satisfies its hypothesis. Put $N_0 = \sum_{j=0}^{B}(mA - Dj)$, $W_0 = \sum_{j=0}^{B} j(mA - Dj)$, $R_0 = \sum_{b=0}^{B'}(m - b)$ and $T_0 = \sum_{b=0}^{B'} b(m - b)$. The hypothesis of Lemma 2.15 is $(H + 1)(N_0 - nR_0) > W_0 - nT_0$. When $nR_0 < N_0$, the least height $H \geq B$ that satisfies it is $\max(B, \lfloor (W_0 - nT_0)/(N_0 - nR_0) \rfloor)$.

At an agreement threshold A, the compiler records the least $\lceil E \rceil$ over $1 \leq m \leq 120$ and $1 \leq B \leq \min(250, 2D + 1, \lfloor (mA - 1)/D \rfloor)$. When A is at least the unique-decoding threshold, the minimum also includes the count n of Lemma 2.14(2). For example, at the hash tableau of Longfellow version 7 ($n = 3230$, $k = 461$) and $A = 1810$, the least count $E = 3041297/135 \approx 22528.1$ is at $m = 1$, $B = 2$ and $H = 3$, and the compiler records 22529. The closed form of Lemma 2.14(1) at the same point is $\approx 2^{21.3}$.

6 Series–parallel BCS

GordieHowe makes $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ non-interactive with an extension of the BCS transformation [BCS16] to series–parallel IOPs (Definition 2.18). We call this extension *series–parallel BCS* (spBCS). The prover commits each oracle string with a Merkle tree and derives each verifier challenge by hashing the statement, a proof salt and the messages that precede the challenge. Theorem 6.6 shows that spBCS preserves the guarantees of a series–parallel IOP, with concrete errors in the QROM and in the ROM. The resulting argument is complete and knowledge sound. It is everlasting zero knowledge for many adaptively chosen proofs, against a distinguisher that receives the whole oracle after the last proof. It is also weakly online simulation extractable and unconditionally witness indistinguishable.

Zero knowledge for many proofs and everlasting privacy matter for credentials, because a credential is presented many times [CL01] and must stay private after the hash function is broken. Simulation extractability matters for signatures built from a proof of knowledge of the secret key, because the unforgeability reduction for such a signature answers signing queries with simulated proofs and extracts a witness from the forgery [Unr17, Thm. 31, p. 40].

6.1 Security notions

Appendix H recalls non-interactive arguments in the random oracle model (ROM) and the quantum random oracle model (QROM) (Definition H.1), with their completeness, adaptive soundness, adaptive knowledge soundness and zero knowledge. For the properties that bound the adversary, we use one definition for both models: the adversary is bounded only in the number of its oracle calls, and in the ROM it is classical.

Definition 6.1 (Query-bounded quantum adversaries)**.** Let $t, Q \in \mathbb{N}$. A (t, Q)-*adversary* A has access to the random oracle H (Definition H.1) and, where the experiment provides one, to a second oracle that maps a classical *proof query* to a classical answer. We call its queries to H *hash queries*. It consists of a Hilbert space $\mathcal{H}$ of its registers, among them a classical register C, an initial density operator ρ on $\mathcal{H}$, and channels $\Phi_1, \Phi_2, \ldots$ on $\mathcal{H}$ [Wat18, Def. 2.13, p. 73]. It runs as follows.

1. Starting from ρ, it applies Φ_1, then reads C and, according to its value, either makes a hash query on its query registers, makes a proof query from a classical register, or halts. If it has not halted, it applies Φ_2, reads C again, and so on.
2. For every sequence of values of C and of answers to its proof queries, it makes at most t hash queries and at most Q proof queries, and halts. We call such a sequence an *execution path*.

3. When it halts, it measures its output register in the computational basis, giving its classical output, and the rest of its registers form its auxiliary output Z.

In the QROM a hash query applies $|u, y\rangle \mapsto |u, y \oplus \mathsf{H}(u)\rangle$ to the query registers. In the ROM, a (t, Q)-adversary is a classical probabilistic oracle algorithm with coins sampled independently of H, the bounds of item 2 for every choice of its coins, and a classical output and an auxiliary output as in item 3.

The class of Definition 6.1 contains the oracle algorithms of Unruh [Unr17, §3] and the t-query algorithms of Chiesa, Manohar and Spooner [CMS19, §3.2]. It contains Unruh's class because an algorithm that runs in time T makes at most T calls. Our class differs from Unruh's in bounding hash queries and proof queries rather than running time. It differs from Chiesa, Manohar and Spooner's in two ways. The initial state may be mixed. The adversary may also measure between calls, and the outcomes may decide the next call. As in the extractability definitions of Unruh [Unr17, Defs. 13–14, pp. 19–21] and the model of Chiesa, Manohar and Spooner [CMS19, §3.2], the initial state ρ is chosen before the oracle is sampled, so it is independent of the oracle. The bound is a worst-case bound: it holds for every outcome of the adversary's measurements.

6.1.1 Privacy

Proofs may be published and examined long after they are made, by an adversary that can by then evaluate the hash function as often as it likes. Zero knowledge against an adversary with no bound on its hash queries before it receives a proof is impossible in the random oracle model unless the language is easy to decide. Chiesa and Yogev show that a simulator that withstands an unbounded number of hash queries yields a decider for the language, with about the running time of the simulator and the verifier [CY26, Lems. 6.6.1 and 6.6.3, pp. 67–70]. Two weaker notions remain achievable. The first keeps zero knowledge (Definition H.6) but bounds the adversary's hash queries until it has received its proofs. Ben-Sasson, Chiesa and Spooner define zero knowledge in this form for one proof, with a distinguisher that receives the whole oracle after the proof [BCS16, §2.3, p. 13]. We extend their definition to adaptively chosen proof queries. The second notion, witness indistinguishability [FS90, Def. 3.1, p. 418] with no bound on the hash queries, asks only that a proof hide which of two witnesses was used.

Definition 6.2 (Everlasting zero knowledge)**.** The argument $(\mathsf{Prv}, \mathsf{Vrf})$ is *everlasting zero knowledge* with respect to Sim, with error $\varepsilon_{\mathrm{zk}}$, a function from $\mathbb{N}^2$ to the nonnegative real numbers, if Sim is as in Definition H.6 and, for all $t, Q \in \mathbb{N}$, the bound of Definition H.6 holds with error $\varepsilon_{\mathrm{zk}}(t, Q)$ for every adversary as in Definition 6.1, except that on every execution path it makes at most Q proof queries, at most t hash queries before its last proof query, and an unbounded number of hash queries after it.

This property follows the everlasting security of Unruh [Unr13] and the long-term security of Müller-Quade and Unruh [MU10, §1.1], which protect against adversaries that are bounded during the protocol and unbounded after it.

Definition 6.3 (Unconditional witness indistinguishability)**.** Let $(\mathsf{Prv}, \mathsf{Vrf})$ be a non-interactive argument for $\mathcal{R}$ (Definition H.1), and fix injective binary encodings of statements and witnesses. Write $|\mathsf{x}|$ and $|\mathsf{w}|$ for the bit lengths of the encodings. Let $Q, L \in \mathbb{N}$. An adversary A chooses, for $j = 0, \ldots, Q-1$ in turn, a triple $(\mathsf{x}_j, \mathsf{w}_{j,0}, \mathsf{w}_{j,1})$ with $(\mathsf{x}_j, \mathsf{w}_{j,0}) \in \mathcal{R}$, $(\mathsf{x}_j, \mathsf{w}_{j,1}) \in \mathcal{R}$ and $|\mathsf{x}_j| + |\mathsf{w}_{j,0}| + |\mathsf{w}_{j,1}| \leq L$, as a function of H and the earlier proofs. For $b \in \{0, 1\}$, let D_b be the distribution of $(\mathsf{H}, \mathsf{pf}_0, \ldots, \mathsf{pf}_{Q-1})$, where $\mathsf{H} \xleftarrow{\$} \mathcal{O}^{\mathcal{Q}}$ and $\mathsf{pf}_j \leftarrow \mathsf{Prv}^{\mathsf{H}}(\mathsf{x}_j, \mathsf{w}_{j,b})$ with fresh coins for each j. Then

$(\mathsf{Prv}, \mathsf{Vrf})$ is *unconditionally witness indistinguishable* with error $\varepsilon_{\mathrm{wi}}$, a function from $\mathbb{N}^2$ to the nonnegative real numbers, if $\mathsf{SD}(D_0, D_1) \le \varepsilon_{\mathrm{wi}}(Q, L)$ for all $Q, L \in \mathbb{N}$ and every such adversary A.

For one proof, Definition 6.3 is Feige and Shamir's non-interactive witness indistinguishability [FS90, Def. 5.2, p. 422], with the random oracle in place of their public random string, in the adaptive, statistical form of Chiesa and Yogev. In that form the adversary chooses the statement and the two witnesses after querying the oracle, and the distinguishers make any number of queries [CY26, Def. 33.1.1, pp. 426–427, and p. 430]. Chiesa and Yogev bound the length of the statement. Definition 6.3 instead bounds the total length L of the statement and both witnesses, and its error depends on L. The adversary's choice may depend on the whole oracle, so the definition covers an adversary that makes any number of hash queries before each choice. The definition applies in both the ROM and the QROM, because D_0 and D_1 include the whole oracle. The bound covers randomized and quantum adversaries, because each is an average of deterministic ones. Feige and Shamir show that witness indistinguishability survives the composition of interactive protocols [FS90, Thm. 3.1, p. 418] and polynomially many non-interactive proofs on one public random string [FS90, Thm. 5.3, p. 423], for statements and witnesses fixed in advance. These results do not give the Q-proof form, in which each triple may depend on the shared oracle. The error depends on L because the choice can depend on the oracle values from which the challenges of each proof are derived. The proof bounds the distance for the worst triple at each oracle, and the number of triples of total length at most L enters through this maximum.

6.1.2 Weak online simulation extractability

In some applications the adversary sees proofs before it produces its own. A signature scheme built from a proof of knowledge of the secret key is one: a forger sees signatures, which are proofs, and must not produce a new one without knowing the key. Simulation extractability asks that extraction succeed even then, for a statement the adversary never received a proof of.

Let $(\mathsf{Prv}, \mathsf{Vrf})$ be a non-interactive argument, Sim a simulator (Definition H.6) and A a (t, Q)-adversary (Definition 6.1) whose proof queries are statements and whose output is a pair $(\mathsf{x}, \mathsf{pf})$. An *execution of A against* Sim answers the hash queries of A, of Sim and of Vrf with one oracle, and the proof queries of A with Sim. On a proof query x_j, it appends x_j to a list Qry of queried statements, runs $\mathsf{Sim}(\mathsf{x}_j)$ and returns the proof of Sim to A. The simulator keeps its state across proof queries. When A outputs $(\mathsf{x}, \mathsf{pf})$ and its auxiliary output Z, the verifier computes $v \leftarrow \mathsf{Vrf}(\mathsf{x}, \mathsf{pf})$. In the *simulated execution*, the oracle is $\mathsf{H}[\mathsf{Reprog}]$, where $\mathsf{H}\colon \mathcal{Q} \to \mathcal{O}$ is uniform and Reprog is the list of assignments that Sim makes. We write $\mathsf{Vrf}^{\mathsf{H}[\mathsf{Reprog}]} \circ A^{\mathsf{H}[\mathsf{Reprog}],\mathsf{Sim}}$ for the simulated execution.

Definition 6.4 (Weak online simulation extractability)**.** Let $(\mathsf{Prv}, \mathsf{Vrf})$ be a non-interactive argument for $\mathcal{R}$ and Sim a simulator. An *online simulation extractor* is an algorithm $\mathcal{E}$ that serves as the oracle of an execution of A against Sim and finally outputs a witness w. We write $\mathsf{Vrf}^{\mathcal{E}} \circ A^{\mathcal{E},\mathsf{Sim}}$ for this execution. The argument $(\mathsf{Prv}, \mathsf{Vrf})$ is *weakly online simulation extractable* with respect to Sim, with oracle simulation error $\varepsilon_{\mathrm{sim}}$ and extraction error $\varepsilon_{\mathrm{SE}}$, functions from $\mathbb{N}^2$ to the nonnegative real numbers, if there is an online simulation extractor $\mathcal{E}$ such that for all $t, Q \in \mathbb{N}$ and every (t, Q)-adversary A whose proof queries are statements and whose output is a pair $(\mathsf{x}, \mathsf{pf})$, the *oracle simulation condition*

$$\mathsf{TD}\big([(\mathsf{x}, \mathsf{pf}, Z, \mathsf{Qry}, v)]_{\mathsf{Vrf}^{\mathsf{H}[\mathsf{Reprog}]} \circ A^{\mathsf{H}[\mathsf{Reprog}],\mathsf{Sim}}},\ [(\mathsf{x}, \mathsf{pf}, Z, \mathsf{Qry}, v)]_{\mathsf{Vrf}^{\mathcal{E}} \circ A^{\mathcal{E},\mathsf{Sim}}}\big) \le \varepsilon_{\mathrm{sim}}(t, Q)$$

and the *extraction condition*

$$\Pr\big[v = 1 \wedge \mathsf{x} \notin \mathsf{Qry} \wedge (\mathsf{x}, \mathsf{w}) \notin \mathcal{R} : (\mathsf{x}, \mathsf{pf}, Z, \mathsf{Qry}; v; \mathsf{w}) \leftarrow \mathsf{Vrf}^{\mathcal{E}} \circ A^{\mathcal{E},\mathsf{Sim}}\big] \le \varepsilon_{\mathrm{SE}}(t, Q)$$

hold. The running time of $\mathcal{E}$ is polynomial in t, Q, the running time of Sim, the length of the statement, the length of the proof and the bit lengths of the oracle's inputs and outputs. In the ROM, the adversaries are those of the ROM and $\mathcal{E}$ is classical.

The oracle simulation condition asks that the extractor's execution match the simulated execution: the joint states of the output, the auxiliary output Z, the queried statements and the verdict in the two executions are within trace distance $\varepsilon_{\mathrm{sim}}$. A reduction can therefore use the extracted witness and continue from the adversary's state. The extraction condition asks that the extractor obtain a witness whenever the verifier accepts a proof of a statement that A never queried. The notion is *online* because the extractor works in the adversary's execution and never rewinds it. It is *weak* because only the statement must be fresh, so a new proof of a queried statement is not a win. Statement freshness suffices for existential unforgeability when the message is part of the statement [Unr17, Defs. 26–27 and Thm. 31, pp. 38–40]. Proof queries may be any statements, with no witness, as in Unruh's simulation oracle [Unr17, p. 21], which is stronger than true-simulation extractability, whose simulator answers only pairs $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$ [DHLW10]. At $Q = 0$, the two conditions are those of Definition H.3. In the ROM, an extractor that answers by lazy sampling and from Reprog has $\varepsilon_{\mathrm{sim}} = 0$ (Theorem I.17(2)).

The notion combines three elements of published definitions. Unruh's simulation-sound online extractability extracts in one execution with simulated proofs [Unr14, Def. 4, p. 7]. It asks only that the pair of statement and proof be fresh, so it also counts a new proof of a queried statement and is the stronger notion. Its extractor is classical and reads a description of the simulator's oracle, and it has no clause on the adversary's state. Don et al.'s online extractability has the oracle simulation condition, with no proof queries [DFMS22, Def. 3.1, p. 10]. The weak form of Unruh's Definition 14, inequality (6), asks that only the statement be fresh [Unr17, p. 21].

Unruh's Definition 14 compares an execution of the adversary against the simulator with an execution of an extractor on the adversary, through a test of the adversary's final state [Unr17, p. 21]. By the oracle simulation condition of Definition 6.4, a test of the final state that makes no oracle queries passes with probabilities within $\varepsilon_{\mathrm{sim}}$ in the simulated execution and in the execution of $\mathcal{E}$ [Wat18, Thm. 3.4, p. 128].

6.2 Series–parallel BCS

The IOP $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ of Theorem 5.3 differs from the IOPs that the analyses of BCS (Section 2.8) assume in three ways. First, it commits one tableau per field in its first prover message, and its later prover messages send values that the verifier reads in full, such as the masked late values and the masked sumcheck messages. Second, its messages carry a series–parallel order, so the subprotocols of different fields are composed in parallel. Third, several of its challenges are uniform elements of sets whose size is not a power of two.

spBCS extends BCS to this format. Each prover message may carry several oracle strings and a non-oracle part, and the prover commits each oracle string with its own Merkle tree. The verifier reads the non-oracle part in full, and the non-oracle part enters the challenge inputs [CY26, Rem. 25.1.2, p. 276]. Each challenge is derived from the statement, the proof salt and the messages of its past in the series–parallel order. Where parallel branches are composed in series with a later part, the transcript of each message of that part replaces each branch by its label and one *final digest* of the branch's whole transcript. Each challenge is computed from one oracle answer, its *stream*. For challenge sets whose size is not a power of two, a sampler maps the stream to a challenge by rejection sampling. The sampler fails if the stream runs out before it accepts, and when it succeeds, the challenge is uniform on its set. The *overrun* $\varepsilon_{\mathrm{ov}}$ is the sum, over the challenges, of the probability that the sampler fails on a uniform stream. It bounds the probability that some sampler fails.

The digest of each challenge's stream enters the transcript before the next challenge is derived. We call this the *write-back*. Because of the write-back, a fresh oracle answer can complete a round of at most one transcript that the extractor reconstructs from the recorded hash queries. Round-by-round knowledge soundness of the IOP therefore bounds the adversary's advantage (Lemma I.12). Unlike the BCS hash chain (Section 2.8), the write-back absorbs each challenge. The proofs of Theorem 6.6 use the write-back for knowledge soundness and for weak online simulation extractability, in both models.

As in BCS, the prover hashes each leaf with its own salt, longer than the digest [BCS16, §3.2, p. 15]. The leaf salts hide the unopened leaves from a distinguisher that holds the whole oracle. The prover also draws one proof salt, and every challenge input and stream point contains it. The proof salt separates these oracle inputs for different proofs, and the proof of zero knowledge for many adaptively chosen proofs relies on this separation. Protocol 6.5 states the transformation. Section I.1 fixes the encodings of the oracle's inputs, the Merkle layout and the samplers, and defines the quantities in the error bounds (Definition I.1).

Protocol 6.5 (Series–parallel BCS, spBCS)**.** Let IOP be a series–parallel IOP (Definition 2.18) for $\mathcal{R}$ with order $\sqsubset$. Fix a digest length of $b \geq 1$ bytes, a stream excess of $e \geq 0$ bytes, proof salts of b_{s} bytes, leaf salts of b_{l} bytes, leaf encodings of at most L_{leaf} bytes, samplers $\mathsf{sample}_i \colon \{0,1\}^{8(b+e)} \to \mathcal{C}_i \cup \{\perp\}$ under which every challenge has the same number of preimages and a uniform preimage of a given challenge can be drawn in polynomial time (Section I.1), the Merkle commitment of Section I.1, and a set of *merges*, each with a label. A merge is a part $(A_0 \sqcup \ldots \sqcup A_{r-1}); D$ of $\sqsubset$ (Definition 2.17) whose *branches* $A_0, \ldots, A_{r-1}$ each have a largest message under $\sqsubset$. A merge may lie inside a branch or inside the part D of another merge.

Notation. A prover message $i \in J_P$ carries a non-oracle part a_i and oracle strings $O_{i,0}, \ldots, O_{i,m_i-1}$, and $\mathrm{past}(i)$ is its past under $\sqsubset$ (Definition 2.18). The random oracle $\mathsf{H} \colon \mathcal{Q} \to \mathcal{O}$, with $\mathcal{O} = \{0,1\}^{8(b+e)}$, has four parts with injective encodings (Section I.1): challenge inputs $\mathsf{ch}(\cdot)$, leaves, internal nodes, and stream points $\mathsf{stream}(\cdot,\cdot)$. The digest $\mathsf{dg}(y)$ is the first $8b$ bits of $y \in \mathcal{O}$. The *item* of a prover message i is $W_i = (a_i, \mathsf{rt}_{i,0}, \ldots, \mathsf{rt}_{i,m_i-1})$, and the item of a verifier message i is the digest $\mathsf{dg}(y_i)$ of its stream (the write-back). For a statement x, a message i and a proof salt s, the *transcript* ℓ_i lists the items of $\mathrm{past}(i)$ in message order, each with its index, except that each branch A of a merge whose part D contains i is replaced by its label and its final digest $d_A = \mathsf{dg}(\mathsf{H}(\mathsf{ch}(s, \mathsf{x}, i_A, \ell_{i_A} \| W)))$, where i_A is the largest message of A under $\sqsubset$ and W its item with its index.

1. *Prover* $\mathsf{Prv}^{\mathsf{H}}(\mathsf{x}, \mathsf{w})$.
   (a) Sample a proof salt s and the IOP prover's coins.
   (b) For each message i in message order: at a prover message, run the IOP prover to obtain a_i and $O_{i,0}, \ldots, O_{i,m_i-1}$, and commit to each $O_{i,j}$ with a salted Merkle tree of root $\mathsf{rt}_{i,j}$. At a verifier message, compute the key $u_i = \mathsf{dg}(\mathsf{H}(\mathsf{ch}(s, \mathsf{x}, i, \ell_i)))$, the stream $y_i = \mathsf{H}(\mathsf{stream}(u_i, s))$ and the challenge $\theta_i = \mathsf{sample}_i(y_i)$, abort with output $\perp$ if $\theta_i = \perp$, and give θ_i to the IOP prover.
   (c) Output $\mathsf{pf} = (s, (W_i)_{i \in J_P}, \text{the salted leaves at } \mathsf{qry}(\theta), \text{their authentication paths})$.
2. *Verifier* $\mathsf{Vrf}^{\mathsf{H}}(\mathsf{x}, \mathsf{pf})$. Recompute every challenge from pf as in step 1(b), and reject if a sampler returns $\perp$. Check every authentication path against its root in W_i, decode the opened leaves, and accept if and only if the IOP verifier accepts on x, the challenges, the non-oracle messages and the opened values.

Our implementation, like Longfellow's [Goo26a], compresses the authentication paths by deduplicating shared ancestor nodes. The compression makes the proof smaller and does not change the

bounds below.

6.3 Security of spBCS

Theorem 6.6 states the security of spBCS for every series–parallel IOP. Each part assumes one property of the IOP and carries it over to the non-interactive argument: completeness, round-by-round knowledge soundness, or honest-verifier zero knowledge. In Parts (1), (2), (3) and (5) the adversary makes at most t hash queries, and in Parts (3) and (5) at most Q proof queries. In Part (1) the bound t counts only the hash queries before the adversary chooses the statement, and in Part (3) only those before the last proof query. In Part (4) the encodings of each statement and its two witnesses have at most L bits in total.

The errors also depend on quantities of the protocol (Definition I.1). Four are counts: the verifier's hash queries k_q, the simulator's hash queries q_S per proof, a bound c on the digests in a challenge input, and the largest number n_J of final digests in the challenge input of one message. The count n_J sums the branches of the merges whose part contains that message. The other three are the sampler overrun $\varepsilon_{\mathrm{ov}}$, the key-collision term $\varepsilon_{\mathrm{key}}$ and the leaf term $\varepsilon_{\mathrm{leaf}}$. The first term of $\varepsilon_{\mathrm{leaf}}$ is $2^{-4(b_{\mathrm{l}}-b)-1}$, and the second is at most $\sqrt{(8L_{\mathrm{leaf}}+1)\ln 2/2^{8b_{\mathrm{l}}+1}}$. Each byte of leaf salt beyond the digest divides both terms by 2^4, so longer leaf salts make $n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}}$ as small as required (Section 7.7).

In the QROM, the instability argument of the knowledge-soundness proof (Lemma I.12) loses a factor quadratic in the number of queries [CMS19, Lem. 5.7], so the extraction errors are dominated by a term $6T^2\varepsilon^{\mathrm{k}}$. Here T counts the hash queries of the adversary, the verifier and the simulator. In Part (5) it counts the simulator's queries twice, because the extractor replays them. Sections I.2 to I.4 prove the parts of Theorem 6.6.

Theorem 6.6 (Security of spBCS)**.** *Let* IOP *be a series–parallel IOP (Definition 2.18) for* $\mathcal{R}$ *with* n_{pos} *oracle positions, and* $(\mathsf{Prv},\mathsf{Vrf})$ *its spBCS transformation (Protocol 6.5) with digests of* b *bytes, streams of* $b+e$ *bytes, proof salts of* b_{s} *bytes, leaf salts of* b_{l} *bytes and leaf encodings of at most* L_{leaf} *bytes, with* k_q, q_S, c, n_J, $\varepsilon_{\mathrm{ov}}$, $\varepsilon_{\mathrm{key}}$ *and* $\varepsilon_{\mathrm{leaf}}$ *as in Definition I.1. Write* $c' = \max(2,c)+n_J+1$, *and for* $t,Q\in\mathbb{N}$, $k(Q)=k_q+Qq_S$, $T(t,Q)=t+k(Q)+Qq_S$ *and* $T^{\mathrm{C}}(t,Q)=t+k(Q)$. *Throughout,* $\mathbf{S}$, $\mathbf{E}$, *membership in* $\mathcal{R}$, *the samplers, and the encodings of Section I.1 and their decoders run in polynomial time. The bit length of an encoded oracle input,* b *and* e *are polynomial in the length of the statement, and so are* N_{msg}, n_B, n_{pos}, q_{IOP} *and* $\mathsf{l}_{\max}$.

1. Completeness. *If* IOP *has completeness error at most* $\varepsilon_{\mathrm{c}}\in[0,1]$ *at every statement, then* $(\mathsf{Prv},\mathsf{Vrf})$ *has completeness error (Definition H.2) at most* $\varepsilon_{\mathrm{cpl}}(t)=\varepsilon_{\mathrm{c}}+\varepsilon_{\mathrm{ov}}+\varepsilon_{\mathrm{key}}+2t/2^{4b_{\mathrm{s}}}$ *in the QROM and* $\varepsilon_{\mathrm{cpl}}(t)=\varepsilon_{\mathrm{c}}+\varepsilon_{\mathrm{ov}}+\varepsilon_{\mathrm{key}}+t/2^{8b_{\mathrm{s}}}$ *in the ROM, for adversaries that make at most* t *hash queries before they choose the statement (Theorem I.3).*

2. Knowledge soundness. *If* IOP *is round-by-round knowledge sound with errors at most* ε^{k}, *then* $(\mathsf{Prv},\mathsf{Vrf})$ *is adaptively knowledge sound (Definition H.3) with oracle simulation error* 0 *and extraction error at most*

$$\varepsilon_{\mathrm{ex}}(t)=2\sqrt{\frac{k_q}{2^{8(b+e)}}}+6(t+k_q)^2\Big(\varepsilon^{\mathrm{k}}+\frac{c'(t+k_q)}{2^{8b}}\Big)$$

in the QROM and

$$\varepsilon^{\mathrm{C}}_{\mathrm{ex}}(t)=(t+k_q)\Big(\varepsilon^{\mathrm{k}}+\frac{c'(t+k_q)}{2^{8b}}\Big)$$

in the ROM (Theorem I.16). It therefore has adaptive soundness error at most $\varepsilon_{\mathrm{ex}}(t)$ *in the QROM and* $\varepsilon^{\mathrm{C}}_{\mathrm{ex}}(t)$ *in the ROM (Definition H.4).*

3. Everlasting zero knowledge. *If the honest prover of* IOP *is honest-verifier zero knowledge with error at most* $\varepsilon_{\mathrm{hvzk}} \in [0,1]$ *at every statement, then* $(\mathsf{Prv}, \mathsf{Vrf})$ *is everlasting zero knowledge (Definition 6.2) with respect to the simulator of Protocol I.4, built from the HVZK simulator of* IOP*, with error at most*

$$\varepsilon_{\mathrm{zk}}(t,Q) = \frac{4Qt}{2^{4b_{\mathrm{s}}}} + \frac{Q(Q-1)}{2^{8b_{\mathrm{s}}}} + Q\big(\varepsilon_{\mathrm{key}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{hvzk}} + n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}}\big)$$

in the QROM and

$$\varepsilon^{\mathrm{C}}_{\mathrm{zk}}(t,Q) = \frac{Qt}{2^{8b_{\mathrm{s}}}} + \frac{Q(Q-1)}{2\cdot 2^{8b_{\mathrm{s}}}} + Q\big(\varepsilon_{\mathrm{key}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{hvzk}} + n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}}\big)$$

in the ROM (Theorem I.5).

4. Unconditional witness indistinguishability. *If the honest prover of* IOP *is honest-verifier zero knowledge with error at most* $\varepsilon_{\mathrm{hvzk}} \in [0,1]$ *at every statement, then* $(\mathsf{Prv}, \mathsf{Vrf})$ *is unconditionally witness indistinguishable (Definition 6.3) with error at most*

$$\varepsilon_{\mathrm{wi}}(Q,L) = Q\Big(2n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}} + 2\varepsilon_{\mathrm{hvzk}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} + \sqrt{\frac{(L+1)\ln 2 + 2\ln(L+1)}{2\cdot 2^{8b_{\mathrm{s}}}}}\Big)$$

(Theorem I.6).

5. Weak online simulation extractability. *Let* $\mathbf{S}$ *be any probabilistic algorithm that maps a statement to challenges* θ*, non-oracle messages and values at the positions* $\mathsf{qry}(\theta)$*, such as the HVZK simulator of Part (3). If* IOP *is round-by-round knowledge sound with errors at most* ε^{k}*, then* $(\mathsf{Prv}, \mathsf{Vrf})$ *is weakly online simulation extractable (Definition 6.4) with respect to the simulator of Protocol I.4 built from* $\mathbf{S}$*, with oracle simulation error* 0 *and extraction error at most*

$$\varepsilon_{\mathrm{SE}}(t,Q) = 2\sqrt{\frac{k(Q)}{2^{8(b+e)}}} + 6T(t,Q)^2\Big(\varepsilon^{\mathrm{k}} + \frac{c'\,T(t,Q)}{2^{8b}}\Big)$$

in the QROM and

$$\varepsilon^{\mathrm{C}}_{\mathrm{SE}}(t,Q) = T^{\mathrm{C}}(t,Q)\Big(\varepsilon^{\mathrm{k}} + \frac{c'\,T^{\mathrm{C}}(t,Q)}{2^{8b}}\Big)$$

in the ROM (Theorem I.17).

Four byte lengths of Protocol 6.5 control these errors: the digest length b, the stream excess e, the leaf salt b_{l} and the proof salt b_{s}. The digest length b sets the collision terms. The stream excess e sets the probability that a sampler fails, and the stream length $b+e$ sets the term $2\sqrt{k/2^{8(b+e)}}$ of knowledge soundness and simulation extractability. The leaf salt sets the leakage of unopened leaves, and the proof salt sets the salt terms of completeness, zero knowledge and witness indistinguishability. Some verifier messages get longer streams, so that the failure probability of each sampler does not depend on the number of challenges. A message with one challenge per row of a linear combination, such as Ligero's linear test, gets four extra bytes per byte of those challenges, twice what rejection sampling reads in expectation. In a message with several challenges, each challenge after the first gets as many extra bytes as the earlier challenges read, so each challenge may read $b+e$ bytes. The model takes $\mathcal{O}$ at the longest stream, and each sampler reads a prefix of it (Section I.1). A longer stream only lowers these terms, so the theorems hold as stated.

Parts (1) and (2) follow known arguments. The knowledge-soundness extractor is Chiesa, Manohar and Spooner's database extractor [CMS19, Thm. 8.6], and the proof of Part (2) adapts their instability argument to spBCS (Lemma I.12).

For BCS in the QROM, Part (3) gives adaptive zero knowledge with a concrete error. Its distinguisher makes hash queries before it chooses each statement and witness. Like Chiesa, Manohar and Spooner's form (Section 2.8), Part (3) also holds against a distinguisher that queries the whole oracle after the last proof, and its error grows with the number of hash queries made before the last proof. The proof relies on the proof salt. Every challenge input and stream point of a proof contains that proof's salt, which the prover samples when it makes the proof. Earlier hash queries hit these inputs only with small probability, so the simulator can program them. Proofs of chosen-message security for Fiat–Shamir signatures reprogram the oracle in the same way, at points that contain fresh randomness [GHHM21, §4.1, Thm. 3]. The proof of Part (3) applies one pair of one-way-to-hiding steps per proof [AHU18, Thm. 3, p. 11].

The adversary of Part (4) makes any number of hash queries before, between and after the proofs, so the error of Part (4) has no term in the number of hash queries. Part (4) therefore does not follow from Part (3).

Part (5) gives a straight-line extractor with a concrete error. The extractor works in the adversary's execution and leaves the joint state of the adversary's output and its quantum auxiliary output as in the simulated execution. To our knowledge, no earlier work proves simulation extractability of BCS in the QROM (Section 9).

Weak online simulation extractability gives the extraction property of Chase and Lysyanskaya's signatures of knowledge. That property bounds the probability that one execution yields an accepted signature on a fresh statement and message from which the extractor obtains no witness [CL06, Def. 2.2, pp. 8–9]. To compare the two, we encode the message in the statement. Their signing oracle checks the witness and returns a simulated signature. An adversary can make this check itself and send a proof query for each statement whose witness is valid, so its queried statements are their list of successful queries. In both models, the extraction error of Definition 6.4 bounds their event. The extractor's simulation of the oracle corresponds to their trapdoor, and Theorem 6.6(3) gives their simulatability. Theorem 6.6(5) therefore gives their extraction property for spBCS.

The database extractor of spBCS answers the adversary's hash queries from an oracle of its own and never queries the random oracle at the inputs of internal Merkle nodes. Our Lean library proves that, along families of complete instances, inequality (6) of Unruh's Definition 14 [Unr17, p. 21] fails for such extractors with polynomially long assignment lists when the test may query the random oracle after the execution (Appendix J).

6.4 Security of GordieHowe

We now apply the spBCS transformation to $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ (Protocol 5.1).

Theorem 6.7 (Security of the non-interactive argument)**.** *Let $(\mathcal{P},\mathcal{F})$ be a valid realized program that realizes $\mathcal{P}_0$, and let $\mathcal{R} \subseteq \mathcal{X}^{\mathrm{st}}_{\mathcal{P}_0} \times \mathcal{W}_{\mathcal{P}_0}$. Suppose that the Ligero parameters of each field satisfy the conditions of Theorem 5.3, and that $\mathcal{P}_0$ and $\mathcal{R}$ satisfy the hypotheses of its Parts (1) and (2), with completeness error ε_{c} and errors $(\varepsilon_\gamma)_\gamma$. Let $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ be Protocol 5.1 on $(\mathcal{P},\mathcal{F})$, with n_{pos} oracle positions (Definition 2.18), and let ε^{k} be the largest of its round errors $\varepsilon^{\mathcal{F}}$ (Theorem 5.3(2)). Let $(\mathsf{Prv},\mathsf{Vrf})$ be the spBCS transformation of $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ (Protocol 6.5) with digests of $b \geq 1$ bytes, streams of $b+e$ bytes, leaf salts of b_{l} bytes, proof salts of b_{s} bytes and leaf encodings of at most L_{leaf} bytes, and c, n_J, k_q, q_S, $\varepsilon_{\mathrm{ov}}$, $\varepsilon_{\mathrm{key}}$ and $\varepsilon_{\mathrm{leaf}}$ its quantities (Definition I.1), with c', $k(Q)$, $T(t,Q)$ and $T^{\mathrm{C}}(t,Q)$ as in Theorem 6.6. Then $(\mathsf{Prv},\mathsf{Vrf})$ is a non-interactive argument for $\mathcal{R}$ (Definition H.1) with the following properties.*

1. *It has completeness error (Definition H.2) at most $\varepsilon_{\mathrm{cpl}}(t) = \varepsilon_{\mathrm{c}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} + 2t/2^{4b_{\mathrm{s}}}$ in the QROM and $\varepsilon_{\mathrm{cpl}}(t) = \varepsilon_{\mathrm{c}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} + t/2^{8b_{\mathrm{s}}}$ in the ROM.*

2. *It is adaptively knowledge sound in the QROM (Definition H.3), with oracle simulation error* 0 *and extraction error at most*

$$\varepsilon_{\mathrm{ex}}(t) = 2\sqrt{\frac{k_q}{2^{8(b+e)}}} + 6(t+k_q)^2\Big(\varepsilon^{\mathrm{k}} + \frac{c'(t+k_q)}{2^{8b}}\Big),$$

and adaptively knowledge sound in the ROM, with oracle simulation error 0 *and extraction error at most*

$$\varepsilon^{\mathrm{C}}_{\mathrm{ex}}(t) = (t+k_q)\Big(\varepsilon^{\mathrm{k}} + \frac{c'(t+k_q)}{2^{8b}}\Big).$$

It therefore has adaptive soundness error at most $\varepsilon_{\mathrm{ex}}(t)$ *in the QROM and* $\varepsilon^{\mathrm{C}}_{\mathrm{ex}}(t)$ *in the ROM (Definition H.4).*

3. *It is everlasting zero knowledge (Definition 6.2) with respect to the simulator of Protocol I.4, built from the HVZK simulator of Protocol 5.4, in the QROM with error at most*

$$\varepsilon_{\mathrm{zk}}(t,Q) = \frac{4Qt}{2^{4b_{\mathrm{s}}}} + \frac{Q(Q-1)}{2^{8b_{\mathrm{s}}}} + Q\big(\varepsilon_{\mathrm{key}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{c}} + n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}}\big),$$

and in the ROM with error at most

$$\varepsilon^{\mathrm{C}}_{\mathrm{zk}}(t,Q) = \frac{Qt}{2^{8b_{\mathrm{s}}}} + \frac{Q(Q-1)}{2\cdot 2^{8b_{\mathrm{s}}}} + Q\big(\varepsilon_{\mathrm{key}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{c}} + n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}}\big).$$

4. *It is unconditionally witness indistinguishable (Definition 6.3) with error at most* $\varepsilon_{\mathrm{wi}}(Q,L)$*, the error of Theorem 6.6(4) at* $\varepsilon_{\mathrm{hvzk}} = \varepsilon_{\mathrm{c}}$*,*

$$\varepsilon_{\mathrm{wi}}(Q,L) = Q\Big(2n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}} + 2\varepsilon_{\mathrm{c}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} + \sqrt{\frac{(L+1)\ln 2 + 2\ln(L+1)}{2\cdot 2^{8b_{\mathrm{s}}}}}\Big).$$

5. *It is weakly online simulation extractable (Definition 6.4) with respect to the simulator of Part (3), with oracle simulation error* 0*, with extraction error at most*

$$\varepsilon_{\mathrm{SE}}(t,Q) = 2\sqrt{\frac{k(Q)}{2^{8(b+e)}}} + 6T(t,Q)^2\Big(\varepsilon^{\mathrm{k}} + \frac{c'\,T(t,Q)}{2^{8b}}\Big)$$

in the QROM, and with extraction error at most

$$\varepsilon^{\mathrm{C}}_{\mathrm{SE}}(t,Q) = T^{\mathrm{C}}(t,Q)\Big(\varepsilon^{\mathrm{k}} + \frac{c'\,T^{\mathrm{C}}(t,Q)}{2^{8b}}\Big)$$

in the ROM.

Proof. For a fixed realized program, $\mathcal{X}^{\mathrm{st}}_{\mathcal{P}_0}$ and $\mathcal{W}_{\mathcal{P}_0}$ are finite, so the running-time conditions of Theorems I.5 and I.16 hold. By Theorem 5.3, $\mathsf{IOP}_{\mathcal{P},\mathcal{F}}$ is a series–parallel IOP for $\mathcal{R}$ with completeness error ε_{c}, round-by-round knowledge errors at most ε^{k}, and honest-verifier zero-knowledge error ε_{c} with respect to the HVZK simulator of Protocol 5.4. Part (1) is Theorem I.3(2). Part (2) is Theorem I.16. Part (3) is Theorem I.5(1) and (2) with $\varepsilon_{\mathrm{hvzk}} = \varepsilon_{\mathrm{c}}$. Part (4) is Theorem I.6 with $\varepsilon_{\mathrm{hvzk}} = \varepsilon_{\mathrm{c}}$. Part (5) is Theorem I.17(1) and (2). □

Every error of Theorem 6.7 is a formula in ε^{k}, ε_{c}, the numbers t and Q of hash and proof queries, the length bound L of witness indistinguishability, the digest and salt lengths, and the protocol quantities of Definition I.1. Section 7.7 evaluates these formulas at the parameter sets of Table 3.

	ROM	QROM
Adversaries	classical	quantum
Transcript and Merkle hash	SHA-256 [Nat15a]	SHA-384
Digest, proof salt, leaf salt (bytes)	32, 32, 64	48, 48, 96
Stream excess e (bytes)	206,848	206,848
Hashing field H	$\mathbb{F}_{2^{128}}$	$\mathbb{F}_{2^{256}}$
Algebra field A	$\mathbb{F}_{p_{\mathrm{G}}^2}$ (RSA-2048, ML-DSA), $\mathbb{F}_{p_{256}}$ (ECDSA mDL)	$\mathbb{F}_{p_{\mathrm{G}}^4}$ (ML-DSA)
Code rate of H	1/12	1/12
Code rate of A	1/5 ($\mathbb{F}_{p_{\mathrm{G}}^2}$), 1/4 ($\mathbb{F}_{p_{256}}$)	1/6
Per-round error target	$2^{-113.5}$	2^{-236}

Table 3: The ROM and QROM parameter sets.

7 Applications and evaluation

We compare with baselines built on Longfellow at commit d5e6be7 [Goo26a], with Binius64 (Section 7.6), with Vega (Section 7.5.1) and with the proof of Baum and Beckmann (Section 7.4). Post-quantum signatures (ML-DSA and Falcon) are the subject of a companion paper [SWPG26], but we include some ML-DSA results here.

We measure SHA-256 and the ML-DSA applications at both parameter sets (Table 3), and RSA-2048 and the ECDSA mDL at the ROM parameters, since RSA and ECDSA are not post-quantum. The *ROM parameters* target classical adversaries, and the *QROM parameters* target quantum adversaries. The per-round error target of the ROM parameters is $2^{-113.5}$. Longfellow version 7 [Goo26a] queries 132 columns at rate 1/7 [Fs26, §5.1, p. 38]. At these parameters, Lemma G.12 bounds every Ligero round error of Longfellow's mDL proof by $2^{-113.46}$ with one attribute and by $2^{-113.23}$ with two to four attributes (Section G.7). Longfellow's sumcheck rounds have smaller errors. The largest of them, at the output point of the hash circuit, is at most $2^{-125.6}$. The ROM target of GordieHowe is therefore at least as strict as the largest round error of each of these Longfellow proofs.

Google's Lean analysis bounds Longfellow's two Ligero rounds jointly for a fixed commitment and weights each combination challenge by the probability that the prover's response still passes the column query [Goo26b]. For Longfellow version 7's tableaus at 132 queried columns, this joint bound is at most 2^{-127} for the signature circuit and 2^{-125} for the hash circuit. Our adversaries (Definition 6.1) may grind commitments, and in the QROM they are quantum. It is not clear how to adapt the joint analysis to either, so we set GordieHowe's target from the round-by-round bound.

We measure on an otherwise idle AMD EPYC 9754 with Linux's `performance` CPU frequency governor. Longfellow and the single-threaded GordieHowe are pinned to one core, and the multi-threaded GordieHowe runs on four. All systems are release builds for the native instruction set, compiled with clang 21.1.8 (GordieHowe and Longfellow) or rustc 1.98.1 (Binius64). Every figure is the median of 9 trials after 2 warm-up proofs, and for GordieHowe and the Longfellow baselines the trials are interleaved across applications. The prover time P runs from the statement and the prover's private inputs to the proof, witness generation included. The verifier time V includes proof parsing and GordieHowe's per-proof computations on the statement, such as ExpandA for ML-DSA. The ML-DSA and ML-DSA mDL baselines verify in memory and parse nothing, and Binius64 receives the expanded ML-DSA key as a public input and reports its witness generation once per process, which we add to its median commitment and proving time. Proof sizes vary by less than 2%

between trials, because the opened columns' authentication paths share a number of Merkle nodes that depends on which columns are queried.

At the QROM parameters, we modify the Longfellow baselines to use SHA-384 for the transcript and the Merkle commitments, as GordieHowe does. The rest of their Ligero and sumcheck code is Longfellow's. At the ROM parameters, every baseline meets GordieHowe's per-round target of $2^{-113.5}$ by Lemma G.12: the ECDSA mDL and ML-DSA-44 baselines with Longfellow's 132 queried columns, the RSA baseline with 139 and 138, and the SHA-256 baseline with the fewest columns that meet it, 139 for the 8 KiB message. The ML-DSA mDL baseline queries 122 columns, for $\approx 2^{-112.8}$, matching GordieHowe's $2^{-112.7}$ for this relation. At the QROM parameters, the baselines query 248 to 256 columns for SHA-256 and 295 at rate 1/6 for the ML-DSA applications, and each meets the target of 2^{-236}. The baselines keep Longfellow's digest-length leaf salts, which makes their proofs smaller.

7.1 Fields

Each application proves its hashes over a binary field H and, except for SHA-256, its arithmetic over a field A of odd characteristic (Table 3). A degree-one check over $\mathbb{F}_{2^{128}}$ has error 2^{-128}. This error meets the per-round target $2^{-113.5}$ of the ROM parameters but not the target 2^{-236} of the QROM parameters (Table 3), so H is $\mathbb{F}_{2^{128}}$ at the ROM parameters and $\mathbb{F}_{2^{256}}$ at the QROM parameters.

For RSA-2048 and ML-DSA, A is an extension of the Goldilocks field $\mathbb{F}_{p_G}$ [Pol26], for the prime $p_G = 2^{64} - 2^{32} + 1$: the quadratic extension at the ROM parameters and, for ML-DSA, the quartic extension at the QROM parameters. An element of $\mathbb{F}_{p_G}$ fits in one 64-bit word. The multiplicative group of $\mathbb{F}_{p_G}$ has a subgroup of order 2^{32}, which holds the evaluation points of the Reed–Solomon code. The extensions contain $\mathbb{F}_{p_G}$, so ring switching packs two or four values below p_G into one committed element (Section 5.5). For the ECDSA mDL, A is the base field $\mathbb{F}_{p_{256}}$ of P-256, as in Longfellow.

At the ROM parameters, the baselines use fields that Longfellow implements: $\mathbb{F}_{2^{128}}$ for hashing, $\mathbb{F}_{p_{256}}$ for ECDSA, the prime field $\mathbb{F}_{p_{128}}$ with $p_{128} = 2^{128} - 2^{108} + 1$ for RSA's arithmetic, and $\mathbb{F}_{q^6}$ for ML-DSA's arithmetic, where q is the ML-DSA modulus. These fields are too small for the target of the QROM parameters, so at those parameters the baselines use $\mathbb{F}_{2^{256}}$ and $\mathbb{F}_{q^{11}}$, which we implemented in Longfellow.

7.2 Proofs of possession of RSA signatures

We construct a proof for the relation

$$\mathcal{R}_{\mathrm{RSA}} = \big\{ \big(N, (m, s)\big) : s \in \mathbb{Z}_N,\ s^{65537} \equiv \mathsf{Encode}(\text{SHA-256}(m)) \pmod N \big\},$$

where the modulus N has exactly 2048 bits and the message m has at most n_m bytes, for a fixed public maximal length n_m. For a digest h,

$$\mathsf{Encode}(h) = \mathtt{0x00} \,\|\, \mathtt{0x01} \,\|\, \mathtt{0xFF} \cdots \mathtt{0xFF} \,\|\, \mathtt{0x00} \,\|\, \text{DigestInfo} \,\|\, h$$

is the EMSA-PKCS1-v1_5 encoding of h in 256 bytes [MKJR16, §9.2], where DigestInfo is the fixed 19-byte prefix of SHA-256. The relation leaves the range of s open. RSASSA-PKCS1-v1_5 verification also requires $0 \le s < N$ [MKJR16, §5.2.2], and every s in the relation reduces modulo N to such a signature. The program declares N as a stmt variable, and the verifier checks that N has exactly 2048 bits before it verifies a proof. It declares m and s as inputs. It proves the hash of m over a binary field H, and the modular arithmetic over the quadratic extension A of $\mathbb{F}_{p_G}$ (Section 7.1).

For a digit width w_d and a number n_d of digits in each limb, the program represents every integer below 2^{2048} as a vector of $L = \lceil 2048/(n_d w_d) \rceil$ limbs of $n_d w_d$ bits. A limb is the affine combination

						against Longfellow		
Application	Parameters	System	P (ms)	V (ms)	KiB	P	V	Proof
RSA-2048, 2 KiB	ROM	Longfellow †	641.7	337.8	314.7			
		GordieHowe (seq)	57.8	26.7	209.8	11.1×	12.7×	1.50×
		GordieHowe (par)	25.9	11.3	209.8	24.8×	29.8×	1.50×
SHA-256, 8 KiB	ROM	Longfellow †	1435.5	714.2	204.3			
		GordieHowe (seq)	60.6	20.9	123.0	23.7×	34.1×	1.66×
		GordieHowe (par)	23.7	9.1	123.0	60.6×	78.9×	1.66×
	QROM	Longfellow †	2746.3	1623.9	456.8			
		GordieHowe (seq)	109.4	46.5	309.5	25.1×	34.9×	1.48×
		GordieHowe (par)	44.7	21.1	309.5	61.4×	76.8×	1.48×
ECDSA mDL	ROM	Longfellow	657.3	318.6	352.0			
		GordieHowe (seq)	116.4	65.0	275.5	5.6×	4.9×	1.28×
		GordieHowe (par)	64.8	38.4	275.5	10.1×	8.3×	1.28×
ML-DSA-44, 32 B	ROM	Longfellow †	2026.0	1194.6	380.7			
		GordieHowe (seq)	62.0	31.9	213.5	32.7×	37.4×	1.78×
		GordieHowe (par)	34.4	18.5	213.5	58.9×	64.6×	1.78×
	QROM	Longfellow †	6129.6	3268.2	777.8			
		GordieHowe (seq)	146.8	72.1	549.2	41.8×	45.3×	1.42×
		GordieHowe (par)	87.3	46.8	549.2	70.2×	69.8×	1.42×
ML-DSA mDL	ROM	Longfellow †	25526.5	14379.9	1451.5			
		GordieHowe (seq)	473.7	247.0	447.6	53.9×	58.2×	3.24×
		GordieHowe (par)	190.9	115.2	447.6	133.7×	124.8×	3.24×
	QROM	Longfellow †	83291.8	43662.2	1892.2			
		GordieHowe (seq)	921.7	486.3	1018.8	90.4×	89.8×	1.86×
		GordieHowe (par)	397.4	281.9	1018.8	209.6×	154.9×	1.86×

Table 4: GordieHowe against Longfellow baselines. Times are in milliseconds and are medians of 9 trials. The prover time P includes witness generation for both systems (Section 7). The last three columns give GordieHowe's speedup over Longfellow and how many times smaller its proof is. GordieHowe (seq) runs on one core and GordieHowe (par) on four, and the (par) rows repeat the proof size of the (seq) rows. Longfellow is single-threaded. † Baseline written or reconfigured for this comparison in Longfellow's code base. Sizes after an application name are message lengths.

$\sum_{j<n_\mathrm{d}} 2^{w_\mathrm{d} j} d_j$ of n_d digits d_j of w_d bits. Each digit is a variable of type $\mathsf{Sub}(\mathbb{F}_{p_\mathrm{G}})$ over A. The limbs of N are public. Since $65537 = 2^{16} + 1$, the exponentiation consists of 16 squarings and one product. The program declares $x_0 = s, x_1, \ldots, x_{16}$ and states 17 modular products $x_b\, y_b \equiv r_b \pmod N$: $y_b = x_b$ and $r_b = x_{b+1}$ for $b < 16$, and $x_{16}\, x_0 \equiv \mathsf{Encode}(\text{SHA-256}(m))$. It states each product as the integer equality $x_b y_b = q_b N + r_b$, with a quotient q_b of $L_q = \lceil (2n_\mathrm{d} w_\mathrm{d} L - 2047)/(n_\mathrm{d} w_\mathrm{d}) \rceil$ limbs, since $x_b y_b < 2^{2n_\mathrm{d} w_\mathrm{d} L}$ and $N \geq 2^{2047}$.

The product of two limb vectors is the acyclic convolution $\mathsf{conv}(x, y)$ of their limbs, of length $2L-1$, with integer coefficients below $L\, 2^{2n_\mathrm{d} w_\mathrm{d}}$. The coefficients of $D = \mathsf{conv}(x_b, y_b) - \mathsf{conv}(q_b, N) - r_b$ are the integer $x_b y_b - q_b N - r_b$ written in base $2^{n_\mathrm{d} w_\mathrm{d}}$ without carries. The convolution $\mathsf{conv}(q_b, N)$ has $L_q + L - 1$ coefficients, so D has $K_D = \max(2L - 1, L_q + L - 1)$ coefficients. Since N is public, $\mathsf{conv}(q_b, N)$ is affine. The program groups the coefficients of D into runs of k consecutive coefficients, for a group size k chosen below, and forms $E_g = \sum_{j<k} 2^{n_\mathrm{d} w_\mathrm{d} j} D_{gk+j}$, and states $D = 0$ with signed carries c_g between groups:

$$\mathsf{int_eq}\big(E_g + c_{g-1},\ 2^{n_\mathrm{d} w_\mathrm{d} k} c_g\big), \qquad c_{-1} = c_{G-1} = 0,$$

for each of the $G = \lceil K_D / k \rceil$ groups of coefficients.

Range checks bound every digit of x_b and q_b to $[0, 2^w_\mathrm{d})$. The program range-checks each shifted carry $c_g + L\, 2^{n_\mathrm{d} w_\mathrm{d}}$ to $[0, W)$, where W is the least power of 2^w_d that is at least $2L\, 2^{n_\mathrm{d} w_\mathrm{d}} + 1$ (Section 4.2.2). The honest carries lie in $[-L\, 2^{n_\mathrm{d} w_\mathrm{d}}, L\, 2^{n_\mathrm{d} w_\mathrm{d}}]$, so they pass. All the range checks use one table of 2^w_d rows.

The program proves SHA-256 of m over H (Section 7.3). Only the 256 bits of the digest are linked to A, by a link predicate (Section 4.1.4). The digits of $\mathsf{Encode}(\text{SHA-256}(m))$ are affine in A's copy of the digest bits.

Realization The compiler batches each convolution $\mathsf{conv}(x, y)$. Let n_NTT be the least power of two that is at least $2L - 1$. The prover commits $z = \mathrm{NTT}_{n_\mathrm{NTT}}(x) \cdot \mathrm{NTT}_{n_\mathrm{NTT}}(y)$. One batched zerocheck (Section 5.4.2) with $P_Q = X_0 X_1 - X_2$ proves the $17 n_\mathrm{NTT}$ products, over the affine operands $\mathrm{NTT}_{n_\mathrm{NTT}}(x)_j$, $\mathrm{NTT}_{n_\mathrm{NTT}}(y)_j$ and z_j. The convolution $\mathsf{conv}(x, y)$ consists of the first $2L-1$ coordinates of $\mathrm{INTT}_{n_\mathrm{NTT}}(z)$, which is affine in z. It is exact modulo p_G, since $n_\mathrm{NTT} \geq 2L - 1$. The transforms are NTTs over $\mathbb{F}_{p_\mathrm{G}}$, so the prover and the verifier compute every convolution, and every affine map built from one, in quasilinear time. The batch requires both operands to be committed phase-0 values of the prime field, and $\mathbb{F}_{p_\mathrm{G}}$ to have a transform of length n_NTT. Both hold here.

The carry equations are int_eq predicates. The compiler realizes them by field equalities with no quotient (Section 4.1.3). It computes the interval of each side through the convolutions. A coefficient of $\mathsf{conv}(x, y)$ is a sum of between 1 and $\min(\ell_x, \ell_y)$ products of entries, for the lengths ℓ_x and ℓ_y of x and y. The compiler requires the span of each carry equation (Section 4.1.3) to be below p_G, and the program's group size is the largest at which every carry equation meets this bound. The bits that A commits for the link predicate are its copies of the digest and of the MAC's pad. They have the Boolean checks $x(x - 1) = 0$. The compiler places these checks in A's GKR circuit, where the cost model rates them cheaper than a batch (Section 4.2.1). So A has one batch, for the products, and its GKR circuit holds the Boolean checks and the LogUp fraction tree of the range checks (Section 5.4.3).

Evaluation We measure RSA-2048 at the ROM parameters, for a 2 KiB message, with the digit width optimized experimentally and the largest admissible group size (Table 4). GordieHowe's prover takes 57.8 ms and its verifier 26.7 ms, and its proof has 209.8 KiB. The baseline proves $\mathcal{R}_\mathrm{RSA}$ in Longfellow's architecture (Section 2.7), with its arithmetic over $\mathbb{F}_{p_{128}}$. Its prover takes 641.7 ms and

its verifier 337.8 ms, and its proof has 314.7 KiB. GordieHowe's prover is 11.1× faster and its verifier 12.7× faster, and its proof is 1.50× smaller. SHA-256 in H takes 26.6 ms of GordieHowe's prover time.

With A over $\mathbb{F}_{p_{128}}$ instead, and its digit width likewise optimized experimentally, the prover takes 61.7 ms and the verifier 30.3 ms, and the proof has 253.1 KiB. The quadratic extension of $\mathbb{F}_{p_G}$ gives a 1.07× faster prover, a 1.13× faster verifier and a 1.21× smaller proof. The prime field $\mathbb{F}_{p_{128}}$ has no subfield into which ring switching could pack the digits, so its tableau for A has 53 rows, against 37 over the quadratic extension of $\mathbb{F}_{p_G}$.

7.3 SHA-256 preimages

We construct a proof for the relation

$$\mathcal{R}_{\mathrm{SHA}} = \big\{(d, m) : \text{SHA-256}(m) = d\big\},$$

for messages m of a fixed public length, with SHA-256 as specified in [Nat15a]. The program has one field, the binary field H. SHA-256 is a library function written over the core language, and the RSA proof and both mDLs call it.

The program commits SHA-256's state so that every predicate is a product of two affine expressions plus an affine expression. In every round, the program commits the new values of the working variables a and e, and it reads the other six working variables from the values of a and e in the three previous rounds or from the chaining value. The rotations, shifts and XORs of Σ_0, Σ_1, σ_0 and σ_1 are affine maps on the bits. They enter the predicates as affine expressions and commit no values. The program commits Ch and Maj in every round, each with one product: $\mathrm{Ch} = e(f \oplus g) \oplus g$ and $\mathrm{Maj} = (a \oplus c)(b \oplus c) \oplus c$. For each addition, it commits the sum $s = x + y \bmod 2^{32}$, whose carries are $s \oplus x \oplus y$. One predicate of degree 2 per bit states that each carry is the majority of the two operand bits and the carry from the bit below. A sum of k words commits $k - 2$ partial sums. For a private message length, as in RSA, the library compresses a buffer of a fixed number of blocks and selects the digest with one-hot flags. Every predicate of a round has degree 2, so the compiler dispatches the predicates of all rounds to one batched zerocheck (Section 5.4.2). For $\mathcal{R}_{\mathrm{SHA}}$, the check that the digest equals d is affine, so the compiler assigns it to the linear test, and H has no GKR circuit.

Evaluation We measure SHA-256 on an 8 KiB message, 129 blocks after padding (Table 4). At the ROM parameters, GordieHowe proves it in 60.6 ms and verifies it in 20.9 ms, with a 123.0 KiB proof. Longfellow proves the same relation with its SHA-256 circuit in 1435.5 ms and verifies it in 714.2 ms, with a 204.3 KiB proof. GordieHowe's prover is 23.7× faster, its verifier 34.1× faster and its proof 1.66× smaller. At the QROM parameters, these factors are 25.1×, 34.9× and 1.48×.

Our program commits more words than Longfellow's circuit. For each block, Longfellow's circuit commits 184 words: the 48 scheduled words, the words a and e of each round, and the 8 words of the chaining value. Its GKR circuit then computes Ch, Maj and the additions. Our program also commits Ch, Maj and the partial sums, 728 words per block in all. This keeps every constraint at degree 2, so the compiler can batch them.

At the ROM parameters, ring switching packs 128 bits into each committed element of $\mathbb{F}_{2^{128}}$, and Longfellow's bit plucker packs four bits into each committed element of $\mathbb{F}_{2^{16}}$. Our program therefore commits 182 elements of $\mathbb{F}_{2^{128}}$ per block for its 728 words, against 1,472 elements of $\mathbb{F}_{2^{16}}$ for Longfellow's 184 words. For these words, the two commit about the same number of bits, 23,296 against 23,552 per block, because the bit plucker spends an element of 16 bits on every four bits.

Longfellow's circuit also commits each message bit as its own element of $\mathbb{F}_{2^{16}}$. Our program packs the message bits as it packs the words, so Longfellow's circuit commits more bits per block once the message is counted. We also wrote Longfellow's SHA-256 circuit in our language, where the compiler sends it to H's GKR circuit. For long messages, the extra words make our proof larger than this GKR version's but keep our prover faster. For 64-byte messages at the ROM parameters, the GKR version's proof is 26% larger than our batched program's and its prover 3.3× slower. For 2 KiB messages, the GKR version's proof is 6% smaller than our batched program's, but its prover is 10.7× slower.

7.4 Proofs of possession of ML-DSA-44 signatures

We construct a proof for the relation

$$\mathcal{R}_{\text{ML-DSA}} = \big\{ \big(pk, (m, \sigma)\big) : \mathsf{ML\text{-}DSA.Verify}(pk, m, \sigma) \big\},$$

for ML-DSA-44 with the empty context [Nat24, Alg. 3] and messages of at most n_m bytes, for a fixed public maximal length n_m. Our measurements use $n_m = 32$. In ML-DSA.Verify, each call of RejNTTPoly in ExpandA reads at most 1008 bytes of SHAKE128 output, and SampleInBall reads at most 272 bytes of SHAKE256 output. Both caps comply with FIPS 204 [Nat24, App. C, Table 3]. The companion paper [SWPG26] describes the construction.

The program has three parts. Over H, it computes the SHAKE256 hashes of verification, among them μ, and SampleInBall. Over A, it computes the polynomial arithmetic of verification. A MAC links the values that both fields use (Section 4.1.4).

Realization At the ROM parameters, the compiler proves the rounds of Keccak-f in one batched zerocheck over H and the Boolean checks over A in one batched zerocheck over A. Each field has one GKR circuit for its other predicates, with 15 layers over H and 16 over A. The tableau of H has $(n_{\text{RS}}, k_{\text{RS}}) = (3392, 282)$ and holds 4,143 committed elements. The tableau of A has $(n_{\text{RS}}, k_{\text{RS}}) = (2560, 512)$ and holds 13,333 committed elements.

Evaluation Our baseline modifies the ML-DSA-44 verification in Longfellow's tests, for two reasons. First, Longfellow's test implementation places the entire ML-DSA computation over $\mathbb{F}_{q^6}$, for the ML-DSA modulus q, including the hashing, and it does not use subfield rows over $\mathbb{F}_{q^6}$. Its prover and verifier are therefore slower, and its proofs larger, than they would be with optimizations that Longfellow uses elsewhere in its repository. Our baseline proves the hashing over $\mathbb{F}_{2^{128}}$, links the two fields with Longfellow's per-chunk MAC, with a committed blind added to each tag so that a zero chunk stays hidden (Section 4.1.4), and uses subfield rows over $\mathbb{F}_{q^6}$. Second, Longfellow's test implementation takes the message representative μ as a private input and does not check that it is computed from the message. With the empty context, μ is the 64-byte output of SHAKE256 on $tr \,\|\, \mathsf{0x00} \,\|\, \mathsf{0x00} \,\|\, m$ [Nat24, Algs. 3 and 8]. A proof of possession, in which m is private, must compute μ inside the proof, so our baseline does so as well, to match the relation of our program. At the ROM parameters, these changes make its prover 1.50× faster, its verifier 1.51× faster and its proof 1.92× smaller than Longfellow's test circuit extended to compute the message representative inside the proof, at the same rate and per-round target. Our comparison therefore understates GordieHowe's advantage over that circuit. The baseline's SampleInBall reads 136 bytes of SHAKE256 output, below 221 bytes, the smallest limit that FIPS 204 allows [Nat24, App. C, Table 3]. Reading less output also favors the baseline. At the QROM parameters, the baseline uses $\mathbb{F}_{q^{11}}$ and $\mathbb{F}_{2^{256}}$ (Section 7.1).

System	Soundness (bits)	P (ms)	V (ms)	KiB	against Baum–Beckmann P	V	Proof
Baum–Beckmann (seq)	≈ 128	1761.0	897.9	988.2			
GordieHowe (seq)	114.0	62.0	31.9	213.5	28.4×	28.1×	4.63×
	236.3	146.8	72.1	549.2	12.0×	12.5×	1.80×
Baum–Beckmann (par)	≈ 128	1426.0	557.0	988.2			
GordieHowe (par)	114.0	34.4	18.5	213.5	41.5×	30.1×	4.63×
	236.3	87.3	46.8	549.2	16.3×	11.9×	1.80×

Table 5: GordieHowe against the proof of Baum and Beckmann [BB26] on $\mathcal{R}_{\text{ML-DSA}}$ with $n_m = 32$. The soundness column gives $-\log_2$ of an error bound. For Baum and Beckmann, it is their union bound over the checks of the whole protocol, with grinding, in the ROM [BB26, §4]. For GordieHowe, it is the largest round error at the ROM parameters (first row) and at the QROM parameters (second row), as in Table 7. The (seq) rows run on one core and the (par) rows on four, and each GordieHowe row is compared with the Baum–Beckmann row of the same thread count. Times are medians, and the prover time P includes witness generation for both systems (Section 7). Baum and Beckmann's proof size is the size their implementation computes for the proof's encoding, and their verifier time includes no proof parsing, since their implementation has no serializer. Baum and Beckmann's rows come from a separate session on the same hardware (AMD EPYC 9754), with medians over two blocks of 9 trials, each after 2 warm-ups. GordieHowe's rows are those of Table 4, and the (par) rows repeat the proof size of the (seq) rows.

At the ROM parameters, GordieHowe proves the relation in 62.0 ms and verifies it in 31.9 ms, single-threaded, with a 213.5 KiB proof (Table 4). Our baseline takes 2026.0 ms and 1194.6 ms, with a 380.7 KiB proof. GordieHowe's prover is 32.7× faster, its verifier 37.4× faster and its proof 1.78× smaller. At the QROM parameters, GordieHowe takes 146.8 ms and 72.1 ms with a 549.2 KiB proof, against 6129.6 ms, 3268.2 ms and 777.8 KiB for the baseline, so its prover is 41.8× faster, its verifier 45.3× faster and its proof 1.42× smaller.

The batched zerocheck also proves the Keccak permutations of this relation faster than a GKR circuit does. We prove the permutations alone over H, at the same rate and per-round target, each with the fewest queried columns that meet the target. At the ROM parameters, the batched permutations then have a 17.1× faster prover and a 20.3× faster verifier than a GKR circuit that commits the state every sixth round, and an 11% larger proof. At the QROM parameters, the factors are 15.2× and 22.0×, and the proofs are within 1% in size.

We also compare with the concurrent work of Baum and Beckmann [BB26], which proves knowledge of an unmodified ML-DSA-44 signature with VOLE-in-the-head [BBD+23b]. We measured their implementation on our host, on the relation $\mathcal{R}_{\text{ML-DSA}}$ with $n_m = 32$ (Table 5). Their soundness target of $\approx 2^{-128}$ is a union bound over the checks of their whole protocol, with grinding [BB26, §4]. It is stronger than our per-round target at the ROM parameters and weaker than ours at the QROM parameters, so we compare at both. At these parameter sets, Theorem 6.7 gives 101 bits of extraction security against classical adversaries and 116 bits against quantum adversaries (Table 7). At the ROM parameters, GordieHowe's prover is 28.4× faster on one core and 41.5× faster on four, its verifier is 28.1× and 30.1× faster, and its proof is 4.63× smaller. At the QROM parameters, its prover is 12.0× and 16.3× faster, its verifier is 12.5× and 11.9× faster, and its proof is 1.80× smaller.

Section 7.6 compares this proof with Binius64.

7.5 Mobile driving licenses

We construct a proof for the relation

$$\mathcal{R}_{\mathrm{mDL}} = \big\{\big((\mathsf{now}, (id, v), pk_I, m_D),\ (\mathsf{MSO}, \mathit{off}, I, \sigma_I, pk_D, \sigma_D)\big) : \text{the five conditions below hold}\big\}$$

for an ISO 18013-5 mobile driving license [ISO21] with one requested attribute. It generalizes the mDL presentation relation [Fs24] of Longfellow version 7 [Goo26a] to an arbitrary signature scheme Sig. The statement is the time now, the requested attribute (id, v) of an identifier and a value, the issuer's public key pk_I, and the device-authentication message m_D. The witness is the mobile security object MSO of private length, the offsets *off* of its fields, the item I of the requested attribute, the issuer's signature σ_I, the device's public key pk_D and the device's signature σ_D. The conditions are:

1. $\mathsf{Sig.Verify}(pk_I, M, \sigma_I)$ for M the COSE prefix followed by MSO [Sch22, §4.4], with MSO of at most n_{MSO} bytes, for a fixed public maximal length n_{MSO}.
2. $\mathsf{MSO}[\mathsf{validFrom}] \le \mathsf{now} \le \mathsf{MSO}[\mathsf{validUntil}]$.
3. $\mathsf{MSO}[\mathsf{deviceKeyInfo}] = pk_D$ and $\mathsf{Sig.Verify}(pk_D, m_D, \sigma_D)$.
4. SHA-256(I) has a window in MSO, $I[\mathsf{elementIdentifier}] = id$ and $I[\mathsf{elementValue}] = v$.
5. Each window lies inside MSO, and the windows are pairwise disjoint.

A *window* is a byte string of MSO at an offset in *off*. The window of validFrom, validUntil or deviceKeyInfo holds the key's encoding followed by a value, which $\mathsf{MSO}[key]$ denotes. The window of valueDigests holds only the key's encoding, and the window of the digest of I holds the digest's CBOR encoding [BH20]. As in Longfellow version 7 [Goo26a], the relation locates these fields by their encodings and does not parse the CBOR map of MSO. Every honest MSO meets condition 5.

7.5.1 With ECDSA

We instantiate Sig with ECDSA [Nat23] over P-256 [CMR+23] with SHA-256 [Nat15a], with public keys $pk_I = Q_I$ and $pk_D = Q_D$. This gives the relation [Fs24] of Longfellow version 7 [Goo26a] with two differences. Longfellow version 7 also requires the issuer's digest and the coordinates of Q_D to be below the order of P-256, so it rejects some valid signatures and keys that GordieHowe accepts. Longfellow version 7 checks only that each window starts inside MSO and does not require the windows to be disjoint, so it accepts some witnesses that condition 5 rejects. We take $n_{\mathsf{MSO}} = 2{,}533$, as Longfellow does. The program proves the hashing and the parsing of MSO over H, and both signatures over $A = \mathbb{F}_{p_{256}}$, the base field of P-256.

The program proves the issuer's digest $e = \text{SHA-256}(M)$ with the private-length SHA-256 of Section 7.3, over a buffer of 40 blocks. The item I has two blocks, and its SHA-256 joins the same batch.

Each window lies in MSO at a private offset. Longfellow proves each window by shifting all of MSO left by the offset and comparing the first bytes with the window. The shift proceeds in rounds of multiplexers, each of which consumes three bits of the offset and selects among eight shifts. For an MSO of n_{MSO} bytes and a window of ℓ bytes, this costs on the order of $n_{\mathsf{MSO}} + \ell \log n_{\mathsf{MSO}}$ multiplexer terms per window. With one requested attribute, the program reads five windows: validFrom, validUntil, deviceKeyInfo, valueDigests and the digest of I. We prove all five windows at once with one filtered permutation check (Section 3.3). In characteristic 2 the integers $0, \ldots, n_{\mathsf{MSO}} - 1$ do not map to distinct field elements, so over H position i is encoded as g^i for a generator g of

the multiplicative group of $\mathbb{F}_{2^{16}}$. The left list holds a record $(g^i, \mathsf{MSO}_i, f_i)$ for each position i at which a window can lie, where the flag f_i is 1 exactly when i is below the private length of MSO. A committed bit marks the records of the bytes inside windows. The right list holds the records $(z_w g^j, B_{w,j}, 1)$ of the windows' bytes $B_{w,j}$, where the program commits $z_w = g^{o_w}$ for the private offset o_w of window w. If the marked records of the left list are a permutation of the right list, each window lies at its offset inside MSO, and the windows are disjoint. For a left list of n_{rec} records, the check costs $2n_{\text{rec}} + \sum_w \ell_w - 2$ multiplications for all windows together: n_{rec} for the filter, and $n_{\text{rec}} - 1$ and $\sum_w \ell_w - 1$ for the two product trees. As in Longfellow, a window starts at byte 7 of MSO or later and ends before byte n_{MSO}, so the left list has $n_{\text{rec}} = 2{,}526$ records, for the positions 7 to 2,532.

The program checks both signatures in the standard form $X = [s^{-1}]([e]G + [r]Q)$, with $x(X)$ mod $n = r$ for the order n of G. Multiplications of the public points G and Q_I by private scalars use tables of their multiples. G's table is fixed, and the verifier computes Q_I's from the statement. Multiplications of private points use doublings and additions: of Q_D by r in the device's signature, and of $[e]G + [r]Q$ by s^{-1} in both signatures. Longfellow instead checks each signature as $[e]G + [r]Q - [s]R = O$, with the signature's point R as a witness. The digest e and the coordinates of Q_D are needed in both fields, 768 bits in all, and three link predicates connect their two copies (Section 4.1.4).

Realization The compiler batches the SHA-256 rounds of MSO and of I, together with the selection of each digest from the chaining values, into one batched zerocheck, as in Section 7.3. The window check, the padding checks, the date comparisons and most of the attribute check go to H's GKR circuit. The attribute check's predicates that fix committed bits to constants are affine, and the compiler assigns them to the linear test as 84 rows. Over $\mathbb{F}_{p256}$, the compiler dispatches the predicates of the signatures to A's GKR circuit, as Longfellow does, except six affine predicates, which it assigns to the linear test. The three link predicates share one MAC over $\mathbb{F}_{2^{128}}$ (Section 4.1.4). Over A, Ligero's linear test proves the MAC's check, a convolution of the committed bits by public matrices, and 256 lookups into a table of 39 entries range-check the quotient digits of its reduction modulo 2.

Evaluation We measure the mDL at the ROM parameters, with one attribute (Table 4). GordieHowe proves the presentation in 116.4 ms and verifies it in 65.0 ms, single-threaded, with a 275.5 KiB proof. Longfellow's prover takes 657.3 ms and its verifier 318.6 ms, with a 352.0 KiB proof. GordieHowe's prover is 5.6× faster, its verifier 4.9× faster and its proof 1.28× smaller. On four cores, GordieHowe's prover is 10.1× faster and its verifier 8.3× faster than Longfellow's, which runs single-threaded. These times include the statement's public computations, the table of Q_I and the point $[e_2]G$ for the digest $e_2 = \text{SHA-256}(m_D)$, for both the prover and the verifier.

Most of the difference is in the hashing. Over H, GordieHowe proves in 38.4 ms and verifies in 17.5 ms, with 96.9 KiB of the proof, against 487.9 ms, 212.0 ms and 157.3 KiB for Longfellow's hash circuit. Over $\mathbb{F}_{p256}$, GordieHowe takes 74.4 ms and 46.2 ms, with 169.3 KiB, against 167.0 ms, 105.2 ms and 194.5 KiB for Longfellow's signature circuit. GordieHowe's prover is therefore 12.7× faster than Longfellow's over H and 2.2× faster over $\mathbb{F}_{p256}$. Neither system's per-field prover time includes witness generation, and neither system's per-field verifier time includes proof parsing.

Vega [KS25] proves mDL presentations with Hyrax commitments [WTs+18] over the curve T-256 [FLM22]. Its soundness rests on discrete logarithms in T-256, a group of $\approx 2^{256}$ elements, so it is not post-quantum. Its prover commits once per credential to the part of the witness that does not depend on the verifier, including the credential's bytes and the issuer's signature. For each presentation, it rerandomizes these commitments and commits only the rest. Vega's measured circuit

checks MSO[validUntil] but not MSO[validFrom]. It proves an age predicate on the birth date in place of the equality of one attribute. We compare with VegaMC, Vega's faster variant [KS25, Fig. 10]. Vega's mDL circuit is not public, so we cannot run VegaMC on our host. Vega reports Longfellow's times on its own host. We ran the same Longfellow benchmark on our host, built with Longfellow's own compiler flags, with 3 warm-ups and 31 trials, and multiply Vega's reported times by the ratios of Longfellow's times on our host to its reported times on Vega's host, 1.66 for proving and 1.62 for verifying. This assumes that Vega timed Longfellow as Longfellow's benchmark does, that the ratio measured on Longfellow's field arithmetic also holds for Vega's elliptic-curve arithmetic, and that the single-threaded ratio holds at every thread count. For this comparison, we measure GordieHowe's mDL with Vega's bound of 1920 bytes on the MSO. At this bound, GordieHowe proves in 109.3 ms and verifies in 62.1 ms on one core, and in 62.3 ms and 38.2 ms on four, with a 269.6 KiB proof. Scaling Vega's reported times, we estimate that VegaMC proves in 393.8 ms and verifies in 92.2 ms on one thread, takes 174.4 ms and 40.4 ms on four, and proves in 152.8 ms on sixteen. GordieHowe's prover is thus 3.6× faster than Vega's per-presentation prover on one core and 2.8× faster on four, and on four cores it is faster than Vega's on sixteen. GordieHowe's verifier is 1.5× faster on one core and 1.1× faster on four. VegaMC's proofs, of 105.5 KiB (108 kB), are 2.6× smaller. The proofs of Vega's other variant, VegaSC, are 88.9 KiB (91 kB).

7.5.2 With ML-DSA

We instantiate Sig with ML-DSA-44 [Nat24] for both the issuer and the device, with one requested attribute. We take $n_{\mathsf{MSO}} = 3{,}780$, which allows for the larger device key. In this relation, each call of RejNTTPoly in ExpandA reads at most 1008 bytes of SHAKE128 output, and SampleInBall reads at most 272 bytes of SHAKE256 output. Both caps comply with FIPS 204 [Nat24, App. C, Table 3]. The baseline follows the construction of the ML-DSA-44 baseline in Section 7.4. Each call of RejNTTPoly in its ExpandA reads 840 bytes of SHAKE128 output and its SampleInBall 136 bytes of SHAKE256 output, below 894 and 221 bytes, the smallest limits that FIPS 204 allows [Nat24, App. C, Table 3]. Both shortfalls favor the baseline.

At the ROM parameters, GordieHowe proves the presentation in 473.7 ms and verifies it in 247.0 ms, single-threaded, with a 447.6 KiB proof (Table 4). The baseline takes 25526.5 ms and 14379.9 ms, with a 1451.5 KiB proof. GordieHowe's prover is 53.9× faster, its verifier 58.2× faster and its proof 3.24× smaller. At the QROM parameters, GordieHowe's prover is 90.4× faster, its verifier 89.8× faster and its proof 1.86× smaller. At the ROM parameters, the largest round error of this proof is $2^{-112.7}$, at the LogUp challenge of the range lookups over A, slightly above the target of $2^{-113.5}$ that the other applications meet.

At the ROM parameters, GordieHowe's prover takes 7.6× longer for the mDL than for an ML-DSA-44 proof of possession (Section 7.4). Besides the issuer's signature and the hash of the MSO, the mDL verifies the device's signature under a private key. The device's public key is private, so the program must expand the key's matrix $\hat{A}$ with ExpandA inside the proof. ExpandA makes 16 SHAKE128 calls of 6 blocks each, 96 Keccak permutations in all. A proof of possession under a public key leaves ExpandA and the hash tr of the public key to the verifier and proves 10 permutations for the remaining hashing required for verification. The expanded $\hat{A}$ must also be carried from H to A. In the baseline, $\hat{A}$ makes up 736 of the MAC's 920 chunks of 128 bits. GordieHowe batches the Keccak permutations (Section 5.4.2) and carries $\hat{A}$ with a single polynomial MAC. Over A, GordieHowe proves the MAC's check as in the ECDSA mDL (Section 7.5.1). Without the device's signature, GordieHowe's prover takes 110.1 ms and its verifier 51.4 ms, with a 247.6 KiB proof. The device's signature therefore accounts for 77% of the prover time and 45% of the proof. In the ECDSA mDL, it accounts for 23% of the prover time and 9% of the proof.

7.6 Comparison with Binius64

We compare GordieHowe with Binius64 [Irr26b] on ML-DSA-44 verification, on SHA3-256 and on SHA-256 (Table 6). Binius64 specifies a zero-knowledge mode [Irr26b, §7], and we measure it on our host. We implemented ML-DSA-44 verification in Binius64 for this comparison, and for the hashes we use Binius64's own circuits for SHA3-256 and SHA-256 of a fixed-length message [Irr26a]. Each message is private and its digest public. Each message has 1, 8 or 32 KiB. SHA-256 pads them into 17, 129 and 513 blocks. Both systems hash their Merkle trees with SHA-256. GordieHowe runs at the ROM parameters, where every round error of these proofs is at most $2^{-113.5}$. Binius64's verifier targets 96 bits of security [Irr26a]. It sets its FRI [BBHR18] query count, without grinding, so that the query phase has error at most 2^{-96} by the unique-decoding analysis of FRI-Binius [DP26]. Binius64 runs at code rate 1/2. This rate gives the smallest sum of prover and verifier times for ML-DSA-44 and, in a separate measurement, for Keccak-f permutations and for SHA-256 on messages of up to about 1 KiB. At rate 1/4, its proofs with zero knowledge are 25 to 27% smaller and its provers slower. Its verifier times exclude the setup that it computes once per circuit, as GordieHowe's exclude compilation.

We compare GordieHowe primarily against Binius64 with zero knowledge. GordieHowe's advantage over it is largest for small workloads. On one core, for SHA3-256 and SHA-256 on 1 KiB, GordieHowe's prover is 27.7× and 28.0× faster, its verifier 48.0× and 69.7× faster, and its proof 6.67× and 6.15× smaller. The advantage shrinks as the workload grows, because most of Binius64's cost for zero knowledge is an outer proof of its verifier, and that proof grows little with the workload. On 8 KiB, GordieHowe's prover is 5.6× faster for SHA3-256 and 6.4× faster for SHA-256, its verifier 10.7× and 14.7× faster, and its proof 3.80× and 3.42× smaller. On 32 KiB, GordieHowe remains faster at both thread counts. On one core, its prover is 2.0× and 2.2× faster, and its verifier 3.9× and 4.6× faster. On four cores, its prover is 3.1× and 4.0× faster, and its verifier 11.2× and 11.5× faster. Its proof is 2.30× and 2.06× smaller. For ML-DSA-44, GordieHowe's prover is 9.9× faster on one core and 12.4× on four, its verifier 12.2× and 20.9× faster, and its proof 2.53× smaller. Even against Binius64 without zero knowledge, GordieHowe's ML-DSA-44 prover is 1.6× faster, on one core and on four, and its proof is 1.47× smaller.

7.7 Soundness and parameters

We evaluate each figure from Theorem 6.7 at the parameters of each application's realized program: its tableaus, circuits, batches, tables and challenges, and the lengths of the transcript's digests, salts and streams. The largest round error is the maximum over the rounds of Theorem 5.3(2). For an error bound $\varepsilon(t)$ against adversaries that make t hash queries, the bit security of ε is $\lambda(\varepsilon) = \min_{t\geq 1} \log_2\bigl(t/\min(1,\varepsilon(t))\bigr)$. The extraction column gives $\lambda(\varepsilon_{\mathrm{ex}}^{\mathrm{C}})$ at the ROM parameters and $\lambda(\varepsilon_{\mathrm{ex}})$ at the QROM parameters (Theorem 6.7(2)). At the ROM parameters the minimum is at $t = 1$, so the extraction column lies about $\log_2(1 + k_q)$ bits below the round-by-round column. At the QROM parameters the extraction column is about $(-\log_2 \varepsilon^{\mathrm{k}} - \log_2 6)/2$. The zero-knowledge column gives the bit security of everlasting zero knowledge (Definition 6.2), for one proof. Its errors are $\varepsilon_{\mathrm{zk}}^{\mathrm{C}}$ at the ROM parameters and $\varepsilon_{\mathrm{zk}}$ at the QROM parameters (Theorem 6.7(3)), with $\varepsilon_{\mathrm{c}} = 0$.

Zero knowledge for Q proofs loses $\log_2 Q$ bits. We estimate that leaf salts 20 bytes longer at the ROM parameters, which add at most 3,420 bytes to each proof in our applications, keep 128 bits at $Q = 2^{64}$.

					against Binius64		
Application	System	P (ms)	V (ms)	KiB	P	V	Proof
ML-DSA-44	Binius64 (par)	427.4	386.7	541.0			
	GordieHowe (par)	34.4	18.5	213.5	12.4×	20.9×	2.53×
	Binius64 (seq)	619.3	388.6	541.0			
	GordieHowe (seq)	62.0	31.9	213.5	10.0×	12.2×	2.53×
SHA3-256, 1 KiB	Binius64 (par)	168.9	166.0	397.9			
	GordieHowe (par)	5.0	2.3	59.8	33.7×	73.3×	6.65×
	Binius64 (seq)	253.3	161.0	397.9			
	GordieHowe (seq)	9.2	3.4	59.6	27.7×	48.0×	6.67×
SHA3-256, 8 KiB	Binius64 (par)	191.2	196.1	419.9			
	GordieHowe (par)	20.7	7.4	110.5	9.2×	26.6×	3.80×
	Binius64 (seq)	289.8	189.8	419.9			
	GordieHowe (seq)	51.9	17.8	110.5	5.6×	10.7×	3.80×
SHA3-256, 32 KiB	Binius64 (par)	227.1	257.7	434.5			
	GordieHowe (par)	73.7	23.1	189.1	3.1×	11.2×	2.30×
	Binius64 (seq)	401.3	254.3	434.5			
	GordieHowe (seq)	198.3	64.4	188.9	2.0×	3.9×	2.30×
SHA-256, 1 KiB	Binius64 (par)	269.9	289.0	398.0			
	GordieHowe (par)	7.2	2.8	64.5	37.5×	103.3×	6.17×
	Binius64 (seq)	360.9	286.9	398.0			
	GordieHowe (seq)	12.9	4.1	64.7	28.0×	69.7×	6.15×
SHA-256, 8 KiB	Binius64 (par)	281.7	312.5	420.0			
	GordieHowe (par)	23.7	9.1	122.8	11.9×	34.5×	3.42×
	Binius64 (seq)	388.1	309.3	420.0			
	GordieHowe (seq)	60.4	21.0	122.8	6.4×	14.7×	3.42×
SHA-256, 32 KiB	Binius64 (par)	307.2	334.8	434.6			
	GordieHowe (par)	77.3	29.1	211.5	4.0×	11.5×	2.05×
	Binius64 (seq)	486.7	350.1	434.6			
	GordieHowe (seq)	221.9	75.8	211.2	2.2×	4.6×	2.06×

Table 6: GordieHowe against Binius64 with zero knowledge. GordieHowe runs at the ROM parameters and Binius64 at its fixed 96-bit parameters, both with SHA-256 Merkle hashing. The (seq) rows run on one core and the (par) rows on four, and each GordieHowe row is compared with the Binius64 row of the same thread count. Times are medians of 9 trials, and the prover time P includes witness generation for both systems (Section 7). Binius64's verifier is single-threaded.

		Ligero, H / A		bits		
Application	Parameters	Rate	Queried columns	Round-by-round	$\lambda(\varepsilon_{\text{ex}})$	$\lambda(\varepsilon_{\text{zk}})$
RSA-2048	ROM	1/12 / 1/5	63 / 96	114.2	101.9	116.3
ECDSA mDL	ROM	1/12 / 1/4	63 / 108	113.9	101.2	116.5
SHA-256, 8 KiB	ROM	1/12 / –	64 / –	115.2	105.1	115.9
	QROM	1/12 / –	162 / –	236.1	116.7	179.6
ML-DSA-44	ROM	1/12 / 1/5	63 / 96	114.0	101.3	116.4
	QROM	1/12 / 1/6	128 / 175	236.3	116.8	180.1
ML-DSA mDL	ROM	1/12 / 1/5	64 / 96	112.7	99.6	115.1
	QROM	1/12 / 1/6	131 / 229	236.0	116.7	178.5

Table 7: Ligero parameters and security of each application. Rate and queried columns are given for the hashing field H / the algebra field A. The round-by-round column gives $-\log_2$ of the largest round error. The last two columns give the bit security of extraction, and of everlasting zero knowledge for one proof (Section 7.7). Every figure assumes that the source program is complete for its relation with error 0 and round-by-round knowledge sound with the errors of its challenges (Theorem 6.7). Lean proves both only for the proof of possession of ML-DSA-44 (Section 8).

7.8 Ablations

We measure each technique by turning it off and keeping everything else, at the ROM parameters on one core. For packing, we compare the ML-DSA-44 proof of possession of Table 4 against two variants. The first commits each bit of H as one element of $\mathbb{F}_{2^{16}}$, in place of sixteen bits per element. Its tableau over H grows from 30 to 77 rows, so each opened column is longer. Each row encodes 842 values in place of 282, so the rows that the prover sends in the linear and quadratic tests are also longer. The proof is 1.45× larger, the prover 5.8× slower and the verifier 4.6× slower. The second commits every subfield value as one entry of Ligero's subfield rows, as Longfellow does, with no packing in either field. Its proof is 1.43× larger, its prover 9.7× slower and its verifier 8.6× slower. Unpacked bits each need a Boolean check, and their Keccak rounds are evaluated on elements of $\mathbb{F}_{2^{16}}$, so the times of both variants include this work. Both variants keep the full system's choice of subprotocols. Turning off the univariate skip in the batched zerocheck makes the prover 3.4× slower for SHA3-256 on 32 KiB and 1.9× slower for SHA-256 on 8 blocks, and changes the proof size by less than 1% and the verifier time by at most 2%.

Dropping the mask test (Section 5.6.2) makes the proofs 30% larger for the ML-DSA-44 proof of possession and for the ECDSA mDL. In these measurements, the compiler chooses its tableaus anew without the mask test and keeps each field's number of low-degree tests. In the proof of possession, the queried columns grow from 63 to 87 over $\mathbb{F}_{2^{128}}$ and from 96 to 168 over $\mathbb{F}_{p_{\text{G}}^2}$. In the ECDSA mDL, they grow from 63 to 87 over $\mathbb{F}_{2^{128}}$ and from 108 to 216 over $\mathbb{F}_{p_{256}}$.

Replacing our polynomial MAC with Longfellow's per-chunk MAC (Section 2.7) on 128-bit chunks makes GordieHowe's proof 1.05× larger, its prover 1.7× slower and its verifier 1.7× slower in the ECDSA mDL, which shares 6 chunks. In the ML-DSA-44 proof of possession of Table 4, which shares 52 chunks, the proof is 1.09× larger, the prover 6.2× slower and the verifier 5.9× slower. In the ML-DSA mDL, which shares 920 chunks, the proof is 1.18× larger, the prover 13.4× slower and the verifier 12.3× slower. As in Longfellow, the circuit over A emulates the product in $\mathbb{F}_{2^{128}}$ of each tag. The tags reveal which chunks are zero (Section 4.1.4), so this variant is not zero knowledge, and we measure it only for comparison. Removing the MACs reduces the prover time of Longfellow's

ECDSA mDL by 13.4% and of our ML-DSA-44 baseline by 51.0%.

8 Formalization

We formalized the security analysis of GordieHowe in Lean 4 [dMU21] with Mathlib [The20]. Appendix J lists the Lean declarations of the machine-checked results, grouped by topic. Our security theorems hold for a Lean model of GordieHowe (Section 8.2) under the assumptions of Section 8.3.

We formalized the analysis to increase confidence in the security of GordieHowe. Our security argument composes reductions for packing, batching, GKR, Ligero and the Fiat–Shamir transformation, over several fields. Security proofs in the QROM are harder than their classical counterparts [BDF+11]. An adversary's queries cannot be copied, so classical techniques such as recording queries do not carry over [Zha18]. Unruh observes that intuition from the classical world makes it easy to err about quantum phenomena, and for this reason he advocates computer-aided verification [Unr20, p. 2]. Published QROM proofs have contained gaps [DFMS21, App. A] [BBD+23a].

8.1 Results

The formalization proves two kinds of results. The first kind holds for every relation. For every series–parallel IOP, it proves the guarantees of the spBCS transformation in the ROM and the QROM (Section 6). It proves the round-by-round soundness of Ligero at the Johnson radius with the mask test (Section 5.6.2), and the security theorem of the proof system (Theorem 5.3) for every valid realized program. One proof of Theorem 5.3 covers the subprotocols, ring switching and Ligero of every field, every admissible packing tree, any number of GKR circuits and batches, and the quadratic test. To prove completeness, it derives the honest prover of the realized program from an accepting execution of the source program, as Lemma 5.7(2) does. Together, these results give the guarantees of Theorem 6.7 for every valid realized program that passes the exported checks and whose source program is complete and round-by-round knowledge sound for its relation. The rows for Theorem 5.3 in Appendix J state the restrictions of its Lean statement.

The second kind concerns particular programs. Lean proves that the source program of the proofs of possession of ML-DSA-44 (Section 7.4) is complete and round-by-round knowledge sound for its verification relation, at both parameter sets. For this application the guarantees of Theorem 6.7 are therefore machine-checked from the source program to the non-interactive argument. For each formalized application, Lean also evaluates the round errors and the error bounds at the exported parameters. We also proved the round-by-round knowledge soundness of Ligero without the mask test in Longfellow's order of challenges (Lemma G.12), and Lean evaluates its round errors at the parameters of Longfellow version 7 for the mDL proof with one attribute.

The library is generic in the fields, the packing and the parameters. Each formalized application instantiates the proof of Theorem 5.3 once, with its validity and parameter facts decided by the kernel. Adding an application therefore requires its field models, its relation, its exported program and, for an end-to-end result, proofs that its source program is complete and round-by-round knowledge sound.

Along the way we formalized results from the literature that our theorems use: Zhandry's compressed oracle [Zha18, Lems. 3–5], the compressed-oracle and BCS lemmas of Chiesa, Manohar and Spooner [CMS19], the one-way-to-hiding lemma of Ambainis, Hamburg and Unruh [AHU18, Thm. 3] for pure and mixed adversaries, the interleaved proximity lemma of Ames et al. [AHIV22, Lem. 4.2] jointly over its repetitions, and Berlekamp–Welch decoding [GRS25, §12.1]. Our analysis of Ligero at the Johnson radius uses the mutual correlated agreement theorem of Dao, Kominers and Thaler [DKT26, Thm. 5.12], and we take its formalization from ArkLib [Ark26a]. At unique decoding,

the analysis uses the correlated agreement theorem of Ben-Sasson et al. for affine spaces [BCI+20, Thm. 1.6]. ArkLib formalizes this theorem for lines, and we prove the form for affine spaces from the form for lines.

8.2 The model

A quantum algorithm in our model consists of a density matrix on finite registers and of completely positive trace-non-increasing maps, one before each oracle query and one after the last. A query applies the unitary $|u, y\rangle \mapsto |u, y \oplus \mathsf{H}(u)\rangle$ of Definition 6.1, from Chiesa, Manohar and Spooner [CMS19, §3.2], and the output is the work register measured in the computational basis. These algorithms are the mixed adversaries of Heidler and Unruh [HU25, Def. 6.3]. A map may lose trace, which models an algorithm that does not halt. Our registers are finite, which the analyses we build on assume implicitly [AHU18, CMS19]. This is the only departure from Heidler and Unruh's model, whose registers may be infinite-dimensional.

As in Unruh's definition of zero knowledge [Unr17, Def. 6], an adversary decides from its earlier measurement outcomes whether its next call is a hash query or a proof query. We translate its interaction with the classical challenger into a single quantum algorithm, and we prove that every quantum algorithm has a unitary query algorithm with the same output law. The lemmas of Chiesa, Manohar and Spooner [CMS19] are stated for unitary query algorithms, and the proofs of the compressed oracle [Zha18, §3] and of the one-way-to-hiding lemma [AHU18, Lem. 8] work with unitary queries on a purified state.

Paper proofs in the QROM often assume without loss of generality that the adversary is pure. That assumption fails for adversaries whose calls to a classical oracle depend on intermediate measurements [ABKM21, p. 4]. Our zero-knowledge and simulation-extraction games contain such calls, because in them the adversary interleaves proof queries with hash queries. We therefore state our theorems for general adversaries (Definition 6.1). The proofs purify an adversary only after they fold the classical parties into it, and purification is valid at that point [AHU18, pp. 8–9].

Because the honest parties are classical oracle programs, one prover and one simulator serve every guarantee of an application, in both models. A quantum register holds finitely many values, so the prover's proofs are those whose format fits a bound, and each application proves that its honest proofs fit. Our online extractors (Definition H.3) use lazy sampling in the ROM and the compressed oracle in the QROM, both with oracle-simulation error 0. In the QROM, the Lean adversary draws a classical auxiliary input, independent of the oracle, and its work register may depend on it. Our definition of knowledge soundness bounds the trace distance at each auxiliary input and averages over it. This implies Definition H.3, and Lemma H.5 uses only the outcomes that Definition H.3 exposes.

Every theorem of the formalization names its extractor and its simulator. The simulators, generic and of each configuration, and the classical part of each extractor are computable definitions, and the simulators read no witness. In the QROM, the extractor also simulates the compressed oracle, a quantum algorithm that Lean defines over Mathlib's complex matrices. We argue its polynomial running time on paper, after Theorem I.17. An extractor or a simulator that a theorem only asserts to exist could choose a witness by the axiom of choice. Zero knowledge would then hold trivially, and knowledge soundness would reduce to soundness.

8.3 Assumptions

The Lean kernel checks every proof. The formalization declares no axioms, and no proof uses `native_decide`, which would make a proof trust Lean's compiler as well as its kernel. Our compiler

exports each realized program as Lean definitions, generated from the compiled object that the prover and the verifier use, together with certificates. The kernel checks from the certificates that the exported program is valid (Definition 4.8) and realizes its source program, so Theorem 5.3 applies to it. These checks cover the compiler's dispatch of the equalities, its range tables, its other realization choices and the Ligero parameters. The realized programs of all our applications pass them.

Beyond the kernel, the guarantees of a deployed proof rest on assumptions about the GKR circuits and on four further assumptions. We do not export the GKR circuits that Longfellow's circuit compiler produces, and the formalization makes three assumptions about each compiled circuit. Every input vector that has the verifier's public inputs and satisfies the circuit satisfies the predicates, Boolean checks, LogUp identities and MAC equations assigned to the circuit. The circuit accepts the honest input whenever these hold. The circuit has at most the number of layers listed for it, and each of its layers, including its output layer, has at most 2^w wires, for the least w such that 2^w is at least the listed numbers of its wires and of its inputs. The GKR error of Lemma 5.9 is therefore at most its value at n_{lay} equal to the listed number of layers and at $m_j = w$ for every j.

The four further assumptions are the following. First, the export records the realized program that the prover and the verifier use. Second, the C++ prover and verifier implement the Lean model. Third, the source-program theorems use Lean specifications of ML-DSA, SHAKE128 and SHAKE256 adapted from the VCVio library [Ver26], and we assume that they follow FIPS 204 [Nat24] and FIPS 202 [Nat15b]. Known-answer tests check SHAKE128 and SHAKE256. The ML-DSA-44 specification accepts a signature produced by OpenSSL, and its ExpandA and SampleInBall agree with the test vectors of our C++ reference implementation. Fourth, the running times of the extractors and simulators are argued on paper.

8.4 Problems in published analyses

While formalizing the analysis of Ligero, we found the gap in the published soundness analyses of its masked tests by Ames et al. [AHIV22] and Cascudo et al. [CHVY25] that Section 5.6.1 describes, with its machine-checked counterexamples and the replacement bound.

Longfellow's leaf salts have the length of the digest [Goo26a]. Frigo and shelat state statistical zero knowledge, with an error proportional to $2^{-\lambda/4}$ for a security parameter λ, for Ligero compiled with the transformation of Ben-Sasson, Chiesa and Spooner, and they attribute this theorem to Ames et al. [Fs24, Thm. 4, pp. 8–9]. Ames et al. state no such bound. They remark that this transformation preserves statistical zero knowledge [AHIV22, §5.2]. The error of Frigo and shelat has the form of the privacy bound of Ben-Sasson, Chiesa and Spooner for salted Merkle trees, $n \cdot 2^{-\lambda/4+2}$ for n leaves, and this bound assumes salts of twice the security parameter [BCS16, §3.2, Lem. 3.4]. For λ-bit digests and salts, the expected statistical distance of an unopened leaf from uniform, over the random oracle, is $(1 - 2^{-\lambda})^{2^\lambda}$, close to $1/e$, so statistical zero knowledge does not follow. This gap concerns the statistical form of the guarantee. It gives no attack on Longfellow's proofs by an adversary with a bounded number of hash queries. We found this gap while formalizing the zero-knowledge proof of the spBCS transformation. Our compilation uses leaf salts of twice the digest length.

9 Related work

We compare GordieHowe with Longfellow throughout the paper, on each application (Section 7) and in the analysis of Ligero (Section 5.6). The body also relates each technique to its closest prior work: late values (Section 4.1.5), the batched zerocheck (Appendix D), ring switching (Section 5.5) and

spBCS (Sections 2.8 and 6), and Section 7.6 compares GordieHowe with Binius64. We discuss other related work below.

Credentials and proofs of signatures. Several systems prove presentations of credentials that issuers sign with standard signatures. Cinderella [DFKP16] proves statements about X.509 certificates signed with RSA, using Pinocchio [PHGR13]. zkLogin [BCJ+24] proves possession of a JWT signed with RS256, using Groth16 [Gro16]. zk-creds turns a passport into a credential at issuance with a Groth16 proof about its RSA signature, and the holder reveals the signature to the issuer [RWGM23]. Crescent proves presentations of JWTs signed with RSA and of mDLs signed with ECDSA, and it prepares a Groth16 proof once per credential and rerandomizes it for each presentation [PPZ24]. Dorian proves possession of RSA, ECDSA and Ed25519 signatures under a public key [WOS+25], and ZKAttest proves an ECDSA signature under a committed key [FLM22]. Vega [KS25] (Section 7.5.1) and Longfellow [Fs24] prove mDL presentations with transparent arguments. BBS and CL signatures have efficient presentations [ASM06, TZ23, CL04], but issuers must adopt them in place of the standard schemes [Fs24].

For post-quantum credentials, Policharla et al. prove knowledge of a signature of a Dilithium [DKL+18] variant, with an arithmetization-friendly hash and an adapted SampleInBall, using a STARK [PWFW23]. Concurrently with this work, Baum and Beckmann prove knowledge of an unmodified ML-DSA-44 signature with VOLE-in-the-head, for blind signatures with issuer hiding [BB26]. Their proof runs two VOLE-in-the-head instances, one over $\mathbb{F}_2$ and one over $\mathbb{F}_q$, and links them with a batch consistency check that they call edaBitE [BB26, p. 15 and §3]. Their security theorem is stated in the ROM [BB26, Thm. 2, p. 28].

Longfellow, GordieHowe and the system of Baum and Beckmann prove unmodified signatures with proof systems built only from symmetric-key primitives, and none of them needs a trusted setup. So do the RS256 and ECDSA circuits in Binius64's repository [Irr26a] and the variant of Cloudflare's VOLE-in-the-head proof of knowledge of an ML-DSA signature that commits the full witness [BB26, p. 10]. These proof systems are plausibly post-quantum. To our knowledge, GordieHowe is the only system that keeps standard signatures and has concrete parameters analyzed in the QROM. On the same host, GordieHowe's prover is faster than Longfellow's (Section 7.5.1), and we estimate that it is faster than Vega's per-presentation prover. GordieHowe's proofs are smaller than Longfellow's but larger than Vega's and than those of the systems based on pairing-based SNARKs.

Compilers and embedded languages. Compilers for proof systems usually compile a program to constraints over one native field. Circom, Noir, gnark, halo2 and zkLLVM do so [ide26, Noi26, Con26, Pri25, nil24]. Like our language, gadget libraries such as bellman, libsnark, arkworks and plonky2 are embedded languages [zkc26, SCI26, ark26b, Pol26]. arkworks, gnark and halo2wrong emulate a second prime field in limbs of the native field [ark26b, Con26, Pri26]. Two languages allow several fields in one relation: the SIEVE IR and ZK-SecreC. The SIEVE IR has conversion gates between field types [SIE23], and the PicoZK frontend emits such gates between several prime fields and $\mathbb{F}_2$ [NDDB25]. ZK-SecreC compiles a program with several moduli to several circuits and asserts equalities between their wires. It suggests MPC-in-the-head [IKOS07] with ring conversions to prove these equalities [BJL+22, p. 24]. In Longfellow, the application author writes the link between the two fields by hand, including the order of its commitments and challenges [Fs24, Goo26a]. Our compiler emits this link (Section 4.1.4). Binius types values by the subfields of a binary tower [DP25], and Plonky3 and Stwo commit trace values in a base field and draw challenges in an extension [Plo26, Sta26b]. Our types allow any subfield of any field of the program, and the compiler

chooses how to pack each value (Section 4.2.3).

The systems also differ in how they treat integers. Most expose integers or elements of an emulated field as a type with its own arithmetic. gnark, arkworks, halo2wrong, Barretenberg, ZoKrates and xJsnark track a bound on each such value and insert reductions or bit-width adjustments automatically [Con26, ark26b, Pri26, Azt26, ZoK26, KPS18], and Cairo's bounded integers reject a result whose interval does not fit the prime [Sta26a]. Our language has no integer type. Its integer expressions evaluate field values over the integers (Definition 3.4), and a congruence and a range check are predicates over them. The compiler uses interval tracking only to decide how to realize them, as one field equality with a range-checked quotient or by parts (Section 4.1.3).

The compilers also differ in how they assign challenges to rounds. A pass in the CirC repository places each challenge one round after its dependencies, and each variable in the latest round before every challenge that depends on it, over one field [Cir26]. Our compiler places every challenge and variable in its least phase, across several fields (Definition 3.7), while halo2 leaves phases to the author [Pri25]. PLONK [GWC19], Mirage [KPPS20] and Dorian [WOS+25, p. 4] commit the part of the witness that depends on challenges in a later phase. GordieHowe instead commits a pad for each late value in phase 0 and sends the masked value (Section 4.1.5).

Statements over several fields. edaBits come from MPC [EGK+20], and designated-verifier proofs based on VOLE use them and related checks to convert values between $\mathbb{Z}_2$ and $\mathbb{Z}_p$ [BBMH+21, WYX+21]. Chase, Ganesh and Mohassel, and Agrawal, Ganesh and Mohassel, prove composite statements that combine algebraic statements over a group with Boolean circuits, linking the two through commitments [CGM16, AGM18]. Zinc+ and BitZ avoid a link because they commit a witness once and prove constraints over several rings by projecting the witness or the claim [AGK+26, BGK+26]. Crescent links committed values across two fields of odd characteristic by revealing a masked random combination of them. A sigma proof over one field checks this combination, and a circuit over the other field recomputes it [PPZ24]. Longfellow links its two fields in a publicly verifiable argument with an information-theoretic MAC over $\mathbb{F}_{2^{128}}$ [Fs24, §3.4]. Its implementation uses one MAC per 128-bit chunk, and each chunk has its own committed prover key share [Goo26a]. Longfellow evaluates each tag in its circuit over the field of odd characteristic, at a multiplication for every exclusive or and every product of bits, and its tags do not hide zero chunks (Section 2.7). Our compiler emits one polynomial MAC for all values that two fields share in a phase, and Ligero's linear test proves its check, with a congruence modulo 2 whose quotient digits are range-checked in a field of odd characteristic (Section 4.1.4). Its masked tag hides every chunk, at a soundness error of $n_{\mathrm{ch}}/2^{\omega}$ (Lemma 4.5) against Longfellow's $1/2^{\omega}$.

Small-field and binary-field proof systems. Binius and its ring-switching reduction pack values of small binary subfields into larger elements of a tower before encoding them [DP25, DP26]. Binius64 and Flock combine this packing with a zerocheck that uses the univariate skip [Irr26b, BRW26], as our proof system does (Sections 5.4.2 and 5.5). Flock is not zero knowledge [BRW26]. Binius64, Longfellow [Fs24, Protocol 2.5] and GordieHowe obtain zero knowledge the same way, by the technique of Ben-Or et al. [BGG+90] and of Cramer and Damgård [CD98]. The prover masks the messages of its proof with committed pads, and a zero-knowledge argument proves the verifier's checks on the masked messages. Binius64 encodes these checks as an R1CS instance and proves it with IronSpartan, a second argument that it compiles together with its core protocol, with one batched polynomial-commitment opening for their committed oracles [Irr26b, §7.1–§7.3]. We prove the checks as further linear claims, product constraints and membership claims (Definition 2.25) on the Ligero tableaus that hold the witness (Section 5.4.3). Ligero's tests prove the linear claims and product

constraints, and our subfield test proves the membership claims (Section 5.5.2). Our verifier's check in ring switching applies powers of the Frobenius map to the masked messages. Subfield masks and squaring-chain masks keep this check provable in Ligero (Section 5.5). Binius64 pads these messages with one-time keys, as it pads its other messages, and proves the verifier's checks on them in its R1CS instance [Irr26b, §7.3]. The commitments of Binius64 and Flock have polylogarithmic proofs. In our proofs, the Ligero part grows with the square root of the number of committed elements, the masked GKR transcript with the depth of the circuit times the logarithm of its width, the zerocheck transcript with the logarithm of the batch size, and the masked late values with their number. Binius, Binius64 and Flock work over binary fields only, and GordieHowe proves statements that combine a binary field with fields of odd characteristic.

Ligero and proximity. Implementations of hash-based arguments set their number of queries from a conjectured proximity bound, from the proven unique-decoding bound or from a bound at the Johnson radius [Sta26b, Pol26, RIS26a, Suc26b, NA25, Irr26a, Plo26, Win26, ACFY26]. Plonky3 evaluates the closed-form exceptional count of Dao, Kominers and Thaler at the Johnson radius, searches the multiplicity of the underlying interpolation for the highest security level, and reports this level for given query counts [Plo26, DKT26]. Our compiler instead evaluates the interpolation bound of Dao, Kominers and Thaler exactly and minimizes it over a bounded range of integer parameters for each tableau (Lemma 2.15). In the example of Section 5.7, this gives an exceptional count about 115 times smaller than the closed form at the same threshold. For the formalized configurations, Lean's kernel checks this bound at each chosen threshold and evaluates the resulting round errors (Section 8). Chiesa, Fenzi and Weissenberg [CFW26, Constr. 7.2, Thm. 7.1 and Lem. 7.4] and Arnon, Boneh and Fenzi [ABF26, §6, Constr. 6.2 and Lem. 6.8] analyze Ligero-style linear tests round by round beyond the unique-decoding radius, with an error term that grows with the list size. Google's machine-checked analysis of Longfellow's Ligero also goes beyond the unique-decoding radius [Goo26b].

Extraction in the quantum random oracle model. Chiesa, Di, Hu and Zheng prove post-quantum straight-line knowledge soundness for BCS-compiled interactive oracle reductions [CDHZ25, Def. 10.3, Thm. 11.3]. Their relation deciders may query the emulated oracle, and their game has no simulation oracle. Don, Fehr, Majenz and Schaffner extract the commitments of commit-and-open sigma protocols online, by answering the prover's queries with the compressed oracle and measuring its database once the prover announces its first message [DFMS21, Thm. 5.4]. For the Fiat–Shamir transform of these protocols, they read the preimages from the database after the prover outputs its proof [DFMS22]. Their definition of online extractability bounds the distance between the joint outputs, including a quantum auxiliary output, of the real execution and of the extractor's execution, and it has no simulated proofs [DFMS22, Def. 3.1]. Our definition of knowledge soundness takes this form (Definition H.3). In the classical ROM, Chiesa and Fenzi prove weak simulation extractability of BCS with a straight-line extractor [CF24, Def. 5.16, p. 36, and Lem. 9.12, p. 76], and SPRINT proves weak simulation extractability of Fiat–Shamir for interactive proofs with a canonical simulator [dLM$^+$26].

Machine-checked proof systems. Our formalization proves knowledge soundness, everlasting zero knowledge and weak online simulation extractability of spBCS in the ROM and the QROM, and its unconditional witness indistinguishability (Section 8). To our knowledge, no earlier mechanized security proof covers the BCS transformation in any model for a general class of IOPs, a Fiat–Shamir-style transformation of an IOP in the QROM, or zero knowledge of a Fiat–Shamir-style

transformation of such a class of IOPs in any model. ArkLib [Ark26a] defines interactive oracle reductions and proves sumcheck soundness and the proximity gaps that our analysis uses, but its BCS transformation and its Fiat–Shamir theorems are not yet proved. Concurrently with this work, Google released a Lean proof of knowledge soundness, zero knowledge and witness indistinguishability for a model of Longfellow version 7 in the classical ROM, under premises that relate the model to Longfellow's code [Goo26b]. Its BCS theorem covers round-by-round sound IOPs with one or two Merkle commitments. Composed with round-by-round bounds for Longfellow's two Ligero rounds, it gives a soundness error of at most $(1 + T) \cdot 2^{-115}$ for a prover that makes T hash queries, and Lean's native code evaluator checks the numerical bounds. The Lean development of VEIL proves partial zero knowledge of an interactive compilation of IOPs, without a random oracle [DHRR26, Suc26a]. Other mechanized work covers sigma protocols in the standard model [BHZ+10, ABB+12, BLAG21, FU23], Fiat–Shamir signatures from sigma protocols in the classical ROM [BBD+23a, TDW+26, FJ25], and the soundness of one STARK-style protocol in the classical ROM [Mar26]. In the QROM, qrhl-tool [Unr19] and EasyPQC verify a variant of the Fujisaki–Okamoto transform [FO99] and full-domain-hash signatures [BR93], among other schemes [Unr20, BBF+21], and Unruh's Isabelle library and Heidler and Unruh formalize the compressed oracle and semi-classical O2H [Unr25, HU25]. We know of no earlier model of the QROM in Lean.

Our compiler exports each realized program to Lean with certificates, and Lean's kernel checks from them that the program is valid and realizes its source program (Section 8.3). The kernel thus checks these outputs of the compiler, but it does not verify the compiler itself. Leo is designed to emit a machine-checkable proof for each execution of its compiler [CWC+21], and ZKCrypt checks each optimized protocol against a proved reference implementation [ABB+12]. powdr's optimizer is verified in Lean to preserve circuit equivalence, relative to a per-VM bus semantics, under stated assumptions on its input circuits [pow26], and zirgen emits its circuits for determinism checks with Picus [PCW+23] [RIS26b].

10 Future work

Several parts of the system lie outside the formalization (Section 8.3), and each could be machine-checked. A model of the compilation to GKR circuits would turn the three assumptions on these circuits into theorems. A refinement proof from the Lean model to the C++ prover and verifier, or code extracted from Lean, would remove the assumption that the C++ implements the model. A cost semantics in Lean would machine-check the running times of the extractors and simulators, which we argue on paper. A proof that our Lean specifications of ML-DSA, SHAKE128 and SHAKE256 match their standards would remove the need to trust them. We embedded the language and implemented the compiler in C++ to reuse Longfellow's circuit compiler. Embedding both in Lean would let us verify the compiler itself and would remove the assumption that the export records the realized program that the prover and the verifier use.

The standard library and the compiler's cost-based choices could both be extended. A richer standard library, with more hash functions, signature schemes and credential encodings, would shorten programs like the mDL presentations. The compiler could choose every edge of the packing tree by the measured cost of its candidate chains, as it already does for the edge from S_0 to the field (Section 4.2.3). It could also decide, for each phase, whether to send the late values masked or to commit them in a second Ligero tableau, using the cost model that dispatches constraints (Section 4.2). A second tableau has a fixed cost in its low-degree test and openings, so it pays off only when a phase has many more late values than our applications have.

Our verifier is not sublinear in the size of the program. The rounds of the batched zerocheck take

time logarithmic in the number of instances, but its opening claim takes time linear in the number of instances, and so does the combination of the constraints that Ligero's tests check. GKR has a succinct verifier for data-parallel circuits [Tha13, WTs+18], but our GKR verifier does not exploit this structure and evaluates the wiring predicates in time linear in the circuit. Preprocessing that commits once to the constraints of each realized program, as in holographic proofs [Set20, CHM+20, COS19], or a uniform description of repeated constraints, as in STARKs [BBHR19], could make this combination sublinear. Even then, Ligero's verifier reads its queried columns and responses, about the square root of the number of committed elements, so a polylogarithmic verifier would also need another commitment.

References

[ABB+12] José Bacelar Almeida, Manuel Barbosa, Endre Bangerter, Gilles Barthe, Stephan Krenn, and Santiago Zanella Béguelin. Full proof cryptography: verifiable compilation of efficient zero-knowledge protocols. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 488–500. ACM Press, October 2012.

[ABF26] Gal Arnon, Dan Boneh, and Giacomo Fenzi. Open problems in list decoding and correlated agreement. Cryptology ePrint Archive, Paper 2026/680, 2026.

[ABKM21] Gorjan Alagic, Chen Bai, Jonathan Katz, and Christian Majenz. Post-quantum security of the Even-Mansour cipher. Cryptology ePrint Archive, Report 2021/1601, 2021.

[ACFY26] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR reference implementation. `https://github.com/worldfnd/whir`, 2026. Accessed 2026-10-14, commit afec1fa.

[AGK+26] Alexander Abdugafarov, Albert Garreta, Amit Kumar, Michał Osadnik, Psi Vesely, Ilia Vlasov, and Kai Zhe Zheng. Zinc+: SNARKs for polynomial rings. Cryptology ePrint Archive, Paper 2026/855, 2026.

[AGM18] Shashank Agrawal, Chaya Ganesh, and Payman Mohassel. Non-interactive zero-knowledge proofs for composite statements. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 643–673. Springer, Cham, August 2018.

[AHIV22] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligero: Lightweight sublinear arguments without a trusted setup. Cryptology ePrint Archive, Report 2022/1608, 2022.

[AHU18] Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. Cryptology ePrint Archive, Report 2018/904, 2018.

[Ark26a] ArkLib contributors. ArkLib: Formally verified arguments of knowledge in Lean. `https://github.com/Verified-zkEVM/ArkLib`, 2026. Accessed 2026-10-09.

[ark26b] arkworks contributors. r1cs-std. `https://github.com/arkworks-rs/r1cs-std`, 2026. Accessed 2026-10-14, commit 45e4e26.

[as26] abhi shelat. Comment on pull request #190, "specify the transcript structure for a sequence of proofs (multi-proof transcript forking)". `https://github.`

com/longfellow-zk/longfellow-zk/pull/190#issuecomment-5932896889, October 2026.

[ASM06] Man Ho Au, Willy Susilo, and Yi Mu. Constant-size dynamic k-TAA. In Roberto De Prisco and Moti Yung, editors, *SCN 06*, volume 4116 of *LNCS*, pages 111–125. Springer, Berlin, Heidelberg, September 2006.

[Azt26] Aztec. Barretenberg. `https://github.com/AztecProtocol/aztec-packages`, directory `barretenberg`, 2026. Accessed 2026-10-14, commit aca4d3e.

[BB26] Carsten Baum and Marvin Beckmann. edaBits in the exponent (edaBitE) and issuer-hiding blind signatures from ML-DSA and VOLE-in-the-head. Cryptology ePrint Archive, Paper 2026/2473, 2026.

[BBD+23a] Manuel Barbosa, Gilles Barthe, Christian Doczkal, Jelle Don, Serge Fehr, Benjamin Grégoire, Yu-Hsuan Huang, Andreas Hülsing, Yi Lee, and Xiaodi Wu. Fixing and mechanizing the security proof of Fiat-Shamir with aborts and Dilithium. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 358–389. Springer, Cham, August 2023.

[BBD+23b] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 581–615. Springer, Cham, August 2023.

[BBF+21] Manuel Barbosa, Gilles Barthe, Xiong Fan, Benjamin Grégoire, Shih-Han Hung, Jonathan Katz, Pierre-Yves Strub, Xiaodi Wu, and Li Zhou. EasyPQC: Verifying post-quantum cryptography. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2564–2586. ACM Press, November 2021.

[BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl, July 2018.

[BBHR19] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable zero knowledge with no trusted setup. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 701–732. Springer, Cham, August 2019.

[BBMH+21] Carsten Baum, Lennart Braun, Alexander Munch-Hansen, Benoît Razet, and Peter Scholl. Appenzeller to brie: Efficient zero-knowledge proofs for mixed-mode arithmetic and Z2k. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 192–211. ACM Press, November 2021.

[BCG+14] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE Computer Society Press, May 2014.

[BCG+19] Eli Ben-Sasson, Alessandro Chiesa, Lior Goldberg, Tom Gur, Michael Riabzev, and Nicholas Spooner. Linear-size constant-query IOPs for delegating computation. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part II*, volume 11892 of *LNCS*, pages 494–521. Springer, Cham, December 2019.

[BCI+20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for reed-solomon codes. Cryptology ePrint Archive, Report 2020/654, 2020.

[BCJ+24] Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Yan Ji, Jonas Lindstrøm, Deepak Maram, Ben Riva, Arnab Roy, Mahdi Sedaghat, and Joy Wang. zkLogin: Privacy-preserving blockchain authentication with existing credentials. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 3182–3196. ACM Press, October 2024.

[BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. Cryptology ePrint Archive, Report 2016/116, 2016.

[BDF+11] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 41–69. Springer, Berlin, Heidelberg, December 2011.

[BEG+91] Manuel Blum, William S. Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. Checking the correctness of memories. In *32nd FOCS*, pages 90–99. IEEE Computer Society Press, October 1991.

[BGG+90] Michael Ben-Or, Oded Goldreich, Shafi Goldwasser, Johan Håstad, Joe Kilian, Silvio Micali, and Phillip Rogaway. Everything provable is provable in zero-knowledge. In Shafi Goldwasser, editor, *CRYPTO'88*, volume 403 of *LNCS*, pages 37–56. Springer, New York, August 1990.

[BGK+26] Remco Bloemen, Albert Garreta, Marcin Kostrzewa, Shreyas Londhe, Lev Soukhanov, and John Wu. BitZ: proofs and commitments in arbitrary rings through binary fields. Cryptology ePrint Archive, Paper 2026/2141, 2026.

[BH20] C. Bormann and P. Hoffman. Concise binary object representation (CBOR). RFC 8949, STD 94, December 2020.

[BHZ+10] Gilles Barthe, Daniel Hedin, Santiago Zanella-Béguelin, Benjamin Grégoire, and Sylvain Heraud. A machine-checked formalization of sigma-protocols. In Andrew Myers and Michael Backes, editors, *CSF 2010 Computer Security Foundations Symposium*, pages 246–260. IEEE Computer Society Press, 2010.

[BJL+22] Dan Bogdanov, Joosep Jääger, Peeter Laud, Härmel Nestra, Martin Pettai, Jaak Randmets, Ville Sokk, Kert Tali, and Sandhra-Mirella Valdma. ZK-SecreC: a domain-specific language for zero-knowledge proofs. arXiv:2203.15448v2 [cs.PL], 2022. Published at CSF 2024.

[BLAG21] David Butler, Andreas Lochbihler, David Aspinall, and Adrià Gascón. Formalising Σ-protocols and commitment schemes using CryptHOL. *Journal of Automated Reasoning*, 65(4):521–567, 2021. ePrint 2019/1185.

[BLM13] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013.

[BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993.

[BRW26] Benedikt Bünz, Ron D. Rothblum, and William Wang. Flock: Fast proving for batch boolean computations. Cryptology ePrint Archive, Paper 2026/1329, 2026.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 499–530. Springer, Cham, April 2023.

[CD98] Ronald Cramer and Ivan Damgård. Zero-knowledge proofs for finite field arithmetic; or: Can zero-knowledge be for free? In Hugo Krawczyk, editor, *CRYPTO'98*, volume 1462 of *LNCS*, pages 424–441. Springer, Berlin, Heidelberg, August 1998.

[CDHZ25] Alessandro Chiesa, Zijing Di, Zihan Hu, and Yuxi Zheng. How to prove post-quantum security for succinct non-interactive reductions. Cryptology ePrint Archive, Paper 2025/2166, 2025. Full version of a paper at EUROCRYPT 2026.

[CET+24] Stefanos Chaliasos, Jens Ernstberger, David Theodore, David Wong, Mohammad Jahanara, and Benjamin Livshits. SoK: What don't we know? Understanding security vulnerabilities in SNARKs. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*, pages 3855–3872. USENIX Association, August 2024.

[CF24] Alessandro Chiesa and Giacomo Fenzi. zkSNARKs in the ROM with unconditional UC-security. Cryptology ePrint Archive, Report 2024/724, 2024.

[CFW26] Alessandro Chiesa, Giacomo Fenzi, and Guy Weissenberg. Zero-knowledge IOPPs for constrained interleaved codes. Cryptology ePrint Archive, Paper 2026/391, 2026.

[CGM16] Melissa Chase, Chaya Ganesh, and Payman Mohassel. Efficient zero-knowledge proof of algebraic and non-algebraic statements with applications to privacy preserving credentials. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 499–530. Springer, Berlin, Heidelberg, August 2016.

[CHM+20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 738–768. Springer, Cham, May 2020.

[CHVY25] Ignacio Cascudo, Carmit Hazay, Muthu Venkitasubramaniam, and Eylon Yogev. Independent security analysis of Longfellow ZK. Security review report on Longfellow ZK, `https://github.com/longfellow-zk/longfellow-zk/blob/main/docs/static/reviews/Longfellow_security_2025_12_15.pdf`, December 2025.

[Cir26] CirC authors. CirC. `https://github.com/circify/circ`, 2026. Accessed 2026-10-09, commit 271f911.

[CL01] Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In Birgit Pfitzmann, editor, *EUROCRYPT 2001*, volume 2045 of *LNCS*, pages 93–118. Springer, Berlin, Heidelberg, May 2001.

[CL04] Jan Camenisch and Anna Lysyanskaya. Signature schemes and anonymous credentials from bilinear maps. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 56–72. Springer, Berlin, Heidelberg, August 2004.

[CL06] Melissa Chase and Anna Lysyanskaya. On signatures of knowledge. Cryptology ePrint Archive, Report 2006/184, 2006.

[CMR+23] Lily Chen, Dustin Moody, Andrew Regenscheid, Angela Robinson, and Karen Randall. Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters. NIST Special Publication 800-186, National Institute of Standards and Technology, February 2023.

[CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. Succinct arguments in the quantum random oracle model. Cryptology ePrint Archive, Report 2019/834, 2019.

[Con26] Consensys. gnark. `https://github.com/Consensys/gnark`, 2026. Accessed 2026-10-09, commit 65da5ef.

[COS19] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. Fractal: Post-quantum and transparent recursive proofs from holography. Cryptology ePrint Archive, Report 2019/1076, 2019.

[CWC+21] Collin Chin, Howard Wu, Raymond Chu, Alessandro Coglio, Eric McCarthy, and Eric Smith. Leo: A programming language for formally verified, zero-knowledge applications. Cryptology ePrint Archive, Report 2021/651, 2021.

[CY26] Alessandro Chiesa and Eylon Yogev. Building cryptographic proofs from hash functions. Book, `https://snargsbook.org/`, 2026. Version of March 25, 2026.

[DDB+26] Quang Dao, Zachary DeStefano, Suyash Bagad, Yuval Domb, and Justin Thaler. Speeding up sum-check proving (extended version). Cryptology ePrint Archive, Paper 2026/587, 2026.

[DFKP16] Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, and Bryan Parno. Cinderella: Turning shabby X.509 certificates into elegant anonymous credentials with the magic of verifiable computation. In *2016 IEEE Symposium on Security and Privacy*, pages 235–254. IEEE Computer Society Press, May 2016.

[DFMS21] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Online-extractability in the quantum random-oracle model. Cryptology ePrint Archive, Report 2021/280, 2021.

[DFMS22] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Efficient NIZKs and signatures from commit-and-open protocols in the QROM. Cryptology ePrint Archive, Report 2022/270, 2022.

[DHLW10] Yevgeniy Dodis, Kristiyan Haralambiev, Adriana López-Alt, and Daniel Wichs. Efficient public-key cryptography in the presence of key leakage. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 613–631. Springer, Berlin, Heidelberg, December 2010.

[DHRR26] Rahul Dalal, Tamir Hemo, Eugene Rabinovich, and Ron D. Rothblum. VEIL: Lightweight zero-knowledge for hash-based multilinear proof systems. Cryptology ePrint Archive, Paper 2026/683, 2026.

[DKL+18] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR TCHES*, 2018(1):238–268, 2018.

[DKT26] Quang Dao, Scott Duke Kominers, and Justin Thaler. Reed-Solomon codes beyond Johnson: Efficient decoding and smaller cryptographic proofs. Cryptology ePrint Archive, Paper 2026/2056, 2026.

[dLM+26] Thomas den Hollander, Shai Levin, Marzio Mula, Robi Pedersen, Daniel Slamanig, and Sebastian A. Spindler. SPRINT: New isogeny proofs of knowledge and isogeny-based signatures. Cryptology ePrint Archive, Report 2026/364, 2026. Full version of a paper at ASIACRYPT 2026.

[dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[DP25] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 93–122. Springer, Cham, May 2025.

[DP26] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinears over binary towers. In Joan Daemen and Emmanuel Thomé, editors, *Advances in Cryptology – EUROCRYPT 2026, Part VII*, volume 16547 of *Lecture Notes in Computer Science*, pages 3–32. Springer, Cham, 2026.

[EGK+20] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 823–852. Springer, Cham, August 2020.

[FJ25] Denis Firsov and Jakub Janků. Forking lemma in EasyCrypt. Cryptology ePrint Archive, Report 2025/573, 2025.

[FLM22] Armando Faz-Hernández, Watson Ladd, and Deepak Maram. ZKAttest: Ring and group signatures for existing ECDSA keys. In Riham AlTawy and Andreas Hülsing, editors, *SAC 2021*, volume 13203 of *LNCS*, pages 68–83. Springer, Cham, September / October 2022.

[FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 537–554. Springer, Berlin, Heidelberg, August 1999.

[FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987.

[FS90] Uriel Feige and Adi Shamir. Witness indistinguishable and witness hiding protocols. In *22nd ACM STOC*, pages 416–426. ACM Press, May 1990.

[Fs24] Matteo Frigo and abhi shelat. Anonymous credentials from ECDSA. Cryptology ePrint Archive, Report 2024/2010, 2024.

[Fs26] Matteo Frigo and abhi shelat. Longfellow ZK. Internet-Draft draft-google-cfrg-libzk-03, IETF, September 2026.

[FU23] Denis Firsov and Dominique Unruh. Zero-knowledge in EasyCrypt. In *CSF 2023 Computer Security Foundations Symposium*, pages 1–16. IEEE Computer Society Press, July 2023.

[Gao93] Shuhong Gao. *Normal Bases over Finite Fields*. PhD thesis, University of Waterloo, 1993.

[GHHM21] Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive reprogramming in the QROM. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 637–667. Springer, Cham, December 2021.

[GKR15] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating computation: Interactive proofs for muggles. *Journal of the ACM*, 62(4):27:1–27:64, 2015.

[GMR85] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In *17th ACM STOC*, pages 291–304. ACM Press, May 1985.

[Goo26a] Google. longfellow-zk. GitHub repository, `https://github.com/longfellow-zk/longfellow-zk`, 2026. Accessed 2026-10-13, commit d5e6be7.

[Goo26b] Google. Machine-checked analysis of Longfellow ZK and Ligero in Lean, in longfellow-zk. GitHub repository, `https://github.com/longfellow-zk/longfellow-zk`, directory `formal`, 2026. Accessed 2026-10-18, commit d9d8d6a.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016.

[GRS25] Venkatesan Guruswami, Atri Rudra, and Madhu Sudan. Essential coding theory. Book draft, version of August 26, 2025, `https://cse.buffalo.edu/faculty/atri/courses/coding-theory/book/`, 2025.

[Gru24] Angus Gruen. Some improvements for the PIOP for ZeroCheck. Cryptology ePrint Archive, Report 2024/108, 2024.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019.

[Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022.

[HK24] Ulrich Haboeck and Al Kindi. A note on adding zero-knowledge to STARKs. Cryptology ePrint Archive, Report 2024/1037, 2024.

[Hoe63] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[HU25] Katharina Heidler and Dominique Unruh. Formalizing the one-way to hiding theorem. In *Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP '25)*, pages 243–256. ACM, 2025.

[ide26] iden3. circom. `https://github.com/iden3/circom`, 2026. Accessed 2026-10-09.

[IKOS07] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In David S. Johnson and Uriel Feige, editors, *39th ACM STOC*, pages 21–30. ACM Press, June 2007.

[Int21] International Civil Aviation Organization. Machine readable travel documents. ICAO Doc 9303, eighth edition, 2021.

[Irr26a] Irreducible. Binius64. `https://github.com/binius-zk/binius64`, 2026. Accessed 2026-10-08, commit 4428e759.

[Irr26b] Irreducible. Binius64 blueprint. `https://www.binius.xyz/spec.pdf`, October 2026. Version of 7 October 2026, accessed 2026-10-17.

[ISO21] ISO/IEC. Personal identification — ISO-compliant driving licence — part 5: Mobile driving licence (mDL) application. ISO/IEC 18013-5:2021, first edition, September 2021.

[KjCP16] John Kelsey, Shu jen Chang, and Ray Perlner. SHA-3 derived functions: cSHAKE, KMAC, TupleHash and ParallelHash. NIST Special Publication 800-185, National Institute of Standards and Technology, December 2016.

[KPPS20] Ahmed E. Kosba, Dimitrios Papadopoulos, Charalampos Papamanthou, and Dawn Song. MIRAGE: Succinct arguments for randomized algorithms with applications to universal zk-SNARKs. In Srdjan Capkun and Franziska Roesner, editors, *USENIX Security 2020*, pages 2129–2146. USENIX Association, August 2020.

[KPS18] Ahmed E. Kosba, Charalampos Papamanthou, and Elaine Shi. xJsnark: A framework for efficient verifiable computation. In *2018 IEEE Symposium on Security and Privacy*, pages 944–961. IEEE Computer Society Press, May 2018.

[KS25] Darya Kaviani and Srinath Setty. Vega: Low-latency zero-knowledge proofs over existing credentials. Cryptology ePrint Archive, Paper 2025/2094, 2025.

[LFKN92] Carsten Lund, Lance Fortnow, Howard J. Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM*, 39(4):859–868, 1992.

[Mar26] Diego Marmsoler. Isabelle/STARK: A formalization of a STARK-style protocol in Isabelle/HOL. arXiv:2608.01965v2, 2026.

[Mer88] Ralph C. Merkle. A digital signature based on a conventional encryption function. In Carl Pomerance, editor, *CRYPTO'87*, volume 293 of *LNCS*, pages 369–378. Springer, Berlin, Heidelberg, August 1988.

[MKJR16] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch. PKCS #1: RSA cryptography specifications version 2.2. RFC 8017, November 2016.

[MU10] Jörn Müller-Quade and Dominique Unruh. Long-term security and universal composability. *Journal of Cryptology*, 23(4):594–671, October 2010.

[NA25] Andrija Novakovic and Guillermo Angeris. Ligerito: A small and concretely fast polynomial commitment scheme. Cryptology ePrint Archive, Paper 2025/1187, 2025.

[Nat15a] National Institute of Standards and Technology. Secure hash standard (SHS). Federal Information Processing Standards Publication 180-4, August 2015.

[Nat15b] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication 202, August 2015.

[Nat23] National Institute of Standards and Technology. Digital signature standard (DSS). Federal Information Processing Standards Publication 186-5, February 2023.

[Nat24] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Federal Information Processing Standards Publication 204, August 2024.

[NDDB25] Joseph P. Near, Onyinye Dibia, David Darais, and Mark Blunk. PicoZK. `https://github.com/uvm-plaid/picozk`, 2025. Accessed 2026-10-09, commit f7e8981.

[nil24] zkLLVM. =nil; Foundation, `https://github.com/NilFoundation/zkLLVM`, 2024. Accessed 2026-10-09.

[Noi26] Noir authors. Noir and the abstract circuit intermediate representation (ACIR). `https://github.com/noir-lang/noir`, 2026. Accessed 2026-10-14.

[PCW+23] Shankara Pailoor, Yanju Chen, Franklyn Wang, Clara Rodríguez, Jacob Van Geffen, Jason Morton, Michael Chu, Brian Gu, Yu Feng, and Işıl Dillig. Automated detection of under-constrained circuits in zero-knowledge proofs. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1510–1532, June 2023.

[PH23] Shahar Papini and Ulrich Haböck. Improving logarithmic derivative lookups using GKR. Cryptology ePrint Archive, Report 2023/1284, 2023.

[PHGR13] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252. IEEE Computer Society Press, May 2013.

[Plo26] Plonky3 authors. Plonky3. `https://github.com/Plonky3/Plonky3`, 2026. Commit f983316, accessed 2026-10-15.

[Pol26] Polygon Zero. plonky2. `https://github.com/0xPolygonZero/plonky2`, 2026. Accessed 2026-10-14.

[pow26] powdr labs. powdr and apc-optimizer. `https://github.com/powdr-labs/powdr` and `https://github.com/powdr-labs/apc-optimizer`, 2026. Accessed 2026-10-14, powdr commit e21205e, apc-optimizer commit 0269fd80.

[PPZ24] Christian Paquin, Guru-Vamsi Policharla, and Greg Zaverucha. Crescent: Stronger privacy for existing credentials. Cryptology ePrint Archive, Report 2024/2013, 2024.

[Pri25] Privacy and Scaling Explorations. halo2 (PSE fork). `https://github.com/privacy-ethereum/halo2`, 2025. Archived; commit 198e9ae, accessed 2026-10-17.

[Pri26] Privacy and Scaling Explorations. halo2wrong. `https://github.com/privacy-scaling-explorations/halo2wrong`, 2026. Accessed 2026-10-14, commit 3eaa2c4.

[PWFW23] Guru-Vamsi Policharla, Bas Westerbaan, Armando Faz-Hernández, and Christopher A Wood. Post-quantum privacy pass via post-quantum anonymous credentials. Cryptology ePrint Archive, Report 2023/414, 2023.

[RIS26a] RISC Zero. RISC Zero. `https://github.com/risc0/risc0`, 2026. Accessed 2026-10-14, commit 3bbcd44.

[RIS26b] RISC Zero. zirgen. `https://github.com/risc0/zirgen`, 2026. Accessed 2026-10-14, commit df6fb9d.

[RRR20] Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. Constant-round interactive proofs for delegating computation. Electronic Colloquium on Computational Complexity (ECCC), Report TR16-061, revision 2, October 2020.

[RWGM23] Michael Rosenberg, Jacob D. White, Christina Garman, and Ian Miers. zk-creds: Flexible anonymous credentials from zkSNARKs and existing identity infrastructure. In *2023 IEEE Symposium on Security and Privacy*, pages 790–808. IEEE Computer Society Press, May 2023.

[Sch79] Jacob T. Schwartz. Probabilistic algorithms for verification of polynomial identities. In *Symbolic and Algebraic Computation, EUROSAM '79*, volume 72 of *Lecture Notes in Computer Science*, pages 200–215. Springer, 1979.

[Sch22] J. Schaad. CBOR object signing and encryption (COSE): Structures and process. RFC 9052, STD 96, August 2022.

[SCI26] SCIPR Lab. libsnark. `https://github.com/scipr-lab/libsnark`, 2026. Accessed 2026-10-14.

[Set20] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 704–737. Springer, Cham, August 2020.

[SIE23] SIEVE IR contributors. SIEVE circuit intermediate representation, version 2.2.0. `https://github.com/sieve-zk/ir`, 2023. Last updated 2023-11-21. Accessed 2026-10-09.

[Sou24] Lev Soukhanov. Hashcaster. Unpublished report, `https://hackmd.io/@levs57/SJ4fuZMD0`, September 2024.

[Sta25a] Alan Stapelberg. Google Wallet launches new age and identity verification features. https://blog.google/products-and-platforms/platforms/google-pay/google-wallet-age-identity-verifications/, April 2025.

[Sta25b] Alan Stapelberg. Opening up 'zero-knowledge proof' technology to promote privacy in age assurance. Google blog, https://blog.google/technology/safety-security/opening-up-zero-knowledge-proof-technology-to-promote-privacy-in-age-assurance/, July 2025.

[Sta26a] StarkWare. Cairo (sierra bounded integers). https://github.com/starkware-libs/cairo, 2026. Accessed 2026-10-14, commit ca912f7e.

[Sta26b] StarkWare. Stwo. https://github.com/starkware-libs/stwo, 2026. Accessed 2026-10-09.

[Suc26a] Succinct. VEIL formal verification. https://github.com/succinctlabs/veil-formal-verification, 2026. Accessed 2026-10-09.

[Suc26b] Succinct Labs. SP1. https://github.com/succinctlabs/sp1, 2026. Accessed 2026-10-14, commit 02fa301.

[SWPG26] Chad Sharp, Anna P. Y. Woo, Chris Peikert, and Paul Grubbs. Enriching standardized lattice signatures, generically, 2026. In preparation.

[TDW+26] Devon Tuma, Quang Dao, James Waters, Alexander Hicks, and Nicholas Hopper. VCVio: Verified cryptography in Lean via oracle effects and handlers. Cryptology ePrint Archive, Paper 2026/899, 2026.

[Tha13] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 71–89. Springer, Berlin, Heidelberg, August 2013.

[Tha23] Justin Thaler. Proofs, arguments, and zero-knowledge. Manuscript, version of July 18, 2023, https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.pdf, July 2023.

[The20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020*, pages 367–381. ACM, 2020.

[TZ23] Stefano Tessaro and Chenzhi Zhu. Revisiting BBS signatures. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 691–721. Springer, Cham, April 2023.

[Unr13] Dominique Unruh. Everlasting multi-party computation. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 380–397. Springer, Berlin, Heidelberg, August 2013.

[Unr14] Dominique Unruh. Non-interactive zero-knowledge proofs in the quantum random oracle model. Cryptology ePrint Archive, Report 2014/587, 2014.

[Unr17] Dominique Unruh. Post-quantum security of Fiat-Shamir. Cryptology ePrint Archive, Report 2017/398, 2017.

[Unr19] Dominique Unruh. Quantum relational Hoare logic. *Proceedings of the ACM on Programming Languages*, 3(POPL):1–31, January 2019.

[Unr20] Dominique Unruh. Post-quantum verification of Fujisaki-Okamoto. Cryptology ePrint Archive, Report 2020/962, 2020.

[Unr25] Dominique Unruh. Compressed oracles. Archive of Formal Proofs, `https://www.isa-afp.org/entries/Compressed_Oracles.html`, December 2025.

[Ver26] Verified-zkEVM. VCVio. GitHub repository, `https://github.com/Verified-zkEVM/VCVio`, 2026. Commit eb0f6e4, accessed 2026-09-22.

[VTL79] Jacobo Valdes, Robert E. Tarjan, and Eugene L. Lawler. The recognition of series parallel digraphs. In *Proceedings of the 11th Annual ACM Symposium on Theory of Computing (STOC)*, pages 1–12, 1979.

[Wat18] John Watrous. *The Theory of Quantum Information*. Cambridge University Press, 2018. Author's draft, `https://cs.uwaterloo.ca/~watrous/TQI/TQI.pdf`.

[WC81] Mark N. Wegman and Larry Carter. New hash functions and their use in authentication and set equality. *Journal of Computer and System Sciences*, 22:265–279, 1981.

[Win26] Winterfell authors. Winterfell. `https://github.com/facebook/winterfell`, 2026. Accessed 2026-10-14, commit 2f78ee9.

[WOS+25] Anna P. Y. Woo, Alex Ozdemir, Chad Sharp, Thomas Pornin, and Paul Grubbs. Efficient proofs of possession for legacy signatures. Cryptology ePrint Archive, Report 2025/538, 2025.

[WTs+18] Riad S. Wahby, Ioanna Tzialla, abhi shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. In *2018 IEEE Symposium on Security and Privacy*, pages 926–943. IEEE Computer Society Press, May 2018.

[WYX+21] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 501–518. USENIX Association, August 2021.

[Zha18] Mark Zhandry. How to record quantum queries, and applications to quantum indifferentiability. Cryptology ePrint Archive, Report 2018/276, 2018.

[Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In *Symbolic and Algebraic Computation, EUROSAM '79*, volume 72 of *Lecture Notes in Computer Science*, pages 216–226. Springer, 1979.

[zkc26] zkcrypto. bellman. `https://github.com/zkcrypto/bellman`, 2026. Accessed 2026-10-14.

[ZoK26] ZoKrates authors. ZoKrates. `https://github.com/Zokrates/ZoKrates`, 2026. Accessed 2026-10-14, commit 8699128.

Appendices

A Proofs from the preliminaries **95**
A.1 Reductions . . . 97
B The language in full **97**
B.1 Degree . . . 98
B.2 The interactive proof of a program . . . 98
B.3 Permutation checks . . . 98
C Security of the phase step **99**
D The batched zerocheck **102**
D.1 Batched predicates and skip domains . . . 102
D.2 Multilinear extensions and Gruen's zerocheck . . . 103
D.3 The skip zerocheck . . . 103
D.4 The masked skip zerocheck . . . 105
D.5 The opening claim on the stored values . . . 106
D.6 The bit sumcheck . . . 106
E The GKR protocol **107**
F Ring switching **109**
F.1 Ring switching on one edge . . . 109
F.2 Ring switching along a packing tree . . . 111
G Security of Ligero **113**
G.1 Agreement lists and correlated agreement . . . 113
G.2 Ligero's tableau and tests as Longfellow runs them . . . 114
G.3 Ligero with the mask test . . . 115
G.4 Mask pieces and the subfield test . . . 117
G.5 Security of Ligero with the mask test . . . 118
G.6 Security of Ligero without the mask test . . . 122
G.7 Ligero in Longfellow's order of challenges . . . 123
G.8 Counterexamples to the published analyses . . . 126
H Non-interactive arguments **127**
H.1 Random oracles and non-interactive arguments . . . 127
H.2 Completeness and soundness . . . 127
H.3 Zero knowledge . . . 129
I Proofs for series–parallel BCS **130**
I.1 Protocol details . . . 130
I.2 Completeness, zero knowledge and witness indistinguishability . . . 132
I.3 Knowledge soundness . . . 137
I.4 Weak online simulation extractability . . . 143
J Machine-checked results **146**

A Proofs from the preliminaries

This appendix proves the lemmas of Section 2 whose proofs are not given there. It also states two lemmas on the reductions of Section 2.6 (Section A.1).

Proof of Lemma 2.7. Reindexing the sum of Frobenius powers in Definition 2.4 gives $\mathrm{Tr}_{T/A} = \mathrm{Tr}_{B/A} \circ \mathrm{Tr}_{T/B}$. Since $\mathrm{Tr}_{T/B}$ is B-linear, $\mathrm{Tr}_{T/A}(\mathfrak{a}_i\mathfrak{b}_j\,\mathfrak{a}_{i'}^*\mathfrak{b}_{j'}^*) = \mathrm{Tr}_{B/A}(\mathfrak{a}_i\mathfrak{a}_{i'}^*\,\mathrm{Tr}_{T/B}(\mathfrak{b}_j\mathfrak{b}_{j'}^*))$. The right side is 1 if $(i,j) = (i',j')$ and 0 otherwise, so the products $\mathfrak{a}_i\mathfrak{b}_j$ and the products $\mathfrak{a}_i^*\mathfrak{b}_j^*$ are each linearly independent over A. Since $[T : A] = dd'$, both families are bases of T over A, and the identity says they are dual. The same identity gives $\mathrm{Tr}_{T/A}(\mathfrak{a}_i^*\mathfrak{b}_j^*U) = \mathrm{Tr}_{B/A}(\mathfrak{a}_i^*\mathrm{Tr}_{T/B}(\mathfrak{b}_j^*U))$. The left side is the entry of $\mathsf{unpack}_{T/A}(U)$ at the product $\mathfrak{a}_i\mathfrak{b}_j$, and the right side is entry i of $\mathsf{unpack}_{B/A}$ applied to entry j of $\mathsf{unpack}_{T/B}(U)$. With the products ordered by j and then i, the two vectors agree. □

Proof of Lemma 2.14. Part (1) is [DKT26, Thm. 5.12] with $D = k-1$ and $\eta_0 = \eta$. These choices make their threshold $\lceil(\sqrt{\rho^-} + \eta_0)n\rceil$ equal to A. Their theorem holds in every characteristic. Its exceptional count, the B_{cf} of [DKT26, eq. (68)], is below the stated bound. For part (2), fix u_0, u_1. If at most n points z have a codeword within the unique-decoding radius of $u_0 + zu_1$, those points form the exceptional set. Otherwise (u_0, u_1) has correlated agreement with codewords p_0, p_1 on all but at most e_{U} positions [BCI+20, Thm. 4.1] (their k is our $k-1$). Each position outside $\mathrm{Agr}(p_0, u_0) \cap \mathrm{Agr}(p_1, u_1)$ gains agreement with $p_0 + zp_1$ at one point z at most. These at most e_{U} points form the exceptional set. Let z be any other point, and let p be a polynomial of degree below k with $|\mathrm{Agr}(p, u_0 + zu_1)| \geq n - e_{\mathrm{U}}$. Unique decoding gives $p = p_0 + zp_1$, so the agreement set of p is $\mathrm{Agr}(p_0, u_0) \cap \mathrm{Agr}(p_1, u_1)$. □

Proof of Lemma 2.15. We follow the argument of [DKT26, App. B] behind their Theorem 5.12, with the support of the interpolating polynomial set by (m, B, H). Fix $u_0, u_1 \in K^{\mathcal{D}}$. For $x \in \mathcal{D}$, let $w_x(Z) = u_{0,x} + Zu_{1,x}$.

First we construct Q. The unknowns are polynomials $q_{ij} \in K[Z]$ of degree at most $H - j$, for $0 \leq j \leq B$ and $0 \leq i < mA - Dj$. Put $Q = \sum_{i,j} q_{ij}(Z)X^iY^j$. For each $x \in \mathcal{D}$ and $a + c < m$, we require the coefficient of U^aV^c in $Q(x + U, Z, w_x(Z) + V)$ to vanish as a polynomial in Z. This coefficient is $\sum_{i,j} \binom{i}{a}\binom{j}{c} x^{i-a} w_x(Z)^{j-c} q_{ij}(Z)$. Its entry at q_{ij} has degree at most $j - c$ in Z and is zero for $c > j$. The conditions with $c > B$ are therefore vacuous, and we drop them. Each remaining condition has $c \leq B'$, and for each x and c, $m - c$ conditions remain. Give the unknown q_{ij} the cost j and the condition (x, a, c) the cost c. The unknowns have $\sum_{j=0}^{B}(mA - Dj)(H + 1 - j)$ scalar coefficients. Each condition (x, a, c) imposes at most $H + 1 - c$ scalar equations. The hypothesis of the lemma is then the strict surplus (20) of [DKT26, Lem. 3.12, p. 30]. That lemma gives a nonzero solution, so $Q \neq 0$, $\deg_Y Q \leq B$ and $\deg_Z Q \leq H$. Every monomial X^iY^j of Q has $i + Dj < mA$.

Next we show that Q vanishes at every close polynomial. We use the argument of [DKT26, Prop. 3.10, p. 29, and Prop. B.1, p. 116]. Let $z \in K$, and let p be a polynomial of degree below k with $|\mathrm{Agr}(p, u_0 + zu_1)| \geq A$. For $x \in \mathrm{Agr}(p, u_0 + zu_1)$, write $p(x + U) = w_x(z) + V_p(U)$. Then U divides V_p. Substitute $V = V_p(U)$ in the expansion of $Q(x + U, z, w_x(z) + V)$. Only terms $U^aV_p(U)^c$ with $a + c \geq m$ remain, so $(X - x)^m$ divides $Q(X, z, p(X))$. Every monomial of Q has $i + Dj < mA$, so $Q(X, z, p(X))$ has degree below mA. It has at least mA roots counted with multiplicity and is therefore zero.

Finally we apply [DKT26, Lem. 5.3, p. 40] to Q and the received line $u_0 + Zu_1$. We take their curve degree $\ell = 1$, their $B_{\mathrm{jet}} = B \geq 1$, their $B_Z = H$ and their $L = D + 1$. Their hypotheses $1 \leq D \leq n - 2$ and $D + 1 \leq A \leq n$ hold, and Lemma 5.3 holds in every characteristic. It gives a set $\mathcal{Z} \subseteq K$ of at most $E_{\mathrm{ord}}^{(1)}(D, A, n; B, H)$ points [DKT26, eqs. (38) and (39)]. Let $z \notin \mathcal{Z}$, and let p be a polynomial of

degree at most D with $Q(X, z, p(X)) = 0$ and $|\mathrm{Agr}(p, u_0 + zu_1)| \geq A$. Lemma 5.3 gives polynomials p_0, p_1 of degree at most D with $p = p_0 + zp_1$ and $\mathrm{Agr}(p, u_0 + zu_1) = \mathrm{Agr}(p_0, u_0) \cap \mathrm{Agr}(p_1, u_1)$ [DKT26, Def. 3.1]. By the previous step, every polynomial p of degree below k with $|\mathrm{Agr}(p, u_0 + zu_1)| \geq A$ has $Q(X, z, p(X)) = 0$. So $\mathcal{Z}$ is an exceptional set as in Definition 2.13. It remains to compute the count. Their eq. (38) with $\ell = 1$ and $L = D + 1$ reads

$$E^{(1)}_{\mathrm{ord}}(D, A, n; B, H) = (2B - 1)H + \frac{n - D}{A - D}\big(B + H\Psi_D(B)\big) + (n - D - 1)B,$$

with $\Psi_D(B) = 1 + (2D - 1)(2B - 1) + 2(B - 2D - 1)_+$ by their eq. (37). The positive part vanishes for $B \leq 2D + 1$, so $E^{(1)}_{\mathrm{ord}}(D, A, n; B, H) = E(n, k, A; m, B, H)$. □

Remark A.1. At the parameters of [DKT26, Thm. 5.12], whenever $B_{\mathrm{jet}} \leq 2D + 1$, the count of Lemma 2.15 is below the closed form of Lemma 2.14(1).

Proof. Let A be within the Johnson radius, and let m, t, B_{jet} and B_Z be as in [DKT26, Thm. 5.12, eqs. (65) and (66)] with $\eta_0 = A/n - \sqrt{\rho^-}$. The support of [DKT26, Prop. B.1] consists of the monomials $X^i Y^j Z^l$ with $i + Dj < t\sqrt{Dn}$ and $j + l < t^2/(3\rho^-)$. Its proof shows $t\sqrt{Dn} \leq mA$, so this support lies in the support of the proof of Lemma 2.15 at $B = B_{\mathrm{jet}}$ and $H = B_Z$. The count of equations of Prop. B.1 [DKT26, eq. (128)] sums over all $b < m$, and the hypothesis of Lemma 2.15 sums over $b \leq B'$ only. Each term of their count is positive, since $t^2/(3\rho^-) \geq t^2/3 > m$. Since $B_{\mathrm{jet}} < t/\sqrt{\rho^-}$, the bound $t\sqrt{Dn} \leq mA$ gives $DB_{\mathrm{jet}} < mA$. Since $t > 3\sqrt{\rho^-}$, we have $t^2/(3\rho^-) > t/\sqrt{\rho^-}$, so $B_{\mathrm{jet}} \leq B_Z$. So the surplus of Prop. B.1 implies the hypothesis of Lemma 2.15 at $(m, B_{\mathrm{jet}}, B_Z)$. Since $m \geq 3$, we have $t \geq 7/2$, and since $\rho^- < 1$, $t/\sqrt{\rho^-} > 7/2$, so $B_{\mathrm{jet}} \geq 1$, as Lemma 2.15 requires. At these parameters $E(n, k, A; m, B_{\mathrm{jet}}, B_Z)$ equals the count B_{cf} of [DKT26, eq. (68)], and that equation bounds B_{cf} below $8nt^3/(3\rho^-)$. □

Proof of Lemma 2.21. Under the total order, the past of a verifier message i is the prefix before i. For a transcript or prefix p containing i, write $p_{<i}$ and $p_{\leq i}$ for its prefixes before and after i.

Let $\mathsf{State}^{\mathrm{CMS}}$ be a state function on prefixes as in [CMS19, Def. 8.5], with extractor $\mathbf{E}$, and let $\mathsf{State}(\mathsf{x}, i, p) = \mathsf{State}^{\mathrm{CMS}}(\mathsf{x}, p)$ for a prefix p before or after i. Along a full transcript tr that $\mathbf{V}$ accepts, $\mathsf{State}^{\mathrm{CMS}}$ is 0 on the empty prefix, stays 0 at prover messages and is 1 on tr. So some verifier message i has $\mathsf{State}(\mathsf{x}, i, \mathsf{tr}_{<i}) = 0$ and $\mathsf{State}(\mathsf{x}, i, \mathsf{tr}_{\leq i}) = 1$, which is condition (a) of Definition 2.20. Condition (b) is the round-by-round condition of [CMS19, Def. 8.5], with the per-round errors indexed as in [CY26, Def. 31.1.6, p. 386].

Conversely, let State and $\mathbf{E}$ satisfy Definition 2.20. For a prefix p, let $\mathsf{State}^{\mathrm{CMS}}(\mathsf{x}, p) = 1$ if some verifier message i in p has $\mathsf{State}(\mathsf{x}, i, p_{<i}) = 0$ and $\mathsf{State}(\mathsf{x}, i, p_{\leq i}) = 1$, or has $(\mathsf{x}, \mathbf{E}(\mathsf{x}, p_{<i})) \in \mathcal{R}$, and $\mathsf{State}^{\mathrm{CMS}}(\mathsf{x}, p) = 0$ otherwise. The empty prefix contains no verifier message, so its state is 0. A prover message adds no verifier message, so the state does not change there. On an accepted full transcript the state is 1 by condition (a). At a prefix p with state 0 that ends before i, the state after θ_i is 1 exactly when $\mathsf{State}(\mathsf{x}, i, p) = 0$ and $\mathsf{State}(\mathsf{x}, i, p \| \theta_i) = 1$, or $\mathbf{E}$ succeeds on p. If $\mathsf{State}(\mathsf{x}, i, p) = 1$ and $\mathbf{E}$ fails on p, the state after θ_i is 0. Otherwise condition (b) gives the per-round condition. □

Proof of Lemma 2.27. (a) At a round of Π_1, let the state be the state of Π_1 on x (series) or x_1 (parallel). At a round of Π_2, let the state be the state of Π_2 on its own statement. In the series case, the past of the round determines this statement. Each round inherits condition (ii) of Definition 2.24(a) from its component. For condition (i) in the series case, let $(\mathsf{x}, \boldsymbol{U}) \notin \mathcal{R}_1$ with output in $\mathcal{R}_2 \wedge \mathcal{R}_{\mathrm{out}}$. If $((\mathsf{out}_{\mathrm{in}}, \mathsf{out}_{\mathrm{out}})(\mathsf{x}, \mathsf{tr}_1), \boldsymbol{U}) \in \mathcal{R}_{\mathrm{in}} \wedge \mathcal{R}_{\mathrm{out}}$, the state of Π_1 moves from 0 to 1 at some round. Otherwise, since $(\mathsf{out}_{\mathrm{out}}(\mathsf{x}, \mathsf{tr}_1), \boldsymbol{U}) \in \mathcal{R}_{\mathrm{out}}$, we have $(\mathsf{out}_{\mathrm{in}}(\mathsf{x}, \mathsf{tr}_1), \boldsymbol{U}) \notin \mathcal{R}_{\mathrm{in}}$, so the state of Π_2 moves. In the parallel case, $(\mathsf{x}_1, \boldsymbol{U}) \notin \mathcal{R}_1$ or $(\mathsf{x}_2, \boldsymbol{U}) \notin \mathcal{R}'_1$, so the state of the corresponding component moves.

(b) Let the oracle input lie in $\mathcal{K}_1 \cap \mathcal{K}_2$, and let E_1 be the event that the output of Π_1 is outside its target relation, so $\Pr[E_1] \le \varepsilon_{\mathrm{c}}^{(1)}$. In the parallel case, Π_2 fails with probability at most $\varepsilon_{\mathrm{c}}^{(2)}$. In the series case, fix tr_1 outside E_1. Then $(\mathsf{out}_{\mathrm{in}}(\mathsf{x}, \mathsf{tr}_1), \boldsymbol{U}) \in \mathcal{R}_{\mathrm{in}}$, and the challenges of Π_2 are uniform and independent of tr_1, so Π_2 fails with probability at most $\varepsilon_{\mathrm{c}}^{(2)}$. A union bound gives $\varepsilon_{\mathrm{c}}^{(1)} + \varepsilon_{\mathrm{c}}^{(2)}$.

(c) The view of Π_1 is uniform. The messages of Π_1 do not read the masks of Π_2, and the masks are drawn independently. Conditioned on the view of Π_1, the masks of Π_2 are therefore distributed as μ_2 and the statement of Π_2 is fixed. The view of Π_2 is then uniform for every value of the masks of Π_1, so the joint view is uniform. □

A.1 Reductions

When $\sqsubset$ is a total order, round-by-round soundness is equivalent to a condition on the prefixes of the transcript, and Chiesa, Di, Hu and Zheng state round-by-round soundness in this form [CDHZ25, Def. 3.4]. Let $\mathcal{T}r_\Pi$ be the set of transcript prefixes, including the empty prefix $\emptyset$. A nonempty prefix ends with a message or with a challenge.

Lemma A.2 (Prefix form of round-by-round soundness)**.** *Let Π be a reduction from $\mathcal{R}_1$ to $\mathcal{R}_2$ whose order is total. Π has round-by-round soundness errors $\varepsilon_0, \dots, \varepsilon_{r-1}$ if and only if there is a function $\mathsf{State}\colon \mathcal{X}_1 \times \mathcal{U} \times \mathcal{T}r_\Pi \to \{0,1\}$ such that for every $\mathsf{x} \in \mathcal{X}_1$ and every $\boldsymbol{U} \in \mathcal{U}$*

(i) $\mathsf{State}(\mathsf{x}, \boldsymbol{U}, \emptyset) = 0$ *if* $(\mathsf{x}, \boldsymbol{U}) \notin \mathcal{R}_1$,

(ii) $\mathsf{State}(\mathsf{x}, \boldsymbol{U}, p \| a_i) = 0$ *for every prefix p of i rounds with* $\mathsf{State}(\mathsf{x}, \boldsymbol{U}, p) = 0$ *and every $a_i \in S_i$,*

(iii) for every prefix p that ends with a message of round i and has $\mathsf{State}(\mathsf{x}, \boldsymbol{U}, p) = 0$,

$$\Pr_{\theta_i \xleftarrow{\$} \mathcal{C}_i} \big[\mathsf{State}(\mathsf{x}, \boldsymbol{U}, p \| \theta_i) = 1\big] \le \varepsilon_i,$$

(iv) $(\mathsf{out}(\mathsf{x}, \mathsf{tr}), \boldsymbol{U}) \notin \mathcal{R}_2$ *for every transcript* tr *of r rounds with* $\mathsf{State}(\mathsf{x}, \boldsymbol{U}, \mathsf{tr}) = 0$.

Proof. Let State satisfy (i)–(iv). Evaluated on the prefixes of a transcript, it is a state function as in Definition 2.24(a). Its condition (iii) is condition (ii) there. For condition (i) there, let tr be a full transcript with $(\mathsf{x}, \boldsymbol{U}) \notin \mathcal{R}_1$ and $(\mathsf{out}(\mathsf{x}, \mathsf{tr}), \boldsymbol{U}) \in \mathcal{R}_2$. The state is 0 on the empty prefix and stays 0 at messages. By (iv) it is 1 on tr, so it moves from 0 to 1 at a challenge. Conversely, let State' be a state function with conditions (i) and (ii) of Definition 2.24(a). Set the state of a prefix to 1 when $(\mathsf{x}, \boldsymbol{U}) \in \mathcal{R}_1$ or when State' moves from 0 to 1 at some round of the prefix, and to 0 otherwise. This function satisfies (i)–(iv). □

Lemma A.3 (Random linear combination)**.** *Let K be a finite field, val a value map to K^I and $n_{\mathrm{cl}} \ge 0$. Consider the reduction with one round, no prover message and a uniform challenge $\pi \in K^{n_{\mathrm{cl}}}$. It maps linear claims $(c^{(l)}, e^{(l)})_{l \in [n_{\mathrm{cl}}]}$ on val to $\big(\sum_l \pi_l c^{(l)}, \sum_l \pi_l e^{(l)}\big)$. It is a reduction from the lists of length n_{cl} in $(\mathcal{R}_{\mathrm{lin}}^{\mathsf{val}})^*$ to $\mathcal{R}_{\mathrm{lin}}^{\mathsf{val}}$ with round-by-round soundness error $1/|K|$, and it is complete on all oracle inputs. It is honest-verifier zero knowledge with error 0 with respect to the uniform simulator.*

Proof. It sends no prover message, so it is honest-verifier zero knowledge with error 0. The soundness error follows from Lemma 2.1, applied to the linear form in π with coefficients $\langle c^{(l)}, \mathsf{val}(\boldsymbol{U})\rangle - e^{(l)}$, which is nonzero when some claim is false. □

B The language in full

This appendix gives the parts of the language that Section 3 only summarizes: the degree of an expression, the interactive proof and the honest prover of a program, and the permutation check.

B.1 Degree

Definition B.1 (Degree)**.** For a set S of variables and an expression e (Definition 3.3), the *degree* $\deg_S(e)$ of e in S is defined by recursion on e as

$$\deg_S(e) = \begin{cases} 1 & \text{if } e = v \text{ for a variable } v \in S, \\ \deg_S(e_v) & \text{if } e = v \text{ for a variable } v \notin S \text{ declared } v := \mathsf{pub}(e_v), \\ 0 & \text{if } e \text{ is another variable or a constant,} \\ \max\big(\deg_S(e_1), \deg_S(e_2)\big) & \text{if } e = e_1 + e_2, \\ \deg_S(e_1) + \deg_S(e_2) & \text{if } e = e_1 \cdot e_2, \\ \deg_S(e_1) & \text{if } e = e_1[I], \\ \deg_S(e_1) + \max_y \deg_S(y) & \text{if } e = \Lambda(e_1), \text{ over the coefficient variables } y \text{ of } \Lambda, \text{ with the empty maximu} \\ s_a \deg_S(e_1) & \text{if } e = \mathsf{prod}_a(e_1), \text{ for the length } s_a \text{ of axis } a. \end{cases}$$

B.2 The interactive proof of a program

Definition B.2 (Interactive proof of a program)**.** Let $\mathcal{P}$ be a program with phases $0, \ldots, n_{\mathrm{ph}}$. The *interactive proof of* $\mathcal{P}$, $\Pi_{\mathcal{P}}$, is the public-coin interactive proof on $\mathsf{x} \in \mathcal{X}^{\mathrm{st}}_{\mathcal{P}}$ with the following messages, in order. The prover first sends the values of the priv variables of phase 0. Then, for $k = 1, \ldots, n_{\mathrm{ph}}$, the verifier sends the value of each chal variable γ of phase k, uniformly on $S_\gamma^{s_\gamma}$, one per message in the order of declaration, and the prover sends the values of the priv variables of phase k. The verifier then sends a final message uniform on a set of one element. The verifier accepts if and only if the resulting assignment is accepting (Definition 3.9).

Let ph' be a map from the variables to $\mathbb{N}$ that satisfies conditions (1), (2) and (4) of Definition 3.7. The interactive proof $\Pi_{\mathcal{P},\mathsf{ph}'}$ orders its messages by ph' in place of the phases. The program $\mathcal{P}$ is round-by-round knowledge sound *under* ph' if $\Pi_{\mathcal{P},\mathsf{ph}'}$ is, in the sense of Definition 3.10(2), with the values of phase 0 under ph' in place of the phase-0 values.

The messages of $\Pi_{\mathcal{P}}$ are totally ordered, so its round-by-round knowledge errors agree with those of Chiesa, Manohar and Spooner (Lemma 2.21).

Let $\mathcal{P}$ be a program without challenges, with the phase function that is 0 on every variable. Let $\mathcal{R}$ be the relation of the pairs (x, w) that some values of the other priv variables extend to an accepting assignment. Then $\mathcal{P}$ is round-by-round knowledge sound for $\mathcal{R}$, with the extractor that outputs the values of the inputs. Indeed, $\Pi_{\mathcal{P}}$ then consists of one prover message and the final message, and the input values of an accepted transcript form a witness, since its assignment is accepting.

Definition B.3 (Honest prover of a program)**.** Let $\mathcal{P}$ be a program, $\mathsf{x} \in \mathcal{X}^{\mathrm{st}}_{\mathcal{P}}$ and $\mathsf{w} \in \mathcal{W}_{\mathcal{P}}$. The *honest prover* of $\mathcal{P}$ on (x, w) is the prover of $\Pi_{\mathcal{P}}$ that computes the execution $\mathsf{Exec}_{\mathcal{P}}(\mathsf{x}, \mathsf{w})$ with each chal variable equal to the challenge that the verifier sends for it. At each of its messages, it sends the values that this execution assigns to the priv variables of the message. Its only coins are the coins of the randomized functions of $\mathcal{P}$.

B.3 Permutation checks

A *record* of arity ℓ is a tuple $(r_0, \ldots, r_{\ell-1})$ of elements of a field F. A list of records consists of ℓ tensors of one length, and record i consists of their i-th entries. Two lists $L = (l_0, \ldots, l_{N_L-1})$ and $R = (r_0, \ldots, r_{N_R-1})$ are *permutations* of each other when $N_L = N_R$ and some bijection σ of $[N_L]$ has $l_i = r_{\sigma(i)}$ for every i.

The permutation check of Section 3.3 encodes each record r as the element $\mathsf{enc}(r) = r_0 + Xr_1 + \cdots + X^{\ell-1}r_{\ell-1}$ of F. The element X is a fixed element of F or a second challenge, as the next paragraph specifies. The check declares a challenge $\delta := \mathsf{chal}$ and the predicate

$$\mathsf{equal}\Big(\mathsf{prod}\big(\delta - \mathsf{enc}(L)\big),\ \mathsf{prod}\big(\delta - \mathsf{enc}(R)\big)\Big).$$

The encoding must be injective on the records that can occur. When every component lies in a subfield E and $\ell \le [F : E]$, the library takes $X = \mathring{u}$, a fixed element with $1, \mathring{u}, \ldots, \mathring{u}^{\ell-1}$ independent over E. Then enc is injective and is also a linear map with constant coefficients. Otherwise X is a second challenge γ, drawn in the same phase as δ.

Lemma B.4. *Let F be a finite field, $\ell \ge 1$, and $L = (l_0, \ldots, l_{N_L-1})$ and $R = (r_0, \ldots, r_{N_R-1})$ lists of records of arity ℓ over F that are not permutations of each other, with $N = \max(N_L, N_R)$. For $X \in F$ let $\mathsf{enc}_X(r) = \sum_{c<\ell} X^c r_c$, and for $\delta, X \in F$ let $\mathsf{MSet}_{\delta,X}(L, R)$ be the equation $\prod_i(\delta - \mathsf{enc}_X(l_i)) = \prod_i(\delta - \mathsf{enc}_X(r_i))$.*

1. *For every $u \in F$ at which enc_u is injective on the records of L and R, $\Pr_{\delta \xleftarrow{\$} F}[\mathsf{MSet}_{\delta,u}(L, R)] \le N/|F|$.*
2. *For uniform γ, the two products of $\mathsf{MSet}_{\delta,\gamma}(L, R)$ differ as polynomials in δ except with probability at most $N(\ell-1)/|F|$. If they differ, $\mathsf{MSet}_{\delta,\gamma}(L, R)$ holds with probability at most $N/|F|$ over a uniform δ. If γ and δ are drawn together, $\mathsf{MSet}_{\delta,\gamma}(L, R)$ holds with probability at most $N\ell/|F|$.*

Proof. (1) Let $\Delta_u(\delta) = \prod_i(\delta - \mathsf{enc}_u(l_i)) - \prod_i(\delta - \mathsf{enc}_u(r_i))$. By injectivity, the encoded lists differ as multisets, so Δ_u is a nonzero polynomial of degree at most N, and $\Pr_\delta[\Delta_u(\delta) = 0] \le N/|F|$ by Lemma 2.1.

(2) Let $\Delta(\gamma, \delta) \in F[\gamma][\delta]$ be the same difference with $X = \gamma$. Distinct records encode to distinct polynomials in γ, so $\Delta \ne 0$ by unique factorization in $F[\gamma][\delta]$. Let $c(\gamma)$ be a nonzero coefficient of Δ in δ. It has degree at most $N(\ell-1)$, so $c(\gamma) = 0$ with probability at most $N(\ell-1)/|F|$ by Lemma 2.1. If $c(\gamma) \ne 0$, then $\Delta(\gamma, \cdot)$ is a nonzero polynomial of degree at most N, and $\Delta(\gamma, \delta) = 0$ with probability at most $N/|F|$ over δ. For γ and δ drawn together, a union bound gives $N(\ell-1)/|F| + N/|F| = N\ell/|F|$. □

The filtered permutation check of Section 3.3 has a flag $a_i \in \{0, 1\}$ for each record l_i of L. It replaces each left factor $\delta - \mathsf{enc}(l_i)$ by $a_i(\delta - \mathsf{enc}(l_i)) + (1 - a_i)$, so an unflagged record contributes 1. Lemma B.4 applies to the flagged records and R, with $N = \max(|L|, |R|)$.

C Security of the phase step

This appendix proves Lemma 5.7.

Proof of Lemma 5.7. Let ph' be the phases that $\mathcal{P}$ assigns to the variables of $\mathcal{P}_0$. For a transcript prefix T of Π_{ph} on $\boldsymbol{U}$, let $\psi(T)$ be the partial transcript of $\Pi_{\mathcal{P}_0,\mathsf{ph}'}$ whose phase-0 values are the values in $\boldsymbol{U}$, whose value at each late value v of $\mathcal{P}_0$ is $\overline{v} - Z_v$, and whose challenges are the challenges of $\mathcal{P}_0$ in T. It omits the values and challenges that the realizations introduce.

(1) Each challenge γ that the realizations introduce has a *bad set* Bad_γ, a function of the values of phase below that of γ. For the key κ of a MAC with binding polynomial G (Lemma 4.5), Bad_κ is the set of roots of G if every entry of the chunks and of the pad copies lies in $\{0, 1\}$ and $G \ne 0$, and $\emptyset$ otherwise. For the challenge α_K of a field K, consider the groups of the tables of K that look up

an entry that is not a row of the table. Then Bad_{α_K} is the set of points $\alpha \in K \setminus \mathcal{N}_\alpha$ at which the LogUp identity of one of these groups holds: the identity of Lemma 5.5(1) for a range table, and the identity with its denominators cleared (Lemma 5.5(3)) for every other table.

The order of $\Pi_{\mathcal{P}_0,\mathsf{ph}'}$ is total, so by Lemma 2.21 its round-by-round knowledge soundness has a state function State_0 on prefixes. While $\mathbf{E}$ fails, State_0 is 0 on the empty prefix, never changes from 0 to 1 at a prover message, changes from 0 to 1 at a challenge γ with probability at most ε_γ, and is 1 on every full transcript that the verifier of $\Pi_{\mathcal{P}_0,\mathsf{ph}'}$ accepts. Fix x and $\boldsymbol{U}$. If $(\mathsf{x}, \boldsymbol{U}) \in \mathcal{R}^{\mathbf{E}}_{\mathcal{R}}$, let $\mathsf{State}(T) = 1$ for every prefix T. Otherwise $\mathbf{E}$ fails on every prefix, since its output is a function of the phase-0 values, and we let $\mathsf{State}(T) = 1$ exactly when

$$\mathsf{State}_0(\psi(T)) = 1, \quad \text{or} \quad \gamma \in \mathrm{Bad}_\gamma \text{ for some challenge } \gamma \text{ in } T \text{ that the realizations introduce.}$$

We check this state on the empty prefix, at the prover messages, at the challenges and on the full transcripts. Items (a) and (b) are immediate, (c) spends the errors of Lemma 5.7(1), and (d), the main step, shows that every full transcript at which the realized predicates hold has state 1.

(a) Before the first challenge, State_0 is 0 and no challenge has been drawn. So $\mathsf{State}(\emptyset) = 0$ when $(\mathsf{x}, \boldsymbol{U}) \notin \mathcal{R}^{\mathbf{E}}_{\mathcal{R}}$.

(b) A message never changes the state from 0 to 1. Its values of $\mathcal{P}_0$ form a message of $\Pi_{\mathcal{P}_0,\mathsf{ph}'}$, and such a message never changes State_0 from 0 to 1. Each bad set is fixed before its challenge, so a message does not change whether a drawn challenge lies in its bad set.

(c) Let T be the prefix before the message of phase k. It fixes $\psi(T)$ up to the first challenge of phase k, and the values of every phase below k. In $\Pi_{\mathcal{P}_0,\mathsf{ph}'}$ the challenges of phase k are consecutive rounds with no prover message between them, so State_0 changes from 0 to 1 at one of them with probability at most $\sum_\gamma \varepsilon_\gamma$ over the challenges γ of $\mathcal{P}_0$ of phase k. At a MAC's key, the chunks and pad copies that Bad_κ depends on have phase below that of the key, and $\kappa \in \mathrm{Bad}_\kappa$ with probability at most $n_{\mathrm{ch}}/2^\omega$ (Lemma 4.5(2)). At α_K, the bad set depends on entries, rows and counts of phase below $\mathsf{ph}(\alpha_K)$, since α_K has the phase after them (Section 5.2). Definition 4.7 requires the grouping of Section 4.1.2, so each group g of a table tb has fewer than $\mathrm{char}\, K$ lookups and contributes at most $(N_g + |\mathsf{tb}| - 1)/(|K| - |\mathcal{N}_\alpha|)$ (Lemma 5.5(1) and (3)). A union bound over the groups gives the error of α_K in Lemma 5.7(1), and Lemma 5.2 gives the error of Lemma 5.7(1) at the message of phase k.

(d) Let T be a full transcript with $((\mathsf{x}, c, \vec{\overline{v}}), \boldsymbol{U}) \in \mathcal{R}_{\mathrm{pred}}$. We show $\mathsf{State}(T) = 1$. If $(\mathsf{x}, \boldsymbol{U}) \in \mathcal{R}^{\mathbf{E}}_{\mathcal{R}}$ or some challenge γ that the realizations introduce lies in Bad_γ, this holds by definition, so suppose neither. Let a be the assignment of T, with each late v equal to $\overline{v} - Z_v$. Then every masked value $\overline{v}$ lies in $\mathcal{X}^{\mathrm{pad}}_v$, the message set of its round in Π_{ph} (Definition 2.23), and every predicate of $\mathsf{Preds}_{\mathcal{F}}$ holds at a. We show that every predicate of $\mathcal{P}_0$ holds at a.

Every priv variable v of type Bit or $\mathsf{Sub}(E)$ lies in its type set at a, by condition (V3). A phase-0 v is stored at the node of its type, or $\mathsf{Preds}_{\mathcal{F}}$ contains its Boolean check $\mathsf{equal}(v \cdot v - v, 0)$. For a late v of type $\mathsf{Sub}(E)$, the pad is stored at the node E and $\overline{v} \in \mathcal{X}^{\mathrm{pad}}_v$, so $\overline{v} - Z_v$ lies in E. A late v of type Bit has its Boolean check.

Every entry looked up is a row of its table, since $\alpha_K \notin \mathrm{Bad}_{\alpha_K}$ and every group's LogUp identity holds. The range checks then hold (Section 4.1.1). The entries of each stmt variable with a declared bound N lie in $[0, N)$, since $\mathsf{x} \in \mathcal{X}^{\mathrm{st}}_{\mathcal{P}_0}$. So every variable lies in its interval (Definition 4.2).

Definition 4.7 requires the condition of each realization of a mod_q to hold, so every mod_q holds at a by Lemma 4.3, whether or not its sum is split. Each MAC's key is not a root of a nonzero

binding polynomial. The check in each of the MAC's two fields holds: a mod_2 by the above, and a check in characteristic 2 or in $\mathbb{F}_{2^\omega}$ by the paragraph after Lemma 4.5. The entries of $Z^{(2)}$ lie in $\{0,1\}$ by condition (V3). Those of $Z^{(1)}$ lie in $\{0,1\}$ by their Boolean checks in odd characteristic, by the check of the tag in a field of characteristic 2 that does not host the MAC, and because $Z^{(1)}$ is one element of $\mathbb{F}_{2^\omega}$ when K_1 hosts the MAC. So both accumulators are congruent to $\mathrm{bits}(\overline{\mathsf{tag}})$ modulo 2, and Lemma 4.5(1) with $t = \overline{\mathsf{tag}}$, the masked tag of T, gives that the two copies of the shared bits are equal. The operands of every $\mathsf{link}(x, y)$ lie in $\{0,1\}$, the type set of Bit, so every $\mathsf{link}(x, y)$ holds.

So the verifier of $\Pi_{\mathcal{P}_0,\mathsf{ph}'}$ accepts $\psi(T)$ completed by its final verifier message, and State_0 is 1 on this completed transcript. The final message has error 0 while $\mathbf{E}$ fails, so State_0 is already 1 on $\psi(T)$, and $\mathsf{State}(T) = 1$.

(2) Each added value is a function of the variables that its declaration names, so the values of S' are those of the honest prover of $\mathcal{P}$ (Definition B.3). The execution S is accepting with probability at least $1 - \varepsilon_\mathrm{c}$ over the challenges and the coins. Where it is accepting, every predicate of $\mathcal{P}_0$ holds at S, and we check that the predicates of the realizations hold at S' at every value of the challenges that $\mathcal{F}$ introduces.

The honest quotient lies in its range (Lemma 4.3(2)), and the honest digits lie in $[0, b)$. Since each group has fewer than $\mathrm{char}\, K$ lookups (Section 4.1.2), the honest counts satisfy each group's identity for a range table at every $\alpha \in K \setminus \mathcal{N}_\alpha$ (Lemma 5.5(2)), and the cleared identity for every other table at every α (Lemma 5.5(3)).

The honest masked tag satisfies the check in each of the MAC's two fields. The copies of the chunks are equal and $Z^{(2)} = Z^{(1)}$, so $\mathsf{acc}_k(\kappa) \equiv \mathrm{bits}(\overline{\mathsf{tag}}) \pmod 2$ for both k (Definition 4.4). This congruence is the check in a field of characteristic 2 that does not host the MAC. In a field of odd characteristic, it gives the mod_2 with its honest quotient digits. In the field that hosts the MAC, the check $\overline{\mathsf{tag}} = Z^{(k)} + \sum_{c<n_\mathrm{ch}} r_c X_{k,c}$ holds by the definition of $\overline{\mathsf{tag}}$ and the equality of the copies.

Every masked value lies in its set. By hypothesis, $Z_v \in \mathcal{X}_v^{\mathrm{pad}}$. A late variable v other than a tag is not an input, so the function of its declaration maps into $\mathcal{X}_v^{\mathsf{type}}$ (Definition 3.6(1)), and $S'_v \in \mathcal{X}_v^{\mathsf{type}} \subseteq \mathcal{X}_v^{\mathrm{pad}}$. For the tag of a MAC, $S'_{\mathsf{tag}} = \sum_{c<n_\mathrm{ch}} r_c X_{1,c}$ lies in $\mathbb{F}_{2^\omega} = \mathcal{X}_{\mathsf{tag}}^{\mathrm{pad}}$. So $\overline{v} = S'_v + Z_v$ lies in $\mathcal{X}_v^{\mathrm{pad}}$. In each predicate a late v takes the value $\overline{v} - Z_v = S'_v$, so $((\mathsf{x}, c, \vec{\overline{v}}), \boldsymbol{U}) \in \mathcal{R}_\mathrm{pred}$.

(3) Fix the input, the phase-0 values other than the pads and the bit copies $Z^{(2)}$ of the MAC pads, the challenges, and the coins of the randomized functions of the late variables. Order the masked values by $\prec$. For $v \in \mathsf{Late}(\mathcal{P})$, let P_v be the pad of v. For a late variable of a field, $P_v = Z_v$, and for the tag of a hosted MAC this pad is $Z^{(1)}$. For the tag of a MAC that no field hosts, P_v is the element Z whose bits are $Z^{(1)}$.

By condition (V1) of Definition 4.8, S'_v is a function of the fixed values and of the pads of the masked values before $\overline{v}$, and not of P_v. For a tag, this holds because the key is a challenge and the chunks are entries of the x_l, and condition (V1) constrains the dependencies of the x_l. An honest prover draws Z_v uniformly from $\mathcal{X}_v^{\mathrm{pad}}$, independent of the other values. No fixed value depends on a pad, since the realizations read a pad only in the declarations of late variables and in the copy $Z^{(2)}$ of a MAC pad (Definition 4.7). The copy $Z^{(1)}$ is uniform on $\mathbb{F}_{2^\omega}$ (Section 4.1.4), so P_v is uniform on $\mathcal{X}_v^{\mathrm{pad}}$ given the fixed values and the earlier pads. The value S'_v lies in $\mathcal{X}_v^{\mathrm{pad}}$, as in (2). So $\overline{v} = S'_v + P_v$ is uniform on $\mathcal{X}_v^{\mathrm{pad}}$ and independent of the earlier masked values.

By the chain rule over $\prec$, the masked values are uniform on $\prod_{v \in \mathsf{Late}(\mathcal{P})} \mathcal{X}_v^{\mathrm{pad}}$ at every witness and every value of the challenges. This distribution is the output of the uniform simulator. The coins are drawn independently of the pads, so the same holds over the coins. □

D The batched zerocheck

This appendix states the objects and protocols of Lemma 5.8: batched predicates, skip domains, the zerocheck with Gruen's univariate skip [Gru24, §5.1], and its masked form Π_{zc}. Our zerocheck differs from Gruen's in four ways: its verifier does not divide, it merges several polynomials by a challenge, it ends with an opening combination where Gruen's ends with the value of the committed constraint function at the sumcheck point, and its masking makes the final check a linear claim on pads and product elements of the oracle input. Ligero proves this claim together with the other claims of the field. Gruen's skip set is a multiplicative subgroup, and he notes that more general sets work [Gru24, footnote 18]. Our skip domains are integer points or bytes of $\mathbb{F}_{2^8}$.

D.1 Batched predicates and skip domains

Definition D.1 (Batched predicate)**.** A *batched predicate* over a field K of a program consists of:

1. polynomials $Q_0, \dots, Q_{n_Q-1} \in K[X_0, \dots, X_{n_{\mathrm{op}}-1}]$, each of total degree at most $D \geq 1$ with $Q_i(0) = 0$;
2. a skip width $k \geq 1$ and a number $m \in \mathbb{N}$ of index variables. The *instances* are the pairs $(b, z) \in \{0,1\}^m \times [k]$, with *table index* b and *skip coordinate* z;
3. a set I_Z of positions of stored values over K (Definition 4.6), whose values we write $Z \in K^{I_Z}$;
4. an *operand map* $\mathsf{op}(Z) = L_{\mathrm{op}} Z + o^0$ with values in $K^{(\{0,1\}^m \times [k]) \times [n_{\mathrm{op}}]}$, where L_{op} is a matrix and o^0 is a vector of constants. We write $o_{(b,z),j}$ for the entry of $\mathsf{op}(Z)$ at the instance (b, z) and operand j.

The predicate *holds* at Z if and only if $Q_i(o_{(b,z),0}, \dots, o_{(b,z),n_{\mathrm{op}}-1}) = 0$ for every i and every instance (b, z). The *batch relation* $\mathcal{R}_{\mathrm{batch}}$ is a relation on oracle inputs with batched predicates $\mathcal{B}$ as statements. It contains $(\mathcal{B}, \boldsymbol{U})$ if and only if $\mathcal{B}$ holds at the values Z of its positions among the stored values of $\boldsymbol{U}$.

Maps that are linear over K are part of the operand map op, so they add no elements to the oracle input. Examples are rotations of bit vectors and, in characteristic two, the exclusive or of bits.

Definition D.2. For a batched predicate of degree D and skip width k, let $N_{\mathrm{sk}} = D(k-1)+1$. A *skip domain* is a sequence of distinct points $y_0, \dots, y_{N_{\mathrm{sk}}-1} \in K$. The first k are the points of the skip coordinates, and the other $(D-1)(k-1)$ are the remaining points. Write ℓ_z for the Lagrange basis polynomials on $y_0, \dots, y_{k-1}$ and ℓ_i^{all} for those on all N_{sk} points.

Our programs use two skip domains. The *integer domain* $y_i = i \cdot 1_K$ is the one-sided domain $\{0, \dots, N_{\mathrm{sk}} - 1\}$. Its points are distinct when the characteristic of K is at least N_{sk}, and it does not need a subgroup or subspace of K. Dao et al. take integer domains in place of Gruen's multiplicative subgroup for small values in large prime fields [DDB+26, §7]. In characteristic two, the *byte domain* $y_i = \mathrm{emb}(\mathsf{enum}(i))$ uses an embedding emb of $\mathbb{F}_{2^8}$ into K and an enumeration $\mathsf{enum}\colon [0, 256) \to \mathbb{F}_{2^8}$, and requires $N_{\mathrm{sk}} \leq 256$. When enum lists the elements by their coordinates in an $\mathbb{F}_2$-basis, its first 2^j points form an $\mathbb{F}_2$-subspace, as in Binius64's skip domain [Irr26b, §4.3.1].

For each batch $\mathcal{B}_j$ of the realized program, the proof system takes the skip width $k = k_j$ of the annotation (Definition 4.7) and chooses a degree bound $D \geq \deg Q_i$ for every i, a number m of index variables and a skip domain (Definition D.2). The proof system also assigns the predicates of $\mathcal{B}_j$ to instances (b, z). When a batch has several polynomials, each Q_i reads its own operands, and an instance holds one equality of each Q_i. An instance with no predicate is a *padding instance*, and its operands $o_{(b,z),0}, \dots, o_{(b,z),n_{\mathrm{op}}-1}$ equal 0. Since every Q_i vanishes at 0, a padding instance satisfies every polynomial. With this data, $\mathcal{B}_j$ is a batched predicate (Definition D.1).

D.2 Multilinear extensions and Gruen's zerocheck

We recall the multilinear extension and the sumcheck-based protocols that the proofs below use.

A function on the hypercube extends to a unique multilinear polynomial.

Definition D.3 (Multilinear extension [Tha23, Def. 3.4, Fact 3.5, Lem. 3.6])**.** Let $\mathbb{F}$ be a field and $n \in \mathbb{N}$. A polynomial is *multilinear* if its degree in each variable is at most one. For $h\colon \{0,1\}^n \to \mathbb{F}$, the *multilinear extension* of h is $\widetilde{h} \in \mathbb{F}[X_0, \dots, X_{n-1}]$,

$$\widetilde{h}(X) = \sum_{b \in \{0,1\}^n} h(b)\, \mathsf{eq}(X, b), \qquad \mathsf{eq}(X, Y) = \prod_{i=0}^{n-1} \big(X_i Y_i + (1 - X_i)(1 - Y_i)\big).$$

We write $\mathbb{F}$, Q, D, $\mathcal{J}$, ξ, τ and ν_i for Gruen's challenge field G and his C, d, D, α, r_0 and r_i. For a number n of variables, a *zerocheck* proves that a polynomial Q of total degree at most D in n_{op} operands vanishes at every index $b \in \{0,1\}^n$ of a table of operands [Gru24, §2.3]. For a random $\xi \in \mathbb{F}^n$, it reduces to a sumcheck [LFKN92], [Tha23, §4.1] of $\mathsf{eq}(\xi, X)\, Q(X)$ with claim 0. In each round i the prover sends only a factor q_i of the round polynomial, and q_i has degree at most D [Gru24, §3.2]. Gruen's *univariate skip* [Gru24, §5.1, §6] replaces the first m_{sk} Boolean variables by one variable over a set $\mathcal{J}$ of size $2^{m_{\mathrm{sk}}}$. The remaining $n - m_{\mathrm{sk}}$ variables correspond to our m index variables. The skip polynomial has degree at most $D(|\mathcal{J}| - 1)$ and is zero on $\mathcal{J}$ when the constraint holds, so its values at $(D-1)(|\mathcal{J}| - 1)$ points outside $\mathcal{J}$ determine it. His soundness error is $\big(D(|\mathcal{J}| - 1) + (n - m_{\mathrm{sk}})(D+1)\big)/|\mathbb{F}|$ [Gru24, App. A.4]. In this error each of the $n - m_{\mathrm{sk}}$ remaining variables contributes $1/|\mathbb{F}|$ for its coordinate of ξ and $D/|\mathbb{F}|$ for its round.

D.3 The skip zerocheck

Definition D.4. Fix a batched predicate and $\varrho \in K$, and let $Q = \sum_{i=0}^{n_Q - 1} \varrho^i Q_i$, the *merged polynomial*, of degree at most D. For a table index b and an operand j, the *index interpolant* is $\hat{o}_{b,j}(X) = \sum_{z<k} o_{(b,z),j}\, \ell_z(X)$. For $\xi \in K^m$, the *skip polynomial* is

$$V_\xi(X) = \sum_{b \in \{0,1\}^m} \mathsf{eq}(\xi, b)\, Q\big(\hat{o}_{b,0}(X), \dots, \hat{o}_{b,n_{\mathrm{op}}-1}(X)\big).$$

The *residual table* of skip coordinate z is $\mathrm{res}_z(b) = Q(o_{(b,z),0}, \dots, o_{(b,z),n_{\mathrm{op}}-1})$. For $\tau \in K$, the *operand table* of operand j at τ is $o_j^\tau\colon \{0,1\}^m \to K$, $o_j^\tau(b) = \hat{o}_{b,j}(\tau)$. We write $\nu = (\nu_0, \dots, \nu_{m-1})$ and $\nu_{<i} = (\nu_0, \dots, \nu_{i-1})$. For challenges $\nu_0, \dots, \nu_{i-1}$, the *round polynomial* of round i is

$$\mathsf{rd}_i(X) = \sum_{x \in \{0,1\}^{m-i-1}} \mathsf{eq}\big(\xi, (\nu_{<i}, X, x)\big)\, Q\big(\widetilde{o_0^\tau}(\nu_{<i}, X, x), \dots, \widetilde{o_{n_{\mathrm{op}}-1}^\tau}(\nu_{<i}, X, x)\big),$$

and its *factor* is

$$q_i(X) = \sum_{x \in \{0,1\}^{m-i-1}} \mathsf{eq}(\xi_{>i}, x)\, Q\big(\widetilde{o_0^\tau}(\nu_{<i}, X, x), \dots, \widetilde{o_{n_{\mathrm{op}}-1}^\tau}(\nu_{<i}, X, x)\big),$$

with $\xi_{>i} = (\xi_{i+1}, \dots, \xi_{m-1})$. Write $\mathsf{eq}_{<i} = \prod_{l<i} \mathsf{eq}(\xi_l, \nu_l)$, with $\mathsf{eq}_{<0} = 1$.

The skip zerocheck follows Gruen's [Gru24, §5.1]. Since eq factors over the coordinates, $\mathsf{rd}_i(X) = \mathsf{eq}_{<i}\, \mathsf{eq}(\xi_i, X)\, q_i(X)$. The index interpolant $\hat{o}_{b,j}$ has degree at most $k-1$, so V_ξ has degree at most $D(k-1)$. The factor q_i has degree at most D, as in Gruen's eq-factored zerocheck (Section D.2). Since $\ell_z(y_{z'}) = 1$ for $z = z'$ and 0 otherwise, $\hat{o}_{b,j}(y_z) = o_{(b,z),j}$, so $V_\xi(y_z) = \widetilde{\mathrm{res}_z}(\xi)$. If some Q_i fails at an instance, then the merged polynomial $\sum_i \varrho^i Q_i$ fails there for all but at most $n_Q - 1$ values of ϱ, and this bound is the merging term of Lemma 5.8.

Protocol D.5 (Skip zerocheck)**.** Common input: a batched predicate (Definition D.1) and a skip domain (Definition D.2). Prover input: the stored values Z.

In the notation of Definition D.4, the prover's messages are the *skip message* $\mathsf{sk}_i = V_\xi(y_{k+i})$ for $i < (D-1)(k-1)$, the coefficients $q_{i,1}, \dots, q_{i,D}$ of $X, \dots, X^D$ in each q_i, and the *terminal claims* $o^\star_0, \dots, o^\star_{n_{\mathrm{op}}-1}$. The terminal claim $o^\star_j$ is the prover's claim for $\widetilde{o^\tau_j}(\nu)$.

	Prover		Verifier
2 :		$\varrho, \xi \longleftarrow$	$\varrho \xleftarrow{\$} K,\ \xi \xleftarrow{\$} K^m$
3 :	$(\mathsf{sk}_i)_{i<(D-1)(k-1)}$	$\longrightarrow (\mathsf{sk}_i)_i$	
4 :		$\tau \longleftarrow$	$\tau \xleftarrow{\$} K$
5 :	$s_0 \leftarrow \sum_i \mathsf{sk}_i\, \ell^{\mathrm{all}}_{k+i}(\tau)$		$s_0 \leftarrow \sum_i \mathsf{sk}_i\, \ell^{\mathrm{all}}_{k+i}(\tau)$
6 :	**for** $i = 0, \dots, m-1$ **do**		
7 :	$q_{i,1}, \dots, q_{i,D}$	$\longrightarrow q_{i,1}, \dots, q_{i,D}$	
8 :		$\nu_i \longleftarrow$	$\nu_i \xleftarrow{\$} K$
9 :	$s_{i+1} \leftarrow \mathsf{eq}(\xi_i, \nu_i)\, s_i + \mathsf{eq}_{<i+1} \sum_{j=1}^{D} (\nu_i^j - \xi_i)\, q_{i,j}$		the same
10 :	$o^\star_0, \dots, o^\star_{n_{\mathrm{op}}-1}$	$\longrightarrow o^\star_0, \dots, o^\star_{n_{\mathrm{op}}-1}$	
11 :		$\varphi \longleftarrow$	$\varphi \xleftarrow{\$} K^n_{\mathrm{op}}$

If $n_Q = 1$, then $\varrho = 1$ and the verifier does not send it. Output claims: the *terminal check* $s_m = \mathsf{eq}(\xi, \nu)\, Q(o^\star_0, \dots, o^\star_{n_{\mathrm{op}}-1})$, and the *opening claim* $\sum_j \varphi_j o^\star_j = \sum_j \varphi_j \widetilde{o^\tau_j}(\nu)$ on Z.

The constant coefficient of q_i is fixed by s_i when $\mathsf{eq}_{<i} \neq 0$, so the prover sends D values per round. Instead of checking each round, the verifier updates the claim to $s_{i+1} = \mathsf{im}_i(\nu_i)$ for the *implied polynomial* $\mathsf{im}_i(X) = \mathsf{eq}(\xi_i, X)\big(s_i + \mathsf{eq}_{<i} \sum_j (X^j - \xi_i)\, q_{i,j}\big)$, and $\mathsf{im}_i(0) + \mathsf{im}_i(1) = s_i$ for every message.

Soundness follows Gruen's analysis [Gru24, App. A.3–A.4], and each term of the error of Lemma 5.8 has a direct source. The state at each point is 1 exactly when the claim current there is true: the predicate before the first challenge, then the vanishing of the skip polynomial on the first k points of the skip domain, then s_0, each s_i, and last the two output claims. The point ξ costs $m/|K|$, and the skip challenge τ costs $D(k-1)/|K|$, the degree of the skip polynomial. The claim s_0 is the value at τ of the polynomial of degree at most $D(k-1)$ that is 0 on the first k points and sk_i on the others. The argument of Gruen's Appendix A.4 uses only that the points of the skip domain are distinct [Gru24, App. A.4 and footnote 18], so it applies to the domains of Definition D.2. Gruen's verifier recovers the round polynomial by dividing by ξ_i and checks a polynomial of degree D, so each of his rounds costs $D/|K|$ [Gru24, §6, step 2(b), footnote 23, and App. A.3]. Our verifier does not divide. A false s_i gives $\mathsf{im}_i \neq \mathsf{rd}_i$, since $\mathsf{im}_i(0) + \mathsf{im}_i(1) = s_i$ and $\mathsf{rd}_i(0) + \mathsf{rd}_i(1)$ is the true sum. Both have degree at most $D+1$, so each round costs $(D+1)/|K|$. The merging challenge ϱ costs $(n_Q - 1)/|K|$. A false terminal vector satisfies the opening claim for at most a fraction $1/|K|$ of the opening combinations φ, so φ costs $1/|K|$. If the terminal vector is true and s_m is false, the terminal check fails for every φ. The total is $\varepsilon^{\Sigma}_{\mathrm{zc}}$ at $m_{\mathcal{B}} = 0$.

D.4 The masked skip zerocheck

Protocol D.6 (Masked skip zerocheck Π_{zc})**.** Common input: as in Protocol D.5. Oracle input: the stored values Z, a pad for each element the prover sends in Protocol D.5 (skip pads P_i^{sk}, round pads $P_{i,j}^{\mathrm{rd}}$ and terminal pads $P_0^{\mathrm{t}}, \ldots, P_{n_{\mathrm{op}}-1}^{\mathrm{t}}$), and a product element P_S for each multiset S of at least two operand indices that is contained in the multiset of operands of a monomial of some Q_i. The honest prover sets $P_S = \prod_{j \in S} P_j^{\mathrm{t}}$. The pads and product elements are stored at the root, among the elements of the oracle input in the field (Protocol 5.1, step 1). The prover runs Protocol D.5 and sends each element v as $\bar{v} = v + P_v$, with P_v its own pad. The verifier applies lines 5–9 of Protocol D.5 to the sent values.

Prover		**Verifier**
	$\xleftarrow{\quad \varrho, \xi \quad}$	$\varrho \xleftarrow{\$} K,\ \xi \xleftarrow{\$} K^m$
$\bar{\mathsf{sk}}_i \leftarrow \mathsf{sk}_i + P_i^{\mathrm{sk}}$	$\xrightarrow{\quad (\bar{\mathsf{sk}}_i)_i \quad}$	
	$\xleftarrow{\quad \tau \quad}$	$\tau \xleftarrow{\$} K$
for $i = 0, \ldots, m-1$ **do**		
$\bar{q}_{i,j} \leftarrow q_{i,j} + P_{i,j}^{\mathrm{rd}}$ for $1 \le j \le D$	$\xrightarrow{\quad (\bar{q}_{i,j})_j \quad}$	
	$\xleftarrow{\quad \nu_i \quad}$	$\nu_i \xleftarrow{\$} K$
$\bar{o}_j^{\star} \leftarrow o_j^{\star} + P_j^{\mathrm{t}}$ for $j < n_{\mathrm{op}}$	$\xrightarrow{\quad (\bar{o}_j^{\star})_j \quad}$	
	$\xleftarrow{\quad \varphi \quad}$	$\varphi \xleftarrow{\$} K_{\mathrm{op}}^n$
		form the terminal constraint

Write $s_m(u, u')$ for the claim that lines 5–9 of Protocol D.5 compute from a skip message u and round coefficients u'. The *terminal constraint* is

$$s_m\big((\bar{\mathsf{sk}}_i)_i, (\bar{q}_{i,j})_{i,j}\big) - s_m\big((P_i^{\mathrm{sk}})_i, (P_{i,j}^{\mathrm{rd}})_{i,j}\big) = \mathsf{eq}(\xi, \nu)\, Q\big(\bar{o}_0^{\star} - P_0^{\mathrm{t}}, \ldots, \bar{o}_{n_{\mathrm{op}}-1}^{\star} - P_{n_{\mathrm{op}}-1}^{\mathrm{t}}\big).$$

The verifier expands its right side over products of terminal pads and replaces each product of two or more terminal pads by its product element. Output claims: the terminal constraint, the opening claim $\sum_j \varphi_j(\bar{o}_j^{\star} - P_j^{\mathrm{t}}) = \sum_j \varphi_j \widetilde{o_j^{\star}}(\nu)$ and, for each product element P_S with $S = \{j_0 \le \cdots \le j_{s-1}\}$, the product constraint $P_S = P_{S'} \cdot P_{j_{s-1}}^{\mathrm{t}}$, where $S' = \{j_0, \ldots, j_{s-2}\}$ and $P_{S'} = P_{j_0}^{\mathrm{t}}$ when $s = 2$.

Proof of Lemma 5.8. The terminal constraint is a linear claim on the pads and product elements, and the verifier computes its coefficients from the challenges and the sent values. At fixed challenges and sent values, $s_m(u, u')$ is linear in (u, u'). Expanding each monomial $\prod_j (\bar{o}_j^{\star} - P_j^{\mathrm{t}})^{k_j}$ of the right side gives products of terminal pads over the sub-multisets of $\{j^{k_j}\}_j$, and each product over a nonempty sub-multiset is a terminal pad or a product element. The set of the multisets S is closed under removing the largest index. So every product constraint reads a terminal pad and a product element of the set, or two terminal pads. Each product element therefore ends a chain of product constraints that starts from a terminal pad. The set is a function of the polynomials Q_i alone, so the product elements are fixed in the oracle input before the merging challenge.

For $Q = X_0X_1 + X_2$ there is one product element, $P_\times = P_0^{\mathrm{t}}P_1^{\mathrm{t}}$, and one product constraint, as in Longfellow's layer check [Fs24, §2.3]. A monomial of degree d contributes at most $2^d - d - 1$ product elements.

(1) Protocol D.6 has the same round errors as Protocol D.5. Its pads are fixed in the oracle input before the first challenge, so subtracting each pad from its sent element gives the messages of a prover of Protocol D.5. Where every product constraint holds, each product element equals the product of its terminal pads. There, by the linearity of s_m, the terminal constraint is the terminal check of Protocol D.5 on these messages, and the opening claim is the opening claim of Protocol D.5.

(2) If the predicate holds, the honest messages of Protocol D.5 pass its terminal check. By induction on i, each s_i equals $\mathsf{rd}_{i-1}(\nu_{i-1})$, the true sum of round i: if $\mathsf{eq}_{<i} \neq 0$, the choice of the constant coefficient of q_i gives $\mathsf{im}_i = \mathsf{rd}_i$, and if $\mathsf{eq}_{<i} = 0$, then $\mathsf{rd}_i = \mathsf{eq}_{<i}\,\mathsf{eq}(\xi_i, X)\,q_i = 0$, $s_i = 0$ and $\mathsf{im}_i = 0$. By the linearity of s_m, the terminal constraint at $P_S = \prod_{j\in S} P_j^{\mathrm{t}}$ is that check, and the opening claim holds. So Protocol D.6 is complete on the oracle inputs with $P_S = \prod_{j\in S} P_j^{\mathrm{t}}$.

(3) The honest prover's messages in Protocol D.5 depend only on Z and the challenges. Each sent element is such a message plus its own pad, and the pads are independent and uniform, so the $(D-1)(k-1) + mD + n_{\mathrm{op}}$ sent elements are independent and uniform on K. The product elements are not sent.

When $m_{\mathcal{B}} > 0$, Protocol D.7 follows in series, and Lemma 2.27 composes the guarantees: the messages of the zerocheck do not read the pads of the bit sumcheck, and Lemma D.8 holds at every value of the zerocheck's pads. By Lemma D.8, it adds an error of $2/|K|$ at each of its $m_{\mathcal{B}}$ challenges and $2m_{\mathcal{B}}$ sent elements, each masked by its own uniform pad. □

D.5 The opening claim on the stored values

The right side of the opening claim is linear in the operands. Write $a^{\mathrm{op}} = (a^{\mathrm{op}}_{b,z})_{(b,z)}$ for the weights $a^{\mathrm{op}}_{b,z} = \mathsf{eq}(\nu, b)\,\ell_z(\tau)$. Then

$$\sum_j \varphi_j \widetilde{o_j^{\tau}}(\nu) = \sum_{b,z,j} a^{\mathrm{op}}_{b,z}\,\varphi_j\,o_{(b,z),j} = \bar{a}^Z + \langle a^Z, Z\rangle, \qquad a^Z = L_{\mathrm{op}}^{\top}(a^{\mathrm{op}} \otimes \varphi), \quad \bar{a}^Z = \langle a^{\mathrm{op}} \otimes \varphi, o^0\rangle.$$

Here $a^{\mathrm{op}} \otimes \varphi$ is the vector indexed by instance and operand whose entry at $((b,z),j)$ is $a^{\mathrm{op}}_{b,z}\,\varphi_j$. The verifier computes a^Z in time linear in the number of nonzero coefficients of L_{op}. For the hash predicates this time is linear in the number of instances, because each operand of an instance reads a fixed number of stored values.

The opening claim becomes $\langle a^Z, Z\rangle + \sum_j \varphi_j P_j^{\mathrm{t}} = \sum_j \varphi_j \bar{o_j^{\star}} - \bar{a}^Z$. Its pad terms read elements of the oracle input at the root. Its terms on Z read stored values. Ring switching along the packing tree reduces this claim together with the field's other linear claims (Protocol F.5). For a batch whose values Z are bits packed into elements of $\mathbb{F}_{2^{16}}$, Protocol D.7 first reduces it, and ring switching receives its output claim.

D.6 The bit sumcheck

For a batch whose values Z are bits packed into elements of $\mathbb{F}_{2^{16}}$, with $2^{\ell_{\mathrm{bit}}} = 16$ bits in each, index the packed elements by ι and the positions within one by $\beta \in \{0,1\}^{\ell_{\mathrm{bit}}}$, and write $t_\iota(\beta)$ for the bit at position β of element ι. The opening claim then reads

$$\sum_{\iota} \sum_{\beta\in\{0,1\}^{\ell_{\mathrm{bit}}}} W_\iota(\beta)\,t_\iota(\beta) = c,$$

where $W_\iota(\beta)$ is the coefficient of that bit in $\langle a^Z, Z\rangle$ and c collects the other terms, which are linear in the pads. Write $\tilde{W}_\iota$ and $\tilde{t}_\iota$ for the multilinear extensions of W_ι and t_ι in the variables β.

Protocol D.7 (Masked bit sumcheck Π_{bit})**.** *Parameters.* The number ℓ_{bit} of bit-index variables, and three distinct points 0, 1 and e_2 of K, as in Longfellow's sumcheck rounds.

Input. The claim above, and pads $P^{\text{bit}}_{i,0}, P^{\text{bit}}_{i,1} \in K$ for $i < \ell_{\text{bit}}$, stored in phase 0 among the elements of the oracle input in K.

Set $c_0 = c$. The verifier does not know the pads, so it treats c and each $u_{i,j} - P^{\text{bit}}_{i,j}$ as affine functions of the elements of the oracle input, and each c_i is such a function. For $i = 0, \dots, \ell_{\text{bit}} - 1$:

1. The prover sends $u_{i,0} = p_i(0) + P^{\text{bit}}_{i,0}$ and $u_{i,1} = p_i(e_2) + P^{\text{bit}}_{i,1}$, for the polynomial
$$p_i(X) = \sum_\iota \sum_{\beta' \in \{0,1\}^{\ell_{\text{bit}} - i - 1}} \tilde{W}_\iota(\nu^{\text{bit}}_0, \dots, \nu^{\text{bit}}_{i-1}, X, \beta')\, \tilde{t}_\iota(\nu^{\text{bit}}_0, \dots, \nu^{\text{bit}}_{i-1}, X, \beta').$$
2. The verifier sends a uniform $\nu^{\text{bit}}_i \in K$.
3. Both set c_{i+1} to the value at ν^{bit}_i of the polynomial of degree at most 2 that takes the values $u_{i,0} - P^{\text{bit}}_{i,0}$, $c_i - (u_{i,0} - P^{\text{bit}}_{i,0})$ and $u_{i,1} - P^{\text{bit}}_{i,1}$ at 0, 1 and e_2.

Output claim. $\sum_\iota \tilde{W}_\iota(\nu^{\text{bit}}) \sum_\beta \mathsf{eq}(\nu^{\text{bit}}, \beta)\, t_\iota(\beta) = c_{\ell_{\text{bit}}}$, where $\nu^{\text{bit}} = (\nu^{\text{bit}}_0, \dots, \nu^{\text{bit}}_{\ell_{\text{bit}}-1})$ and $\tilde{W}_\iota(\nu^{\text{bit}}) = \sum_\beta \mathsf{eq}(\nu^{\text{bit}}, \beta)\, W_\iota(\beta)$. It is linear in the stored bits and in the pads.

Lemma D.8. *Protocol D.7 is a reduction from its input claim to its output claim with round-by-round soundness error $2/|K|$ at each ν^{bit}_i, so $2\ell_{\text{bit}}/|K|$ in total. It is complete. When the prover draws the pads independently and uniformly from K, it is honest-verifier zero knowledge with error 0 with respect to the uniform simulator.*

Proof. The summand $\sum_\iota \tilde{W}_\iota \tilde{t}_\iota$ has degree at most 2 in each variable, so each p_i has degree at most 2, and the honest p_i satisfies $p_i(0) + p_i(1) = c_i$. Let $\hat{p}_i$ be the polynomial of degree at most 2 that the verifier interpolates in round i, so that $\hat{p}_i(0) + \hat{p}_i(1) = c_i$ and $c_{i+1} = \hat{p}_i(\nu^{\text{bit}}_i)$. If c_i is not the true sum, then $\hat{p}_i \neq p_i$, since $p_i(0) + p_i(1)$ is the true sum. Two distinct polynomials of degree at most 2 agree at no more than 2 points, so $c_{i+1} = p_i(\nu^{\text{bit}}_i)$, the true sum of round $i+1$, with probability at most $2/|K|$ over ν^{bit}_i. After the last round the true value is $\sum_\iota \tilde{W}_\iota(\nu^{\text{bit}})\, \tilde{t}_\iota(\nu^{\text{bit}}) = \sum_\iota \tilde{W}_\iota(\nu^{\text{bit}}) \sum_\beta \mathsf{eq}(\nu^{\text{bit}}, \beta)\, t_\iota(\beta)$, which gives the output claim, and on the honest messages it gives completeness. Each sent element is masked by its own uniform pad, so the sent elements are uniform and independent of the witness. The verifier's checks are linear in the pads, so they are linear claims on elements of the oracle input, as in Protocol D.6. □

E The GKR protocol

This appendix recalls layered circuits with assertions and gives the masked GKR protocol of Lemma 5.9.

Definition E.1 (Layered circuit with assertions [Fs26, §§6.3.1–6.3.3])**.** Let $\mathbb{F}$ be a finite field. A *layered circuit* over $\mathbb{F}$ with $n_{\text{lay}} \geq 1$ layers is a tuple $C = ((m_j)_{0 \leq j \leq n_{\text{lay}}}, (Q_j, Z_j)_{0 \leq j < n_{\text{lay}}})$ of numbers of variables $m_j \in \mathbb{N}$ and, for each $j < n_{\text{lay}}$, arrays $Q_j, Z_j \colon \{0,1\}^{m_j} \times \{0,1\}^{m_{j+1}} \times \{0,1\}^{m_{j+1}} \to \mathbb{F}$, such that for every $j < n_{\text{lay}}$ and $g \in \{0,1\}^{m_j}$, $Q_j(g, \cdot, \cdot) = 0$ or $Z_j(g, \cdot, \cdot) = 0$, and Z_j takes values in

$\{0,1\}$. Its *evaluation* on an input $\mathsf{In} \in \mathbb{F}^{\{0,1\}^{m_{n_{\mathrm{lay}}}}}$ is the sequence $V_{n_{\mathrm{lay}}}, \ldots, V_0$, $V_j \in \mathbb{F}^{\{0,1\}^{m_j}}$, with $V_{n_{\mathrm{lay}}} = \mathsf{In}$ and, for $j < n_{\mathrm{lay}}$ and $g \in \{0,1\}^{m_j}$,

$$V_j(g) = \sum_{l,r \in \{0,1\}^{m_{j+1}}} Q_j(g,l,r)\, V_{j+1}(l)\, V_{j+1}(r).$$

The *assertion table* of layer j is $A_j(g) = \sum_{l,r\in\{0,1\}^{m_{j+1}}} Z_j(g,l,r)\, V_{j+1}(l)\, V_{j+1}(r)$ for $g \in \{0,1\}^{m_j}$. The circuit C is *satisfied* on In if and only if $V_0 = 0$ and $A_j = 0$ for every $j < n_{\mathrm{lay}}$. The equations $A_j(g) = 0$ are the *assertions* of layer j.

Longfellow's protocol checks the combination $Q_j + \sigma Z_j$ for a challenge σ [Fs26, §6.3.3].

In Longfellow's form of GKR [Fs24, §2.1, Protocol 2.2], layer j starts from two claims on $\widetilde{V_j}$ at two points, combines them by a challenge, and runs a sumcheck over the variables of its left and right inputs. The sumcheck ends in two claims on $\widetilde{V_{j+1}}$.

For points $\nu, \nu' \in K^{m_j}$ and $\zeta, \sigma \in K$, the *bound quad* of layer j is the polynomial in $2m_{j+1}$ variables

$$\hat{Q}_j(L,R) = \sum_{g\in\{0,1\}^{m_j}} \big(\mathsf{eq}(\nu,g) + \zeta\, \mathsf{eq}(\nu',g)\big)\, \big(\widetilde{Q_j}(g,L,R) + \sigma\, \widetilde{Z_j}(g,L,R)\big).$$

It is multilinear in (L,R). The polynomial $f_j(L,R) = \hat{Q}_j(L,R)\, \widetilde{V_{j+1}}(L)\, \widetilde{V_{j+1}}(R)$ has degree at most 2 in each variable. By Definition D.3,

$$\sum_{l,r\in\{0,1\}^{m_{j+1}}} f_j(l,r) = \widetilde{V_j}(\nu) + \sigma\, \widetilde{A_j}(\nu) + \zeta\, \big(\widetilde{V_j}(\nu') + \sigma\, \widetilde{A_j}(\nu')\big).$$

The sumcheck of layer j has $2m_{j+1}$ rounds. Round k binds coordinate $\lfloor k/2 \rfloor$ of L if k is even and of R if k is odd. This is the order of Longfellow's code. With the variables of rounds $0, \ldots, k-1$ fixed to the challenges $\nu_{j,0}, \ldots, \nu_{j,k-1}$, the *round polynomial* $p_{j,k}(X)$ is the sum of f_j over the Boolean values of the variables of rounds $k+1, \ldots, 2m_{j+1}-1$, with the variable of round k set to X. It has degree at most 2 [Tha23, §4.6.5].

Protocol E.2 (Masked GKR Π_{GKR})**.** Common input: a layered circuit with assertions $C = ((m_j)_{0\le j\le n_{\mathrm{lay}}}, (Q_j, Z_j)_{j<n_{\mathrm{lay}}})$ over K (Definition E.1), an input layout, the assignment pv of the public values that the layout reads, and a point $e_2 \in K \setminus \{0,1\}$, with $e_2 = 2$ in odd characteristic. The input layout assigns to each input wire $b \in \{0,1\}^{m_{n_{\mathrm{lay}}}}$ a public value, a stored value over K or 0. Both parties know pv before the protocol starts. Oracle input: the stored values the layout reads, and, for each layer $j < n_{\mathrm{lay}}$, the round pads $P^{\mathrm{rd}}_{j,k,0}, P^{\mathrm{rd}}_{j,k,\mathrm{e}}$ for $k < 2m_{j+1}$, the claim pads $P^{\mathrm{c}}_{j,0}, P^{\mathrm{c}}_{j,1}$ and the product element $P^{\mathrm{c}}_{j,\times}$, all stored at the root. Every value u the prover sends below is sent as $\bar{u} = u + P_u$, with P_u its own pad: $P^{\mathrm{rd}}_{j,k,0}$ for $p_{j,k}(0)$, $P^{\mathrm{rd}}_{j,k,\mathrm{e}}$ for $p_{j,k}(e_2)$, and $P^{\mathrm{c}}_{j,b}$ for $c_{j+1,b}$. The verifier does not know P_u. It treats $u = \bar{u} - P_u$ as an affine function of the oracle input. Both parties set $\nu^{(0,0)} = \nu^{(0,1)} = \nu^{(0)}$, $c_{0,0} = c_{0,1} = 0$ and, at layer j, $s_{j,0} = c_{j,0} + \zeta_j\, c_{j,1}$. Round k of layer j is a round of the sumcheck of f_j with $\nu = \nu^{(j,0)}$, $\nu' = \nu^{(j,1)}$, $\zeta = \zeta_j$ and $\sigma = \sigma_j$. After it, both parties set $s_{j,k+1} = \hat{p}_{j,k}(\nu_{j,k})$, for the polynomial $\hat{p}_{j,k}$ of degree at most 2 with values $p_{j,k}(0)$ at 0, $s_{j,k} - p_{j,k}(0)$ at 1 and $p_{j,k}(e_2)$ at e_2. After the rounds of layer j, $\nu^{(j+1,0)} = (\nu_{j,0}, \nu_{j,2}, \ldots, \nu_{j,2m_{j+1}-2})$ and $\nu^{(j+1,1)} = (\nu_{j,1}, \nu_{j,3}, \ldots, \nu_{j,2m_{j+1}-1})$.

Prover		**Verifier**
	$\nu^{(0)}$ $\longleftarrow$	$\nu^{(0)} \xleftarrow{\$} K^{m_0}$
for $j = 0, \dots, n_{\text{lay}} - 1$ **do**		
	ζ_j, σ_j $\longleftarrow$	$\zeta_j, \sigma_j \xleftarrow{\$} K$
for $k = 0, \dots, 2m_{j+1} - 1$ **do**		
$p_{j,k}(0),\ p_{j,k}(e_2)$	$p_{j,k}(0), p_{j,k}(e_2)$ $\longrightarrow$	
	$\nu_{j,k}$ $\longleftarrow$	$\nu_{j,k} \xleftarrow{\$} K$
$c_{j+1,b} \leftarrow \widetilde{V_{j+1}}(\nu^{(j+1,b)})$ for $b \in \{0,1\}$	$c_{j+1,0}, c_{j+1,1}$ $\longrightarrow$	form the layer check
	ζ_{in} $\longleftarrow$	$\zeta_{\text{in}} \xleftarrow{\$} K$

The *layer check* of layer j is $s_{j,2m_{j+1}} = \hat{Q}_j(\nu^{(j+1,0)}, \nu^{(j+1,1)})\, c_{j+1,0}\, c_{j+1,1}$. Output claims: the layer checks, the product constraints $P^{\text{c}}_{j,0}\, P^{\text{c}}_{j,1} = P^{\text{c}}_{j,\times}$ for $j < n_{\text{lay}}$, and the *input claim*

$$\sum_{b \in \{0,1\}^{m_{n_{\text{lay}}}}} \big(\mathsf{eq}(\nu^{(n_{\text{lay}},0)}, b) + \zeta_{\text{in}}\, \mathsf{eq}(\nu^{(n_{\text{lay}},1)}, b)\big)\, \mathsf{In}_b = c_{n_{\text{lay}},0} + \zeta_{\text{in}}\, c_{n_{\text{lay}},1},$$

where In_b is the value the layout assigns to wire b.

Write $c_b = c_{j+1,b}$. At fixed challenges and sent values, $s_{j,2m_{j+1}}$ is affine in the round values and in $c_{j,0}, c_{j,1}$. The right side of the layer check is $\hat{Q}_j(\cdot)\big(\bar{c}_0\bar{c}_1 - \bar{c}_0 P^{\text{c}}_{j,1} - \bar{c}_1 P^{\text{c}}_{j,0} + P^{\text{c}}_{j,\times}\big)$, with $P^{\text{c}}_{j,\times}$ in place of $P^{\text{c}}_{j,0} P^{\text{c}}_{j,1}$. So each layer check is one linear claim on the pads and the product element, and the verifier computes its coefficients [Fs24, §2.3]. The verifier moves the public values to the right side of the input claim. The input claim is then a linear claim on the stored values that the layout reads and on the claim pads of the last layer. The rounds add no output claims, because the verifier infers the value of $\hat{p}_{j,k}$ at 1 from the running claim [Fs24, §2.3].

F Ring switching

This appendix gives the two ring-switching protocols of Section 5.5.2 on one edge of a packing tree, proves their lemmas, and composes them along the tree.

F.1 Ring switching on one edge

Throughout this subsection, $S_0 \subseteq K$ and $B \subseteq T \subseteq K$ are finite fields with $d = [T : B]$. Fix a basis of T over B and a basis of K over T, with their dual bases. The oracle input contains $U \in K^m$ and the masks of the protocol. The vector $V \in K^{m'}$ is a fixed function of the entries of the oracle input other than these masks, such as entries of the oracle input or values unpacked from them. The common input is the positions of elements $W \in T^g$ among the entries of $\mathsf{unpack}_{K/T}(U)$ ($W = U$ when $T = K$) and a linear claim $\langle a, \mathsf{unpack}_{T/B}(W)\rangle + \langle b, V\rangle = e$ with $a \in K^{dg}$, $b \in K^{m'}$ and $e \in K$. Both parties write $a_h \in K^d$ for the part of a on $\mathsf{unpack}(W_h)$ and compute the twisted coefficients $f_{k,h} = f_k(a_h)$ for $k < d$ and $h \in [g]$. In both protocols the prover sends $M_k = \sum_h f_{k,h} W_h + R_k$ for

$k < d$. The output claims of both include the d *twisted-sum claims* $\sum_h f_{k,h} W_h + R_k = M_k$, and one further linear claim on V and the mask entries or chain elements, the *remainder claim*. The protocols differ in how they place the masks R_k in the oracle input.

Prover		**Verifier**
$M_k \leftarrow \sum_h f_{k,h} W_h + R_k$ for $k < d$	$\xrightarrow{M_0, \ldots, M_{d-1}}$	

Protocol F.1 (Ring switching with subfield masks)**.** This is $\Pi^{\text{sub}}_{\text{RS}}(B, T)$, for $S_0 \subseteq B$ and a basis $\mathfrak{g}_0, \ldots, \mathfrak{g}_{d_0-1}$ of K over S_0, with $d_0 = [K : S_0]$. Oracle input: the entries of the setting and mask entries $r_{k,i} \in K$ for $0 \le k < d$ and $i \in [d_0]$, which the honest prover draws uniformly from S_0. The masks are $R_k = \sum_i \mathfrak{g}_i \, r_{k,i}$.

Output claims: the d twisted-sum claims, the linear claim

$$\langle b, V \rangle - \sum_k \sum_i \text{Frob}^k_B(\mathfrak{g}_i) \, r_{k,i} = e - \sum_k \text{Frob}^k_B(M_k),$$

and the membership claims $r_{k,i} \in S_0$ for all k, i.

Lemma F.2. *In the setting above, with $S_0 \subseteq B$, let $m, g, m' \ge 0$, and let the oracle input of Protocol F.1 contain the mask entries $r_{k,i}$.*

1. *Let $\mathsf{val} = (\mathsf{unpack}_{T/B}(W), V)$ be the value map. Protocol F.1 is a reduction (Definition 2.23) from $\mathcal{R}^{\mathsf{val}}_{\text{lin}}$ to $(\mathcal{R}^{\mathsf{val}'}_{\text{lin}})^* \wedge \mathcal{R}^*_{\text{mem}}$ (Definition 2.25), for the value map val' that returns W, V and the mask entries, with round-by-round soundness error 0. Its output claims are $d+1$ linear claims and the membership claims $r_{k,i} \in S_0$.*
2. *It is complete on the oracle inputs at which every mask entry lies in S_0.*
3. *When the prover draws the mask entries uniformly from $S_0^{d \times d_0}$, it is honest-verifier zero knowledge with error 0 with respect to the uniform simulator, whose prover messages are $(M_0, \ldots, M_{d-1}) \in K^d$.*

The soundness argument needs the membership claims. If a mask entry lies outside B, then $\sum_i \text{Frob}^k_B(\mathfrak{g}_i) \, r_{k,i}$ can differ from $\text{Frob}^k_B(R_k)$, and the output claims need not imply the input claim.

Proof of Lemma F.2. Write $R'_k = \sum_i \text{Frob}^k_B(\mathfrak{g}_i) \, r_{k,i}$. The Frobenius power Frob^k_B is a field automorphism of K that fixes $B \supseteq S_0$. So if every $r_{k,i}$ lies in S_0, then $\text{Frob}^k_B(R_k) = R'_k$. The map $r \mapsto \sum_i \mathfrak{g}_i r_i$ from $S_0^{d_0}$ to K is a bijection because $(\mathfrak{g}_i)_i$ is a basis. So independent uniform $r_{k,i}$ on S_0 give $(R_0, \ldots, R_{d-1})$ uniform on K^d.

(1) The protocol has one round with a singleton challenge set (Definition 2.23), so it suffices that the output claims imply the input claim. Then no transcript leads from a false input claim to true output claims, and the state function that is 0 at every prefix when the input claim is false satisfies Definition 2.24(a) with error 0. The membership claims give $R'_k = \text{Frob}^k_B(R_k)$. The twisted-sum claims give $\sum_h f_{k,h} W_h = M_k - R_k$, so $\text{Frob}^k_B(\sum_h f_{k,h} W_h) = \text{Frob}^k_B(M_k) - R'_k$. The entries of W lie in T (Definition 2.6, Lemma 2.7), so summing over k, Lemma 2.9 with $L = K$ gives $\langle a, \mathsf{unpack}(W) \rangle = \sum_k \text{Frob}^k_B(M_k) - \sum_k R'_k$. The remainder claim then gives the input claim.

(2) If every mask entry lies in S_0, the membership claims hold. The twisted-sum claims hold by the definition of M_k. The remainder claim follows from the input claim, Lemma 2.9 applied to W, and $R'_k = \text{Frob}^k_B(R_k)$.

(3) Each M_k is $\sum_h f_{k,h} W_h$ plus R_k. For fixed W and V the masks are uniform on K^d, so $(M_0, \ldots, M_{d-1})$ is uniform on K^d. The challenge set is a singleton, so this vector determines the view. □

Protocol F.3 (Ring switching with squaring-chain masks)**.** Let K have characteristic 2 and $|B| = 2^\nu$. This is $\Pi^{\mathrm{sq}}_{\mathrm{RS}}(B, T)$. Oracle input: the entries of the setting, masks $R_k \in K$ for $0 \le k < d$, and *chain elements* $R_{k,j} \in K$ for $0 \le k < d$ and $1 \le j \le \nu k$, with $R_{k,0} = R_k$. The honest prover draws $(R_0, \dots, R_{d-1})$ uniformly from K^d and sets $R_{k,j} = R_k^{2^j}$.

Output claims: the d twisted-sum claims, the linear claim $\langle b, V\rangle - \sum_k R_{k,\nu k} = e - \sum_k \mathrm{Frob}^k_B(M_k)$, and the product constraints $R_{k,j-1} \cdot R_{k,j-1} = R_{k,j}$ for all k and $1 \le j \le \nu k$.

Lemma F.4. *In the setting above, with K, B, ν and d as in Protocol F.3, $m, g, m' \ge 0$, let the oracle input contain the masks R_k and the chain elements $R_{k,j}$.*

1. *Let $\mathsf{val} = (\mathsf{unpack}_{T/B}(W), V)$ be the value map. Protocol F.3 is a reduction (Definition 2.23) from $\mathcal{R}^{\mathsf{val}}_{\mathrm{lin}}$ to $(\mathcal{R}^{\mathsf{val}'}_{\mathrm{lin}})^* \wedge \mathcal{R}^*_{\mathrm{prod}}$ (Definition 2.25), for the value map val' that returns W, V, the masks and the chain elements, with round-by-round soundness error 0. Its output claims are $d+1$ linear claims and the product constraints.*
2. *It is complete on the oracle inputs at which every product constraint holds.*
3. *When the prover draws $(R_0, \dots, R_{d-1})$ uniformly from K^d and sets $R_{k,j} = R_k^{2^j}$, it is honest-verifier zero knowledge with error 0 with respect to the uniform simulator, whose prover messages are $(M_0, \dots, M_{d-1}) \in K^d$.*

Proof of Lemma F.4. Where the product constraints hold, $R_{k,\nu k} = R_k^{2^{\nu k}} = R_k^{|B|^k} = \mathrm{Frob}^k_B(R_k)$ for every value of R_k. So parts (1) and (2) follow as for Lemma F.2, with $R_{k,\nu k}$ in place of R'_k and without membership claims. For part (3), the honest masks are uniform on K^d and the chain elements are functions of them, so the sent values are uniform as in Lemma F.2(3). □

F.2 Ring switching along a packing tree

For each non-root node B of a packing tree (Definition 4.6), give K over $\mathsf{par}(B)$ the product basis of the bases along the path from K to $\mathsf{par}(B)$. By Lemma 2.7, the packed elements of B are then entries of $\mathsf{unpack}_{K/\mathsf{par}(B)}(U)$. Protocols F.1 and F.3 take their input W in this form.

Protocol F.5 (Ring switching along a packing tree Π_{RS})**.** Common input: an admissible packing tree over K (Definitions 4.6 and 5.11) with non-root nodes $B_0, \dots, B_{N-1}$, ordered so that every node comes before its parent, and linear claims on the stored values. Oracle input: $U \in K^m$, and for each $i \in [N]$ the mask entries $r_{k,l}$, or the masks R_k and chain elements $R_{k,j}$, of the step on B_i.

Let L_i be the outstanding linear claims on the values at B_i, that is, on the entries of X_{B_i}, and let Π_i be $\Pi^{\mathrm{sub}}_{\mathrm{RS}}(B_i, \mathsf{par}(B_i))$ (Protocol F.1) if $S_0 \subseteq B_i$ and $\Pi^{\mathrm{sq}}_{\mathrm{RS}}(B_i, \mathsf{par}(B_i))$ (Protocol F.3) otherwise. Step i forms one claim from L_i: it combines L_i by a challenge $\pi^{(i)}$ (Lemma A.3) when $|L_i| \ge 2$, takes the single claim when $|L_i| = 1$, and takes the claim $0 = 0$ when L_i is empty. It then runs Π_i on this claim, with the packed elements of B_i as W.

Prover		**Verifier**
for $i = 0, \dots, N-1$ **do**		
	$\xleftarrow{\quad \pi^{(i)} \quad}$	$\pi^{(i)} \xleftarrow{\$} K^{\lvert L_i\rvert}$ if $\lvert L_i\rvert \ge 2$
combine L_i by $\pi^{(i)}$		combine L_i by $\pi^{(i)}$
run Π_i		run Π_i

The other outstanding claims pass to step $i+1$ unchanged, as the part $\mathcal{R}_{\mathrm{out}}$ of Lemma 2.27, and the linear output claims of Π_i join them. Step i takes as V the entries that the combined claim reads

in U, in the masks, mask entries and chain elements of steps $0, \ldots, i-1$, and at the later nodes $B_{i+1}, \ldots, B_{N-1}$. Lemmas F.2 and F.4 hold for every such V. The protocol Π_{RS} composes the N steps in series (Lemma 2.27, series case). Output claims: the outstanding linear claims, and the membership claims and product constraints of the steps.

Theorem F.6. *Let $\mathcal{T}$ be an admissible packing tree (Definitions 4.6 and 5.11) over a finite field K with evaluation subfield S_0 and N non-root nodes, and let Π_{RS} be Protocol F.5 on it.*

1. *Π_{RS} is a reduction (Definition 2.23) from $(\mathcal{R}^{\mathsf{st}}_{\mathrm{lin}})^*$, the linear claims on the stored values (Definitions 2.25 and 4.6), to $\mathcal{R}^*_{\mathrm{lin}} \wedge \mathcal{R}^*_{\mathrm{prod}} \wedge \mathcal{R}^*_{\mathrm{mem}}$. The output claims are linear claims on U and on the masks, mask entries and chain elements of the steps, the product constraints of the steps with squaring-chain masks, and the membership claims $r_{k,l} \in S_0$ of the steps with subfield masks. The round-by-round soundness error of Π_{RS} is at most $1/|K|$ at each of its at most N challenges $\pi^{(i)}$, and its soundness error is at most $N/|K|$.*
2. *For every value of the masks, mask entries and chain elements that the honest prover draws (Protocols F.1 and F.3), Π_{RS} is complete on the oracle inputs that hold them and satisfy the input claims.*
3. *When the prover draws the masks, mask entries and chain elements of all steps from the product of the steps' honest distributions (Protocols F.1 and F.3), Π_{RS} is honest-verifier zero knowledge with error 0 with respect to the uniform simulator, whose prover messages are its $\sum_i [\mathsf{par}(B_i) : B_i]$ sent elements of K.*

Proof of Theorem F.6. (1) Let $\mathcal{N}_i = \{K, B_i, \ldots, B_{N-1}\}$. Let val_i be the value map that returns the entries of X_T for $T \in \mathcal{N}_i$ and the masks, mask entries and chain elements of steps $0, \ldots, i-1$. We show by induction on i that before step i every outstanding linear claim is a linear claim on val_i. This holds for $i = 0$ because the claims are on the stored values. Suppose that it holds at step i. The combined claim is $\langle a, X_{B_i} \rangle + \langle b, V \rangle = e$ with $X_{B_i} = \mathsf{unpack}(W)$. The packed elements W of B_i are entries of $\mathsf{unpack}_{K/\mathsf{par}(B_i)}(U)$ (Lemma 2.7), and V holds the other values that the claim reads (Protocol F.5). So Π_i applies. Its twisted-sum claims read W and the masks or mask entries of step i. The elements of W are entries of $X_{\mathsf{par}(B_i)}$, and $\mathsf{par}(B_i) \in \mathcal{N}_{i+1}$ because a parent follows its child in the order or is K. Its remainder claim reads V and the mask entries or chain elements of step i. So the statement holds at $i+1$. Step i is thus a reduction from linear claims on val_i to linear claims on val_{i+1}, with its product constraints or membership claims. At $i = N$, $\mathcal{N}_N = \{K\}$ and $X_K = U$, so val_N is the projection onto U and the masks, mask entries and chain elements of all steps. Each step, for every transcript of the earlier steps, is a reduction with round-by-round soundness error $1/|K|$ at its challenge $\pi^{(i)}$ (Lemmas A.3, F.2 and F.4). Series composition (Lemma 2.27, series case, part (a)) keeps each round's error, so the soundness error is at most $N/|K|$.

(2) By Lemmas A.3, F.2 and F.4 and Lemma 2.27, series case, part (b).

(3) The combinations send no prover message. Each step's messages depend on U and, among the masks, only on its own, and its sent values are uniform for every U and every value of the other steps' masks (Lemmas F.2 and F.4). Lemma 2.27, series case, part (c), then gives the joint uniformity of the $\sum_i [\mathsf{par}(B_i) : B_i]$ sent values. □

Ligero proves the output claims. Its evaluation points lie in S_0, and its subfield test covers the rows that hold the mask entries, so it proves the membership claims. The soundness error of ring switching followed by Ligero is at most $N/|K|$ plus Ligero's round errors (Definition 5.12). The column error of Ligero already includes the bound $(k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}/(n_{\mathrm{RS}})_{n_{\mathrm{col}}}$ of the subfield test (Lemma 5.10).

Corollary F.7 is Theorem F.6 for the tree $K \supset S_0 \supset \mathbb{F}_2$ in characteristic 2. It is the simplest tree for bits. The compiler may also split the edge from S_0 to K (Section 4.2.3).

Corollary F.7 (Ring switching for bits in characteristic 2)**.** *Let $K = \mathbb{F}_{2^w}$, and let its evaluation subfield be $S_0 = \mathbb{F}_{2^s}$ with $1 < s < w$. Fix a basis of S_0 over $\mathbb{F}_2$ and a basis of K over S_0, with their dual bases, and give K over $\mathbb{F}_2$ their product basis (Lemma 2.7). The oracle input holds $U \in K^m$, whose entries pack bits, and unpacked values $V \in K^{m'}$. It also holds the masks $R_0, \dots, R_{s-1}$ and chain elements of $\Pi^{\mathrm{sq}}_{\mathrm{RS}}(\mathbb{F}_2, S_0)$ (Protocol F.3) and the mask entries $r_{k,i}$ of $\Pi^{\mathrm{sub}}_{\mathrm{RS}}(S_0, K)$ (Protocol F.1). Let Π be Protocol F.5 on the tree $K \supset S_0 \supset \mathbb{F}_2$ with evaluation subfield S_0. In it K stores V, S_0 stores nothing, and the packed elements of S_0 are U. It has two steps. The first combines the outstanding linear claims (Lemma A.3) and runs $\Pi^{\mathrm{sq}}_{\mathrm{RS}}(\mathbb{F}_2, S_0)$ on the elements $\mathsf{unpack}_{K/S_0}(U)$. The second combines and runs $\Pi^{\mathrm{sub}}_{\mathrm{RS}}(S_0, K)$ on U. Then:*

1. *Π is a reduction (Definition 2.23) from $(\mathcal{R}^{\mathsf{val}}_{\mathrm{lin}})^*$, for the value map $\mathsf{val} = (\mathsf{unpack}_{K/\mathbb{F}_2}(U), V)$, to $\mathcal{R}^*_{\mathrm{lin}} \wedge \mathcal{R}^*_{\mathrm{prod}} \wedge \mathcal{R}^*_{\mathrm{mem}}$ (Definition 2.25). The output claims are linear claims on U, V, the masks, the chain elements and the mask entries, product constraints on the chain elements, and membership claims $r_{k,i} \in S_0$. The round-by-round soundness error is at most $1/|K|$ at each of its at most two challenges, and the soundness error is at most $2/|K|$.*
2. *Π is complete on the oracle inputs at which every mask entry lies in S_0 and every product constraint holds.*
3. *When the prover draws the masks, chain elements and mask entries from the honest distributions of Protocols F.1 and F.3, Π is honest-verifier zero knowledge with error 0 with respect to the uniform simulator, whose prover messages are its $s + w/s$ sent elements of K.*

Proof of Corollary F.7. Apply Theorem F.6 to this tree. The field K has characteristic 2, so the tree is admissible (Definition 5.11). The node $\mathbb{F}_2$ does not contain S_0, so its step is $\Pi^{\mathrm{sq}}_{\mathrm{RS}}(\mathbb{F}_2, S_0)$. The node S_0 contains S_0, so its step is $\Pi^{\mathrm{sub}}_{\mathrm{RS}}(S_0, K)$. The tree has $N = 2$ non-root nodes, so the soundness error is at most $2/|K|$. Since $[S_0 : \mathbb{F}_2] = s$ and $[K : S_0] = w/s$, the prover sends $s + w/s$ elements of K. □

With $S_0 = \mathbb{F}_{2^{16}}$, so $s = 16$, the prover sends 32 elements over the field $\mathbb{F}_{2^{256}}$ of the QROM parameters and 24 over the field $\mathbb{F}_{2^{128}}$ of the ROM parameters. The trees of the ML-DSA-44 proofs of possession add the node $\mathbb{F}_{2^{64}}$ between $\mathbb{F}_{2^{16}}$ and the field, so they have $N = 3$ non-root nodes, soundness error at most $3/|K|$, and $16 + 4 + [K : \mathbb{F}_{2^{64}}]$ sent elements, 24 over $\mathbb{F}_{2^{256}}$ and 22 over $\mathbb{F}_{2^{128}}$.

G Security of Ligero

This appendix proves the soundness, completeness and zero knowledge of Ligero with the mask test (Lemma 5.13). Soundness needs only $n_{\mathrm{col}} \leq k_{\mathrm{RS}}$, and zero knowledge needs $2n_{\mathrm{col}} \leq k_{\mathrm{RS}}$ and $n_{\mathrm{col}} \leq k_{\mathrm{RS}} - \ell$. The appendix also bounds the soundness of Ligero without the mask test (Lemma G.10), including in the order of challenges in which Longfellow runs it (Lemma G.12), and gives counterexamples to the published bounds for this variant (Section G.8).

G.1 Agreement lists and correlated agreement

Definition G.1 (Agreement list [DKT26, eq. (3)])**.** For a word $u \in K^{\mathcal{D}}$, a degree bound $d \geq 0$ and an integer A, the *agreement list* is

$$\mathsf{List}_d(u, A) = \{p \in K[X] : \deg p \leq d,\ |\mathrm{Agr}(p, u)| \geq A\}.$$

Lemma G.2 (Johnson bound [DKT26, Thm. 5.12, eq. (67), and App. B.1])**.** *Let $d \geq 0$ and $1 \leq A \leq n$ be integers with $A^2 > nd$. For every $u \in K^{\mathcal{D}}$,*

$$|\mathsf{List}_d(u, A)| \leq \frac{n\,(A-d)}{A^2 - nd}.$$

Dao, Kominers and Thaler state Lemma G.2 as the second term of their eq. (67), under the hypotheses of their Theorem 5.12, but their pairwise argument in Appendix B.1 needs only $A^2 > nd$. Lemma G.10 applies the bound with $d = 2k_{\mathrm{RS}} - 2$, the degree of the responses of Ligero's tests.

Definition G.3 (Johnson agreement)**.** For $1 \leq d \leq n-1$, the *Johnson agreement* of degree d is the least $A_J \in \mathbb{N}$ with $d(n-1) \leq A_J(A_J+1)$.

The Johnson agreement of degree d is the least a at which the bound of Lemma G.2 allows at most n polynomials of degree at most d to agree with a word on more than a positions. To see this, let $a \in \mathbb{N}$ satisfy $d(n-1) \leq a(a+1)$. If $a+1 \leq d$, the inequality would give $d(n-1) \leq (d-1)d$, so $n \leq d$, contradicting $d \leq n-1$. So $a+1 > d$, and $nd \leq a(a+1) + d < (a+1)^2$, so the denominator of the bound at $A = a+1$ is positive. With a positive denominator, the bound is at most n exactly when $n(a+1-d) \leq n((a+1)^2 - nd)$, which rearranges to $d(n-1) \leq a(a+1)$. The Johnson agreement is at most $n-1$, since $a = n-1$ satisfies this inequality for $d \leq n-1$, and it is $\approx \sqrt{dn}$.

Theorem G.4 (Mutual correlated agreement for affine families [DKT26, Cor. 7.2])**.** *Let $\mathrm{RS}[K, \mathcal{D}, k]$ have full agreement-set line MCA at an agreement threshold $A \geq k$ (Definition 2.12) with exceptional count E, let $t \geq 1$, $a, u_0, \ldots, u_{t-1} \in K^{\mathcal{D}}$, and $w(\chi) = a + \sum_{j=0}^{t-1} \chi_j u_j$ for $\chi \in K^t$. There is a set $\mathcal{Z}_t \subseteq K^t$ of at most $E\,|K|^t/(|K|-1)$ points such that, for every $\chi \notin \mathcal{Z}_t$ and every polynomial p of degree below k with $|\mathrm{Agr}(p, w(\chi))| \geq A$, there are polynomials $p_a, p_0, \ldots, p_{t-1}$ of degree below k with*

$$p = p_a + \sum_{j=0}^{t-1} \chi_j p_j, \qquad \mathrm{Agr}(p, w(\chi)) = \mathrm{Agr}(p_a, a) \cap \bigcap_{j=0}^{t-1} \mathrm{Agr}(p_j, u_j).$$

In Ligero, a is the mask row of the low-degree test, $u_0, \ldots, u_{t-1}$ are the rows of the tableau, and χ is the low-degree challenge. For $\chi \notin \mathcal{Z}_t$, once the low-degree response agrees with $w(\chi)$ on at least A columns, every row agrees with a codeword on that whole agreement set. The density $E/(|K|-1)$ of $\mathcal{Z}_t$ does not depend on t, so the number of rows does not enter the error.

G.2 Ligero's tableau and tests as Longfellow runs them

Definition G.5 (Ligero tableau [Fs26, §4.3], [AHIV22, §4.7])**.** Let K be a finite field and $1 \leq \ell \leq k_{\mathrm{RS}}$. Fix distinct *interpolation points* $\zeta_0, \ldots, \zeta_{2k_{\mathrm{RS}}-2} \in K$, of which $\zeta_{k_{\mathrm{RS}}-\ell}, \ldots, \zeta_{k_{\mathrm{RS}}-1}$ are the *entry points*, and a set $\mathcal{D} \subseteq K$ of n_{RS} evaluation points that contains none of them. A *tableau* with m rows is a matrix in $K^{(m+3)\times\mathcal{D}}$ with rows $M_{\mathrm{ldt}}, M_{\mathrm{lin}}, M_{\mathrm{quad}}, u_0, \ldots, u_{m-1}$. The *honest encoding* of $x \in K^{m\ell}$ is the random tableau of the values at $\mathcal{D}$ of the following polynomials, drawn independently:

1. For each $i \in [m]$, the polynomial u_i has degree below k_{RS}, with $u_i(\zeta_{k_{\mathrm{RS}}-\ell+s}) = x_{i,s}$ for $s \in [\ell]$ and uniform values at $\zeta_0, \ldots, \zeta_{k_{\mathrm{RS}}-\ell-1}$.
2. M_{ldt} is uniform of degree below k_{RS}.
3. M_{lin} is uniform of degree at most $2k_{\mathrm{RS}} - 2$ subject to $\sum_{s=0}^{\ell-1} M_{\mathrm{lin}}(\zeta_{k_{\mathrm{RS}}-\ell+s}) = 0$.
4. M_{quad} is uniform of degree at most $2k_{\mathrm{RS}} - 2$ subject to $M_{\mathrm{quad}}(\zeta_{k_{\mathrm{RS}}-\ell+s}) = 0$ for $s \in [\ell]$.

The rows u_i of an honest encoding are codewords of $\mathrm{RS}[K, \mathcal{D}, k_{\mathrm{RS}}]$ (Definition 2.10), while a cheating prover may commit any matrix of this shape.

Protocol G.6 (Ligero's tests [Fs26, §4.4, §4.5], [AHIV22, §4.7, Figs. 10–11])**.** Oracle input: a tableau with m rows (Definition G.5), committed as an oracle. We write k for k_{RS}. The honest prover commits the honest encoding of $x \in K^{m\ell}$. Common input: linear claims $\langle a_c, x\rangle = e_c$ for $c \in [n_{\mathrm{lin}}]$, with $a_c \in K^{m\ell}$ and $e_c \in K$. Row triples (g_h, g'_h, g''_h) for $h \in [H]$, each claiming $x_{g_h,s}\, x_{g'_h,s} = x_{g''_h,s}$ for $s \in [\ell]$.

Both parties compute from υ the polynomial r_i of degree below k with $r_i(\zeta_{k-\ell+s}) = \sum_c \upsilon_c\, a_{c,(i,s)}$ for $s < \ell$ and $r_i(\zeta_s) = 0$ for $s < k - \ell$.

Prover		**Verifier**
	$\xleftarrow{\quad \chi, \upsilon, \varpi \quad}$	$\chi \xleftarrow{\$} K^m,\ \upsilon \xleftarrow{\$} K^{n_{\mathrm{lin}}},\ \varpi \xleftarrow{\$} K^H$
$w \leftarrow M_{\mathrm{ldt}} + \sum_i \chi_i u_i$ $q \leftarrow M_{\mathrm{lin}} + \sum_i r_i u_i$		
$p \leftarrow M_{\mathrm{quad}} + \sum_h \varpi_h (u_{g''_h} - u_{g_h} u_{g'_h})$	$\xrightarrow{\quad w, q, p \quad}$	$p(\zeta_{k-\ell+s}) \leftarrow 0$ for $s < \ell$
	$\xleftarrow{\quad z_0, \ldots, z_{n_{\mathrm{col}}-1} \quad}$	$z_0, \ldots, z_{n_{\mathrm{col}}-1} \xleftarrow{\$} \mathcal{D}$ distinct
columns at $z_0, \ldots, z_{n_{\mathrm{col}}-1}$	$\xrightarrow{\quad \text{columns} \quad}$	

The prover sends w, of degree below k, as its values at $\zeta_0, \ldots, \zeta_{k-1}$. It sends q, of degree at most $2k-2$, as its values at $\zeta_0, \ldots, \zeta_{2k-2}$. It sends p, of degree at most $2k-2$, as its values at the interpolation points other than the entry points. The verifier accepts if:

1. $\sum_{s=0}^{\ell-1} q(\zeta_{k-\ell+s}) = \sum_c \upsilon_c\, e_c$.
2. $w(z_j) = M_{\mathrm{ldt}}(z_j) + \sum_i \chi_i u_i(z_j)$ for every j.
3. $q(z_j) = M_{\mathrm{lin}}(z_j) + \sum_i r_i(z_j)\, u_i(z_j)$ for every j.
4. $p(z_j) = M_{\mathrm{quad}}(z_j) + \sum_h \varpi_h \big(u_{g''_h}(z_j) - u_{g_h}(z_j)\, u_{g'_h}(z_j)\big)$ for every j.

The first check is the algebraic check of the linear test. On the honest encoding of x, $\sum_i r_i u_i$ takes the value $\sum_i \sum_c \upsilon_c a_{c,(i,s)} x_{i,s}$ at the entry point $\zeta_{k-\ell+s}$. These values sum to $\sum_c \upsilon_c \langle a_c, x\rangle$, and the values of M_{lin} there sum to 0. The algebraic check of the quadratic test requires p to vanish at the entry points, and the format of the response enforces it: the prover omits these values and the verifier sets them to 0. The other three checks are column checks, made at each queried column.

The linear test differs from that of Ames et al. [AHIV22, Fig. 10] in the degree of r_i. Here r_i has degree below k, so q has degree at most $2k-2$, while their response has degree below $k + \ell - 1$.

The low-degree test covers M_{ldt} and the rows u_i. It leaves out the masks M_{lin} and M_{quad} because their degree is above $k-1$, as Longfellow does [Fs26, §4.4.1] and as Ames et al. do [AHIV22, §4.7]. This protocol is Ligero without the mask test in Longfellow's order of challenges (Section G.7).

We lay out a product constraint $x_\iota x_{\iota'} = x_{\iota''}$ on arbitrary entries as Ames et al. [AHIV22, §4.4, §4.7] and Longfellow [Fs26, §4.4.2] do: we copy its operands into the same position of three rows $u_{g_h}, u_{g'_h}, u_{g''_h}$ and add the copy constraints, that each copy equals its original, to the linear claims.

G.3 Ligero with the mask test

Protocol G.7 (Ligero with the mask test, Π_{Lig})**.** Parameters: the finite fields $S_0 \subseteq K$, the interpolation points, entry points and a set $\mathcal{D} \subseteq S_0$ of n_{RS} evaluation points as in Definition G.5

with $\zeta_0, \ldots, \zeta_{k_{\mathrm{RS}}-1} \in S_0$, the number $n_{\mathrm{col}} \le k_{\mathrm{RS}}$ of queried columns, and the number $n_{\mathrm{rep}} \ge 1$ of low-degree challenges.

Oracle input: a tableau (Definition G.5) with two changes. The row M_{ldt} is replaced by n_{rep} rows $M_{\mathrm{ldt}}^{(1)}, \ldots, M_{\mathrm{ldt}}^{(n_{\mathrm{rep}})}$, each drawn as M_{ldt} is. Each of M_{lin} and M_{quad}, written M, is replaced by three *piece rows* M_0, M_1, M_2 of degrees below k_{RS}, k_{RS} and $k_{\mathrm{RS}}-1$. The honest prover draws M as in Definition G.5, and draws (M_0, M_1, M_2) uniformly among the triples within these degree bounds with $M_0 + X^{k_{\mathrm{RS}}-n_{\mathrm{col}}} M_1 + X^{k_{\mathrm{RS}}} M_2 = M$. Some of the rows $u_0, \ldots, u_{m-1}$ are S_0-*subfield rows*, and the honest prover draws their uniform values from S_0. We list the six piece rows as $u_m, \ldots, u_{m+5}$.

Common input: linear claims $\langle a_c, x \rangle = e_c$ for $c \in [n_{\mathrm{lin}}]$, with $a_c \in K^{m\ell}$ and $e_c \in K$. Row triples (g_h, g'_h, g''_h) for $h \in [H]$, each claiming $x_{g_h,s}\, x_{g'_h,s} = x_{g''_h,s}$ for $s \in [\ell]$. The set of S_0-subfield rows. Both parties compute r_i, for $i < m$, as in Protocol G.6 from υ.

Prover		**Verifier**
	$\chi^{(1)}, \ldots, \chi^{(n_{\mathrm{rep}})} \longleftarrow$	$\chi^{(r)} \xleftarrow{\$} K^{m+6}$
$w^{(r)} \leftarrow M_{\mathrm{ldt}}^{(r)} + \sum_{i<m+6} \chi_i^{(r)} u_i$	$\longrightarrow w^{(1)}, \ldots, w^{(n_{\mathrm{rep}})}$	
	$\upsilon, \varpi \longleftarrow$	$\upsilon \xleftarrow{\$} K^{n_{\mathrm{lin}}},\ \varpi \xleftarrow{\$} K^H$
$q \leftarrow M_{\mathrm{lin}} + \sum_{i<m} r_i u_i$		
$p \leftarrow M_{\mathrm{quad}} + \sum_h \varpi_h (u_{g''_h} - u_{g_h} u_{g'_h})$	$\longrightarrow q, p$	$p(\zeta_{k_{\mathrm{RS}}-\ell+s}) \leftarrow 0$ for $s < \ell$
	$z_0, \ldots, z_{n_{\mathrm{col}}-1} \longleftarrow$	$z_0, \ldots, z_{n_{\mathrm{col}}-1} \xleftarrow{\$} \mathcal{D}$ distinct
columns at $z_0, \ldots, z_{n_{\mathrm{col}}-1}$	$\longrightarrow$ columns	

The exchange of the low-degree challenges and responses is the *commit phase*, and the rest is the *proof phase*. The prover sends each $w^{(r)}$ as its values at $\zeta_0, \ldots, \zeta_{k_{\mathrm{RS}}-1}$, q as its values at $\zeta_0, \ldots, \zeta_{2k_{\mathrm{RS}}-2}$, and p as its values at the interpolation points other than the entry points. The pair (υ, ϖ) is the *constraint challenge*. The verifier accepts if $\sum_{s=0}^{\ell-1} q(\zeta_{k_{\mathrm{RS}}-\ell+s}) = \sum_c \upsilon_c\, e_c$ and, at every z_j:

1. $w^{(r)}(z_j) = M_{\mathrm{ldt}}^{(r)}(z_j) + \sum_{i<m+6} \chi_i^{(r)} u_i(z_j)$ for every r.
2. With $M(z_j) = M_0(z_j) + z_j^{k_{\mathrm{RS}}-n_{\mathrm{col}}} M_1(z_j) + z_j^{k_{\mathrm{RS}}} M_2(z_j)$ for each of M_{lin} and M_{quad}, $q(z_j) = M_{\mathrm{lin}}(z_j) + \sum_{i<m} r_i(z_j)\, u_i(z_j)$ and $p(z_j) = M_{\mathrm{quad}}(z_j) + \sum_h \varpi_h \big(u_{g''_h}(z_j) - u_{g_h}(z_j)\, u_{g'_h}(z_j)\big)$.
3. $u_i(z_j) \in S_0$ for every S_0-subfield row u_i (the subfield test).

Protocol G.7 differs from Ligero as Longfellow runs it (Protocol G.6) in four places. The low-degree challenges and responses come before the constraint challenge. The low-degree test is repeated n_{rep} times, each time with its own mask row. The masks of the linear and quadratic tests are split into piece rows, and the low-degree test covers the piece rows. The verifier adds the subfield test.

Protocol G.8 (Simulator of Protocol G.7)**.** Input: the common input cl of Protocol G.7, including the set of S_0-subfield rows, the low-degree challenges $\chi^{(r)}$, the constraint challenge (υ, ϖ) and the points $z_0, \ldots, z_{n_{\mathrm{col}}-1}$ of the column query. Let $T_i = S_0$ for an S_0-subfield row u_i and $T_i = K$ otherwise.

1 : **for** $r = 1, \ldots, n_{\mathrm{rep}}$ **do** $w^{(r)} \xleftarrow{\$} \{f \in K[X] : \deg f < k_{\mathrm{RS}}\}$

2 : $q \xleftarrow{\$} \{f \in K[X] : \deg f \le 2k_{\mathrm{RS}} - 2,\ \sum_{s<\ell} f(\zeta_{k_{\mathrm{RS}}-\ell+s}) = \sum_c v_c e_c\}$

3 : $p \xleftarrow{\$} \{f \in K[X] : \deg f \le 2k_{\mathrm{RS}} - 2,\ f(\zeta_{k_{\mathrm{RS}}-\ell+s}) = 0 \text{ for } s < \ell\}$

4 : **for** $j < n_{\mathrm{col}}$ **do**

5 : **for** $i < m$ **do** $u_i(z_j) \xleftarrow{\$} T_i$

6 : **for** $M \in \{M_{\mathrm{lin}}, M_{\mathrm{quad}}\}$ **do** $M_1(z_j), M_2(z_j) \xleftarrow{\$} K$

7 : $M_{\mathrm{lin}}(z_j) \leftarrow q(z_j) - \sum_{i<m} r_i(z_j)\, u_i(z_j)$

8 : $M_{\mathrm{quad}}(z_j) \leftarrow p(z_j) - \sum_h \varpi_h \big(u_{g_h''}(z_j) - u_{g_h}(z_j)\, u_{g_h'}(z_j)\big)$

9 : **for** $M \in \{M_{\mathrm{lin}}, M_{\mathrm{quad}}\}$ **do** $M_0(z_j) \leftarrow M(z_j) - z_j^{k_{\mathrm{RS}}-n_{\mathrm{col}}} M_1(z_j) - z_j^{k_{\mathrm{RS}}} M_2(z_j)$

10 : **for** $r = 1, \ldots, n_{\mathrm{rep}}$ **do** $M_{\mathrm{ldt}}^{(r)}(z_j) \leftarrow w^{(r)}(z_j) - \sum_{i<m+6} \chi_i^{(r)} u_i(z_j)$

11 : **return** $(w^{(r)})_r, q, p$ in the format of Protocol G.7, and the columns at $z_0, \ldots, z_{n_{\mathrm{col}}-1}$

Lines 1–6 draw from K and S_0. Lines 7–10 solve the verifier's column checks of Protocol G.7 for the values of the masks, the entries of the piece M_0 of each mask and the entries of the rows $M_{\mathrm{ldt}}^{(r)}$. The simulator runs in time polynomial in m, n_{lin}, H, k_{RS}, n_{col} and n_{rep}, since it evaluates w, q, p and $r_0, \ldots, r_{m-1}$ at n_{col} points by Lagrange interpolation over at most $2k_{\mathrm{RS}} - 1$ points.

The simulator of Ames et al. has the same form. It draws the responses uniformly among the polynomials that pass the algebraic checks, draws the queried entries of the rows uniformly from the field, and solves the column checks for the queried entries of the mask rows [AHIV22, Lem. 4.15, pp. 26–27]. The simulator here differs from theirs in two draws. The entries of an S_0-subfield row come from S_0, since they lie in S_0 on the honest tableau. The entries of the pieces M_1 and M_2 are uniform on K.

G.4 Mask pieces and the subfield test

Lemma G.9 (Decomposition into shifted pieces)**.** *Let $\mathbb{F}$ be a field, $r, n_{\mathrm{col}}, D \ge 1$, and let shifts $\tau_0, \ldots, \tau_{r-1}$ and degree bounds $d_0, \ldots, d_{r-1}$ satisfy: for every e with $e + n_{\mathrm{col}} < D$ there is a t with $\tau_t \le e$ and $e + n_{\mathrm{col}} < \tau_t + d_t$. Let $x_0, \ldots, x_{n_{\mathrm{col}}-1} \in \mathbb{F}$ be distinct.*

1. *Every polynomial h of degree below D that vanishes at $x_0, \ldots, x_{n_{\mathrm{col}}-1}$ equals $\sum_t X^{\tau_t} p_t$ for polynomials p_t of degree below d_t that all vanish at $x_0, \ldots, x_{n_{\mathrm{col}}-1}$.*
2. *If moreover $n_{\mathrm{col}} \le d_t$ and $\tau_t + n_{\mathrm{col}} \le D$ for every t, then for all values $y_{t,j} \in \mathbb{F}$ with $\sum_t x_j^{\tau_t} y_{t,j} = 0$ for every j, there are polynomials p_t of degree below d_t with $\sum_t X^{\tau_t} p_t = 0$ and $p_t(x_j) = y_{t,j}$ for all t, j.*
3. *The shifts $0, k_{\mathrm{RS}} - n_{\mathrm{col}}, k_{\mathrm{RS}}$ and bounds $k_{\mathrm{RS}}, k_{\mathrm{RS}}, k_{\mathrm{RS}} - 1$ satisfy the condition for $D = 2k_{\mathrm{RS}} - 1$ if $2n_{\mathrm{col}} \le k_{\mathrm{RS}}$. For two pieces with bounds at most k_{RS} and any shifts, the condition fails for $D = 2k_{\mathrm{RS}} - 1$ if $2 \le n_{\mathrm{col}} \le 2k_{\mathrm{RS}} - 2$.*

Proof. (1) Write $h = ZQ$ with $Z = \prod_j (X - x_j)$ and $\deg Q < D - n_{\mathrm{col}}$. Assign each exponent e of Q to a piece t given by the condition, and let $p_t = Z \sum_{e \mapsto t} Q_e X^{e-\tau_t}$. Then $\deg p_t < n_{\mathrm{col}} + (\tau_t + d_t - n_{\mathrm{col}}) - \tau_t = d_t$, $\sum_t X^{\tau_t} p_t = ZQ = h$, and every p_t vanishes at the points. (2) Let R_t interpolate $y_{t,\cdot}$ at the points, of degree below $n_{\mathrm{col}} \le d_t$. Then $\sum_t X^{\tau_t} R_t$ has degree below D and vanishes at the points. Split it by (1) into pieces p_t' vanishing at the points, and take $p_t = R_t - p_t'$. (3) The intervals $[0, k_{\mathrm{RS}} - n_{\mathrm{col}} - 1]$, $[k_{\mathrm{RS}} - n_{\mathrm{col}}, 2k_{\mathrm{RS}} - 2n_{\mathrm{col}} - 1]$ and $[k_{\mathrm{RS}}, 2k_{\mathrm{RS}} - 2 - n_{\mathrm{col}}]$ cover $[0, 2k_{\mathrm{RS}} - 2 - n_{\mathrm{col}}]$ when $2k_{\mathrm{RS}} - 2n_{\mathrm{col}} - 1 \ge k_{\mathrm{RS}} - 1$. Under the condition, the exponents $0, \ldots, D - 1 - n_{\mathrm{col}}$ lie in the

union of the intervals $[\tau_t, \tau_t + d_t - n_{\mathrm{col}} - 1]$, so $D - n_{\mathrm{col}} \le \sum_t \max(0, d_t - n_{\mathrm{col}})$. For two pieces this becomes $2k_{\mathrm{RS}} - 1 - n_{\mathrm{col}} \le 2\max(0, k_{\mathrm{RS}} - n_{\mathrm{col}})$, which is false when $2 \le n_{\mathrm{col}} \le 2k_{\mathrm{RS}} - 2$. □

The condition of part (3) needs the three pieces and the bound $2n_{\mathrm{col}} \le k_{\mathrm{RS}}$. The split $M_0 + X^{k_{\mathrm{RS}}} M_2$ with bounds k_{RS} and $k_{\mathrm{RS}} - 1$ fails the condition for every $0 < n_{\mathrm{col}} < k_{\mathrm{RS}}$, by the count in the proof of part (3). For the three pieces with $k_{\mathrm{RS}}/2 < n_{\mathrm{col}} < k_{\mathrm{RS}}$, the exponent $k_{\mathrm{RS}} - 1$ satisfies $(k_{\mathrm{RS}} - 1) + n_{\mathrm{col}} < 2k_{\mathrm{RS}} - 1$ but lies in none of the three intervals. For $k_{\mathrm{RS}} \le n_{\mathrm{col}} \le 2k_{\mathrm{RS}} - 2$, the exponent 0 satisfies $n_{\mathrm{col}} < 2k_{\mathrm{RS}} - 1$, but every piece has $\tau_t > 0$ or $\tau_t + d_t \le n_{\mathrm{col}}$.

Lemma 5.10 bounds the subfield test of Protocol G.7. We prove it here.

Proof of Lemma 5.10. Let $p = p_i$ have a coefficient outside S_0. Write $p = \sum_b \mathfrak{e}_b p^{(b)}$ in a basis $\mathfrak{e}_0 = 1, \mathfrak{e}_1, \ldots$ of K over S_0, with each $p^{(b)} \in S_0[X]$ of degree below k_{RS}. Some $p^{(b)}$ with $b \ge 1$ is nonzero. For $x \in S_0$, $p(x)$ lies in S_0 exactly when $p^{(b)}(x) = 0$ for every $b \ge 1$. A nonzero polynomial of degree below k_{RS} has fewer than k_{RS} roots, so p takes values in S_0 at fewer than k_{RS} points of S_0. A drawn column $x \in \mathcal{D}'$ with $v_i(x) \in S_0$ has $p(x) = v_i(x) \in S_0$, so it is one of these points. The n_{col} distinct drawn columns all lie among these fewer than k_{RS} points with probability at most $(k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}/(n_{\mathrm{RS}})_{n_{\mathrm{col}}}$. □

G.5 Security of Ligero with the mask test

Throughout, $\widehat{U}$ is a committed tableau with rows $M_{\mathrm{ldt}}^{(1)}, \ldots, M_{\mathrm{ldt}}^{(n_{\mathrm{rep}})}, u_0, \ldots, u_{m+5}$, where $u_m, \ldots, u_{m+5}$ are the piece rows. For low-degree challenges $\chi = (\chi^{(1)}, \ldots, \chi^{(n_{\mathrm{rep}})})$ with $\chi^{(r)} \in K^{m+6}$, the low-degree combination of repetition r is

$$c_r(\chi) = M_{\mathrm{ldt}}^{(r)} + \sum_{i<m+6} \chi_i^{(r)}\, u_i \in K^{\mathcal{D}}.$$

For low-degree responses $w = (w^{(r)})_r$, the common agreement set is $\mathcal{D}_w = \bigcap_r \mathrm{Agr}(w^{(r)}, c_r(\chi))$. When $|\mathcal{D}_w| \ge A$, let u_i^*, for $i < m+6$, be the polynomial of degree below k_{RS} that interpolates u_i on the first k_{RS} columns of $\mathcal{D}_w$ in a fixed order of $\mathcal{D}$. These columns exist since $A \ge k_{\mathrm{RS}}$ (Definition 2.12). For each of M_{lin} and M_{quad}, written M, with pieces M_0, M_1, M_2, let $M^* = M_0^* + X^{k_{\mathrm{RS}} - n_{\mathrm{col}}} M_1^* + X^{k_{\mathrm{RS}}} M_2^*$. The map Ext of Lemma 5.13(1) sends $(\widehat{U}, \chi, w)$ to the vector $x^* \in K^{m\ell}$ with entries $x_{i,s}^* = u_i^*(\zeta_{k_{\mathrm{RS}} - \ell + s})$ for $i < m$ and $s < \ell$ if $|\mathcal{D}_w| \ge A$, and to 0 otherwise.

Proof of Lemma 5.13. (1) *Step 1.* Fix $\widehat{U}$. For each r, apply Theorem G.4 with $t = m + 6$, $a = M_{\mathrm{ldt}}^{(r)}$ and the rows $u_0, \ldots, u_{m+5}$, and let $\mathcal{Z}^{(r)}$ be its set $\mathcal{Z}_{m+6}$, of density at most $n_{\mathrm{bad}}/(|K| - 1)$. Let $\mathcal{Z}_{\widehat{U}}$ be the set of tuples χ with $\chi^{(r)} \in \mathcal{Z}^{(r)}$ for every r. It depends on $\widehat{U}$ only. The n_{rep} challenges are independent, so its density is at most $(n_{\mathrm{bad}}/(|K| - 1))^{n_{\mathrm{rep}}} = \varepsilon_{\mathrm{com,ldt}}$. Fix $\chi \notin \mathcal{Z}_{\widehat{U}}$ and w with $|\mathcal{D}_w| \ge A$. Some repetition r has $\chi^{(r)} \notin \mathcal{Z}^{(r)}$. The response $w^{(r)}$ has degree below k_{RS} by its format, and $|\mathrm{Agr}(w^{(r)}, c_r(\chi))| \ge |\mathcal{D}_w| \ge A$, so Theorem G.4 gives polynomials $p_a, p_0, \ldots, p_{m+5}$ of degree below k_{RS} with $\mathrm{Agr}(w^{(r)}, c_r(\chi)) = \mathrm{Agr}(p_a, M_{\mathrm{ldt}}^{(r)}) \cap \bigcap_i \mathrm{Agr}(p_i, u_i)$. This set contains $\mathcal{D}_w$, so each u_i agrees with p_i on $\mathcal{D}_w$, in particular on the k_{RS} columns that define u_i^*, and $p_i = u_i^*$. Hence:

(F1) $u_i(x) = u_i^*(x)$ for every $x \in \mathcal{D}_w$ and every $i < m + 6$.

(F2) For every $x \in \mathcal{D}_w$, the mask that the verifier recombines at x, $M_0(x) + x^{k_{\mathrm{RS}} - n_{\mathrm{col}}} M_1(x) + x^{k_{\mathrm{RS}}} M_2(x)$, is $M^*(x)$.

The low-degree test covers the piece rows at the degree bound k_{RS}, so M_2^* has degree below k_{RS}, although the honest M_2 has degree below $k_{\mathrm{RS}} - 1$. So M^* has degree at most $2k_{\mathrm{RS}} - 1$, one more than the bound $2k_{\mathrm{RS}} - 2$ on an honest mask.

Step 2. The map Ext runs in time polynomial in m, n_{RS} and k_{RS}. It computes $\mathcal{D}_w$ by n_{RS} evaluations, the first k_{RS} columns of $\mathcal{D}_w$ by sorting, and each entry $x^*_{i,s}$ by a Lagrange evaluation over these columns. It needs no list decoding, since the challenges and responses (χ, w) fix the one candidate tableau $(u^*_i)_i$. The bad set and the counts below are taken against this tableau, so no round error carries a factor for the number of codewords that agree with a word on at least A positions.

Step 3. Fix cl, $\widehat{U}$, $\chi \notin \mathcal{Z}_{\widehat{U}}$ and w. In the form of Definition 2.23, the proof phase has two rounds. The first is the empty prover message and the constraint challenge (υ, ϖ). The second is the responses (q, p) and the column query $z = (z_0, \ldots, z_{n_{\mathrm{col}}-1})$. The output statement is cl with (χ, w), the challenges and the responses. The output relation holds at it exactly when the verifier accepts. When $|\mathcal{D}_w| \geq A$, let

$$q^* = M^*_{\mathrm{lin}} + \sum_{i<m} r_i\, u^*_i, \qquad p^* = M^*_{\mathrm{quad}} + \sum_h \varpi_h \big(u^*_{g''_h} - u^*_{g_h} u^*_{g'_h}\big),$$

the responses computed from the interpolated rows and masks, of degree at most $2k_{\mathrm{RS}} - 1$. When moreover cl does not hold at x^* (Lemma 5.13), the cases below select a test by the claims that fail at x^*, and the set $\mathcal{B}$ of *bad* constraint challenges is the set at which the computed response of the selected test passes its algebraic check:

(A) If a linear claim fails at x^*, $\mathcal{B}$ is the set of challenges at which $\sum_{s=0}^{\ell-1} q^*(\zeta_{k_{\mathrm{RS}}-\ell+s}) = \sum_c \upsilon_c\, e_c$.

(B) If every linear claim holds at x^* and some product constraint fails, $\mathcal{B}$ is the set of challenges at which $p^*(\zeta_{k_{\mathrm{RS}}-\ell+s}) = 0$ for every $s < \ell$.

(C) Otherwise some membership claim fails at x^*, and $\mathcal{B} = \emptyset$.

The set $\mathcal{B}$ depends only on cl, $\widehat{U}$, χ and w. On the empty prefix and after the empty message, the state is 1 if and only if cl holds at x^*. After the constraint challenge, and after the responses, it is 1 if and only if cl holds at x^*, or $|\mathcal{D}_w| \geq A$ and the challenge lies in $\mathcal{B}$. On a full transcript, it is 1 if and only if the verifier accepts. Condition (i) of Lemma A.2 holds because the state on the empty prefix is 0 when cl fails at x^*. Condition (ii) holds because prover messages do not change the state. Condition (iv) holds because the state on a full transcript is 1 exactly when the verifier accepts. Steps 4 and 5 show condition (iii) at the constraint challenge and at the column query.

Step 4. Let the state before the constraint challenge be 0, so cl does not hold at x^*. The state becomes 1 only if $|\mathcal{D}_w| \geq A$ and the challenge lies in $\mathcal{B}$. In case (A), since $u^*_i(\zeta_{k_{\mathrm{RS}}-\ell+s}) = x^*_{i,s}$ and $r_i(\zeta_{k_{\mathrm{RS}}-\ell+s}) = \sum_c \upsilon_c\, a_{c,(i,s)}$,

$$\sum_{s=0}^{\ell-1} q^*(\zeta_{k_{\mathrm{RS}}-\ell+s}) - \sum_c \upsilon_c\, e_c = \sum_c \upsilon_c \big(\langle a_c, x^*\rangle - e_c\big) + \sum_{s=0}^{\ell-1} M^*_{\mathrm{lin}}(\zeta_{k_{\mathrm{RS}}-\ell+s}).$$

This is affine in υ with the nonzero coefficient vector $(\langle a_c, x^*\rangle - e_c)_c$, and does not depend on ϖ, so $\mathcal{B}$ has density $1/|K|$. In case (B), let the product constraint of the triple h_0 fail at the entry s_0. Then

$$p^*(\zeta_{k_{\mathrm{RS}}-\ell+s_0}) = \sum_h \varpi_h \big(x^*_{g''_h,s_0} - x^*_{g_h,s_0}\, x^*_{g'_h,s_0}\big) + M^*_{\mathrm{quad}}(\zeta_{k_{\mathrm{RS}}-\ell+s_0})$$

is affine in ϖ with a nonzero coefficient at h_0, so $\mathcal{B}$ lies in a set of density $1/|K|$. In case (C), $\mathcal{B}$ is empty. So the state changes from 0 to 1 with probability at most $1/|K| = \varepsilon_{\mathrm{prf,con}}$.

Step 5. Let the state before the column query be 0. Then cl does not hold at x^*, and if $|\mathcal{D}_w| \geq A$ the constraint challenge lies outside $\mathcal{B}$. The responses q and p are fixed, and the state becomes 1

exactly when the verifier accepts. The low-degree column checks at z_j are $w^{(r)}(z_j) = c_r(\chi)(z_j)$ for every r. They hold exactly when $z_j \in \mathcal{D}_w$, so every accepted tuple consists of distinct columns of $\mathcal{D}_w$. We bound the accepted tuples in each case.

- If $|\mathcal{D}_w| < A$, there are $(|\mathcal{D}_w|)_{n_{\mathrm{col}}} \le (A-1)_{n_{\mathrm{col}}}$ tuples of distinct columns of $\mathcal{D}_w$.
- If $|\mathcal{D}_w| \ge A$, there are three cases.
  - Case (A). If q fails the linear test's algebraic check, no tuple is accepted. Otherwise $q \ne q^*$, since q^* fails the check outside $\mathcal{B}$. By (F1) and (F2), the linear column check at an accepted z_j becomes $q(z_j) = q^*(z_j)$. Distinct polynomials of degrees at most $2k_{\mathrm{RS}} - 2$ and $2k_{\mathrm{RS}} - 1$ agree on at most $2k_{\mathrm{RS}} - 1$ points, so at most $(2k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}$ tuples are accepted.
  - Case (B). The polynomial p vanishes at the entry points by its format, and p^* does not outside $\mathcal{B}$, so $p \ne p^*$. By (F1) and (F2), the quadratic column check at an accepted z_j becomes $p(z_j) = p^*(z_j)$. As in case (A), at most $(2k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}$ tuples are accepted.
  - Case (C). Some S_0-subfield row u_i has an entry $x^*_{i,s} = u^*_i(\zeta_{k_{\mathrm{RS}}-\ell+s}) \notin S_0$. Since $\zeta_{k_{\mathrm{RS}}-\ell+s} \in S_0$, $u^*_i \notin S_0[X]$. By (F1), u_i agrees with u^*_i on $\mathcal{D}_w$. So Lemma 5.10, with the one row u_i and $\mathcal{D}' = \mathcal{D}_w$, shows that at most $(k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}$ of the $(n_{\mathrm{RS}})_{n_{\mathrm{col}}}$ tuples lie in $\mathcal{D}_w$ and have every entry of u_i in S_0. The subfield test accepts only these.

The column query is uniform on the $(n_{\mathrm{RS}})_{n_{\mathrm{col}}}$ tuples of distinct columns, so in each case the state changes from 0 to 1 with probability at most one of the three terms of $\varepsilon_{\mathrm{prf,col}}$. This proves (1).

(2) Let cl hold at x. The honest rows $u_0, \dots, u_{m-1}$ and M_{ldt} have degree below k_{RS}. The honest pieces of each mask M are the blocks $(M \bmod X^{k_{\mathrm{RS}}}, 0, M \operatorname{div} X^{k_{\mathrm{RS}}})$ plus a kernel element, all within the degree bounds k_{RS}, k_{RS} and $k_{\mathrm{RS}} - 1$. So every row has degree below k_{RS}. Then each $c_r(\chi)$ is the word of the honest $w^{(r)}$, and $\mathcal{D}_w = \mathcal{D}$. The kernel element recombines to 0, so at every column the recombined masks are the values of M_{lin} and M_{quad}, and the column checks of q and p hold.

The response q has degree at most $2k_{\mathrm{RS}} - 2$, so its values at the $2k_{\mathrm{RS}} - 1$ interpolation points determine it. It passes the algebraic check, since $\sum_s q(\zeta_{k_{\mathrm{RS}}-\ell+s}) = \sum_s M_{\mathrm{lin}}(\zeta_{k_{\mathrm{RS}}-\ell+s}) + \sum_c \upsilon_c \langle a_c, x\rangle = \sum_c \upsilon_c e_c$. The response p has degree at most $2k_{\mathrm{RS}} - 2$ and vanishes at the entry points, since M_{quad} does, $u_g(\zeta_{k_{\mathrm{RS}}-\ell+s}) = x_{g,s}$ and the product constraints hold at x. So the values the prover sends, with the zeros the verifier sets, determine p.

An S_0-subfield row takes values in S_0 at $\zeta_0, \dots, \zeta_{k_{\mathrm{RS}}-1}$, at the entry points by the membership claims and at the other points by the honest prover's draw. These points lie in S_0, so the row, a polynomial of degree below k_{RS}, has coefficients in S_0 and takes values in S_0 on $\mathcal{D} \subseteq S_0$. So the subfield test passes, and the verifier accepts with probability 1.

(3) Let $n_{\mathrm{col}} \le k_{\mathrm{RS}} - \ell$, fix x at which cl holds, and fix the challenges χ, (υ, ϖ) and z. Read as an IOP, Protocol G.7 sends its oracle input, the tableau, as an oracle message, so the view at these challenges consists of the non-oracle messages w, q and p and the answers to the column query (Definition 2.22). The answers are the entries of the $m + 6 + n_{\mathrm{rep}}$ rows at $z_0, \dots, z_{n_{\mathrm{col}}-1}$. Merkle paths appear only in the compiled protocol. Let T_i be as in Protocol G.8. The honest prover's masks are independent and uniform. They are the values $\sigma_i \in T_i^{k_{\mathrm{RS}}-\ell}$ of each row u_i at $\zeta_0, \dots, \zeta_{k_{\mathrm{RS}}-\ell-1}$, the rows $M^{(r)}_{\mathrm{ldt}}$, the polynomials M_{lin} and M_{quad} on their sets of Definition G.5, and the kernel elements κ^{lin} and κ^{quad} of the two splits. Each kernel element is uniform on the kernel $\mathcal{K}$ of $(M_0, M_1, M_2) \mapsto M_0 + X^{k_{\mathrm{RS}}-n_{\mathrm{col}}} M_1 + X^{k_{\mathrm{RS}}} M_2$ on triples of degrees below k_{RS}, k_{RS} and $k_{\mathrm{RS}} - 1$.

By the definitions of w, q, p and of the recombined masks in Protocol G.7, at each z_j the values of M_{lin} and M_{quad} and the entries of the pieces M_0 and of M_{ldt} are those that lines 7–10 of Protocol G.8 compute from w, q, p, the queried row entries $O_R = (u_i(z_j))_{i<m,\, j<n_{\mathrm{col}}}$ and the queried piece entries $O_P = (M_1(z_j), M_2(z_j))_{M,\, j}$. So the view is the image of (w, q, p, O_R, O_P) under the

map of lines 7–11, and it suffices to show that (w, q, p, O_R, O_P) has the distribution of lines 1–6: the five components are independent, with the following distributions. Each $w^{(r)}$ is uniform on the polynomials of degree below k_{RS}. The polynomial q is uniform on the set $\mathcal{Q}_e$ of polynomials of degree at most $2k_{\mathrm{RS}} - 2$ with $\sum_s q(\zeta_{k_{\mathrm{RS}}-\ell+s}) = \sum_c \upsilon_c e_c$. The polynomial p is uniform on the set $\mathcal{P}_0$ of polynomials of degree at most $2k_{\mathrm{RS}} - 2$ that vanish at the entry points. The queried row entries O_R are uniform on $\prod_{i<m} T_i^{n_{\mathrm{col}}}$. The queried piece entries O_P are uniform on $K^{4n_{\mathrm{col}}}$. In (a) and (c) we use that a surjective linear map sends the uniform distribution to the uniform distribution, since its fibres are cosets of its kernel.

(a) Fix $i < m$ and write $T = T_i$. Let $L_0, \ldots, L_{k_{\mathrm{RS}}-1}$ be the Lagrange basis on $\zeta_0, \ldots, \zeta_{k_{\mathrm{RS}}-1}$, which has coefficients in S_0 since these points lie in S_0. Then $u_i = u_i^0 + \sum_{s<k_{\mathrm{RS}}-\ell} \sigma_{i,s} L_s$ with $u_i^0 = \sum_{s<\ell} x_{i,s} L_{k_{\mathrm{RS}}-\ell+s}$. The points z_j lie in $\mathcal{D} \subseteq S_0$, and for an S_0-subfield row the entries $x_{i,s}$ lie in S_0 by the membership claims, so $u_i^0(z_j) \in T$, and $\psi\colon \sigma \mapsto (\sum_s \sigma_s L_s(z_j))_j$ is a T-linear map $T^{k_{\mathrm{RS}}-\ell} \to T^{n_{\mathrm{col}}}$. It is surjective. For $y \in T^{n_{\mathrm{col}}}$, let $Z = \prod_{s<\ell}(X - \zeta_{k_{\mathrm{RS}}-\ell+s}) \in S_0[X]$, which is nonzero at every z_j since $\mathcal{D}$ contains no interpolation point, and let $g \in T[X]$ of degree below n_{col} take the values $y_j/Z(z_j)$ at the points z_j. Then Zg has coefficients in T and degree below $\ell + n_{\mathrm{col}} \le k_{\mathrm{RS}}$, and vanishes at the entry points, so $Zg = \sum_{s<k_{\mathrm{RS}}-\ell}(Zg)(\zeta_s)L_s$ with $(Zg)(\zeta_s) \in T$, and ψ maps these values to y. So the queried entries $u_i^0(z) + \psi(\sigma_i)$ of u_i are uniform on $T^{n_{\mathrm{col}}}$. The rows use disjoint coins, so O_R is uniform on $\prod_i T_i^{n_{\mathrm{col}}}$.

(b) Given $\sigma = (\sigma_i)_i$, the rows $u_0, \ldots, u_{m-1}$ are fixed. The polynomial $f = \sum_i r_i u_i$ has degree at most $2k_{\mathrm{RS}} - 2$ and $\sum_s f(\zeta_{k_{\mathrm{RS}}-\ell+s}) = \sum_c \upsilon_c \langle a_c, x\rangle = \sum_c \upsilon_c e_c$, since the linear claims hold. M_{lin} is uniform on the set $\mathcal{Q}_0$ of polynomials of degree at most $2k_{\mathrm{RS}} - 2$ whose values at the entry points sum to 0, so $q = M_{\mathrm{lin}} + f$ is uniform on $f + \mathcal{Q}_0 = \mathcal{Q}_e$. The polynomial $f' = \sum_h \varpi_h(u_{g_h''} - u_{g_h} u_{g_h'})$ lies in $\mathcal{P}_0$, since the product constraints hold at x, so $p = M_{\mathrm{quad}} + f'$ is uniform on $\mathcal{P}_0$. M_{lin} and M_{quad} are independent, so given σ, (q, p) is uniform on $\mathcal{Q}_e \times \mathcal{P}_0$. So (q, p) is independent of σ, and so of O_R.

(c) Given σ, M_{lin} and M_{quad}, the queried entries of the pieces M_1 and M_2 of a mask M are the queried entries of its blocks, which are fixed, plus $\phi(\kappa)$ for its kernel element κ, where $\phi\colon \mathcal{K} \to K^{2n_{\mathrm{col}}}$ maps κ to $(\kappa_1(z_j), \kappa_2(z_j))_j$. The map ϕ is linear, and Lemma G.9 shows that it is surjective. By part (3) of the lemma, the shifts $0, k_{\mathrm{RS}} - n_{\mathrm{col}}, k_{\mathrm{RS}}$ and bounds $k_{\mathrm{RS}}, k_{\mathrm{RS}}, k_{\mathrm{RS}} - 1$ satisfy its condition for $D = 2k_{\mathrm{RS}} - 1$, since $2n_{\mathrm{col}} \le k_{\mathrm{RS}}$. Also, $2n_{\mathrm{col}} \le k_{\mathrm{RS}}$ with $n_{\mathrm{col}} \ge 1$ gives $n_{\mathrm{col}} \le k_{\mathrm{RS}} - 1$, so $n_{\mathrm{col}} \le d_t$ and $\tau_t + n_{\mathrm{col}} \le D$ for every piece. For $(y_1, y_2) \in K^{2n_{\mathrm{col}}}$, its part (2) with $y_{0,j} = -z_j^{k_{\mathrm{RS}}-n_{\mathrm{col}}} y_{1,j} - z_j^{k_{\mathrm{RS}}} y_{2,j}$ gives pieces within the degree bounds that recombine to 0, an element of $\mathcal{K}$, with values $y_{t,j}$ at z_j. So the queried entries of M_1 and M_2 are uniform on $K^{2n_{\mathrm{col}}}$ whatever M. The two kernel elements are independent, so given $(\sigma, M_{\mathrm{lin}}, M_{\mathrm{quad}})$, O_P is uniform on $K^{4n_{\mathrm{col}}}$, and so independent of (q, p, O_R).

(d) Given all coins other than the rows $M_{\mathrm{ldt}}^{(r)}$, each polynomial $\sum_{i<m+6} \chi_i^{(r)} u_i$ is fixed, of degree below k_{RS}, and the rows $M_{\mathrm{ldt}}^{(r)}$ are independent and uniform on the polynomials of degree below k_{RS}. So the responses $w^{(r)}$ are independent and uniform on them, and independent of (q, p, O_R, O_P).

By (a)–(d), (w, q, p, O_R, O_P) has the distribution of lines 1–6, so at all challenges the view has the distribution of the output of Protocol G.8. The simulator of Lemma 5.13(3) draws the challenges uniformly and runs Protocol G.8, in polynomial time, so the honest-verifier zero-knowledge error is 0. The proof uses $n_{\mathrm{col}} \le k_{\mathrm{RS}} - \ell$ in (a) and $2n_{\mathrm{col}} \le k_{\mathrm{RS}}$ in (c). □

G.6 Security of Ligero without the mask test

This subsection bounds the soundness of Ligero without the mask test and evaluates the bound at Longfellow's parameters. Ligero without the mask test is Protocol G.7, with the low-degree test in the commit phase, except that it commits M_{lin} and M_{quad} as single rows, as in Definition G.5. It leaves them out of the low-degree test, so $\chi \in K^m$, and its column checks use their committed entries directly. Lemma G.10 bounds the linear and quadratic tests with these untested mask rows.

Lemma G.10 (Linear and quadratic tests with an untested mask row)**.** *Let* $\mathrm{RS}[K, \mathcal{D}, k_{\mathrm{RS}}]$ *have block length* n_{RS}*, let* $d = 2k_{\mathrm{RS}} - 2$*, let* A_J *be an integer with* $0 \le A_J$*,* $A_J + 1 \le n_{\mathrm{RS}}$ *and* $(A_J+1)^2 > n_{\mathrm{RS}} d$*, and let*

$$L_{\mathrm{m}} = \left\lfloor \frac{n_{\mathrm{RS}}\,(A_J + 1 - d)}{(A_J+1)^2 - n_{\mathrm{RS}} d} \right\rfloor .$$

Take the linear test or the quadratic test of Protocol G.6, with its claims, on a committed tableau. Let the rows $u_0, \ldots, u_{m-1}$ *agree on a set* $\mathcal{D}' \subseteq \mathcal{D}$ *with polynomials* $p_0, \ldots, p_{m-1}$ *of degree below* k_{RS}*, and let some claim of the test (a linear claim, respectively a product constraint) fail at the entries of the* p_i*. Let the mask row of the test be an arbitrary word* $u \in K^{\mathcal{D}}$*. The set* $\mathcal{D}'$*, the polynomials* p_i *and the word* u *are fixed before the test's challenge. Then for all but a fraction at most* $L_{\mathrm{m}}/|K|$ *of the test's challenges, every response of degree at most* d *that passes the test's algebraic check passes its column check at no more than* A_J *columns of* $\mathcal{D}'$*. So at most* $(A_J)_{n_{\mathrm{col}}}$ *of the* $(n_{\mathrm{RS}})_{n_{\mathrm{col}}}$ *tuples of* n_{col} *distinct columns of* $\mathcal{D}$ *lie in* $\mathcal{D}'$ *and pass. If* $2 \le k_{\mathrm{RS}}$*,* $2k_{\mathrm{RS}} - 1 \le n_{\mathrm{RS}}$ *and* A_J *is the Johnson agreement of degree* d *(Definition G.3), then* $L_{\mathrm{m}} \le n_{\mathrm{RS}}$*.*

Proof of Lemma G.10. Let γ be the test's challenge and f_γ the polynomial computed from $p_0, \ldots, p_{m-1}$: $f_\gamma = \sum_i r_i p_i$ for the linear test and $f_\gamma = \sum_h \varpi_h (p_{g''_h} - p_{g_h} p_{g'_h})$ for the quadratic test, of degree at most d in both. At a column $z \in \mathcal{D}'$ the column check of a response q is $q(z) = u(z) + f_\gamma(z)$. If q has degree at most d and passes the column check at more than A_J columns of $\mathcal{D}'$, then $\mu = q - f_\gamma$ has degree at most d and agrees with u on more than A_J columns, so $\mu \in \mathsf{List}_d(u, A_J + 1)$ (Definition G.1), which has at most L_{m} members by Lemma G.2 at $A = A_J + 1$. This list depends on u only, so it is fixed before γ.

Fix a member μ. Write x for the entries of $p_0, \ldots, p_{m-1}$. For the linear test, $q = f_\gamma + \mu$ passes the algebraic check exactly when $\sum_c \upsilon_c(\langle a_c, x\rangle - e_c) + \sum_s \mu(\zeta_{k_{\mathrm{RS}}-\ell+s}) = 0$. For the quadratic test, it passes exactly when $\sum_h \varpi_h (x_{g''_h,s} - x_{g_h,s} x_{g'_h,s}) + \mu(\zeta_{k_{\mathrm{RS}}-\ell+s}) = 0$ for every s. A claim of the test fails at x, so one of these equations is affine in γ with a nonzero coefficient vector and holds for a fraction at most $1/|K|$ of the challenges. A union over the at most L_{m} members gives the fraction $L_{\mathrm{m}}/|K|$. At every other challenge, every response of degree at most d that passes the algebraic check passes the column check on a set of at most A_J columns of $\mathcal{D}'$, which contains at most $(A_J)_{n_{\mathrm{col}}}$ tuples of n_{col} distinct columns. At the Johnson agreement of degree d, $L_{\mathrm{m}} \le n_{\mathrm{RS}}$ by the paragraph after Definition G.3. □

Corollary G.11 (Ligero without the mask test)**.** *Lemma 5.13 holds for Ligero without the mask test with tableaus in* $K^{(m+2+n_{\mathrm{rep}})\times\mathcal{D}}$*, low-degree challenges in* K^m*, and the round errors*

$$\varepsilon_{\mathrm{com,ldt}} = \Big(\frac{n_{\mathrm{bad}}}{|K|-1}\Big)^{n_{\mathrm{rep}}}, \qquad \varepsilon_{\mathrm{prf,con}} = \frac{L_{\mathrm{m}}}{|K|}, \qquad \varepsilon_{\mathrm{prf,col}} = \frac{(A-1)_{n_{\mathrm{col}}} + (A_J)_{n_{\mathrm{col}}} + (k_{\mathrm{RS}}-1)_{n_{\mathrm{col}}}}{(n_{\mathrm{RS}})_{n_{\mathrm{col}}}},$$

for A_J *and* L_{m} *as in Lemma G.10, and without the condition* $2n_{\mathrm{col}} \le k_{\mathrm{RS}}$*. As in Definition 5.12, the term* $(k_{\mathrm{RS}} - 1)_{n_{\mathrm{col}}}$ *of* $\varepsilon_{\mathrm{prf,col}}$ *is present only when the tableau has* S_0*-subfield rows.*

Proof. The proof is that of Section G.5, with Lemma G.10 in place of Steps 4 and 5 for cases (A) and (B) of part (1). In these cases, $\mathcal{B}$ is the set of challenges of the selected test that Lemma G.10

excludes for the rows u_i^*, $\mathcal{D}' = \mathcal{D}_w$ and the committed mask row. The set $\mathcal{B}$ has density at most $L_{\mathrm{m}}/|K|$, and outside it at most $(A_J)_{n_{\mathrm{col}}}$ tuples are accepted. Part (3) uses Protocol G.8 without the draws of the pieces (line 6) and their recombination (line 9), and with the sum of line 10 over $i < m$. The entries of the mask rows M_{lin} and M_{quad} are those that lines 7 and 8 compute. Part (c) of the proof and the condition $2n_{\mathrm{col}} \le k_{\mathrm{RS}}$ are not needed. □

The cost of omitting the mask test depends on the rate. The figures in this paragraph are at the unique-decoding threshold. At rate 1/7, at the parameters of Cascudo et al.'s Section 6 ($n_{\mathrm{RS}} = 2945$ and 4096, $n_{\mathrm{col}} = 140$) [CHVY25, p. 23], the column term of Corollary G.11 differs from the published term by less than 0.001 bit ($2^{-116.875}$ and $2^{-115.734}$ at the two block lengths, with Cascudo et al.'s $\lfloor (1+\rho)n_{\mathrm{RS}}/2 \rfloor$ in place of $A-1$ and $\lfloor 2\rho n_{\mathrm{RS}} \rfloor - 2$ in place of $2k_{\mathrm{RS}} - 2$, for their rate ρ). At the five tableaus of Longfellow version 7 [Goo26a], the hash tableaus of the mDL proofs with one to four attributes and the signature tableau (132 queried columns) [Fs26, §5.1, p. 38], the Johnson agreement lies below the unique-decoding threshold, so the mask term is negligible. The column term of Corollary G.11 gives 109.5 to 109.7 bits there, and Cascudo et al.'s formula gives 109.4 to 109.6. At four of the five tableaus the formula counts tuples within $\lfloor (n_{\mathrm{RS}} + k_{\mathrm{RS}})/2 \rfloor$ columns, one more than the $A-1$ of Corollary G.11, and gives about 0.1 bit less. At the fifth the two terms differ by less than 0.001 bit.

At the rate-1/4 parameter sets of versions 5 and 6 (128 queried columns) [Goo26a], the column term of Corollary G.11 gives 65.9 to 66.8 bits, where Cascudo et al.'s formula gives 89.3 to 90.6. The formula of Longfellow's own analysis [Fs24, §2.2, p. 9] gives 86.7 to 86.8 bits, and 64.1 to 64.3 with its terms for the linear and quadratic tests replaced by the mask term $(A_J/n_{\mathrm{RS}})^{n_{\mathrm{col}}}$ (Section 5.6.1).

G.7 Ligero in Longfellow's order of challenges

This subsection bounds Ligero without the mask test in Longfellow's order of challenges and evaluates the bound at the parameters of Longfellow version 7. In this order, Longfellow sends the low-degree challenge together with the constraint challenges, before any response [Fs26, §4.4.4, p. 28].

Lemma G.12 (Ligero in Longfellow's order of challenges)**.** *Let K, the interpolation points $\zeta_0, \ldots, \zeta_{2k_{\mathrm{RS}}-2}$, the entry points, $\mathcal{D}$, k_{RS} and ℓ be as in Definition G.5, with $n_{\mathrm{RS}} = |\mathcal{D}|$, and let $1 \le n_{\mathrm{col}} \le n_{\mathrm{RS}}$. Let A be an agreement threshold of $\mathrm{RS}[K, \mathcal{D}, k_{\mathrm{RS}}]$ within the Johnson radius (Definition 2.12) at which the code has full agreement-set line MCA with exceptional count n_{bad} (Definition 2.13). Let A_J be an integer with $0 \le A_J$, $A_J + 1 \le n_{\mathrm{RS}}$ and $(A_J+1)^2 > n_{\mathrm{RS}}(2k_{\mathrm{RS}} - 2)$, and let L_{m} be as in Lemma G.10. Let*

$$L(A) = \left\lfloor \frac{n_{\mathrm{RS}}\,(A - k_{\mathrm{RS}} + 1)}{A^2 - n_{\mathrm{RS}}(k_{\mathrm{RS}} - 1)} \right\rfloor,$$

$$\varepsilon_{\mathrm{prf,joint}} = \frac{n_{\mathrm{bad}}}{|K| - 1} + \frac{L(A)\,L_{\mathrm{m}}}{|K|}, \qquad \varepsilon_{\mathrm{prf,col}} = \frac{(A-1)_{n_{\mathrm{col}}} + (A_J)_{n_{\mathrm{col}}}}{(n_{\mathrm{RS}})_{n_{\mathrm{col}}}}.$$

For a tableau $\widehat{U}$ with rows $M_{\mathrm{ldt}}, M_{\mathrm{lin}}, M_{\mathrm{quad}}, u_0, \ldots, u_{m-1}$, the interleaved agreement list $\mathsf{IList}_{k_{\mathrm{RS}}-1}(\widehat{U}, A)$ *is the set of tuples $P = (p_0, \ldots, p_{m-1})$ of polynomials of degree below k_{RS} whose common agreement set $\mathcal{D}_P = \bigcap_{i<m} \mathrm{Agr}(p_i, u_i)$ has at least A columns. The* entries *of such a tuple P are the vector $x(P) \in K^{m\ell}$ with $x(P)_{i,s} = p_i(\zeta_{k_{\mathrm{RS}}-\ell+s})$ for $i < m$ and $s < \ell$. A common input cl of Protocol G.6 is a statement of $\mathcal{R}^*_{\mathrm{lin}} \wedge \mathcal{R}^*_{\mathrm{prod}}$ on x (Definition 2.25), and* cl holds at x *if $(\mathsf{cl}, x) \in \mathcal{R}^*_{\mathrm{lin}} \wedge \mathcal{R}^*_{\mathrm{prod}}$. For a low-degree challenge χ and a low-degree response w, let $\mathcal{D}_w = \mathrm{Agr}(w, M_{\mathrm{ldt}} + \sum_{i<m} \chi_i u_i)$. If $|\mathcal{D}_w| \ge A$, let P_w be the tuple of the polynomials of degree below k_{RS} that interpolate $u_0, \ldots, u_{m-1}$ on the first k_{RS} columns of $\mathcal{D}_w$.*

1. *For every tableau* $\widehat{U}$, $|\mathsf{IList}_{k_{\mathrm{RS}}-1}(\widehat{U}, A)| \le L(A)$. *If* $2A > n_{\mathrm{RS}}+k_{\mathrm{RS}}-1$, *then* $|\mathsf{IList}_{k_{\mathrm{RS}}-1}(\widehat{U}, A)| \le 1$.
2. *View Protocol G.6 as a series–parallel IOP (Definition 2.18) for* $\mathcal{R}^*_{\mathrm{lin}} \wedge \mathcal{R}^*_{\mathrm{prod}}$ *with the total order on four messages. The first is the tableau* $\widehat{U}$ *as an oracle string. The second is the* joint challenge (χ, υ, ϖ). *The third is the responses* (w, q, p), *and the fourth is the column query. Let* $\mathbf{E}$ *be the extractor that outputs* 0 *at the joint challenge. At the column query it outputs* $x(P_w)$ *if* $|\mathcal{D}_w| \ge A$, *and* 0 *otherwise. Then the IOP has round-by-round knowledge errors* $\varepsilon_{\mathrm{prf,joint}}$ *at the joint challenge and* $\varepsilon_{\mathrm{prf,col}}$ *at the column query (Definition 2.20), with the extractor* $\mathbf{E}$. *If* $2A > n_{\mathrm{RS}} + k_{\mathrm{RS}} - 1$, *it has these errors with* 1 *in place of* $L(A)$.

By part (2), for a statement cl that holds at no x, the verifier of Protocol G.6 accepts every prover with probability at most $\varepsilon_{\mathrm{prf,joint}} + \varepsilon_{\mathrm{prf,col}}$. On such a statement the extractor always fails, so by condition (a) of Definition 2.20 every accepted transcript has a challenge at which the state moves from 0 to 1, and by condition (b) a union bound over the two challenges gives the sum. The errors are those of Corollary G.11 with one low-degree challenge and no subfield term, with one change: the constraint term $L_{\mathrm{m}}/|K|$ is multiplied by $L(A)$ and joins the low-degree term in one round. The extractor uses the low-degree challenge and the low-degree response, which precede the column query, but the response w selects the candidate tuple P_w only after the constraint challenge. So the state after the joint challenge covers the bad constraint challenges of every member of the list. The column query needs no union over the list, since w fixes P_w before it.

Proof of Lemma G.12. (1) Let $P^1, \dots, P^N$ be distinct members of $\mathsf{IList}_{k_{\mathrm{RS}}-1}(\widehat{U}, A)$, with $P^j = (p^j_0, \dots, p^j_{m-1})$. For each j, choose $S_j \subseteq \mathcal{D}_{P^j}$ with $|S_j| = A$. Let $j \ne j'$. Some row i has $p^j_i \ne p^{j'}_i$. At every column $x \in S_j \cap S_{j'}$, $p^j_i(x) = u_i(x) = p^{j'}_i(x)$. Two distinct polynomials of degree below k_{RS} agree on at most $k_{\mathrm{RS}} - 1$ points, so $|S_j \cap S_{j'}| \le k_{\mathrm{RS}} - 1$. For $x \in \mathcal{D}$, let c_x be the number of indices j with $x \in S_j$. Then

$$\sum_x c_x = NA, \qquad \sum_x c_x(c_x - 1) = \sum_{j \ne j'} |S_j \cap S_{j'}| \le N(N-1)(k_{\mathrm{RS}} - 1).$$

The Cauchy–Schwarz inequality gives $\sum_x c_x^2 \ge (NA)^2/n_{\mathrm{RS}}$. So $N^2A^2/n_{\mathrm{RS}} \le NA + N(N-1)(k_{\mathrm{RS}}-1)$. Dividing by N and rearranging gives $N\,(A^2 - n_{\mathrm{RS}}(k_{\mathrm{RS}} - 1)) \le n_{\mathrm{RS}}\,(A - k_{\mathrm{RS}} + 1)$. The threshold is within the Johnson radius, so $A^2 > n_{\mathrm{RS}}(k_{\mathrm{RS}} - 1)$, and $N \le n_{\mathrm{RS}}(A - k_{\mathrm{RS}} + 1)/(A^2 - n_{\mathrm{RS}}(k_{\mathrm{RS}} - 1))$. The number N is an integer, so $N \le L(A)$. This is the classical pairwise Johnson argument, as presented by Dao, Kominers and Thaler [DKT26, App. B.1], applied to the interleaved code. In that code, distinct tuples agree on at most $k_{\mathrm{RS}} - 1$ columns.

Let $2A > n_{\mathrm{RS}} + k_{\mathrm{RS}} - 1$, and let $P \ne P'$ be members of the list. Then $|\mathcal{D}_P \cap \mathcal{D}_{P'}| \ge 2A - n_{\mathrm{RS}} \ge k_{\mathrm{RS}}$. Some row i has $p_i \ne p'_i$, and both equal u_i on $\mathcal{D}_P \cap \mathcal{D}_{P'}$. Two distinct polynomials of degree below k_{RS} would then agree on k_{RS} points, which is impossible. So the list has at most one member.

(2) Fix cl and $\widehat{U}$. Write $\gamma = (\upsilon, \varpi)$ for the constraint part of the joint challenge and

$$c(\chi) = M_{\mathrm{ldt}} + \sum_{i<m} \chi_i\, u_i \in K^{\mathcal{D}}$$

for the low-degree combination. Write $L^* = L(A)$, or $L^* = 1$ if $2A > n_{\mathrm{RS}} + k_{\mathrm{RS}} - 1$. By part (1), $|\mathsf{IList}_{k_{\mathrm{RS}}-1}(\widehat{U}, A)| \le L^*$.

Step 1. Apply Theorem G.4 with $t = m$, $a = M_{\mathrm{ldt}}$ and the rows $u_0, \dots, u_{m-1}$, and let $\mathcal{Z}_{\widehat{U}}$ be its set $\mathcal{Z}_m$. The set $\mathcal{Z}_{\widehat{U}}$ depends on $\widehat{U}$ only, and its density in K^m is at most $n_{\mathrm{bad}}/(|K| - 1)$. Fix

$\chi \notin \mathcal{Z}_{\widehat{U}}$ and a polynomial w of degree below k_{RS} with $|\mathcal{D}_w| \geq A$. Theorem G.4 gives polynomials $p_a, p_0, \ldots, p_{m-1}$ of degree below k_{RS} with

$$\mathcal{D}_w = \mathrm{Agr}(p_a, M_{\mathrm{ldt}}) \cap \bigcap_{i<m} \mathrm{Agr}(p_i, u_i).$$

Each p_i agrees with u_i on $\mathcal{D}_w$, and $|\mathcal{D}_w| \geq A \geq k_{\mathrm{RS}}$. So p_i is the polynomial that interpolates u_i on the first k_{RS} columns of $\mathcal{D}_w$, and $(p_0, \ldots, p_{m-1}) = P_w$. The common agreement set $\mathcal{D}_{P_w}$ contains $\mathcal{D}_w$, so $P_w \in \mathsf{IList}_{k_{\mathrm{RS}}-1}(\widehat{U}, A)$.

Step 2. Let $P \in \mathsf{IList}_{k_{\mathrm{RS}}-1}(\widehat{U}, A)$. If cl holds at $x(P)$, let $\mathcal{B}(P) = \emptyset$. Otherwise, select the linear test if some linear claim fails at $x(P)$, and the quadratic test if every linear claim holds at $x(P)$. In the second case some product constraint fails at $x(P)$. Apply Lemma G.10 to the selected test, with the rows u_i, the polynomials p_i of P, the set $\mathcal{D}' = \mathcal{D}_P$ and the word u of the test's mask row in $\widehat{U}$, M_{lin} or M_{quad}. These are determined by $\widehat{U}$ and P, so they are fixed before the joint challenge. Let $\mathcal{B}(P)$ be the set of constraint challenges γ whose component for the selected test, υ or ϖ, lies in the fraction that Lemma G.10 excludes. Then $\mathcal{B}(P)$ has density at most $L_{\mathrm{m}}/|K|$. For every $\gamma \notin \mathcal{B}(P)$, every response of degree at most $2k_{\mathrm{RS}} - 2$ that passes the algebraic check of the selected test passes its column check at no more than A_J columns of $\mathcal{D}_P$.

Step 3. Call the joint challenge *bad* if $\chi \in \mathcal{Z}_{\widehat{U}}$ or $\gamma \in \mathcal{B}(P)$ for some $P \in \mathsf{IList}_{k_{\mathrm{RS}}-1}(\widehat{U}, A)$. This event depends on $\widehat{U}$ and the joint challenge only. Before the joint challenge, the state is 0. After the joint challenge, and before the column query, the state is 1 if and only if the joint challenge is bad. After the column query, it is 1 if and only if the joint challenge is bad or the verifier accepts. Condition (a) of Definition 2.20 holds. Let the verifier accept a full transcript. If its joint challenge is bad, the state moves from 0 to 1 at the joint challenge. Otherwise the state is 0 before the column query and 1 after it. The extractor $\mathbf{E}$ is the map Ext of Section G.5 with one low-degree challenge and no piece rows, so it runs in time polynomial in m, n_{RS} and k_{RS}. Condition (b) is shown at the two challenges below.

Step 4. The state before the joint challenge is 0. The state becomes 1 only if $\chi \in \mathcal{Z}_{\widehat{U}}$ or $\gamma \in \mathcal{B}(P)$ for some member P of the list. The set $\mathcal{Z}_{\widehat{U}}$ and the sets $\mathcal{B}(P)$ are fixed by $\widehat{U}$, and χ and γ are uniform and independent. A union bound over the event $\chi \in \mathcal{Z}_{\widehat{U}}$ and the at most L^* members of the list gives the probability

$$\frac{n_{\mathrm{bad}}}{|K| - 1} + L^* \frac{L_{\mathrm{m}}}{|K|}.$$

This is $\varepsilon_{\mathrm{prf,joint}}$, with 1 in place of $L(A)$ if $2A > n_{\mathrm{RS}} + k_{\mathrm{RS}} - 1$. The bound holds whatever the extractor outputs.

Step 5. Let the state before the column query be 0, so the joint challenge is not bad. Let the extractor fail, so cl does not hold at $\mathbf{E}(\mathsf{cl}, \mathsf{tr})$ for the transcript tr of the first three messages. The responses are fixed, and the state becomes 1 exactly when the verifier accepts. The format of the responses gives $\deg w < k_{\mathrm{RS}}$ and $\deg q \leq 2k_{\mathrm{RS}} - 2$. It also gives $\deg p \leq 2k_{\mathrm{RS}} - 2$ and $p(\zeta_{k_{\mathrm{RS}}-\ell+s}) = 0$ for $s < \ell$. The low-degree column check at z_j holds exactly when $z_j \in \mathcal{D}_w$, so every accepted tuple consists of distinct columns of $\mathcal{D}_w$. We bound the accepted tuples in two cases.

- If $|\mathcal{D}_w| < A$, there are $(|\mathcal{D}_w|)_{n_{\mathrm{col}}} \leq (A-1)_{n_{\mathrm{col}}}$ tuples of distinct columns of $\mathcal{D}_w$.
- If $|\mathcal{D}_w| \geq A$, Step 1 gives the member $P_w = (p_0, \ldots, p_{m-1})$ of the list, with $\mathcal{D}_w \subseteq \mathcal{D}_{P_w}$. The extractor outputs $x(P_w)$, so cl fails at $x(P_w)$, and $\gamma \notin \mathcal{B}(P_w)$. At a column $z \in \mathcal{D}_{P_w}$, $u_i(z) = p_i(z)$ for every i. So there the linear column check becomes $q(z) = M_{\mathrm{lin}}(z) + f(z)$ with $f = \sum_i r_i p_i$, and the quadratic column check becomes $p(z) = M_{\mathrm{quad}}(z) + f'(z)$ with $f' = \sum_h \varpi_h (p_{g''_h} - p_{g_h} p_{g'_h})$. These are the column checks of Lemma G.10 for P_w. If the linear

test is selected for P_w and q fails its algebraic check, the verifier accepts no tuple. If the linear test is selected and q passes it, then q passes the linear column check at no more than A_J columns of $\mathcal{D}_{P_w}$, by Step 2. If the quadratic test is selected, p passes its algebraic check by its format, and p passes the quadratic column check at no more than A_J columns of $\mathcal{D}_{P_w}$, by Step 2. An accepted tuple consists of distinct columns of $\mathcal{D}_w \subseteq \mathcal{D}_{P_w}$ at which the selected column check holds. So at most $(A_J)_{n_{\mathrm{col}}}$ tuples are accepted.

The column query is uniform on the $(n_{\mathrm{RS}})_{n_{\mathrm{col}}}$ tuples of distinct columns. So the state changes from 0 to 1 with probability at most $\max((A-1)_{n_{\mathrm{col}}}, (A_J)_{n_{\mathrm{col}}})/(n_{\mathrm{RS}})_{n_{\mathrm{col}}} \leq \varepsilon_{\mathrm{prf,col}}$. This is condition (b) at the column query. □

At the parameters of Longfellow version 7 [Goo26a], rate 1/7 and 132 queried columns [Fs26, §5.1, p. 38], Lemma G.12 bounds the round errors of the two tableaus of the mDL proof with one attribute. The hash tableau has $n_{\mathrm{RS}} = 3230$ and $k_{\mathrm{RS}} = 461$ over $\mathbb{F}_{2^{128}}$. At $A = 1810$ its code has full agreement-set line MCA with exceptional count 22529 (Lemma 2.15), and $L(A) = 2$. For its untested mask rows, $A_J = 1724$ is the Johnson agreement of degree $2k_{\mathrm{RS}} - 2$, and $L_{\mathrm{m}} = 646$. The joint challenge then has error at most $2^{-113.46}$, and the column query at most $2^{-113.54}$. The signature tableau has $n_{\mathrm{RS}} = 3187$ and $k_{\mathrm{RS}} = 455$ over $\mathbb{F}_{p_{256}}$. At $A = 1213$ its code has full agreement-set line MCA with exceptional count 10977760134 (Lemma 2.15), and $L(A) = 98$. For its untested mask rows, $A_J = 1701$ and $L_{\mathrm{m}} = 841$. Its joint challenge has error at most $2^{-222.6}$, and its column query at most $2^{-123.12}$. The mask term $(A_J)_{n_{\mathrm{col}}}/(n_{\mathrm{RS}})_{n_{\mathrm{col}}}$ sets this column error, and a smaller A does not lower it. So the largest Ligero round error of the mDL proof with one attribute is at most $2^{-113.46}$, at the joint challenge of the hash tableau. The mDL proofs with two, three and four attributes have hash tableaus with $(n_{\mathrm{RS}}, k_{\mathrm{RS}}) = (3318, 474)$, $(3352, 478)$ and $(3436, 490)$. At $A = 1861$, 1878 and 1925, with the exceptional counts of Lemma 2.15, the lemma bounds their largest round errors by $2^{-113.23}$, $2^{-113.36}$ and $2^{-113.34}$. At the unique-decoding threshold, the interleaved agreement list has at most one member (Lemma G.12(1)), so the figures of Section G.6 also hold in Longfellow's order.

G.8 Counterexamples to the published analyses

In Ames et al.'s notation, n and k are the code's block length and dimension, $d_{\min} = n - k + 1$ its distance, e a bound on the number of corrupted columns, ℓ the number of entries in a row, σ the number of repetitions and t the number of queried columns. Each counterexample below is machine-checked against a model of the published test. The models fix one repetition ($\sigma = 1$). They bound response degrees by $2k - 2$ and take deterministic provers. They read "rejected except with probability ε" as acceptance probability at most ε, and Cascudo et al.'s hypothesis that a codeword "exists" as exact codewords.

Ames et al.'s soundness lemma for the quadratic test for $e < d_{\min}/2$ [AHIV22, Lem. B.9, p. 44], and the quadratic term of their Appendix C applied to the masked test [AHIV22, App. C, p. 47], fail over $\mathbb{F}_{37}$ with $n = 34$, $k = 4$, $\ell = 1$ and $\sigma = 1$. Take a tableau whose rows are exact codewords and at which a product constraint fails. A prover that commits a chosen mask row for the quadratic test passes with probability $2060/20757 \approx 0.0992$ at $t = 2$. The rows are exact codewords, so Lemma B.9 applies with $e = 1$. At $t = 2$ its bound $1/|\mathbb{F}|^{\sigma} + ((e + 2k)/n)^t$ is 0.0971, and Appendix C's term $(2k/n)^t + 1/|\mathbb{F}|$ is 0.0824. At $t = 1$ the acceptance is $409/1258 \approx 0.3251$, against 0.2917 and 0.2623.

Ames et al.'s headline bounds for $e < d_{\min}/4$ [AHIV22, Thm. 4.7, Lem. 4.14] hold, but the published proofs omit one step. The step bounds the agreement of a cheating response with a fixed mask row: apart from one candidate response per repetition, such a response agrees with the mask row on at most $\lfloor (n + 2k - 2)/2 \rfloor$ columns, and the headline bounds already charge this agreement. We machine-check the argument with this step for the protocol of their Section 4.7, with arbitrary

mask rows, an arbitrary blinding row and their constraint system in place of the circuit. It gives the field term $(e+1)/|\mathbb{F}|^{\sigma}$ in place of $(d_{\min}+2)/|\mathbb{F}|^{\sigma}$. For $e < d_{\min}/2$, we know of no proof of the total of their Appendix C above rate $3-2\sqrt{2} \approx 0.172$. Above this rate the Johnson agreement of degree $2k-2$ exceeds $(n+k)/2$, so the mask term $(A_J/n)^t$, which replaces their terms for the linear and quadratic tests, exceeds the code test's term. Their recommended parameters set $e = k$ and $n = 3k$ [AHIV22, §5.3], and their implementation targets soundness 2^{-128} [AHIV22, §6]. At these parameters, with $t = 222$, the least t with $3(2/3)^t \le 2^{-128}$, the total of their Appendix C with its terms for the linear and quadratic tests replaced by the mask term $(A_J/n)^t$ gives $2^{-65.0}$ at $k = 2^{10}$ and $2^{-64.9}$ at $k = 2^{16}$. No attack at these parameters is known.

Theorem 4.4 of Cascudo et al. [CHVY25, pp. 18–19] fails over $\mathbb{F}_{101}$ with block length $n = 5$, dimension $k = 2$, one entry per row and $t = 1$. Cascudo et al. do not state the tests formally, so the model follows Longfellow's checks [Fs26, §4.4.1]. The witness row is zero and the linear claim requires its entry to be 1. The prover commits the values of $(X-1)^3$ as the linear test's mask row and passes with probability 211/505, against the bound $\binom{2k-2}{t}/\binom{n}{t} + 1/|\mathbb{F}| = 207/505$ of their Theorem 4.4. The bound fails whether its code length is the number n of columns or Longfellow's encoded length $n + 2k - 1$.

The sum of the bounds of Theorems 4.3 and 4.4 of Cascudo et al. [CHVY25, pp. 17–19] fails over $\mathbb{F}_{1217}$ for every k from 13 to 30, in Longfellow's layout with $k-1$ random values and one entry per row, $n = 2k+1$ columns and $t = k-1$ queried columns. The witness row is zero and the linear claim requires its entry to be 1. The prover commits the values of X^{2k-1} as the linear test's mask row. For each challenge it answers with a polynomial of degree at most $2k-2$ that passes the algebraic check and agrees with the mask row on $2k-1$ columns for g of the challenges and on $2k-2$ columns for the others. The acceptance is then at least $(g\binom{2k-1}{t} + (|\mathbb{F}|-g)\binom{2k-2}{t})/(|\mathbb{F}|\binom{n}{t})$, which exceeds the summed bound $\left(\binom{\lceil (n+k)/2 \rceil}{t} + \binom{2k-2}{t}\right)/\binom{n}{t} + (n+2)/|\mathbb{F}|$. At $k = 30$, with $g = 988$, it is at least 0.2459 against the bound 0.1896. The bound fails whether its code length is the number n of columns or Longfellow's encoded length $n + 2k - 1$.

H Non-interactive arguments

H.1 Random oracles and non-interactive arguments

Our spBCS transformation compiles a series–parallel IOP into a non-interactive argument in the (quantum) random oracle model.

Definition H.1 (Non-interactive argument [DFMS22, §3.1, p. 10, and footnote 6])**.** Let $\mathcal{Q}$ be a finite set and $\mathcal{O}$ a set of bit strings of one length. A *non-interactive argument* for a relation $\mathcal{R}$ is a pair $(\mathsf{Prv}, \mathsf{Vrf})$ of oracle algorithms with oracle $\mathsf{H}\colon \mathcal{Q} \to \mathcal{O}$, drawn uniformly. The prover $\mathsf{Prv}^{\mathsf{H}}(\mathsf{x}, \mathsf{w})$ outputs a proof pf, and the verifier $\mathsf{Vrf}^{\mathsf{H}}(\mathsf{x}, \mathsf{pf})$ outputs a bit. In the *random oracle model* (ROM) every party queries H classically. In the *quantum random oracle model* (QROM) an adversary may also query H on superpositions of inputs, by the unitary $|u, y\rangle \mapsto |u, y \oplus \mathsf{H}(u)\rangle$ [CMS19, §3.2, p. 14].

H.2 Completeness and soundness

Completeness asks that honestly generated proofs verify, even when the statement is chosen after querying the oracle.

Definition H.2 (Completeness [Unr17, Def. 5, p. 14])**.** A non-interactive argument $(\mathsf{Prv}, \mathsf{Vrf})$ for $\mathcal{R}$ has *completeness error* $\varepsilon_{\mathrm{cpl}}$, a function from $\mathbb{N}$ to the nonnegative real numbers, if for every $t \in \mathbb{N}$

and every $(t, 0)$-adversary A (Definition 6.1) whose output is a pair (x, w),

$$\Pr\big[(\mathsf{x},\mathsf{w}) \in \mathcal{R} \wedge \mathsf{Vrf}^{\mathsf{H}}(\mathsf{x},\mathsf{pf}) = 0 : \mathsf{H} \xleftarrow{\$} \mathcal{O}^{\mathcal{Q}},\ (\mathsf{x},\mathsf{w}) \leftarrow A^{\mathsf{H}},\ \mathsf{pf} \leftarrow \mathsf{Prv}^{\mathsf{H}}(\mathsf{x},\mathsf{w})\big] \leq \varepsilon_{\mathrm{cpl}}(t).$$

The statement may depend on the oracle. This dependence captures protocols in which the statement is itself computed with the same hash function, for example from earlier messages. With $t = 0$ the definition is completeness for a fixed statement. Don et al. state this case and call $\varepsilon_{\mathrm{cpl}}$ the correctness error [DFMS22, §3.1, p. 10].

Knowledge soundness asks that an adversary who produces an accepted proof knows a witness: an extractor that answers the adversary's oracle queries obtains one, without changing the adversary's view.

Definition H.3 (Adaptive knowledge soundness [DFMS22, Def. 3.1, p. 10])**.** Let $(\mathsf{Prv}, \mathsf{Vrf})$ be a non-interactive argument for $\mathcal{R}$. For an adversary A whose output is a pair $(\mathsf{x}, \mathsf{pf})$, let $\mathsf{Vrf}^{\mathsf{H}} \circ A^{\mathsf{H}}$ be the execution in which A outputs $(\mathsf{x}, \mathsf{pf})$ and its auxiliary output Z, and the verifier then computes $v \leftarrow \mathsf{Vrf}^{\mathsf{H}}(\mathsf{x}, \mathsf{pf})$. An *online extractor* is an algorithm $\mathcal{E}$ that answers every oracle query of this execution and finally outputs a witness w. We write $\mathsf{Vrf}^{\mathcal{E}} \circ A^{\mathcal{E}}$ for the execution with $\mathcal{E}$ answering the queries. The argument $(\mathsf{Prv}, \mathsf{Vrf})$ is *adaptively knowledge sound* with oracle simulation error $\varepsilon_{\mathrm{sim}}$ and extraction error $\varepsilon_{\mathrm{ex}}$, functions from $\mathbb{N}$ to the nonnegative real numbers, if there is an online extractor $\mathcal{E}$ such that for every t and every $(t, 0)$-adversary A (Definition 6.1) whose output is a pair $(\mathsf{x}, \mathsf{pf})$,

$$\mathsf{TD}\big([(\mathsf{x},\mathsf{pf},Z,v)]_{\mathsf{Vrf}^{\mathsf{H}} \circ A^{\mathsf{H}}},\ [(\mathsf{x},\mathsf{pf},Z,v)]_{\mathsf{Vrf}^{\mathcal{E}} \circ A^{\mathcal{E}}}\big) \leq \varepsilon_{\mathrm{sim}}(t),$$

$$\Pr\big[v = 1 \wedge (\mathsf{x},\mathsf{w}) \notin \mathcal{R} : (\mathsf{x},\mathsf{pf},Z;v;\mathsf{w}) \leftarrow \mathsf{Vrf}^{\mathcal{E}} \circ A^{\mathcal{E}}\big] \leq \varepsilon_{\mathrm{ex}}(t).$$

The running time of $\mathcal{E}$ is polynomial in t, the length of the statement, the length of the proof and the bit lengths of the oracle's inputs and outputs. In the ROM, the adversaries are those of the ROM and $\mathcal{E}$ is classical.

Don et al. call this property online extractability. With errors negligible at polynomially many queries, Definition H.3 implies theirs. It quantifies over the larger class of adversaries of Definition 6.1. It also bounds the errors at every query bound, whereas Don et al. ask for errors negligible in a security parameter. Chiesa, Manohar and Spooner's knowledge soundness fixes the statement in advance, and its extractor has black-box access to the prover in an execution of its own [CMS19, §3.3, p. 15]. Definition H.3 therefore also implies their notion.

Neither Unruh's extractability nor Definition H.3 is a special case of the other. Unruh's notion considers a measurement of the adversary's final state and bounds from below the probability that the extractor obtains a witness and the measurement still passes. The lower bound is a power of the probability that the proof is accepted and the measurement passes, divided by a polynomial [Unr17, Def. 13, pp. 19–20]. Unruh's notion therefore tolerates a polynomial loss, and Definition H.3 does not. Unruh's notion also covers final measurements that query the oracle, and Definition H.3 does not. Definition H.3, like Don et al.'s notion, extracts in the same execution as the adversary, with additive errors, and it leaves the execution and the auxiliary output unchanged.

Soundness asks only that no adversary produces an accepted proof for a false statement.

Definition H.4 (Adaptive soundness [DFMS22, §3.1, p. 10])**.** A non-interactive argument $(\mathsf{Prv}, \mathsf{Vrf})$ for $\mathcal{R}$ has *adaptive soundness error* $\varepsilon_{\mathrm{snd}}$, a function from $\mathbb{N}$ to the nonnegative real numbers, if for every $t \in \mathbb{N}$ and every $(t, 0)$-adversary A (Definition 6.1) whose output is a pair $(\mathsf{x}, \mathsf{pf})$,

$$\Pr\big[\mathsf{Vrf}^{\mathsf{H}}(\mathsf{x},\mathsf{pf}) = 1 \wedge \mathsf{x} \notin \mathcal{L}(\mathcal{R}) : \mathsf{H} \xleftarrow{\$} \mathcal{O}^{\mathcal{Q}},\ (\mathsf{x},\mathsf{pf}) \leftarrow A^{\mathsf{H}}\big] \leq \varepsilon_{\mathrm{snd}}(t).$$

Don et al. call soundness adaptive when the adversary chooses the statement after querying the oracle. Definition H.4 implies and generalizes two published notions. Chiesa, Manohar and Spooner's soundness fixes the statement in advance [CMS19, §3.3, p. 15], and Chiesa and Yogev's adaptive soundness bounds the size of the statement [CY26, Def. 4.1.3, pp. 41–42].

Lemma H.5 (Soundness from knowledge soundness)**.** *If a non-interactive argument is adaptively knowledge sound (Definition H.3) with oracle simulation error $\varepsilon_{\mathrm{sim}}$ and extraction error $\varepsilon_{\mathrm{ex}}$, it has adaptive soundness error $\varepsilon_{\mathrm{sim}} + \varepsilon_{\mathrm{ex}}$ in the same model.*

Proof. Fix t, a $(t, 0)$-adversary A and the online extractor $\mathcal{E}$ of Definition H.3. The event $v = 1 \wedge \mathsf{x} \notin \mathcal{L}(\mathcal{R})$ is an outcome of a measurement of (x, v). Its probabilities in $\mathsf{Vrf}^{\mathsf{H}} \circ A^{\mathsf{H}}$ and in $\mathsf{Vrf}^{\mathcal{E}} \circ A^{\mathcal{E}}$ therefore differ by at most the trace distance of the joint states of $(\mathsf{x}, \mathsf{pf}, Z, v)$ [Wat18, Thm. 3.4, p. 128]. By the first condition of Definition H.3, this trace distance is at most $\varepsilon_{\mathrm{sim}}(t)$. In $\mathsf{Vrf}^{\mathcal{E}} \circ A^{\mathcal{E}}$, a statement $\mathsf{x} \notin \mathcal{L}(\mathcal{R})$ has $(\mathsf{x}, \mathsf{w}) \notin \mathcal{R}$ for the extracted w, so the event has probability at most $\Pr[v = 1 \wedge (\mathsf{x}, \mathsf{w}) \notin \mathcal{R}] \leq \varepsilon_{\mathrm{ex}}(t)$. In the ROM the joint states are distributions and the same two steps apply. □

H.3 Zero knowledge

Zero knowledge asks that proofs reveal nothing beyond the truth of their statements: a simulator without the witness reprograms the random oracle and produces proofs that no adversary can tell from real ones.

Definition H.6 (Zero knowledge [Unr17, Def. 6, p. 14])**.** Let $(\mathsf{Prv}, \mathsf{Vrf})$ be a non-interactive argument for $\mathcal{R}$. A *simulator* is a classical stateful algorithm Sim that, on a statement x, returns a proof and may reprogram the random oracle: it may output an *assignment list* Reprog, a list of pairs $(u, y) \in \mathcal{Q} \times \mathcal{O}$. The current oracle H' is then replaced by $\mathsf{H}'[\mathsf{Reprog}]$, the oracle that maps each assigned input to its last assigned value and agrees with H' elsewhere. The proof oracles P' and S' answer a proof query $(\mathsf{x}, \mathsf{w}) \notin \mathcal{R}$ with $\perp$, as in Unruh's earlier definition [Unr14, Def. 2, p. 6]. On $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$, with fresh coins, P' returns $\mathsf{Prv}^{\mathsf{H}}(\mathsf{x}, \mathsf{w})$ and S' returns the proof of $\mathsf{Sim}(\mathsf{x})$. The argument $(\mathsf{Prv}, \mathsf{Vrf})$ is *zero knowledge* with respect to Sim, with error $\varepsilon_{\mathrm{zk}}$, a function from $\mathbb{N}^2$ to the nonnegative real numbers, if Sim runs in time polynomial in the length of the statement and the bit lengths of the oracle's inputs and outputs, and for all $t, Q \in \mathbb{N}$ and every (t, Q)-adversary A (Definition 6.1) whose output is a bit,

$$\left| \Pr\left[b = 1 : b \leftarrow A^{\mathsf{H}, P'}\right] - \Pr\left[b = 1 : b \leftarrow A^{\mathsf{H}, S'}\right] \right| \leq \varepsilon_{\mathrm{zk}}(t, Q),$$

over a uniform H, where in the second experiment A's hash queries are answered by the current oracle.

The notion is adaptive and multi-theorem. The adversary chooses each statement and witness after its hash queries and earlier proofs, and it makes up to Q proof queries. Unruh asks for polynomial-time adversaries and a negligible difference. With errors negligible at polynomially many queries, Definition H.6 implies Unruh's notion. It generalizes Unruh's notion in two ways: it quantifies over the larger class of Definition 6.1, and it bounds the error at every t and Q. The larger class includes adversaries with quantum auxiliary input, and Unruh's zero knowledge excludes them [Unr17, footnote 16, p. 20].

I Proofs for series–parallel BCS

I.1 Protocol details

This subsection fixes the details of Protocol 6.5 that the proofs use: the quantities in the error bounds (Definition I.1), the oracle's domain, the hash functions, the challenge inputs, leaves, samplers and Merkle trees, and the prover's and verifier's handling of failures.

The error bounds charge the protocol's features as follows. Knowledge soundness and weak online simulation extractability count the final digests as n_J more digests per challenge input. The write-back makes a fresh stream answer complete a round of at most one reconstructed transcript (Lemma I.12). Key collisions cost $\varepsilon_{\mathrm{key}}$ in completeness, zero knowledge and witness indistinguishability. Completeness for a fixed statement, knowledge soundness and weak online simulation extractability hold at every salt length, including 0, while zero knowledge and witness indistinguishability use the proof salt and the leaf salts.

Definition I.1 (Quantities of spBCS)**.** For Protocol 6.5 on a series–parallel IOP with N_{msg} messages, n_{pos} oracle positions, at most m oracle strings per message, strings of length at most $\mathsf{l}_{\max}$ and at most q_{IOP} queries (Definition 2.18), with n_M merges $(A_0^{(u)} \sqcup \ldots \sqcup A_{r_u-1}^{(u)}); D_u$, $u < n_M$, with $n_B = \sum_{u<n_M} r_u$ branches in all, and $n_J = \max_{i\in[N_{\mathrm{msg}}]} \sum_{u:i\in D_u} r_u$ (both 0 without merges), with digests of b bytes, leaf salts of b_{l} bytes and leaf encodings of at most L_{leaf} bytes:

1. the digest bound $c = N_{\mathrm{msg}} \max(1, m)$, the verifier's hash-query bound $k_q = 2N_{\mathrm{msg}} + n_B + q_{\mathsf{IOP}}(1 + \lfloor \log_2(2\max(1, \mathsf{l}_{\max}))\rfloor)$ and the simulator's hash-query bound $q_S = 2n_{\mathrm{pos}} + N_{\mathrm{msg}} + n_B$;
2. the overrun $\varepsilon_{\mathrm{ov}} = \sum_{i\in J_V} \Pr_y[\mathsf{sample}_i(y) = \perp]$, for y uniform on $\mathcal{O}$, the key-collision term $\varepsilon_{\mathrm{key}} = \binom{N_{\mathrm{msg}}}{2} 2^{-8b}$, and the leaf term

$$\varepsilon_{\mathrm{leaf}} = \frac{1}{2}\sqrt{\frac{2^{8b}}{2^{8b_{\mathsf{l}}}}} + \sqrt{\frac{\ln N_{\mathrm{leaf}}}{2\cdot 2^{8b_{\mathsf{l}}}}},$$

   where $N_{\mathrm{leaf}} = \sum_{j=0}^{L_{\mathrm{leaf}}} 2^{8j}$ is the number of byte strings of length at most L_{leaf}.

An encoded challenge input contains at most c digests of items, one per verifier message and at most $\max(1, m_i)$ per prover message of its past. It also contains at most n_J final digests, one for each branch of each merge whose part D contains the message, nested or not: a merge may lie inside a branch or inside the part of another merge. The verifier makes at most k_q hash queries: one challenge-input query and one stream query per verifier message, one challenge-input query per branch for its final digest, and at most $1 + \lfloor \log_2(2\max(1, \mathsf{l}_{\max}))\rfloor$ Merkle queries per opened position. The simulator of Protocol I.4 makes at most q_S.

Lemma I.2 (Maximal inequality)**.** *Let $n \geq 2$ and $\sigma > 0$, and let $X_0, \ldots, X_{n-1}$ be centered random variables, not necessarily independent, with $\mathbb{E}[e^{\kappa X_i}] \leq e^{\kappa^2\sigma^2/2}$ for every real κ. Then $\mathbb{E}[\max_i X_i] \leq \sigma\sqrt{2\ln n}$, by the standard Chernoff argument (cf. [CY26, Def. 1.2.6 and Lem. 1.2.7, p. 7]).*

For a fixed oracle and leaf content, consider the statistical distance between the digest of the leaf under a uniform salt and a uniform digest. The leaf term bounds the expectation over the oracle of the largest such distance over the contents, and it is small only when $b_{\mathsf{l}} > b$. Its first term is Chiesa and Yogev's bound for a fixed content [CY26, Claim 3.4.1, p. 35]. Its second term follows the proof of their Claim 3.4.2 [CY26, pp. 36–37]. McDiarmid's inequality in moment-generating-function form [BLM13, proof of Thm. 6.2] makes the distance at each content, less its expectation, sub-Gaussian with variance proxy $1/(4\cdot 2^{8b_{\mathsf{l}}})$. Lemma I.2 over the N_{leaf} contents then bounds the expected maximum by $\sqrt{\ln N_{\mathrm{leaf}}/(2\cdot 2^{8b_{\mathsf{l}}})}$.

The oracle's domain. The domain has four disjoint parts, which extend BCS's split of its oracle by a prefix bit [BCS16, §6, p. 25]: $\mathsf{ch}(v)$ for a byte string v, $\mathsf{leaf}(r, v)$ for a leaf salt r and a byte string v, $\mathsf{node}(u, u')$ for digests $u, u' \in \{0,1\}^{8b}$, and $\mathsf{stream}(u, s)$ for a digest u and a proof salt s. Byte strings have a fixed maximum length, so the domain is finite. The four encodings are injective with disjoint images. Leaf salts and the proof salt have fixed lengths, possibly 0.

Hash functions. The implementation hashes challenge inputs, leaves and internal nodes with one SHA-2 function [Nat15a], SHA-256 at the ROM parameters and SHA-384 at the QROM parameters (Table 3), and each of these inputs begins with a byte naming its part. It evaluates stream points with cSHAKE256 [KjCP16] under a fixed customization string. We model both functions as the one random oracle H, with these input sets as its disjoint domain parts. Statement digests, such as circuit identifiers, are part of the statement x, and the verifier reads them from $\mathsf{pre}(\mathsf{x})$ without recomputing them.

Challenge inputs. The challenge input of a verifier message i is $\mathsf{ch}(\mathsf{label} \,\|\, \mathsf{salt}(s) \,\|\, \mathsf{pre}(\mathsf{x}) \,\|\, \mathsf{items}(i, \ell_i))$ for a fixed protocol label. The final input of a branch A is the same with i_A and ℓ_{i_A} followed by the item of i_A. The function items writes the labels of the parts of the order that contain i, then the items of ℓ_i, each with its index, and the label and final digest of each merged branch encoded with a tag distinct from every item. For the total order without merges the index is determined by the number of items and is not written. The statement encoding is prefix-free, and prover items, written-back digests and the encoded labels and final digests of merged branches are uniquely decodable in the fixed schedule, so a challenge input determines $(s, \mathsf{x}, i, \ell_i)$. The database algorithm decodes challenge inputs with a polynomial-time left inverse of the statement encoding on its image. Each challenge is derived from its past, as in Chiesa and Yogev's variants [CY26, Rem. 25.1.3, p. 276], and the per-proof salt follows Fractal's seed [COS19, §10.6] and Chiesa and Yogev's per-round salts [CY26, Constr. 25.1.1, p. 275].

Leaves. Each alphabet $\mathcal{A}_{i,j}$ has an injective codec ser and a default value. A leaf's bytes that are not exactly one encoded value, and a missing leaf, decode to the default. Authentication operates on the bytes, before decoding.

Samplers. The sampler sample_i is deterministic and returns $\perp$ on a rejection. Every element of $\mathcal{C}_i$ has the same number of preimages, so a uniform stream gives a uniform challenge conditional on success. The samplers read a prefix of the stream: a preimage of a challenge is a byte string on which the sampler returns the challenge after reading exactly that string, followed by arbitrary bytes. A stream uniform on the preimages of a challenge is drawn in polynomial time by counting and indexing these strings. The written-back digest $\mathsf{dg}(y_i)$ is the first $8b$ bits of the same stream.

Merkle trees. A string of length $\mathsf{l} > 0$ is committed by the binary tree on the nodes $1, \ldots, 2\mathsf{l} - 1$ in which node $v < \mathsf{l}$ has children $2v$ and $2v+1$ and leaf q is node $\mathsf{l} + q$. Every leaf has at most $\lfloor \log_2(2\mathsf{l}) \rfloor$ ancestors. Leaf q has digest $\mathsf{dg}(\mathsf{H}(\mathsf{leaf}(r_q, \mathsf{ser}(O[q]))))$ for an independent uniform salt r_q, and an internal node has digest $\mathsf{dg}(\mathsf{H}(\mathsf{node}(d, d')))$ for its children's digests d, d'. An empty string has root $\mathsf{dg}(\mathsf{H}(\mathsf{leaf}(0, \epsilon)))$. For a set of opened leaves, the implementation's compressed proof (Section 6.2) contains the salted leaves and the digests of the children outside their root paths. Verification recomputes the root.

Prover and verifier. On a sampler rejection the prover aborts with output $\perp$, and the verifier rejects. The verifier requires exactly one salted leaf per position of $\mathsf{qry}(\theta)$ and fills unopened leaves with defaults, which the IOP verifier does not read.

I.2 Completeness, zero knowledge and witness indistinguishability

This subsection and the next two prove the parts of Theorem 6.6.

Theorem I.3 (Completeness of spBCS)**.** *Let* IOP *be a series–parallel IOP (Definition 2.18) for* $\mathcal{R}$*, with* N_{msg} *messages, and let* $(\mathsf{Prv}, \mathsf{Vrf})$ *be Protocol 6.5 with digests of* $b \geq 1$ *bytes and proof salts of* b_{s} *bytes.*

1. *For every statement* x*, every IOP prover with fixed coins, every proof salt* s *and all leaf salts, and every* $\varepsilon_{\mathrm{rej}} \in [0,1]$ *such that the IOP verifier rejects that prover's transcript on* x *with probability at most* $\varepsilon_{\mathrm{rej}}$ *over uniform challenges,* $\mathsf{Vrf}^{\mathsf{H}}$ *rejects the proof of* $\mathsf{Prv}^{\mathsf{H}}$ *on* x *with probability at most* $\varepsilon_{\mathrm{rej}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}}$*, for every fixing of* H *outside the challenge inputs and stream points that carry* s*, over a uniform* H *on these inputs.*
2. *If* IOP *has completeness error at most* $\varepsilon_{\mathrm{c}} \in [0,1]$ *at every statement for* $\mathcal{R}$ *(Definition 2.19), then* $(\mathsf{Prv}, \mathsf{Vrf})$ *has completeness error (Definition H.2) at most*

$$\varepsilon_{\mathrm{cpl}}(t) = \varepsilon_{\mathrm{c}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} + \frac{2t}{2^{4b_{\mathrm{s}}}}$$

in the QROM, and

$$\varepsilon_{\mathrm{cpl}}(t) = \varepsilon_{\mathrm{c}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} + \frac{t}{2^{8b_{\mathrm{s}}}}$$

in the ROM.

Proof of Theorem I.3. (1) Fix x, the prover's coins, the proof salt s, the leaf salts and H outside the set S_s of challenge inputs and stream points that carry s, and let H be uniform on S_s. The challenge inputs of the proof are distinct, so their answers are uniform and independent. Two of the at most $\binom{N_{\mathrm{msg}}}{2}$ pairs of keys collide with probability at most $\varepsilon_{\mathrm{key}}$. Without a key collision, the stream answers are uniform and independent, so some sampler fails with probability at most $\varepsilon_{\mathrm{ov}}$, and otherwise the challenges are uniform and the IOP verifier rejects with probability at most $\varepsilon_{\mathrm{rej}}$. The verifier of spBCS rejects only on one of these three events.

(2) Let the adversary choose x after t hash queries, and let the proof salt s be uniform and drawn after this choice. Averaging part (1) over the prover's coins, with $\varepsilon_{\mathrm{rej}}$ the rejection probability of the IOP prover at those coins, bounds the probability of rejection by $\varepsilon_{\mathrm{c}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}}$ at every fixing of H outside S_s, when H is uniform on S_s and independent of the adversary's view. In the ROM, let E be the event that one of the adversary's queries lies in S_s. Each query carries s with probability at most $2^{-8b_{\mathrm{s}}}$, so $\Pr[E] \leq t/2^{8b_{\mathrm{s}}}$, and outside E the answers on S_s are uniform given the adversary's view. In the QROM, one-way to hiding [AHU18, Thm. 3, p. 11], applied to the pair of s and the oracle with fresh values on S_s as in the proof of Theorem I.5(1), bounds the change by $2t/2^{4b_{\mathrm{s}}}$. □

Protocol I.4 (Simulator of spBCS)**.** Let $\mathbf{S}$ be a probabilistic algorithm that maps a statement to challenges θ, non-oracle messages and values at the positions $\mathsf{qry}(\theta)$, such as an HVZK simulator (Definition 2.22). On a statement x, with classical access to the current oracle H', the simulator Sim returns a proof and a list of assignments:

1. Sample a proof salt s, coins r for $\mathbf{S}$, a salt for every position, and a uniform digest d_p for every position $p \in \mathcal{P}_{\mathrm{or}}$. Run $\mathbf{S}(\mathsf{x}; r)$, which outputs the challenges θ. For $i \in J_V$, sample a stream y_i uniform on the streams that sample_i maps to θ_i (Section I.1).
2. Build every Merkle tree with H': a position in $\mathsf{qry}(\theta)$ gets the digest of its salted simulated leaf, every other position the sampled d_p.

3. Build the transcripts as the prover does, with the written-back digests $\mathsf{dg}(y_i)$, querying H' for the final digest of every branch. At each verifier message query the key $u_i = \mathsf{dg}(\mathsf{H}'(\mathsf{ch}(s, \mathsf{x}, i, \ell_i)))$ and record the assignment $\mathsf{stream}(u_i, s) \mapsto y_i$.

4. Return the proof assembled as in Protocol 6.5 and the assignments.

The simulator makes at most $q_S = 2n_{\mathrm{pos}} + N_{\mathrm{msg}} + n_B$ hash queries: at most n_{pos} leaf queries, at most n_{pos} internal-node queries, at most one query for the root of an empty tree, shared by all empty trees, one challenge-input query per verifier message, and one per branch. An empty tree belongs to a prover message, so when one exists $|J_V| + 1 \le N_{\mathrm{msg}}$ and the root query fits in the bound. The simulator assigns only stream points, one per verifier message. If $\mathbf{S}$ omits a non-oracle message, Sim returns $\perp$ and makes no assignment. An omitted leaf value is the empty string. The simulator's own queries use the current oracle, and its assignments take effect at the end of the call.

Theorem I.5 (Everlasting zero knowledge of spBCS)**.** *Let* IOP *be a series–parallel IOP (Definition 2.18) for* $\mathcal{R}$*, with* N_{msg} *messages and* n_{pos} *oracle positions, whose honest prover has honest-verifier zero-knowledge error at most* $\varepsilon_{\mathrm{hvzk}} \in [0, 1]$ *at every statement with respect to an HVZK simulator* $\mathbf{S}$ *(Definition 2.22). Let* $(\mathsf{Prv}, \mathsf{Vrf})$ *be Protocol 6.5 at this prover, with digests of* $b \ge 1$ *bytes, leaf salts of* b_{l} *bytes, proof salts of* b_{s} *bytes and leaf encodings of at most* L_{leaf} *bytes, and* Sim *the simulator of Protocol I.4 built from* $\mathbf{S}$*. Then* $(\mathsf{Prv}, \mathsf{Vrf})$ *is everlasting zero knowledge with respect to* Sim *(Definition 6.2):*

1. *in the QROM, with error at most*

$$\varepsilon_{\mathrm{zk}}(t, Q) = \frac{4Qt}{2^{4b_{\mathrm{s}}}} + \frac{Q(Q-1)}{2^{8b_{\mathrm{s}}}} + Q\big(\varepsilon_{\mathrm{key}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{hvzk}} + n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}}\big);$$

2. *in the ROM, with error at most*

$$\varepsilon^{\mathrm{C}}_{\mathrm{zk}}(t, Q) = \frac{Qt}{2^{8b_{\mathrm{s}}}} + \frac{Q(Q-1)}{2 \cdot 2^{8b_{\mathrm{s}}}} + Q\big(\varepsilon_{\mathrm{key}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{hvzk}} + n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}}\big).$$

If N_{msg}*,* n_B *and* n_{pos} *are polynomial in the length of the statement,* Sim *runs in time polynomial in the length of the statement and in* $L_{\mathcal{Q}}$*, the bit length of an encoded oracle input.*

Proof. (1) Hybrid $j \le Q$ answers the proof queries before proof query j with Sim and the others with Prv. From hybrid j to hybrid $j + 1$:

1. Answer the oracle inputs that carry the proof salt s_j of proof query j by a fresh uniform function. The distribution is unchanged. Let F_j be the event that an earlier execution of the simulator drew the salt s_j, so $\Pr[F_j] \le j/2^{8b_{\mathrm{s}}}$. Outside F_j, no earlier execution of the simulator queries an input that carries s_j.

2. Let S be the set of oracle inputs that carry the proof salt s_j of proof query j, H_j the base oracle of hybrid j, and H'_j the oracle of step 1, which agrees with H_j outside S. We apply one-way to hiding [AHU18, Thm. 3, p. 11] to the oracles $O_0 = \mathsf{H}_j$ and $O_1 = \mathsf{H}'_j$, which differ only on S, with the auxiliary input $z = (\mathsf{H}'_j, s_j)$ and the following algorithm $A'^{O}(z)$, which draws every other coin of the game itself.

   (i) Run the game, sending the distinguisher's hash queries to O until proof query j or until the distinguisher has made t hash queries, whichever comes first. The t-th hash query also goes to O.

(ii) Apply the assignments of the simulator's executions before proof query j on top of O, as the game does, so that a query to $O[\mathsf{Reprog}]$ is one query to O. On an assigned input, replace the input by an internal-node input and the answer register by the uniform superposition before the query, restore both after it, and add the assigned value (Lemma I.10).

(iii) Answer every other hash query with H'_j: the distinguisher's hash queries after the switch, and those of the simulator's executions before proof query j.

(iv) Output the distinguisher's output.

Four facts give the bound.

- A'^{O_1} is the game of step 1, and A'^{O_0} is that game with the change made only after proof query j. On an execution path with proof query j, at most t hash queries of the distinguisher precede it, since proof query j comes no later than the last proof query. On an execution path without proof query j, both games and the game that step 4 starts from use the same switch rule.
- A' answers the simulator's queries as the game's current oracle does, the base oracle with the assignments Reprog made so far applied (Definition H.6). Outside F_j, the simulator's executions before proof query j query no input of S: they query leaves and internal nodes, which carry no proof salt, and challenge inputs with their own salts.
- A' has query depth t. By Lemma I.10 in the standard form, with an internal-node input as the dummy input, the part of A' before the switch is a fixed alternation of channels with t queries to O.
- The probability P_{find} that a query of $A'^{O_1}(z)$ before the switch, measured, lies in S is at most 2^{-8b_s}. Before the switch A'^{O_1} uses only H'_j, a uniform function independent of s_j, and it uses s_j only at proof query j, and every input carries at most one salt.

One-way to hiding allows S, the two oracles and z any joint distribution, and it gives

$$\left|\Pr[A'^{O_0}(z) = 1] - \Pr[A'^{O_1}(z) = 1]\right| \le 2t\sqrt{P_{\text{find}}} \le 2t \cdot 2^{-4b_s}.$$

3. Replace the honest proof for proof query j by the simulated proof with its assignments. Outside F_j, the honest proof uses a fresh part of the oracle. Jointly with the oracle outside S, the distinguisher's state and its proof query, the distributions of the proof and of the current oracle after proof query j, the base oracle with the assignments made so far applied, are within $\varepsilon_{\text{key}} + \varepsilon_{\text{ov}} + \varepsilon_{\text{hvzk}} + n_{\text{pos}}\varepsilon_{\text{leaf}}$ in statistical distance. The rest of the game is a channel applied to these, so the trace distance of the whole game is within the same bound [Wat18, Cor. 3.40 and (3.241), p. 168]. The four terms arise as follows. The keys are distinct except with probability ε_{key}. Every sampler succeeds except with probability ε_{ov}, and a successful uniform stream is a uniform preimage of a uniform challenge, so uniform streams and streams drawn uniformly among the preimages of uniform challenges are within ε_{ov}. With the streams drawn as preimages of the challenges, the IOP view and $\mathbf{S}$'s output are within $\varepsilon_{\text{hvzk}}$ (Definition 2.22). Given the oracle, each of the n_{pos} leaves has a digest whose distance from uniform, maximized over the leaf's content, has expectation over the oracle at most $\varepsilon_{\text{leaf}}$ (Definition I.1), and the simulator draws uniform digests for the unopened leaves.
4. Undo steps 2 and 1 with the simulator's salt, whose event F'_j has the same bound.

On an execution path without proof query j, hybrids j and $j+1$ are the same execution, and by the switch rule of step 2 so are the games between them. The hybrid argument therefore

needs no fixed number of proof queries. Summing over $j < Q$, the two applications of one-way to hiding give $4Qt/2^{4b_s}$, the events F_j and F'_j give $\sum_{j<Q} 2j/2^{8b_s} = Q(Q-1)/2^{8b_s}$, and step 3 gives $Q(\varepsilon_{\mathrm{key}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{hvzk}} + n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}})$. (2) The hybrids are those of part (1). Classically, steps 2 and 4 are replaced by the event that one of the distinguisher's at most t hash queries before proof query j carries the proof salt of proof query j. The salt is drawn uniformly after these queries, so the event has probability at most $t/2^{8b_s}$. Hybrids j and $j+1$ agree up to proof query j, and each draws the salt of proof query j uniformly at that query. We draw this salt once and use it in both hybrids, so the event is the same in both. If it does not occur, no input that carries the salt was queried before proof query j, so step 3 bounds the change from the honest to the simulated proof. The two hybrids therefore differ by at most $t/2^{8b_s}$ plus the error of step 3. The salt event of step 1 is likewise charged once, $\sum_{j<Q} j/2^{8b_s} = Q(Q-1)/(2 \cdot 2^{8b_s})$, and step 3 is unchanged. □

The proof follows Chiesa and Yogev's proof of zero knowledge for BCS in the ROM [CY26, Thm. 26.2.1, p. 297, and its proof, pp. 298–301]. That proof bounds the probability that the adversary queries a programmed point before its salt is sampled.

Theorem I.6 (Unconditional witness indistinguishability of spBCS)**.** *In the setting of Theorem I.5, for all $Q, L \in \mathbb{N}$, $(\mathsf{Prv}, \mathsf{Vrf})$ is unconditionally witness indistinguishable (Definition 6.3) with error at most*

$$\varepsilon_{\mathrm{wi}}(Q, L) = Q\Big(2n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}} + 2\varepsilon_{\mathrm{hvzk}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} + \sqrt{\frac{(L+1)\ln 2 + 2\ln(L+1)}{2 \cdot 2^{8b_s}}}\Big).$$

We prove Theorem I.6 with two lemmas. Lemma I.7 bounds one proof at a fixed triple, and Lemma I.8 accounts for a triple that the adversary chooses as a function of the oracle.

For a triple $\theta = (\mathsf{x}, \mathsf{w}_0, \mathsf{w}_1)$ with $(\mathsf{x}, \mathsf{w}_0), (\mathsf{x}, \mathsf{w}_1) \in \mathcal{R}$ and a proof salt s, let $Y^b_{\theta,s} = (v, d, \theta)$ consist of the IOP view v for w_b, uniform leaf digests d, and the challenges θ that the challenge inputs derive from v and d at the salt s, and let $\Delta_\theta(\mathsf{H}, s) = \mathsf{SD}(Y^0_{\theta,s}, Y^1_{\theta,s}) \in [0, 1]$. The challenge inputs and stream points that carry s form a part $\mathcal{Q}_s$ of $\mathcal{Q}$, and the parts for distinct salts are disjoint. Write H_s for the restriction of H to $\mathcal{Q}_s$ and $\mathsf{H}_\circ$ for its restriction to the leaves and internal nodes. The distance $\Delta_\theta(\mathsf{H}, s)$ depends on H only through H_s and $\mathsf{H}_\circ$.

Lemma I.7 (One proof at a fixed triple)**.** *In the setting of Theorem I.5, fix a triple $\theta = (\mathsf{x}, \mathsf{w}_0, \mathsf{w}_1)$ as above.*

1. *For every H, the distributions of the proof at H for w_0 and for w_1 are within $2\lambda(\mathsf{H}) + \bar{\Delta}_\theta(\mathsf{H})$, where $\bar{\Delta}_\theta(\mathsf{H}) = 2^{-8b_s} \sum_s \Delta_\theta(\mathsf{H}, s)$, and $\lambda(\mathsf{H}) \geq 0$ does not depend on θ and has mean at most $n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}}$ over H.*
2. *For every salt s and every fixing of H outside $\mathcal{Q}_s$, the mean of $\Delta_\theta(\mathsf{H}, s)$ over a uniform H_s is at most $p = 2\varepsilon_{\mathrm{hvzk}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}}$.*

Proof. (1) Fix H. We pass from the honest proof for w_0 to the honest proof for w_1 through three hybrids. For a leaf content c and a leaf salt σ, write $d_{\mathsf{H}}(\sigma, c)$ for the digest of the leaf with salt σ and content c, let D_c be the distribution of $d_{\mathsf{H}}(\sigma, c)$ for uniform σ, and let U be uniform on digests. Let

$$\delta(\mathsf{H}) = \max_c \mathsf{SD}(D_c, U), \qquad \lambda(\mathsf{H}) = n_{\mathrm{pos}}\, \delta(\mathsf{H}),$$

so that $\mathbb{E}_{\mathsf{H}}[\lambda(\mathsf{H})] \leq n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}}$ (Definition I.1).

- Let H_1 be the proof for w_0 in which each leaf position π, with content c_π, receives a uniform digest d_π and a salt σ_π uniform on $\{\sigma : d_{\mathsf{H}}(\sigma, c_\pi) = d_\pi\}$, or uniform if this set is empty. Fix the proof salt and the prover's coins, and replace the positions message by message in message

order. Given the execution before a prover message, the contents c_π of its positions are fixed, since they are a function of the coins and of the challenges of its past, and its honest leaf salts are fresh. Given d_π, an honest salt is uniform on $\{\sigma : d_{\mathsf{H}}(\sigma, c_\pi) = d_\pi\}$, so the honest pair (σ_π, d_π) and the replaced one are within $\mathsf{SD}(D_{c_\pi}, U) \le \delta(\mathsf{H})$. In both proofs the rest of the execution applies the same randomized map to these pairs. Summing over the n_{pos} positions, the honest proof and H_1 are within $\lambda(\mathsf{H})$.

- Let H_2 be the proof of the first step for w_1. In H_1 and H_2, every root is a function of the uniform digests and H, and at the salt s the proof is $F(Y^b_{\theta,s})$, where the randomized map F draws the opened salts from their preimages with coins independent of the witness. So at s the two proofs are within $\mathsf{SD}(Y^0_{\theta,s}, Y^1_{\theta,s}) = \Delta_\theta(\mathsf{H}, s)$. The salt is uniform and independent of the prover's other coins, so by the convexity of SD, H_1 and H_2 are within $\bar{\Delta}_\theta(\mathsf{H})$.
- As in the first step, H_2 and the honest proof for w_1 are within $\lambda(\mathsf{H})$.

(2) Fix s and H outside $\mathcal{Q}_s$, and let H_s be uniform. Write $o = (v, d)$ for a pair of an IOP view and leaf digests, and call (o, θ) *consistent* if the challenge inputs derive θ from v and d. Consistency depends on H alone, so it is the same for both witnesses. At given coins, each derived challenge is a function of the messages before it, so by induction over the rounds exactly one consistent pair is revealed. Let μ_θ and μ'_θ be the distributions of o at the fixed challenges θ for w_0 and for w_1, so that $\Pr[Y^0_{\theta,s} = (o, \theta)] = \mu_\theta(o)$ and $\Pr[Y^1_{\theta,s} = (o, \theta)] = \mu'_\theta(o)$ for consistent (o, θ), and both are 0 otherwise. A proof that aborts outputs $\perp$, and it aborts only on a sampler failure. Let

$$\phi(o, \theta) = \frac{\max(0, \mu_\theta(o) - \mu'_\theta(o))}{\mu_\theta(o)}, \qquad \phi(\perp) = 1,$$

with $\phi(o, \theta) = 0$ where $\mu_\theta(o) = 0$, so that ϕ takes values in $[0, 1]$. Write $o(\theta)$ for the pair revealed at the fixed challenges θ, θ^{der} for the derived challenges and θ^{u} for uniform challenges. For fixed coins of the prover and fixed digests, the challenge inputs of the proof are distinct, so, as in the proof of Theorem I.3(1), $\mathsf{SD}(\theta^{\mathrm{der}}, \theta^{\mathrm{u}}) \le \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}}$, and $\perp$ occurs only within this distance. Then

$$\begin{aligned}
\Delta_\theta(\mathsf{H}, s) &\le \mathbb{E}\big[\phi(o(\theta^{\mathrm{der}}), \theta^{\mathrm{der}})\big] && \text{by the definition of } \phi,\\
&\le \mathbb{E}\big[\phi(o(\theta^{\mathrm{u}}), \theta^{\mathrm{u}})\big] + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} && \text{since } \phi \in [0, 1],\\
&= \mathbb{E}_\theta\big[\mathsf{SD}(\mu_\theta, \mu'_\theta)\big] + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} && \text{since } o(\theta) \sim \mu_\theta,\\
&= \mathsf{SD}\big(\mathrm{View}(\mathbf{P}, \mathbf{V}, \mathsf{x}, \mathsf{w}_0), \mathrm{View}(\mathbf{P}, \mathbf{V}, \mathsf{x}, \mathsf{w}_1)\big) + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} &&\\
&\le 2\varepsilon_{\mathrm{hvzk}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} && \text{by Definition 2.22.}
\end{aligned}$$

The equality before the last step holds because the digests are uniform and independent of the view, and θ is a component of the view with the same uniform distribution for both witnesses. Only the challenges derived for w_0 enter, so $\varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}}$ is charged once. □

Lemma I.8 (Adaptive choice of the triple)**.** *Let $\mathcal{T}$ be a finite set of triples, fixed before H is drawn, with $|\mathcal{T}| \le N$, and let $p \ge 0$ bound the mean of $\Delta_\theta(\mathsf{H}, s)$ over a uniform H_s for every $\theta \in \mathcal{T}$, every salt s and every fixing of H outside $\mathcal{Q}_s$. Then*

$$\mathbb{E}\Big[\max_{\theta \in \mathcal{T}} \bar{\Delta}_\theta(\mathsf{H})\Big] \le p + \sqrt{\frac{\ln N}{2 \cdot 2^{8b_{\mathrm{s}}}}}.$$

Proof. Given $\mathsf{H}_\circ$, the restrictions H_s for the $2^{8b_{\mathrm{s}}}$ salts are independent and uniform. So, given $\mathsf{H}_\circ$, each $\bar{\Delta}_\theta(\mathsf{H})$ is an average of $2^{8b_{\mathrm{s}}}$ independent variables with values in $[0, 1]$, with conditional mean

at most p by hypothesis. Let $X_\theta = \bar{\Delta}_\theta(\mathsf{H}) - \mathbb{E}[\bar{\Delta}_\theta(\mathsf{H}) \mid \mathsf{H}_\circ]$. Hoeffding's lemma [Hoe63, (4.16), p. 22] bounds the conditional moment generating function of each centered summand, and by the independence of the summands,

$$\mathbb{E}\big[e^{\kappa X_\theta} \mid \mathsf{H}_\circ\big] \le e^{\kappa^2/(8\cdot 2^{8b_\mathrm{s}})} \quad \text{for every real } \kappa.$$

Given $\mathsf{H}_\circ$, X_θ is therefore sub-Gaussian with variance proxy $1/(4\cdot 2^{8b_\mathrm{s}})$ [CY26, Def. 1.2.6, p. 7]. If $|\mathcal{T}| \le 1$, the conditional mean of $\max_\theta X_\theta$ is at most 0. Otherwise Lemma I.2 with $n = |\mathcal{T}| \le N$ bounds it by $\sqrt{\ln N/(2\cdot 2^{8b_\mathrm{s}})}$. Adding the conditional means, at most p, and taking the expectation over $\mathsf{H}_\circ$ gives the bound. □

Proof of Theorem I.6. Let hybrid $j \le Q$ answer the first j proof queries with w_1 and the others with w_0. Hybrids j and $j+1$ differ only in proof query $j+1$, and we bound their distance at every value of the execution before that query. Let $\mathcal{T}$ be the set of triples $(\mathsf{x}, \mathsf{w}_0, \mathsf{w}_1)$ with $(\mathsf{x}, \mathsf{w}_0), (\mathsf{x}, \mathsf{w}_1) \in \mathcal{R}$ and $|\mathsf{x}| + |\mathsf{w}_0| + |\mathsf{w}_1| \le L$. It depends only on $\mathcal{R}$, the encodings and L, so it is fixed before H is drawn. A triple of $\mathcal{T}$ is determined by $|\mathsf{x}|$, $|\mathsf{w}_0|$ and the concatenation of the three encodings, a bit string of length at most L, so

$$|\mathcal{T}| \le N_C = (L+1)^2 \cdot 2^{L+1}, \qquad \ln N_C = (L+1)\ln 2 + 2\ln(L+1).$$

Each proof is drawn given the oracle, so H is uniform in every hybrid. The adversary chooses the triple of the step in $\mathcal{T}$ as a function of H and the earlier proofs, so by Lemma I.7(1) the step's distance at H is at most $2\lambda(\mathsf{H}) + \max_{\theta\in\mathcal{T}} \bar{\Delta}_\theta(\mathsf{H})$. By Lemma I.7(1), and by Lemma I.8 with $N = N_C$ and $p = 2\varepsilon_{\mathrm{hvzk}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}}$ from Lemma I.7(2), its mean over H is at most

$$2n_{\mathrm{pos}}\varepsilon_{\mathrm{leaf}} + 2\varepsilon_{\mathrm{hvzk}} + \varepsilon_{\mathrm{ov}} + \varepsilon_{\mathrm{key}} + \sqrt{\frac{\ln N_C}{2\cdot 2^{8b_\mathrm{s}}}}.$$

Summing over the Q steps gives $\varepsilon_{\mathrm{wi}}(Q, L)$. □

Theorem I.6 holds against distinguishers with any number of hash queries. By the triangle inequality with the simulator, Theorem I.5(1) gives witness indistinguishability only against adversaries with at most t hash queries before the last proof query, with error $2\varepsilon_{\mathrm{zk}}(t, Q)$. Chiesa and Yogev state adaptive witness indistinguishability of BCS for one proof and omit its proof as similar to that of their Lemma 33.3.3 [CY26, Def. 33.1.1, pp. 426–427, and Lem. 33.3.4, p. 434]. Their bound stays small for distinguishers that make any number of queries. Theorem I.6 states the property for Q proofs, each chosen after the earlier ones. The length bound L enters $\varepsilon_{\mathrm{wi}}$ only through its last term, which is $\approx \sqrt{L\ln 2/(2\cdot 2^{8b_\mathrm{s}})}$ for large L.

I.3 Knowledge soundness

The extractor is Chiesa, Manohar and Spooner's database extractor [CMS19, §8.5]. It runs the adversary once against Zhandry's compressed oracle [Zha18]. That oracle records the queries in a quantum register, the *database*, and the extractor measures the database once at the end. It reconstructs the IOP transcript from the measured table and applies the extractor of the IOP's round-by-round knowledge soundness. The instability of the set of tables from which extraction fails bounds the probability that the extractor fails [CMS19, Def. 5.6 and Lem. 5.7].

Protocol I.9 (Database extractor)**.** The extractor $\mathcal{E}_{\mathrm{SL}}$ is built from the extractor $\mathbf{E}$ of round-by-round knowledge soundness of the IOP (Definition 2.20). It runs the adversary A against a compressed oracle with one database slot for each hash query of the execution: those of A, of the simulator, of the verifier and of the replay below. It takes this number from the execution and receives no bound on the number of hash queries.

1. The execution applies A's channels in order. Before each call, $\mathcal{E}_{\mathrm{SL}}$ reads the classical register in which A holds its choice of oracle for that call.

2. Answer each query with Zhandry's compressed oracle in the phase form of Chiesa, Manohar and Spooner [CMS19, §3.6 and §4.2], skipped on the inputs that have an assignment. Controlled on the input having an assignment, swap out the phase register, so that the compressed oracle acts as the identity [CMS19, Lem. 3.2, p. 17], and apply the phase of the assigned value directly. This is a query to $\mathsf{H}[\mathsf{Reprog}]$.

3. On a proof query, read the queried statement, run the simulator of Protocol I.4 on it classically with its queries answered as in step 2, append the statement to the list Qry of queried statements and the assignments to the assignment list, and return the simulator's proof.

4. After A's output, make at most $k = k_q + Qq_S$ further queries to the base oracle H. These are queries at the verifier's inputs for $(\mathsf{x}, \mathsf{pf})$ and a replay of the simulator's recorded queries. The verifier reads their answers with the final assignments applied.

5. Measure the output $(\mathsf{x}, \mathsf{pf})$, the lists of queried statements and of assignments, and the database, obtaining a table db. Leave A's other registers untouched.

6. Run the database algorithm of Chiesa, Manohar and Spooner [CMS19, §6.1 and §8.5.2], adapted to Protocol 6.5, on db. For the target statement x, consider each recorded challenge input that decodes to $(s, \mathsf{x}, i, \ell_i)$. Reconstruct the transcript of past(i) in message order. At the label and final digest of a merged branch, follow the digest to its unique recorded preimage, the final input of the branch, and reconstruct the branch's messages from it in the same way. Read each prover item's non-oracle message, and recover leaves from its roots by following each digest through its unique recorded preimage. A node preimage descends the tree, and a leaf preimage gives a leaf's salt and bytes. A missing or ambiguous preimage leaves the leaf undefined, and it decodes to the default. At each verifier message, require the challenge-input and stream queries in db, the stream's digest equal to the written-back digest, and sampler success. Otherwise discard the transcript. Add $\mathbf{E}(\mathsf{x}, \mathsf{tr})$ for each reconstructed transcript tr to the candidates. Output the first candidate w with $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$, or $\perp$.

Reading a classical register in the computational basis leaves the reduced state of the execution unchanged (Lemma I.11). The extractor replays the simulator's queries so that it can recognize each programmed point as coming from a challenge input of a queried statement. The base oracle answers the replayed queries with the values that the simulator saw, because the simulator queries only challenge inputs, leaves and internal nodes, and the assignments are only at stream points.

The lemmas of Chiesa, Manohar and Spooner and of Zhandry are stated for T-query quantum oracle algorithms: T unitaries applied to a pure initial state, alternating with T queries in a fixed order [CMS19, §3.2, p. 14]. An adversary of Definition 6.1, with the simulator or the verifier folded into it, applies channels, starts from a mixed state and chooses during its execution whether its next call is a query. Lemmas I.10 and I.11 connect the two models.

Lemma I.10 (Padding to a fixed query order)**.** *Let B be a quantum algorithm whose only oracle is H, with operations as in Definition 6.1, that runs in consecutive segments $0, \ldots, r-1$, whose boundaries fall between its calls and are decided by its classical registers, and makes at most t_i queries to H in segment i on every execution path, and let $T = t_0 + \cdots + t_{r-1}$. There is a T-query quantum oracle algorithm $\hat{B}$ [CMS19, §3.2, p. 14] whose queries, numbered from 0, with numbers in $[t_0 + \cdots + t_{i-1},\ t_0 + \cdots + t_i)$ are made in segment i, such that for every oracle H, and also for the compressed phase oracle with capacity at least T, the reduced state of $\hat{B}$'s execution on B's registers*

and on the oracle's register is that of B's execution. In the standard form, the queries that $\hat{B}$ adds may be made on any fixed input $x_0 \in \mathcal{Q}$.

Proof. Ambainis, Hamburg and Unruh state the unitary form for oracle algorithms with a fixed order of queries [AHU18, pp. 8–9]. Two steps reduce B to that form. First, group everything B does between two consecutive queries of a segment, including measurements, choices of the next call and the end of the segment, into one channel controlled on B's classical registers, and fill the query slots that a segment leaves unused on an execution path with dummy queries. In the phase form a dummy query has phase register 0, on which the compressed phase oracle acts as the identity [CMS19, Lem. 3.2, p. 17], as does the phase oracle of every H. In the standard form it holds the fixed input x_0 and the uniform superposition over $\mathcal{O}$ in the answer register, which every query leaves unchanged. Second, Stinespring dilations of the channels [Wat18, Cor. 2.27, p. 89], with each measurement outcome in a further register [Wat18, Thm. 2.42, p. 109], and a purification of the initial state [Wat18, Def. 2.9 and Thm. 2.10, pp. 69–71] give unitaries and a pure initial state: the dilation of the channel before the first query, applied to the purification, is the initial state, and each later isometry extends to a unitary on ancillas that start in $|0\rangle$. No later operation acts on the further registers or the purifying register, so tracing them out gives B's state against every oracle that acts only on the query registers and its own register. □

The dummy queries extend the unitary form to algorithms that choose their next call during the execution. The lemma applies because the only oracle of B is H, and the classical parties are folded into B. Purification fails for an adversary whose calls to a classical oracle depend on intermediate measurements [ABKM21, p. 4].

Lemma I.11 (Block-diagonal oracle)**.** *Let B be a quantum algorithm whose only oracle is H, with operations as in Definition 6.1 and at most T queries on every execution path. The reduced state of B's registers at the end of its execution against a uniform H equals their reduced state at the end of its execution against the compressed oracle with capacity T. Reading a register of B that holds a classical value, one that a computational-basis measurement wrote, in the computational basis, leaves this state unchanged, and a measurement of the compressed oracle's database acts on the oracle's register alone.*

Proof. Purify the oracle as the uniform superposition over H in an oracle register [Zha18, §3, proof of Lem. 4, p. 15]. Every query is then controlled on that register and diagonal in its computational basis, and every channel of B acts as the identity on it. So the reduced state of B's registers is the average over H of the state of the execution with the fixed oracle H, that is, the state of the execution against a uniform H. This holds for any channels, measurements and choices made during the execution, without purifying B. The compressed oracle is the image of the purified oracle under an isometry on the oracle register alone, applied between calls, where it commutes with B's operations [Zha18, §3, proof of Lem. 4, p. 15]. For a T-query algorithm this is Chiesa, Manohar and Spooner's Lemma 3.3 [CMS19, p. 17]. With at most T queries on every execution path the database is never full at a query, so the compressed oracle with capacity T acts as Zhandry's on every state the execution reaches. An isometry on the oracle register does not change the reduced state of the other registers. For each fixed H of the purified execution, the joint state is block-diagonal in the computational basis of a register that holds a classical value, since a computational-basis measurement wrote it. The isometry to the compressed oracle acts on the oracle register alone and preserves this, so reading the register does not change the state. A channel on the database register leaves the reduced state of B's registers unchanged and commutes with every operation on B's registers. □

Lemma I.12 (Instability of the bad set)**.** *Let B be the set of databases* db *with a* digest collision*, two distinct recorded inputs whose answers have the same digest, or with a* knowledge failure*: some recorded challenge input for some statement* x *and a verifier message i encodes a past whose transcript* tr *the database extractor reconstructs from* db *(Protocol I.9), and* $\mathsf{State}(\mathsf{x}, i, \mathsf{tr}) = 0$*,* $\mathsf{State}(\mathsf{x}, i, \mathsf{tr} \| \theta_i) = 1$ *for the recorded challenge θ_i, and* $(\mathsf{x}, \mathbf{E}(\mathsf{x}, \mathsf{tr})) \notin \mathcal{R}$ *(Definition 2.20). For every database with fewer than T entries and every input it does not contain, the probability over a uniform answer y that adding the entry changes whether the database is in B is at most* $\varepsilon^{\mathrm{k}} + (\max(2, c) + n_J + 1)T/2^{8b}$*. That is, the instability $I(B, T)$ [CMS19, Defs. 5.4 and 5.6, pp. 21–22] is at most this quantity.*

Proof. Fix a database db with fewer than T entries and an input u that db does not contain. Let y be uniform on $\mathcal{O}$, and let $\mathsf{db}' = \mathsf{db} \cup \{(u, y)\}$. Let F be the event that $\mathsf{dg}(y)$ equals the digest of a recorded answer of db, a digest of an item that a recorded input carries, or a final digest that a recorded input carries for a merged branch. A recorded input carries at most $\max(2, c)$ digests of items, counting a stream point's key and a node's two child digests among them, and at most n_J final digests, so

$$\Pr[F] \leq \frac{(\max(2, c) + n_J + 1)\, T}{2^{8b}}.$$

Outside F, the entry (u, y) creates no digest collision, and no recorded input of db contains $\mathsf{dg}(y)$. So the entry changes no transcript, leaf or challenge that the database extractor already reconstructs from db: the entries such a reconstruction uses are in db and u is not, and outside F the entry creates no second preimage of a recorded digest. In particular, if u is the final input of a branch, it gives no recorded challenge input of a later part a new reconstructed past. It remains to bound the probability, outside F, that db and db' differ in membership in B.

If $\mathsf{db} \in B$, then $\mathsf{db}' \in B$. A digest collision of db is one of db'. A knowledge failure of db is one of db', since the entry changes no reconstructed transcript or challenge.

Let $\mathsf{db} \notin B$. Outside F, db' has no digest collision, so $\mathsf{db}' \in B$ only through a knowledge failure at a transcript that the entry extends at its end. If u is a challenge input, a leaf or a node, it extends no reconstructed transcript, and a new challenge input completes no round, since no stream point at its key $\mathsf{dg}(y)$ is recorded. Let $u = \mathsf{stream}(v, s)$. The key v is the digest of at most one recorded answer, so the entry fixes the challenge $\theta_i = \mathsf{sample}_i(y)$ at one reconstructed past tr of one challenge input, for one statement x and one verifier message i. By the write-back, every challenge input whose past contains i contains $\mathsf{dg}(y)$, directly or through the final input of a branch. No recorded input contains $\mathsf{dg}(y)$, so the entry completes no later round. Hence $\mathsf{db}' \in B$ only if

$$\mathsf{State}(\mathsf{x}, i, \mathsf{tr}) = 0, \qquad \mathsf{State}(\mathsf{x}, i, \mathsf{tr} \| \theta_i) = 1 \qquad \text{and} \qquad (\mathsf{x}, \mathbf{E}(\mathsf{x}, \mathsf{tr})) \notin \mathcal{R}.$$

A sampler failure fixes no challenge. Conditional on success, θ_i is uniform on $\mathcal{C}_i$, so by Definition 2.20 these hold with probability at most ε^{k}.

In both cases, the membership changes with probability at most $\Pr[F] + \varepsilon^{\mathrm{k}}$, which is the stated bound. □

This adapts Proposition 8.14 and Lemma 6.11 of Chiesa, Manohar and Spooner [CMS19, pp. 44 and 33]. Their Proposition 8.14 bounds by $2t + 1$ the digests that recorded pairs carry, and their Lemma 6.11 adds t for collisions. In Lemma I.12, $(\max(2, c) + n_J)T$ bounds the digests that recorded inputs carry, because a challenge input carries up to c digests of items and n_J final digests against two in their pair-hash chain, and T bounds collisions. With the write-back, ε^{k} bounds the instability. Without it, one stream answer could complete rounds of many recorded transcripts, and Chiesa,

Manohar and Spooner need a stronger round-by-round notion in that case [CMS19, §8.6, p. 45]. The empty database is not in B, since it has no collision and no recorded challenge input.

A *base game* [CMS19, §4] is a set G of tuples $((x_0, \ldots, x_{k-1}), (y_0, \ldots, y_{k-1}))$ of oracle inputs and outputs, possibly with extra classical data. An algorithm wins the oracle game if it outputs a tuple of G with $y_j = \mathsf{H}(x_j)$ for every j. It wins the *simulated oracle game* [CMS19, Def. 4.7, p. 20] if, executed against the compressed oracle, it outputs a tuple of G with which the measured database agrees. Chiesa, Manohar and Spooner's database game asks only for some tuple of G that agrees with the database. Lemma I.14 needs the algorithm's actual output, so the proofs use the simulated oracle game.

Definition I.13 (Base game of spBCS)**.** The *base game* of spBCS is the condition under which a simulation-extractability adversary wins, in the form of [CMS19, §4]. The *appended queries* of Protocol I.9 are its k final queries to the base oracle. Let G be the set of tuples (a statement x, a proof, the lists of queried statements and of assignments, and the appended inputs with answers) that satisfy three conditions. First, x is fresh. Second, there is an oracle under which the verifier accepts with all its queries among the appended ones, answered as recorded with the final assignments applied. Third, every assignment is a stream point $\mathsf{stream}(\mathsf{dg}(y), s)$, where y is the recorded answer to a challenge-input query on an input that encodes the salt s and a queried statement.

Lemma I.14 (Extraction outside the bad set)**.** *If a database* db *agrees with a tuple of* G *for the statement* x*, then* $\mathcal{E}_{\mathrm{SL}}$ *outputs a witness for* x *from* db*, or* $\mathsf{db} \in B$*.*

Proof. Let $\mathsf{db} \notin B$ agree with a tuple of G with statement x, proof pf and queried statements $\mathsf{x}_1, \ldots, \mathsf{x}_Q$. We show that $\mathcal{E}_{\mathrm{SL}}$ outputs a witness for x.

Suppose the verifier's execution meets an assigned point $\mathsf{stream}(u, s)$. By the third condition of G, the simulator's challenge input for some x_j has digest u, and so does the verifier's challenge input for x. Since x is fresh, $\mathsf{x} \neq \mathsf{x}_j$, and since the encoding is injective, the two inputs are distinct. They form a digest collision, a contradiction. So every answer that the verifier reads is a base answer recorded in db. In particular, at every verifier message $i \in J_V$, the challenge input and the stream point are recorded, the stream's digest equals the written-back digest, and the sampler succeeds, since the verifier computed them.

Let tr be the full transcript whose non-oracle messages and challenges are those of pf and of the verifier's execution, and whose oracle strings hold at each position the leaf that the database extractor reconstructs from db, or the default where it reconstructs none. Since db has no digest collision, each recorded digest has a unique preimage. So the leaf at every position that pf opens is the opened leaf. The reconstructions of a message from different pasts also agree, since each comes from the same item of pf or from the unique preimage of a final digest. Hence $\mathsf{tr}|_{\mathrm{past}(i)}$ is the past that the database extractor reconstructs at the verifier's challenge input for i, for every $i \in J_V$. The IOP verifier reads only the non-oracle messages, the challenges and the opened positions, which agree with those of the spBCS verifier, so it accepts tr.

If $\mathbf{E}(\mathsf{x}, \mathsf{tr}|_{\mathrm{past}(i)})$ is a witness for x for some $i \in J_V$, then $\mathcal{E}_{\mathrm{SL}}$ outputs a witness, since it tries every reconstructed past. Otherwise, by condition (a) of Definition 2.20, some $i \in J_V$ has

$$\mathsf{State}(\mathsf{x}, i, \mathsf{tr}|_{\mathrm{past}(i)}) = 0 \qquad \text{and} \qquad \mathsf{State}(\mathsf{x}, i, \mathsf{tr}|_{\mathrm{past}(i) \cup \{i\}}) = 1.$$

This is a knowledge failure of db at the verifier's challenge input for i, so $\mathsf{db} \in B$, a contradiction. □

Lemma I.15 (Oracle games and lifting)**.** *Let* G *be a base game whose tuples have* k *oracle inputs, and let* $\hat{A}$ *be a* T*-query quantum oracle algorithm [CMS19, §3.2, p. 14] for an oracle with outputs of* $8(b + e)$ *bits.*

1. *The values of the oracle game and of the simulated oracle game of $\hat{A}$ satisfy* $\Pr[G : \mathit{oracle}] \leq \Pr[G : \mathit{simulated}] + 2\sqrt{k/2^{8(b+e)}}$.

2. *For every set B of databases that does not contain the empty database, the database measured at the end of the execution of $\hat{A}$ against the compressed oracle lies in B with probability at most $6T^2 \cdot I(B,T)$.*

Proof. (1) By Lemma 4.9 of Chiesa, Manohar and Spooner, with k oracle inputs in the output and outputs of $8(b+e)$ bits [CMS19, p. 20], $\sqrt{\Pr[G : \text{oracle}]} \leq \sqrt{\Pr[G : \text{simulated}]} + \sqrt{k/2^{8(b+e)}}$. For $a, a' \leq 1$ and $\delta \geq 0$, $\sqrt{a} \leq \sqrt{a'} + \delta$ implies $a \leq a' + 2\delta$. If $\sqrt{a'} + \delta \geq 1$, then $a' + 2\delta \geq 1 + (1 - \sqrt{a'})^2 \geq 1 \geq a$. Otherwise squaring gives $a \leq a' + \delta(2\sqrt{a'} + \delta) \leq a' + 2\delta$. (2) Lemmas 5.9 and 5.10 of Chiesa, Manohar and Spooner give the bound of their Lemma 5.7 for sets of databases that do not contain the empty database [CMS19, p. 22]. □

Theorem I.16 (Knowledge soundness of spBCS)**.** *Let* IOP *be a series–parallel IOP (Definition 2.18) for $\mathcal{R}$, with round-by-round knowledge errors at most ε^{k} with an extractor $\mathbf{E}$ (Definition 2.20), and let* $(\mathsf{Prv}, \mathsf{Vrf})$ *be Protocol 6.5 with digests of $b \geq 1$ bytes and streams of $b + e$ bytes, with c, n_J and k_q as in Definition I.1. Then:*

1. *in the QROM,* $(\mathsf{Prv}, \mathsf{Vrf})$ *is adaptively knowledge sound (Definition H.3) with the database extractor $\mathcal{E}_{\mathrm{SL}}$ built from $\mathbf{E}$ (Protocol I.9), oracle simulation error 0 and extraction error at most*

$$\varepsilon_{\mathrm{ex}}(t) = 2\sqrt{\frac{k_q}{2^{8(b+e)}}} + 6(t+k_q)^2\Big(\varepsilon^{\mathrm{k}} + \frac{(\max(2,c) + n_J + 1)(t+k_q)}{2^{8b}}\Big),$$

   and it has adaptive soundness error at most $\varepsilon_{\mathrm{ex}}(t)$ (Definition H.4);

2. *in the ROM,* $(\mathsf{Prv}, \mathsf{Vrf})$ *is adaptively knowledge sound (Definition H.3) with oracle simulation error 0 and extraction error at most*

$$\varepsilon^{\mathrm{C}}_{\mathrm{ex}}(t) = (t+k_q)\Big(\varepsilon^{\mathrm{k}} + \frac{(\max(2,c) + n_J + 1)(t+k_q)}{2^{8b}}\Big),$$

   and it has adaptive soundness error at most $\varepsilon^{\mathrm{C}}_{\mathrm{ex}}(t)$ (Definition H.4) against $(t,0)$-adversaries of the ROM. The extractor answers the queries of the adversary and the verifier by lazy sampling and applies the database algorithm of Protocol I.9, built from $\mathbf{E}$, to the table of these queries and answers.

If membership in $\mathcal{R}$ is decidable in polynomial time and N_{msg}, n_B, n_{pos}, q_{IOP} and $\mathsf{l}_{\max}$ are polynomial in the length of the statement, both extractors run in time polynomial in t, k_q, $L_{\mathcal{Q}}$, $b + e$ and the length of the statement.

Proof. (1) Let $T = t + k_q$. We bound the oracle simulation error, then the extraction error.

$\mathcal{E}_{\mathrm{SL}}$ answers every query of A and of the verifier with the compressed oracle, reads A's classical register that holds the choice of its next call, and measures the database only after the verifier's queries. At each query its database has fewer entries than slots, so its compressed oracle never acts in its full case [CMS19, Lem. 3.2, p. 17], and it acts on the measured table as the compressed oracle with capacity T does. By Lemma I.11, applied to A followed by the verifier, the joint state of $(\mathsf{x}, \mathsf{pf}, Z, v)$ is that of the real execution, including the quantum register Z. So the oracle simulation error is 0.

For the extraction error, take G of Definition I.13 with no proof query, so that every statement is fresh, and let db be the measured database. By Lemma I.10, with dummy queries of phase 0, the

execution of A followed by the verifier's queries is a T-query algorithm with the same reduced state and the same measured database as the execution of $\mathcal{E}_{\mathrm{SL}}$. In the extraction execution, $v = 1$ exactly when the output and the appended queries form a tuple of G, and by Lemma I.11 this event has probability $\Pr[G : \text{oracle}]$. A win of the simulated oracle game is an output tuple of G with which db agrees, so it implies $v = 1$, and by Lemma I.14, if db agrees with the output tuple and $\mathcal{E}_{\mathrm{SL}}$ returns no witness, then $\mathsf{db} \in B$. Measuring the output before the database does not change $\Pr[\mathsf{db} \in B]$, since the two measurements act on different registers and commute. So

$$\begin{aligned}
\Pr[v = 1 \wedge (\mathsf{x}, \mathsf{w}) \notin \mathcal{R}] &\leq \Pr[G : \text{oracle}] - \Pr[G : \text{simulated}] + \Pr[\mathsf{db} \in B] \\
&\leq 2\sqrt{k_q / 2^{8(b+e)}} + 6T^2\, I(B, T) && \text{by Lemma I.15 with } k = k_q, \text{ since } \emptyset \notin B \\
&\leq \varepsilon_{\mathrm{ex}}(t) && \text{by Lemma I.12.}
\end{aligned}$$

The running time is argued after Theorem I.17. (2) Lazy sampling gives the distribution of a uniform oracle, so the oracle simulation error is 0. The table of a classical execution is the database, so step 4 is not needed. Classically, the probability that the table of a T-query execution lies in B is at most T times the instability [CMS19, p. 22 and footnote 6], by a union bound over the T entries, and Lemmas I.12 and I.14 apply with $T = t + k_q$. The union bound holds for each choice of the adversary's coins: stopping the execution before its i-th query [CMS19, footnote 6, p. 22] gives the same execution as stopping it before query $i + 1$ on execution paths with fewer than i queries, and the coins are independent of H, so the bound survives averaging over them. In both models the oracle simulation error is 0, so Lemma H.5 gives the soundness errors. □

Part (1) extends Chiesa, Manohar and Spooner's QROM knowledge soundness [CMS19, Thm. 8.6(2)], which they prove for a fixed statement, with an asymptotic extraction probability and one oracle string per message. The statement here has the online form of Definition H.3, explicit constants and a statement chosen by the adversary. Part (2) is Chiesa and Yogev's straight-line knowledge soundness [CY26, Def. 5.1.1, p. 48] for the database algorithm applied to the query traces.

Chiesa, Manohar and Spooner prove soundness of BCS in the QROM from round-by-round soundness [CMS19, Thm. 8.6(1), p. 40]. The soundness bound of Theorem I.16 assumes the stronger round-by-round knowledge soundness and lets the adversary choose the statement.

I.4 Weak online simulation extractability

This subsection proves Theorem I.17. Its extractor is straight-line: it runs the adversary once, without rewinding, and answers the hash queries of the adversary, the simulator and the verifier itself. Chiesa, Manohar and Spooner's extractor for BCS is straight-line in the same way [CMS19, §8.5], and the theorem holds for every adversary bounded only in its numbers of hash queries and proof queries (Definition 6.1).

Compared with knowledge soundness, the simulator adds one case to the argument. It reprograms the oracle only at stream points whose keys are digests of the answers to its own challenge inputs, and each of those inputs encodes the queried statement. An execution of the verifier for a statement of no proof query hashes different inputs, so it reaches a reprogrammed point only if the answers to two different inputs have the same digest. Outside that event the verifier sees the unprogrammed oracle and the knowledge-soundness argument applies unchanged.

Theorem I.17 (Weak online simulation extractability of spBCS)**.** *Let* IOP, $\mathbf{E}$, ε^{k}, $(\mathsf{Prv}, \mathsf{Vrf})$, c, n_J *and* k_q *be as in Theorem I.16, and* q_S *as in Definition I.1. Let* $\mathbf{S}$ *be any probabilistic algorithm that maps a statement to challenges* θ*, non-oracle messages and values at the positions* $\mathsf{qry}(\theta)$*. Let* Sim *be*

the simulator of Protocol I.4 built from $\mathbf{S}$*, and* $\mathcal{E}_{\mathrm{SL}}$ *the database extractor of Protocol I.9 built from* $\mathbf{E}$*. Put* $k = k_q + Qq_S$ *and* $T = t + k + Qq_S$*. Then:*

1. *in the QROM,* $(\mathsf{Prv}, \mathsf{Vrf})$ *is weakly online simulation extractable with respect to* Sim *(Definition 6.4), with the extractor* $\mathcal{E}_{\mathrm{SL}}$*, oracle simulation error* 0 *and extraction error at most*

$$\varepsilon_{\mathrm{SE}}(t, Q) = 2\sqrt{\frac{k}{2^{8(b+e)}}} + 6T^2\Big(\varepsilon^{\mathrm{k}} + \frac{(\max(2, c) + n_J + 1)\,T}{2^{8b}}\Big);$$

2. *in the ROM,* $(\mathsf{Prv}, \mathsf{Vrf})$ *is weakly online simulation extractable with respect to* Sim *(Definition 6.4), with oracle simulation error* 0 *and extraction error at most*

$$\varepsilon^{\mathrm{C}}_{\mathrm{SE}}(t, Q) = (t + Qq_S + k_q)\Big(\varepsilon^{\mathrm{k}} + \frac{(\max(2, c) + n_J + 1)(t + Qq_S + k_q)}{2^{8b}}\Big),$$

the extraction error $\varepsilon^{\mathrm{C}}_{\mathrm{ex}}(t + Qq_S)$ *of Theorem I.16(2). Its extractor answers the unassigned hash queries by lazy sampling and the assigned ones from the assignment list. It applies the database algorithm of Protocol I.9, built from* $\mathbf{E}$*, to the statement and to the table* τ *of the queries that the base oracle* H *answers, each with its answer.*

At $Q = 0$, the error of Theorem I.17(1) equals the extraction error $\varepsilon_{\mathrm{ex}}(t)$ of Theorem I.16(1). In the ROM, the extractor of Theorem I.17(2) uses the table of the execution's hash queries. Its error is the classical knowledge error of Theorem I.16(2) with the simulator's hash queries added to the adversary's.

The number T bounds the hash queries of the expanded execution: t queries of the adversary, Qq_S of the simulator answering proof queries, and at most $k = k_q + Qq_S$ appended queries, of which k_q are the verifier's and Qq_S replay the simulator's queries. In the ROM the table already holds the simulator's queries, so the count is $t + Qq_S + k_q$.

The extractor $\mathcal{E}_{\mathrm{SL}}$ runs in quantum polynomial time, as Definition 6.4 requires. The bound T and the lookup count depend on N_{msg}, n_B, n_{pos}, q_{IOP} and $\mathsf{l}_{\max}$, which are polynomial in the length of the statement. With Zhandry's sparse encoding of the database [Zha18, pp. 13–14], each query to the compressed oracle is an efficiently computable isometry [DFMS21, App. B, p. 26]. On a database of at most T entries of $L_{\mathcal{Q}} + 8(b + e)$ bits each, a query takes $O(T(L_{\mathcal{Q}} + 8(b + e)))$ elementary operations. A query to $\mathsf{H}[\mathsf{Reprog}]$ is one such query controlled on membership in the assignment list, which has at most $Q|J_V|$ entries. The database algorithm makes $O(T^2(N_{\mathrm{msg}} + n_{\mathrm{pos}} \log n_{\mathrm{pos}}))$ lookups in all, and at most T calls of $\mathbf{E}$ and of membership in $\mathcal{R}$.

The proof follows Chiesa, Manohar and Spooner's proof of knowledge for BCS [CMS19, §8.5.2], with their Lemma 4.9 (from Zhandry) and Lemmas 5.9 and 5.10 as black boxes (Lemma I.15). Three steps differ from theirs. The base game includes the simulator (steps 1 and 3), the clause on the joint state is that of Theorem I.16(1) with the simulator folded into the adversary (step 2), and the cover lemma handles programmed points (step 5).

Proof of Theorem I.17(1). Fix $t, Q \in \mathbb{N}$ and a (t, Q)-adversary A whose proof queries are statements.

1. The adversary together with the simulator is one algorithm A_{Sim} against the base oracle H, with at most $t + Qq_S$ hash queries on every execution path: at most t of the adversary, and at most Q proof queries of at most q_S hash queries each. A query to $\mathsf{H}[\mathsf{Reprog}]$ is one query to H, skipped on assigned inputs as in Protocol I.9 and controlled on the classical register that holds Reprog. The lists of queried statements and of assignments and the simulator's state are classical registers of A_{Sim}. Let A'_{Sim} be A_{Sim} followed by the appended queries: the verifier's

queries on $(\mathsf{x}, \mathsf{pf})$ and the replay of the simulator's queries. Then A'_{Sim} makes at most T queries to H on every execution path. By Lemma I.10, A'_{Sim} is a T-query algorithm, whose reduced state and measured database are those of the execution of $\mathcal{E}_{\mathrm{SL}}$.

2. At an assigned input, $\mathcal{E}_{\mathrm{SL}}$ swaps out the phase register, so that the compressed oracle acts as the identity [CMS19, Lem. 3.2, p. 17], and applies the phase of the assigned value. So the only oracle of A'_{Sim} is H, and in the execution of $\mathcal{E}_{\mathrm{SL}}$ the compressed oracle answers it. The simulator's classical queries and their measured answers are reads of classical registers of A'_{Sim}. By Lemma I.11, the reduced state of the registers of A'_{Sim} at the end of the execution of $\mathcal{E}_{\mathrm{SL}}$ equals their reduced state at the end of the execution of A'_{Sim} against a uniform H. These registers hold x, pf, Z, Qry, Reprog, the verdict v and the replayed answers. Against a uniform H, the verifier's queries are answered by $\mathsf{H}[\mathsf{Reprog}]$, so the execution of A'_{Sim} up to the verdict is the simulated execution. The replay acts only on the oracle's register and on registers outside x, pf, Z, Qry and v, so it does not change their reduced state. The trace distance in the oracle simulation condition of Definition 6.4 is therefore 0. This is the argument of Theorem I.16(1), applied to A'_{Sim}.

3. Let G be the base game of Definition I.13, and write G also for the event that the measured output and appended queries form a tuple of G. In the execution of $\mathcal{E}_{\mathrm{SL}}$, $v = 1$ and $\mathsf{x} \notin \mathsf{Qry}$ imply the first two conditions of G. Freshness is the first condition. The verifier's queries are among the appended queries, and the verifier reads their recorded answers with the final assignments applied, which gives the second. Against a uniform H, the third condition holds on every execution. The simulator assigns only stream points whose keys are digests of the answers to its own challenge-input queries, and the replayed queries receive the answers that the simulator saw (Protocol I.9). By step 2, the classical registers that decide the third condition have the same distribution in the execution of $\mathcal{E}_{\mathrm{SL}}$, so the third condition holds there with probability 1. Hence
$$\Pr\big[v = 1 \wedge \mathsf{x} \notin \mathsf{Qry} \wedge (\mathsf{x}, \mathsf{w}) \notin \mathcal{R}\big] \leq \Pr\big[G \wedge (\mathsf{x}, \mathsf{w}) \notin \mathcal{R}\big].$$

4. Write agree for the event that the measured database agrees with the measured tuple. Then $\Pr[G \wedge (\mathsf{x}, \mathsf{w}) \notin \mathcal{R}] \leq \Pr[G \wedge \neg\mathsf{agree}] + \Pr[G \wedge \mathsf{agree} \wedge (\mathsf{x}, \mathsf{w}) \notin \mathcal{R}]$. Measuring the output before the database does not change these probabilities, since the two measurements act on different registers and commute. By Lemma I.11, $\Pr[G]$ in the execution of $\mathcal{E}_{\mathrm{SL}}$ is the value $\Pr[G : \text{oracle}]$ of the oracle game of the T-query algorithm of step 1, and $\Pr[G \wedge \mathsf{agree}]$ is the value $\Pr[G : \text{simulated}]$ of its simulated oracle game. So $\Pr[G \wedge \neg\mathsf{agree}] = \Pr[G : \text{oracle}] - \Pr[G : \text{simulated}]$. By Lemma I.15(1), with the k appended queries, $\Pr[G : \text{oracle}] \leq \Pr[G : \text{simulated}] + 2\sqrt{k/2^{8(b+e)}}$, so $\Pr[G \wedge \neg\mathsf{agree}] \leq 2\sqrt{k/2^{8(b+e)}}$.

5. A database agreeing with a tuple of G yields a witness for x through $\mathcal{E}_{\mathrm{SL}}$, or lies in the bad set B of Lemma I.12. So $\Pr[G \wedge \mathsf{agree} \wedge (\mathsf{x}, \mathsf{w}) \notin \mathcal{R}] \leq \Pr[\mathsf{db} \in B]$.

6. The empty database is not in B. So, by Lemma I.15(2), the measured database of the T-query execution of step 1 lies in B with probability at most $6T^2 \cdot I(B, T)$, and $I(B, T) \leq \varepsilon^{\mathrm{k}} + (\max(2, c) + n_J + 1)T/2^{8b}$ by Lemma I.12.

By steps 3 to 6, the probability in the extraction condition of Definition 6.4 is
$$\begin{aligned}
\Pr\big[v = 1 \wedge \mathsf{x} \notin \mathsf{Qry} \wedge (\mathsf{x}, \mathsf{w}) \notin \mathcal{R}\big] &\leq \Pr[G \wedge \neg\mathsf{agree}] + \Pr[\mathsf{db} \in B] && \text{by steps 3 to 5,}\\
&\leq 2\sqrt{k/2^{8(b+e)}} + 6T^2\, I(B, T) && \text{by steps 4 and 6,}\\
&\leq \varepsilon_{\mathrm{SE}}(t, Q) && \text{by Lemma I.12.}
\end{aligned}$$

□

Proof of Theorem I.17(2). Lazy sampling gives the distribution of a uniform oracle, and the extractor answers the assigned inputs from the assignment list, as the simulated execution does. So the extractor's execution is the simulated execution, and the oracle simulation error is 0. For the extraction error, fix the coins of the adversary and of the simulator. The adversary and the simulator form one classical algorithm against the base oracle H, with at most $t + Qq_S$ hash queries, and the verifier adds at most k_q. So the table τ records at most $t + Qq_S + k_q$ queries, each with the answer of H. The simulator queries only challenge inputs, leaves and internal nodes, and it assigns only stream points, so its recorded answers are the answers it received. Every assignment is a stream point $\mathsf{stream}(\mathsf{dg}(y), s)$, where y is the recorded answer to a challenge-input query of the simulator whose input encodes the salt s and a queried statement. If the verifier of an accepted proof for a fresh statement queries an assigned point, its challenge input and the simulator's challenge input for a queried statement are distinct inputs in τ with one digest, so $\tau \in B$. Otherwise every verifier query lies in τ, and the accepted proof gives a tuple of the base game G of Definition I.13 with which τ agrees. By Lemma I.14, the database algorithm outputs a witness for x from τ, or $\tau \in B$. By the union bound of the proof of Theorem I.16(2) over the entries of τ, the table lies in B with probability at most $t + Qq_S + k_q$ times the instability, which Lemma I.12 bounds. The bound holds for every choice of the coins, and so on average over them. □

J Machine-checked results

This appendix lists the Lean declarations of the machine-checked results, grouped by topic in the order of the paper's sections. The statement column names the result and the restrictions of its Lean statement. The names are relative to the root namespace `GordieHowe` of our Lean library, except ArkLib's and Mathlib's.

Statement	Lean declaration
Background (Section 2 and Appendix A)	
Lemma 2.2	`MAC.binding`, `MAC.probUniform_bindingEvent_le`
Lemma 2.3, with the denominators cleared	`Range.committedBadPoly_ne_zero_of_outside`, `Range.committedBadPoly_rowHistogram`
Definition 2.4	`RelativePacking.relTrace`
Definition 2.5	`PackingStage.StageBasis`
Definition 2.6	`PackingStage.StageBasis.pack`, `PackingStage.StageBasis.coord`
Lemma 2.7, with the basis as independence and membership in T	`PackingStage.unpack_tower`
Definition 2.8	`PackingStage.StageBasis.form`
Lemma 2.9	`PackingStage.StageBasis.linear_identity`
Definition 2.10, as the code's parameters with a unique-decoding radius	`RSCode`
Definition 2.11	ArkLib's `Code.uniqueDecodingRadius`, with `ReedSolomon.uniqueDecodingRadius_RS_eq`
Definition 2.13	`RSCode.LineCertificate`
Lemma 2.14	part (1): `RSCode.lineCertificate_johnson`, with ArkLib's closed form `ReedSolomon.HiddenDerivative.johnsonExceptionCount_lt_closed`; part (2): `RSCode.lineCertificate_uniqueDecoding`
Lemma 2.15	`RSCode.lineCertificate_of_surplus`

Statement	Lean declaration
Definition 2.16, the verifier of a public-coin IOP with challenge-determined queries	`IOP.IOP`
Definition 2.17	`IOP.SPTerm.Leaf.Before`
Definition 2.18	`IOP.SPTerm`, `IOP.SPIOP`
Definition 2.19	`IOP.causalCompletenessCoins`
Definition 2.20	`IOP.CausalRBRKnowledgeSoundStmt`
Lemma 2.21	`IOP.rbrKnowledgeSoundness_total_iff`
Definition 2.22	`IOP.CausalHVZKStmt`
Definitions 2.23 and 2.24	`IOP.Reduction`, `IOP.Reduction.RBR`, `IOP.Reduction.Complete`, `IOP.Reduction.HVZK`
Definition 2.25(1), (2) and (4)	`RowsHold` (the projection onto the elements of the oracle input in one field), `FieldProof.MaskedReduction.OutHolds` (claims at a value map), `IOP.relConj`
Definition 2.26	`IOP.Reduction.seq`, `IOP.Reduction.par`
Lemma 2.27	`IOP.Reduction.RBR.seq`, `IOP.Reduction.RBR.par`, `IOP.Reduction.Complete.seq`, `IOP.Reduction.Complete.par`, `IOP.Reduction.HVZK.seq`, `IOP.Reduction.HVZK.par`
Lemma 2.27, for public-coin IOPs with input and output claims and no pass-through part	`IOP.CausalRBRStmt.seq`, `IOP.CausalRBRStmt.par`, `IOP.CausalComplete.seq`, `IOP.CausalComplete.par`, `IOP.CausalHVZK.seq`, `IOP.CausalHVZK.par`
Lemma A.2	`IOP.Reduction.rbr_iff_prefixRBR`
Lemma A.3, composed in series after a reduction	`FieldProof.MaskedReduction.Sound.combine`, `FieldProof.MaskedReduction.Honest.combine`
The relation language (Section 3 and Appendix B)	
Definition 3.1	`RealizedProgram.IndexMap`
Definition 3.2	`RealizedProgram.Value`
Definition 3.3	`RealizedProgram.Expr`
Definition 3.4	`RealizedProgram.evalIntWith`
Definition 3.5, with $\mathsf{link}(x, y)$ stating that every entry of y is below 2^w and that, if every entry of x is below 2^w, x and y represent equal naturals. When x holds bits, this is $\mathsf{link}(x, y)$	`RealizedProgram.HoldsWith`
Definition 3.6, with the honest prover's randomized functions supplied	`RealizedProgram`
Definition 3.7	`RealizedProgram.ChallengeOrder`
Definition 3.8	`RealizedProgram.IsLate`
Definition 3.9, with the honest prover's randomized functions supplied, with statements that pass the program's membership checks and with no type-set condition on their values	`RealizedProgram.StatementIn`, `RealizedProgram.SourceAcceptingAt`, `RealizedProgram.SourceProver.execEnvAt`

Statement	Lean declaration
Definition B.3 and Definition 3.10(1), with the honest prover's randomized functions supplied, with acceptance including the field equations of the field-form congruences	`RealizedProgram.SourceProver`, `RealizedProgram.SourceComplete`
Definition 3.10(2)	`IOP.rbrKnowledgeSoundness` at `RealizedProgram.sourceVerifier` and `RealizedProgram.sourceExtractor`; for the paper's $\Pi_{\mathcal{P}_0}$, `RealizedProgram.rbr_paper_of_source`, `RealizedProgram.rbr_source_of_paper`
Definition B.1, in one challenge	`RealizedProgram.exprDeg`
Definition B.2, with the pads of the late values in the first message	`RealizedProgram.sourceSpec`, `RealizedProgram.sourceVerifier`
Lemma B.4(1), for records of arity 1	`PermutationEvent.multiset_eq_or_badAt`, with the bound `ChallengeEvent.probUniform_badAt_le`
Lemma B.4(2)	`PermutationEvent.probUniform_prodPoly_eq_le`, `PermutationEvent.probUniform_mset_le_of_prodPoly_ne`, `PermutationEvent.probUniform_mset_joint_le`
The compiler (Section 4)	
Definition 4.2, with one interval for all entries of an expression, for linear maps among the library's operators, and without pub coefficients	`RealizedProgram.Expr.ivl`
Lemma 4.3(1), (3) and (4), from the equation in K to the predicate	part (1), for a quotient computed from e and e': `RealizedProgram.holdsWith_mod_of_oneDigit`; for a quotient committed as digits: `RealizedProgram.holdsWith_mod_of_digits`, `RealizedProgram.holdsWith_mod_of_committed`, `RealizedProgram.holdsWith_mod_of_grouped`; part (3): `RealizedProgram.holdsWith_mod_of_fieldEq`; part (4): `RealizedProgram.holdsWith_mod_of_field`
Section 4.1.3, for the accumulators of a MAC	`RealizedProgram.GroupedMod`
Definition 4.4	`Conv.matrix`, `Conv.convMax`
Section 4.1.4	`RealizedProgram.MacRealization`
Lemma 4.5, part (1) at the MACs of a realized program, with $\mathsf{link}(x, y)$ stating that every entry of y is below 2^w and that, if every entry of x is below 2^w, x and y represent equal naturals	part (1): `RealizedProgram.crossHolds_or_macBad_of`, `RealizedProgram.crossHolds_or_macBad_twoTail_of`; part (2): `MAC.probUniform_bindingEvent_le`
Definition 4.6	`RingSwitching.Tree`
Definition 4.7	`RealizedProgram`
Definition 4.7, the condition that the realized program realizes its source program, checked from the certificates	`RealizedProgram.Derivable`
Definition 4.8, with further well-formedness conditions	`RealizedProgram.Valid`

Statement	Lean declaration
The proof system (Section 5 and Appendices D to F)	
Theorem 5.3(1)	`RealizedProgram.ProofSystem.FieldBranches.completeness_coins`, `RealizedProgram.ProofSystem.FieldBranches.hvzk_coins`, from `RealizedProgram.SourceComplete.coinComplete`; for a deterministic honest prover, at every value of the pads, `RealizedProgram.ProofSystem.FieldBranches.completeness`, `RealizedProgram.ProofSystem.FieldBranches.hvzk`; `RealizedProgram.HonestStrategy.complete`
Theorem 5.3(2)	`RealizedProgram.ProofSystem.FieldBranches.causal_rbr_knowledge_soundness`
Theorem 5.3, for every finite field with an admissible packing tree (Definition 4.6), with tableaus with or without the mask test	`RealizedProgram.ProofSystem.fieldBranches`, from `RealizedProgram.fieldProof`, `RealizedProgram.fieldSound` and `RealizedProgram.ProofSystem.ofFieldProofs`; the class `RealizedProgram.FieldInterp.Packing`, with every finite field in it by `RealizedProgram.FieldInterp.exists_packing_realizes`; the exported checks `RealizedProgram.fieldOkB_mono`, `RealizedProgram.fieldOkB_generic`, `RealizedProgram.FieldReady.generic`; the masking of each tableau, with its round errors, `Ligero.MaskRow.TreeLayout.Masking`; under the hypothesis `RealizedProgram.Interp.Model` that Lean's fields follow their specifications, the MAC fields have polynomial bases and the functions computed from the statement have their stated ranges, proved for each configuration
Section 8.3, the assumptions on the compiled GKR circuits	`RealizedProgram.CircuitsEnforce`, `RealizedProgram.CircuitsAccept`
Lemma 5.5	parts (1) and (2): `Range.weightedArgument_prob_le`, `Range.weightedArgument_complete`; part (3): `Range.committedArgument_prob_le`, `Range.committedArgument_complete`
Lemma 5.7(1), as the bound on the events of the realization step in the proof of Theorem 5.3	`RealizedProgram.ProofSystem.eventBound_lowering`, with the errors `RealizedProgram.realError`; each group's LogUp identity, for a range table and by Lemma 5.5(3) for any other table, `RealizedProgram.needlesIn_or_logupBad`; each direct lookup $\mathsf{lookup}(e, e')$ from its groups' identities, for e' a range, a table, or an expression whose coordinates are the table's rows, `RealizedProgram.memHolds_of_lookup`
Lemma 5.7(2)	`RealizedProgram.HonestCore.ofCompletion`
Lemma 5.7(3)	`RealizedProgram.honestRun_uniform_of_respectsReads`, from `RealizedProgram.ProofSystem.PhaseLayout.PadOrder.honestRun_uniform`

Statement	Lean declaration
Lemma 5.8	`BatchedZerocheck.MergedAffineBatch.fusedSound`, `BatchedZerocheck.MergedAffineBatch.fusedHonest`, `BatchedZerocheck.MergedAffineBatch.fusedHvzk`, with the errors `BatchedZerocheck.MergedAffineBatch.fusedError_gammaRho`, `BatchedZerocheck.MergedAffineBatch.fusedError_sum`; without merging, `BatchedZerocheck.AffineBatch.sound`, `BatchedZerocheck.AffineBatch.honest`; in the proof system, with the merging challenge and the zerocheck point as separate verifier messages of errors $(n_Q - 1)/\|K\|$ and $m/\|K\|$, `BatchedZerocheck.MergedAffineBatch.sound`, `BatchedZerocheck.MergedAffineBatch.honest`, `BatchedZerocheck.MergedAffineBatch.hvzk`, with the errors `BatchedZerocheck.MergedAffineBatch.error_sum`
Lemma 5.8, for operands that are bits	`BatchedZerocheck.MergedAffineBatch.bitSumcheckSound`, `BatchedZerocheck.MergedAffineBatch.bitHvzk`, with the errors `BatchedZerocheck.MergedAffineBatch.bitError_sum`; without merging, `BatchedZerocheck.AffineBatch.bitSumcheckSound`, `BatchedZerocheck.AffineBatch.bitHvzk`
Lemma D.8	`FieldProof.MaskedReduction.Sound.bitSumcheck`, from `BitSumcheck.sound`; `BitSumcheck.claims_honest`; `FieldProof.MaskedReduction.Honest.SimBijective.bitSumcheck`
Lemma 5.9	`GKR.LayeredCircuit.maskedSound`, `GKR.LayeredCircuit.maskedHonest`, `GKR.LayeredCircuit.hvzk`, with the errors `GKR.LayeredCircuit.error_sum`
Lemma 5.10, for one row, as a count of samples	`Ligero.SubfieldTest.card_filter_mem_le_of_agrees`
Definition 5.11	`TreeSpec`
Lemma F.2, composed in series after a reduction	`FieldProof.MaskedReduction.Sound.measure`, `FieldProof.MaskedReduction.Honest.measure`; the honest masks `MaskKit.subfield`
Protocol F.3, the measurement after the combination	`FieldProof.MaskedReduction.measure`, with `MaskScheme.squaringChainOver`
Lemma F.4, composed in series after a reduction	`FieldProof.MaskedReduction.Sound.measure`, `FieldProof.MaskedReduction.Honest.measure`; the honest masks `MaskScheme.squaringChainOverHonest`, on all data with the product constraints by `MaskScheme.squaringChainOverHonest_commits`; as a kit of a packing tree, `MaskKit.chain`
Theorem F.6	`RingSwitching.Tree.reduces`, `RingSwitching.Tree.complete`, `RingSwitching.Tree.hvzk`
Corollary F.7(1), with the soundness error equal to the bound	`RingSwitching.TowerSetup.reduces`
Corollary F.7(2)	`RingSwitching.TowerSetup.complete`
Corollary F.7(3)	`RingSwitching.TowerSetup.hvzk`
Definition D.1	`BatchedZerocheck.MergedAffineBatch`, `BatchedZerocheck.MergedAffineBatch.Holds`
Definition D.2	`Sumcheck.FieldSkip.SkipPoints`
Definition D.3	`GKR.ClaimEvaluation.mle`, `EqPolynomial.eqPoly`
Definition D.4	`Sumcheck.FieldSkip.zeroPoly`
Definition E.1	`GKR.LayeredCircuit`, `GKR.LayeredCircuit.Satisfied`

Statement	Lean declaration
Ligero (Section 5.6 and Appendix G)	
Definition 5.12	`Ligero.AgreementThreshold.Threshold.commitError`, `Ligero.MaskTest.error`, `Ligero.UntestedMask.error`
Lemma 5.13(1)	`Ligero.MaskTest.sound`
Lemma 5.13(2)	`Ligero.MaskTest.complete`
Lemma 5.13(3)	`Ligero.MaskTest.hvzk`
Definition G.1, as $\mathsf{List}_d(u, a+1)$	`PolynomialAgreementList.candidates`
Definition G.3	`Ligero.MaskedConstraintList.johnsonAgreement`
Definition G.5	`Ligero.Tableau.Parameters`, `Ligero.Tableau.Commitment`, `Ligero.HVZK.commitOf`
Lemma G.9	part (1): `Ligero.MaskPieces.exists_splitting_vanishing`; part (2): `Ligero.MaskPieces.exists_kernel_eval`; part (3): `Ligero.MaskPieces.covers_three`, `Ligero.MaskPieces.not_covers_fin_two`
Lemma G.10	the linear test: `Ligero.UntestedMaskRow.linear_query_count`, `Ligero.UntestedMaskRow.linear_bad_count`; the quadratic test: `Ligero.UntestedMaskRow.quad_query_count`, `Ligero.UntestedMaskRow.quad_bad_count`; for the test that the verifier selects, `Ligero.UntestedMaskRow.selected_query_count`, `Ligero.UntestedMaskRow.constraint_bad_count`; its last sentence: `Ligero.UntestedMaskRow.Parameters.of_johnsonAgreement`
Corollary G.11, part (1) of Lemma 5.13	`Ligero.UntestedMask.sound`
Corollary G.11, part (2) of Lemma 5.13	`Ligero.UntestedMask.complete`
Corollary G.11, part (3) of Lemma 5.13	`Ligero.UntestedMask.hvzk`
Lemma G.12, in the round-by-round form of Chiesa, Manohar and Spooner, equivalent to Definition 2.20 by Lemma 2.21	`Ligero.LongfellowOrder.rbr_knowledge_soundness`, with the extractor `Ligero.LongfellowOrder.extractor` and the state function `Ligero.LongfellowOrder.stateFn`; part (1): `Ligero.LongfellowOrder.card_dataList_le`, `Ligero.LongfellowOrder.card_dataList_le_one`, from `Ligero.InterleavedAgreementList.card_le`, `Ligero.InterleavedAgreementList.card_le_one`; the state function's list, over the data rows and the low-degree mask row, projects onto the list of part (1) by `Ligero.LongfellowOrder.succ_mem_dataList`
Section G.7, the round errors at Longfellow version 7's parameters	`Ligero.LongfellowOrder.V7.hashJoint_le`, `Ligero.LongfellowOrder.V7.hashColumn_le`, `Ligero.LongfellowOrder.V7.hash_error_le` (hash tableau); `Ligero.LongfellowOrder.V7.sigJoint_le`, `Ligero.LongfellowOrder.V7.sigColumn_le`, `Ligero.LongfellowOrder.V7.sig_error_le` (signature tableau)
Counterexamples to the published analyses (Section G.8)	
Ames et al.'s Lemma B.9 fails, at $t = 1$ and $t = 2$	`Errata.AHIV22.lemmaB9_false_one`, `Errata.AHIV22.lemmaB9_false_two`
The quadratic term of Ames et al.'s Appendix C fails, at $t = 1$ and $t = 2$	`Errata.AHIV22.appCQuadratic_false_one`, `Errata.AHIV22.appCQuadratic_false_two`

Statement	Lean declaration
Ames et al.'s Lemma 4.14 holds with arbitrary mask rows and an arbitrary blinding row, also with the field term $(e+1)/\lvert\mathbb{F}\rvert^{\sigma}$	`Errata.AHIV22.UntestedMasks.lemma414`, from `Errata.AHIV22.UntestedMasks.Setup.acceptProb_le_of_not_satisfiable`
Cascudo et al.'s Theorem 4.4 fails, at code length n and at Longfellow's encoded length $n+2k-1$	`Errata.CHVY25.theorem44_false`, `Errata.CHVY25.theorem44_false_ncol`
The sum of Cascudo et al.'s Theorems 4.3 and 4.4 fails for every k from 13 to 30, at both code lengths	`Errata.CHVY25.Family.theorem44Summed_false_family`, `Errata.CHVY25.Family.theorem44Summed_false_family_ncol`
Section 5.6.1, the repaired bound, is Lemma G.10	`Ligero.UntestedMaskRow.selected_query_count`, `Ligero.UntestedMaskRow.constraint_bad_count`
Series–parallel BCS (Section 6)	
Definition 6.1	`Quantum.QueryBoundedAdversary`, `ROM.QueryBoundedAdversary`
Definition 6.2, in the QROM for finite statement, witness and proof sets, to which the configurations map theirs	`NARG.QROM.EverlastingZeroKnowledge`, `NARG.ROM.EverlastingZeroKnowledge`, with the proof queries `QROM.ProofQuery`
Definition 6.3, for the arguments of spBCS	`SPBCS.WitnessIndistinguishableOfLength`
Definition 6.4, for the arguments of spBCS, at the database extractor and $\varepsilon_{\mathrm{sim}} = 0$	`SPBCS.WeakOnlineSimulationExtractable`, with its two conditions in `SPBCS.WeakOnlineSimExtracts`; `SPBCS.WeakOnlineSimulationExtractableROM`, with its two conditions in `SPBCS.WeakOnlineSimExtractsROM`
Theorem 6.6	`SPBCS.complete_qrom`, `SPBCS.complete_rom` (part (1)), `SPBCS.privacyGuarantees` (parts (3) and (4)), `SPBCS.knowledgeGuarantees` (parts (2) and (5)), for every Merkle opening format (`SPBCS.MerkleOpening`), among them the authentication paths (`SPBCS.PerLeaf.opening`)
Theorem 6.6, with the authentication paths of each tree compressed into one multiproof	the declarations for Theorem 6.6 at the multiproof format (`SPBCS.Multiproof.opening`), which the configurations use. The honest multiproof is the compression of the honest authentication paths (`SPBCS.Multiproof.compress_path`). An accepted multiproof yields accepted authentication paths (`SPBCS.Multiproof.check_decompress`) whose queries are among its own (`SPBCS.Multiproof.trace_decompress_subset`)
Definition H.1	`NARG.Prover`; `NARG.QROM.Verifier`, `NARG.ROM.Verifier`
Definition H.2	`NARG.QROM.Complete`, `NARG.ROM.Complete`
Definition H.3, in the QROM with a classical auxiliary input of the adversary, the trace distance bounded at each value of it and averaged	`NARG.QROM.Adversary.OnlineExtracts`, `NARG.ROM.Adversary.OnlineExtracts`
Definition H.4	`NARG.QROM.Adversary.acceptsFalse`, `NARG.ROM.Adversary.acceptsFalse`
Lemma H.5	`NARG.QROM.Verifier.acceptsFalse_le_of_outcomeDistLE`, `NARG.ROM.Adversary.acceptsFalse_le`

Statement	Lean declaration
Definition H.6, in the QROM for finite statement, witness and proof sets, to which the configurations map theirs	`NARG.QROM.ZeroKnowledge`, `NARG.ROM.ZeroKnowledge`
Definition I.1	`SPBCS.kqOf`, `SPBCS.simQueries`, `SPBCS.Merges.nJ`, `SPBCS.Base.overrun`, `SPBCS.keyCollision`, `SPBCS.leafBound`
Theorem I.3(1)	`SPBCS.prob_not_accepts_prove_le`
Theorem I.3(2)	`SPBCS.complete_qrom`, `SPBCS.complete_rom`, `SPBCS.prob_not_accepts_prove_coins_le` (a fixed statement)
Theorem I.5(1)	`SPBCS.everlastingZeroKnowledge_qrom`
Theorem I.5(2)	`SPBCS.everlastingZeroKnowledge_rom`
Theorem I.6	`SPBCS.unconditionallyWitnessIndistinguishable`, from `SPBCS.wiAdvantage_le`
Lemma I.10, in a per-execution-path form: at most T queries other than dummy queries on each execution path, with [CMS19, Lem. 5.9] and [AHU18, Thm. 3] proved for this form	`Quantum.Alg.toQueryAlgCoh` with `QROM.QueryCount`; `QROM.Lifting.measured_le_of_queryCount`, `Quantum.O2HSetting.o2h_fresh_secret_of_count`
Lemma I.11, for the algorithm of the proof of Theorem I.16, at each value of its output and verdict	`SPBCS.QROM.extractionOutputState_eq`
Lemma I.12	`SPBCS.instab_bad_union_le`, from `SPBCS.condInstab_bad_le` and `QROM.instab_digestCollision_le`; with step 6 of the proof of Theorem I.17, `SPBCS.dbProb_bad_union_le_of_queryCount` (at the per-execution-path query count `Quantum.AdaptiveIAlg.tpQueryCount`)
Theorem I.16(1)	`SPBCS.onlineExtracts_qrom`, from `SPBCS.knowledgeBound` and `SPBCS.QROM.onlineExtracts`; the clause on the joint state in the field `state_clause_qrom` of `SPBCS.KnowledgeGuarantees`, which `SPBCS.knowledgeGuarantees` proves
Theorem I.16(2)	`SPBCS.onlineExtracts_rom`, from `SPBCS.jointBound` and `SPBCS.ROM.onlineExtracts`
Theorem 6.6(2), soundness	`SPBCS.soundness_qrom`, from `SPBCS.QROM.soundness`; `SPBCS.soundness_rom`, from `SPBCS.ROM.soundness`
Theorem I.17(1)	`SPBCS.weak_online_simulation_extractability_qrom`
Theorem I.17(2)	`SPBCS.weak_online_simulation_extractability_rom`
Definition I.13	`SPBCS.simExtGame`
Lemma I.14	`SPBCS.cover_simExtGame`
Definition 6.4 implies Unruh's Definition 14, inequality (6), at error $\varepsilon_{\mathrm{sim}} + \varepsilon_{\mathrm{SE}}$, for tests that are events of the adversary's measured output (Section 6.1.2), for the extractor of Theorem I.17, at $\varepsilon_{\mathrm{sim}} = 0$	`SPBCS.WeakOnlineSimulationExtraction.unruhSimExtraction`, `SPBCS.WeakOnlineSimulationExtractable.unruhSimExtraction_of_queryFreeTest`

Statement	Lean declaration
Definition 6.4 gives Chase and Lysyanskaya's extraction property in the QROM (Section 6.3), for the arguments of spBCS at the database extractor and $\varepsilon_{\mathrm{sim}} = 0$	`SPBCS.WeakOnlineSimulationExtractable.cl06Extraction`
Section 6.3: for the database extractor and tests that query the oracle, inequality (6) of Unruh's Definition 14 fails along families of complete instances	`SPBCS.oracleTest_eq6_fails_eventually`
End to end (Section 6.4)	
Theorem 6.7	`RealizedProgram.ProofSystem.FieldBranches.e2e_source`, from Definition 3.10(1) at every ε_{c}, with the guarantees `SPBCS.ProofSystemTranscript.CoinGuarantees`; for any honest prover, `RealizedProgram.ProofSystem.FieldBranches.e2e_coins`; for a deterministic honest prover, `RealizedProgram.ProofSystem.FieldBranches.e2e`, with the guarantees `SPBCS.ProofSystemTranscript.Guarantees`; part (2) alone, `RealizedProgram.ProofSystem.FieldBranches.e2e_knowledge`; the error of part (2) in the QROM, `SPBCS.knowledgeError_eq`
End to end, for the proofs of possession of ML-DSA-44 (Section 6.4)	
Theorem 6.7, its hypotheses on $\mathcal{P}_0$: round-by-round knowledge soundness, and completeness with $\varepsilon_{\mathrm{c}} = 0$	`MLDSA44.Emulated.ROMParams.source_rbr_knowledge_soundness`, `MLDSA44.Emulated.QROMParams.source_rbr_knowledge_soundness`; the honest strategies `MLDSA44.Emulated.ROMParams.honestStrategy`, `MLDSA44.Emulated.QROMParams.honestStrategy`
Theorem 6.7, the largest round error ε^{k}	`MLDSA44.Emulated.ROMParams.maxRoundError_le`, `MLDSA44.Emulated.QROMParams.maxRoundError_le`
Theorem 6.7(1), with numerical upper bounds on $\varepsilon_{\mathrm{ov}}$ and $\varepsilon_{\mathrm{key}}$	`Configuration.AdaptiveCompleteness.completeness_adaptive_qrom`, `Configuration.AdaptiveCompleteness.completeness_adaptive_rom`; for a fixed statement, `Configuration.Common.completeness`, and on the honest inputs, `Configuration.Common.completeness_honest`; at every pair of the relation, `Configuration.PaperRelation.completeness_of_rel`, at the honest input of the pair (`Configuration.PaperRelation.honest_input_of_rel`), valid with no pair excluded (`MLDSA44.PoP.HonestInput.valid_ofWitness`)
Theorem 6.7(2), at every t, with numerical upper bounds on ε^{k} and c' and, in the QROM, on $2\sqrt{k_q/2^{8(b+e)}}$, and on k_q, with statements the encoded keys at which each call of RejNTTPoly in ExpandA finishes within 1008 bytes of SHAKE128 output; at any other key no signature verifies	`Configuration.Common.knowledge_soundness_qrom_at`, `Configuration.Common.soundness_qrom_at`; `Configuration.Common.knowledge_soundness_rom_at`, `Configuration.Common.soundness_rom_at`

Statement	Lean declaration
Theorem 6.7(3)	`Configuration.Common.everlasting_zero_knowledge_qrom`, `Configuration.Common.everlasting_zero_knowledge_rom`; on the honest inputs, `Configuration.Common.everlasting_zero_knowledge_honest_qrom`, which every pair of the relation has at its statement (`Configuration.PaperRelation.honest_input_of_rel`)
Table 7, the zero-knowledge column for ML-DSA-44: $\varepsilon_{\mathrm{zk}}^{\mathrm{C}}(t,Q) \le Qt\,2^{-116.46}$ at the ROM parameters and $\varepsilon_{\mathrm{zk}}(t,Q) \le Qt\,2^{-180.19}$ at the QROM parameters, for every $t \ge 1$ and $Q \le 2^{64}$, with $\varepsilon_{\mathrm{c}} = 0$	`MLDSA44.Emulated.ROMParams.zero_knowledge_bits_rom`, `MLDSA44.Emulated.QROMParams.zero_knowledge_bits_qrom`
Theorem 6.7(4)	`Configuration.Common.unconditional_witness_indistinguishability`, on the honest inputs, which every pair of the relation has at its statement (`Configuration.PaperRelation.honest_input_of_rel`)
Theorem 6.7(5)	`Configuration.Common.weak_online_simulation_extractability_qrom`, `Configuration.Common.weak_online_simulation_extractability_rom`; in Unruh's weak fresh form, `Configuration.Common.unruh_simulation_extractability_qrom`
Results from the literature (Section 8.1)	
Schwartz–Zippel [Sch79, Zip79] (Lemma 2.1)	Mathlib's `MvPolynomial.schwartz_zippel_totalDegree`
Dao, Kominers and Thaler's Theorem 5.12 [DKT26], for lines (Lemma 2.14(1))	ArkLib's `ReedSolomon.exists_johnson_line_exactCorrelatedPair_ceil`
The interpolation certificate of Dao, Kominers and Thaler [DKT26, Lem. 3.12 and App. B], at the parameters (m, B, H) (Lemma 2.15)	ArkLib's `ReedSolomon.HiddenDerivative.exists_weighted_johnson_symbolic_certificate`
Dao, Kominers and Thaler's Johnson bound [DKT26, App. B.1] (Lemma G.2)	`PolynomialAgreementList.card_candidates_le`
Dao, Kominers and Thaler's Corollary 7.2 [DKT26] (Theorem G.4)	ArkLib's `ReedSolomon.exists_affine_exceptionalSet_full_agreement_of_exactLine`
Zhandry's Lemma 3 [Zha18]	`QROM.stO_eq`
Zhandry's Lemma 4 [Zha18]	`QROM.reduce_simState`; for the measured output, `QROM.simLaw_out`
Zhandry's Lemma 5 [Zha18]	`QROM.oracle_sim_sets`
Chiesa, Manohar and Spooner's Lemma 5.7 [CMS19], for properties that exclude the empty database	`QROM.Lifting.basic_lifting`
Chiesa, Manohar and Spooner's Theorem 8.6 [CMS19]	`SPBCS.QROM.soundness`, `SPBCS.QROM.onlineExtracts`
Ambainis, Hamburg and Unruh's one-way-to-hiding lemma [AHU18, Thm. 3], for pure and for mixed adversaries	`Quantum.O2HSetting.o2h`, `Quantum.O2HSetting.o2h_mixed`

Statement	Lean declaration
The interleaved proximity lemma of Ames et al. [AHIV22, Lem. 4.2], jointly over its repetitions	`InterleavedProximity.card_close_mul_le`
Berlekamp–Welch decoding	`RSCode.BerlekampWelch.decode_eq_some_iff`
Correlated agreement for affine spaces at unique decoding [BCI$^+$20, Thm. 1.6]	`RSCode.affineProximity`

Lean states Theorem 6.7 for each configuration from Theorems 5.3 and 6.6, which hold for every valid realized program and every series–parallel IOP. Each configuration restates the declarations under `Configuration` in the rows for Theorem 6.7 in its own namespace, with the same last name component.